
XFaceMaker/UIMS

Version 4.0/Motif 2.1

User's Guide

Nova Software Labs

© Copyright Nova Software Labs S.A.R.L. 1993 -2002.

All rights reserved. No part of this document may be stored in a retrieval system, transmitted, or otherwise copied or reproduced by any electronic, mechanical, photographic, or recording process without express written permission from Nova Software Labs S.A.R.L.

Published by:



Nova Software Labs

8, rue de Hanovre

75002 Paris, France

Tel: +33 1 58 18 61 61

Fax: +33 1 58 18 61 60

email: sales@NovaSoftwareLabs.com or sales@nsl.fr.

Web: <http://www.NovaSoftwareLabs.com> or <http://www.nsl.fr>.

Every precaution has been taken to ensure the accuracy of this document. Nova Software Labs S.A.R.L. assumes no responsibility for errors or omissions, or for damages resulting from the use of this document.

XFaceMaker is a registered trademark of Nova Software Labs S.A.R.L. X Window System is a trademark of the X Consortium. UNIX is a registered trademark of Novell. OSF/Motif and Motif are trademarks of The Open Group, Inc. All trademarks are the property of their respective owners.

Document Number: XFMUIMSUserMan v. 7.1.1.

Document date: November 29, 2002

Acknowledgments

The following people have contributed to the development of XFaceMaker:

Alain Bastard, Genevieve Beaujard, Nicole Brion, Alan Char, Eric Durocher, Marc Durocher, France Geze, Eric Herbaut, Elizabeth Huffer, Ion Filotti, Jérôme Maloberti, Sara Sautin, Eric Touchard, Philippe Volle.

Many thanks to all of them and to several others.

Table of Contents

Table of Contents	v
List of Figures	xvii
List of Tables	xxi
PREFACE	xix
1. Introduction	1
1.1. What are Widgets?	2
1.2. Designing GUI's with XFaceMaker	3
1.2.1. The X/Motif application in the absence of a GUI builder	3
1.2.2. The Seeheim model of an application and its GUI	4
1.2.3. Classification of GUI builders	5
1.2.4. Implementation of the Seeheim model in XFaceMaker	7
1.2.5. Advantages of separating the GUI and the application	7
1.3. Examples of Interfaces Built with XFaceMaker	8
1.4. The Phases of Interface Design	13
1.5. Other NSL Products Used with XFaceMaker	14
1.6. The XFaceMaker Product Family	15
1.7. Using This Manual	16
2. Overview	17
2.1. How to Use XFaceMaker	18
2.2. Laying Out the Interface	19
2.3. Specify and Test Interface Behavior	23
2.4. Interface Description Formats	28
2.5. Building the Application	29
2.6. Creating Reusable Components	32
2.7. The Dual-Process Architecture	35
2.8. Internationalization	37
3. Getting Started	39

3.1. Starting XFaceMaker	40
3.2. The Command Window	41
3.2.1. The Editing Section	42
3.2.2. The Toolkit Section	47
3.2.3. The Message Section	49
3.3. Opening Files	50
3.4. Saving Your Work	52
3.4.1. Saving Project Files	54
3.5. Save Options	55
3.6. Clearing the Interface	56
3.7. Quitting XFaceMaker	56
4. Laying out the Interface	57
4.1. Choosing and Placing Widgets	58
4.2. Selecting Widgets and Gadgets	59
4.2.1. Selecting Primitive Widgets	60
4.2.2. Selecting Composite or Shell Widgets	60
4.2.3. Selecting Multiple Widgets	60
4.3. Naming Widgets	62
4.4. The Widget Tree and Widget List	64
4.4.1. The Widget Tree	64
4.4.2. The Widget List	66
4.5. Moving Widgets	67
4.6. Cut, Copy, Paste, Duplicate, Delete	68
4.6.1. Cutting Widgets	68
4.6.2. Copying Widgets	68
4.6.3. Pasting Widgets	69
4.6.4. Duplicating Widgets	69
4.6.5. Deleting Widgets	69
4.6.6. Gadgets	70
4.7. Resizing a Widget	71
4.8. Tracking a Widget Name	71
4.9. Raising and Lowering Widgets	72
4.10. Hiding and Showing Widgets	73

4.11. Aligning Widgets	74
4.11.1. Aligning Widgets Relative to Other Widgets	75
4.11.2. Aligning Widgets by Size	75
4.11.3. Aligning Widgets on a Grid	76
4.12. Printing an Interface or Widget Tree	77
4.13. Setting Print Options	78
5. Resource Editing	81
5.1. General Procedures	82
5.2. The Widget Resources Window	84
5.2.1. The Widget Identification Fields	85
5.2.2. The Resource List	88
5.2.3. Using Drag and Drop in the Resource List	89
5.2.4. The Resource List Icons	90
5.2.5. The “Fast” Resource List	91
5.2.6. Overriding the Default Apply Options	92
5.2.7. Selecting Sort Order and Display Options	94
5.2.8. The Resource Editor	96
5.2.9. The Resource Dialog Boxes	97
5.3. Editing Global Objects	100
5.4. Editing Color Resources	101
5.4.1. Using the Color Editor	101
5.4.2. Using the RGB Box	102
5.4.3. The Color Description File	103
5.5. Selecting Fonts	105
5.6. Creating and Editing Pixmap	106
5.6.1. Using the Pixmap Editor	107
5.6.2. Pixmap Tables	109
5.6.3. Pixmap and Bitmaps in XFaceMaker	110
5.7. Translations	111
5.7.1. Defining a Translation	112
5.7.2. Using Eval in Translations	114
5.8. Attachments	116
5.8.1. Using the Attachments Editor	116

5.9. Setting Default Resources for Your Interface	121
6. Editing and Debugging Scripts.	123
6.1. Writing FACE Scripts	124
6.2. Using a Custom Text Editor	126
6.3. Callback Scripts	127
6.4. Creation Scripts	131
6.5. FACE Functions and Named Scripts	132
6.6. Active Value Scripts	135
6.7. Debugging Scripts	136
6.8. Error Recovery.	139
6.8.1. Design Process Termination	139
6.8.2. Restarting the Design Process.	140
6.8.3. The -restore Option	141
6.8.4. Launching a Different Design Process	142
7. Menus	143
7.1. Menu Types	143
7.2. Accelerators and Mnemonics.	144
7.3. Using the Menu Editor.	145
7.3.1. The Representation Field	146
7.3.2. The Dialog Section	146
7.3.3. The Pushbuttons	147
7.4. Example—Editing a Menu	148
7.5. Creating Menus without the Menu Editor	152
7.5.1. Creating a Pulldown Menu	152
7.5.2. Creating a Popup menu	155
7.5.3. Creating an Option Menu	156
8. Active Values	157
8.1. Writing Active Value Scripts.	159
8.2. Allocating Storage	160
8.3. Defining a New Active Value	163
8.3.1. Interface.	164
8.3.2. Drag Source.	166

8.3.3. Drop Site	167
8.3.4. Widget Class	169
8.3.5. C++ Class.	171
8.4. Active Value Functions	172
8.4.1. New Active Value Functions.	172
8.4.2. Old-style Active Value Functions	176
8.5. Using Active Values—Examples.	178
8.5.1. Defining an Active Value	178
8.5.2. Manipulating strings	178
8.5.3. Getting a Widget ID.	180
8.5.4. Defining Multiple Active Values.	180
8.5.5. Using Per-Widget and Per-Call Active Values	181
8.5.6. FileSel.fm.	186
9. Modules: Groups and Templates	189
9.1. Using Groups	190
9.1.1. Defining a Group	192
9.1.2. Predefined Dialog Groups	194
9.1.3. Predefined Interface Elements.	194
9.2. Templates	196
9.2.1. Defining a Template	197
9.2.2. Choosing a Template Icon.	199
9.2.3. Loading an Existing Template.	199
9.2.4. Deleting a Template.	200
9.2.5. Modifying A Template	200
9.2.6. Editing a Template Instance	201
9.2.7. Template Resources.	201
9.2.8. Applying a Template	203
9.2.9. Interfaces Containing Templates	204
10. The Application	205
10.1. Generating Application Files Automatically	206
10.1.1. The Main File.	206
10.1.2. The Functions File	206
10.1.3. The Include File	207

10.1.4. The MakeFile	207
10.1.5. The Pattern Files	207
10.2. Building the Application	209
10.2.1. Modifying Application Files	211
10.2.2. Modifying Application Patterns	216
10.3. Write and Build Your Application.	217
10.4. Interpreted Interface.	218
10.4.1. Minimal Main Program.	218
10.4.2. Using Active Values and Application Functions	219
10.4.3. Loading Several Interface Files	220
10.4.4. Loading an Interface with Groups.	222
10.5. Compiled Interface	224
10.5.1. Minimal Main Program.	224
10.5.2. Using Active Values and Application Functions	225
10.5.3. Loading Two Interface Files	226
10.5.4. Loading an Interface With Groups	228
10.5.5. Stand-alone C-Code	230
10.5.6. C-Code With UIL	230
10.6. Building a New XFM	232
10.7. Compatibility with libFm_c and libFm_e	233
11. Generating C-Code	235
11.1. C-Code Interfaces	236
11.2. Generating the C-Code File	237
11.3. C Code Options	239
11.3.1. Description	240
11.3.2. The Command Line Options Table.	245
11.4. Specifying C/C++ Type Names.	247
11.4.1. Declare Types and Prototypes in FACE Scripts	247
11.4.2. Declaring Types and Prototypes from the Application.	248
11.5. C Code for a New Widget Class	249
11.6. Stand-alone C Code	250
11.7. C and Fm Mode Differences	253
12. Building Projects	255

12.1. Sub-dividing a Complex Interface	255
12.2. Combining Interface Files	258
12.3. Working with a Project.	258
12.4. Projects and Application Files	259
12.5. Undoing a Project.	259
13. Creating New Widget Classes.	261
13.1. Defining and Saving a Widget Class	262
13.1.1. Saving a Class in the Modules Menu.	262
13.1.2. Defining a Class in the Active Values Editor	264
13.1.3. Defining a New Resource	264
13.1.4. Xt Intrinsic Class Methods	266
13.1.5. Loading a Widget Class	267
13.1.6. Editing a Widget Class	267
13.1.7. Deleting a Widget Class	267
13.2. Generating C Code for a New Widget Class	268
13.3. Contents of the Class File.	269
13.4. Hints	271
13.4.1. Widget Size	271
13.4.2. Compound Widgets	271
13.4.3. File Organization	271
13.4.4. Stand-alone C for a New Class	273
13.4.5. Callbacks	273
13.4.6. Building A New XFaceMaker	274
13.4.7. Sub-Classing At Arbitrary Levels.	276
13.5. How XFM Generates Sub-Classes.	277
13.5.1. The Class Record.	277
13.5.2. The Widget Instance Record	277
13.6. Examples	278
13.6.1. Alert Label	278
13.6.2. The NumField Class	281
14. Developing in C++ with XFM	285
14.1. Generating C++ Code.	285
14.2. Calling Member Functions from FACE Scripts.	285

14.2.1. A Complete Example	287
14.3. Limitations	290
14.3.1. Argument Passing Rules	290
14.3.2. C++ Member Functions	290
15. Creating C++ Classes	293
15.1. Defining a C++ Class	294
15.1.1. Data Members	295
15.1.2. Member Functions	296
15.1.3. The this Pointer	297
15.2. Saving a C++ Class	298
15.3. Predefined Styles	299
15.3.1. The NSL Style	299
15.3.2. The Rogue Wave Style	299
15.4. Example	302
15.4.1. Building the Widget Tree	303
15.4.2. Defining the Class Members	303
15.4.3. Callbacks	306
15.4.4. Saving the Class	306
15.4.5. Using the Counter Class	307
16. Extending XFaceMaker	311
16.1. Loading an Extension	312
16.2. Extension Libraries	314
16.3. The “xfmextend” Command	315
16.4. Defining a New Extension	317
16.4.1. Build Information Declarations	317
16.4.2. Declaring Extension Objects	320
16.4.3. Additional Source Files	323
16.4.4. New Resource Editors	324
16.4.5. New Table Resource Editors	325
16.5. Example - Adding an Extension	326
16.6. XFaceMaker Extension Functions	331
16.6.1. New Widget Class	331
16.6.2. New Resources	331

16.6.3. Using A New Resource Type in XFaceMaker	334
16.6.4. Adding a New Resource Editing Group	335
16.7. Building a New XFaceMaker	338
16.7.1. Linking with Alternate Libraries	339
16.7.2. Generating and Linking C-Code Files	339
16.7.3. Subclassing User-Defined Widgets	340
16.8. Calling Extension Functions Indirectly	341
17. Controlling the Design Process	343
17.1. The Process Functions	343
17.2. Connection	343
17.3. Socket Addresses	345
17.4. Error Protection	346
17.4.1. Design Process Termination	346
17.4.2. Restarting the Design Process	347
17.4.3. Restoring the Design Process State	348
17.4.4. Launching a New Design Process	349
18. Working with UIL	351
18.1. Generating UIL Files	352
18.1.1. The Save UIL file Option	352
18.1.2. The -uil Command Line Option.	353
18.2. Saving UIL and C Code Files	354
18.3. Contents of the Generated UIL File	355
18.3.1. Structure of the UIL File	355
18.3.2. Widget Names	356
18.3.3. Shell Widgets	356
18.3.4. FACE Scripts	357
18.3.5. Active Values	358
18.3.6. Create Callbacks	358
18.3.7. Dialogs	359
18.3.8. Menus	359
18.3.9. User-defined widgets	359
18.4. Changing the Application	361
18.5. Translating UIL Files to Fm files	362

18.5.1. uil2fm Syntax and Options	362
18.5.2. UIL to Fm Translation Rules	363
18.5.3. UIL and the Application	370
19. RDB Files	377
19.1. Setting RDB Options	378
19.2. Saving an RDB file	380
19.3. Generating Multiple Files	381
20. Using Drag and Drop	383
20.1. Simple Drag-and-Drop	384
20.1.1. Simple XFM Drag and Drop Functions	384
20.1.2. Transferring Data Using Active Values	386
20.2. Defining Drag and Drop Operations	389
20.2.1. Drag Source.	389
20.2.2. Drop Site	390
20.3. Drag and Drop Functions.	392
20.3.1. Argument List Functions.	392
20.3.2. Drag Context Functions	393
20.3.3. Drop Site Functions.	394
20.3.4. Drop Transfer Functions	395
20.3.5. Display and Screen Functions.	397
20.3.6. Conversion and Transfer Structures	397
21. Internationalization.	401
21.1. General Concepts of GLS	402
21.2. Internationalizing Strings.	404
21.3. Saving Message Catalogs	405
21.4. Loading Message Catalogs	407
21.4.1. Within XFaceMaker	407
21.4.2. In Your Application	407
21.5. Internationalization Functions	408
22. Command Line Options	411
22.1. The Command Line Options	411

22.1.1. General Options	411
22.1.2. Environment Options	411
22.1.3. Editor Options	413
22.1.4. Compilation Options	414
22.1.5. Dual Process Options	417
22.1.6. Application Generation Options	418
22.1.7. Debug Options	418
22.1.8. XFaceMaker Extension Options	418
22.1.9. Standard X client	419
23. Environment Variables	421
24. The Fm File Structure	423
25. Editing Graphics in XFaceMaker	425
25.1. Graphics in XFaceMaker Format	426
25.2. Graphics in Draw File Format	427
25.3. Recommended Procedure	427
25.4. Graphics and C-code Generation	428
. Index	429

List of Figures

Figure 1.5:	NSL's Widget Tree extends the Motif widget set.....	12
Figure 3.1:	The Load Window	39
Figure 3.2:	The Command Window	41
Figure 3.3:	The Editing Section	42
Figure 3.4:	The Display and Edit Icons	44
Figure 3.5:	The Widget List and Tree	46
Figure 3.6:	The Toolkit.	47
Figure 3.7:	The Message Section	49
Figure 3.8:	The Message Icons	49
Figure 3.9:	The File Selector	50
Figure 3.10:	The Alert Window	56
Figure 4.1:	The Selection Handles	59
Figure 4.2:	The Widget Tree	64
Figure 4.3:	The Widget List	66
Figure 4.4:	The Cut, Copy, Paste, Delete Icons	68
Figure 4.5:	The Track Icon	71
Figure 4.6:	The Raise and Lower Icons	72
Figure 4.7:	The Alignment Box	74
Figure 4.8:	The Print Options Window	78
Figure 5.1:	The Widget Resources Window.....	84
Figure 5.2:	The Identification Fields	85
Figure 5.3:	The Resource List	88
Figure 5.4:	The Resource List Icons.....	90
Figure 5.5:	The "Fast" Resource List	91
Figure 5.6:	The Fast List Letters	91
Figure 5.7:	The Apply Options Box	92
Figure 5.8:	The Sort Order Options	94
Figure 5.9:	The Resource Editor	96
Figure 5.10:	The Integer Edit Box	97
Figure 5.11:	The Color Editor (Monochrome Screens)	101

Figure 5.12:	The Colors Box (Color Screens)	102
Figure 5.13:	The RGB Box	103
Figure 5.14:	The Fonts Editor	105
Figure 5.15:	The Pixmap Editor.	106
Figure 5.16:	The Pixmap Table Editor.	109
Figure 5.17:	The Translations Editor	112
Figure 5.18:	The Eval Script Editor	114
Figure 5.19:	The Attachments Editor.	117
Figure 6.1:	Editing a Callback Resource	129
Figure 6.2:	The Script Debugger	136
Figure 6.3:	The Restart Design Process Box	140
Figure 6.4:	The Mono-process Alert Box	141
Figure 7.1:	The Menu Editor	145
Figure 7.2:	Example: Using the Menu Editor	148
Figure 7.3:	The Widget List and Tree for the Example.	153
Figure 8.1:	Editing an “Interface” Active Value	164
Figure 8.2:	Drag Source.	166
Figure 8.3:	Drop site	167
Figure 8.4:	Widget Class	169
Figure 8.5:	Editing a C++ Class Type Active Value	171
Figure 8.6:	The “Edit” Main Window	181
Figure 8.7:	The Active Values for “Edit”	182
Figure 8.8:	The Resources List	184
Figure 9.1:	The File Selector and the Group Name Window	191
Figure 9.2:	Saving a Template	198
Figure 11.1:	The C Code Options Window	239
Figure 11.2:	Saving a C Code Version of a Class	249
Figure 12.1:	The Complete Interface	255
Figure 12.2:	Save Whole Interface Question Dialog	256
Figure 13.1:	The Save Class File Selector	263
Figure 13.2:	Defining a Class	264
Figure 13.3:	Saving a C code version of a Class	268

Figure 15.1:	Defining a C++ Class	294
Figure 15.2:	The Counter	302
Figure 15.3:	The Hierarchy.	303
Figure 17.1:	The Restart Process Box	347
Figure 18.1:	The UIL Options Window	352
Figure 19.1:	The RDB Options Window	378
Figure 19.2:	The RDB File Selector.	380
Figure 20.1:	Drag source	389
Figure 20.2:	Drop site.	391
Figure 21.1:	The Message Catalog Box	405

List of Tables

Predefined Group Widgets	194
Predefined Group Interface Elements	195
C Code Command Line Options	245
UIL Command Line Options	353
UIL: Dialogs and Convenience Objects	369
RDB Command Line Options	378
C-Code Command Line Options	416

PREFACE

Before You Start

This *User's Guide* is designed to give you practical assistance in using XFaceMaker. We assume that you are familiar with the C programming language and, to a lesser extent, with the Unix operating system, the X Window system, and the Motif widget set.

The XFaceMaker distribution includes a number of examples that illustrate how to do things. Please refer to the **demos** directory in the installed tree.

You should have access to a reference manual for the Motif widgets. The *Motif Style Guide* can help you with ideas on what you should be trying to achieve with your interface. Some of the books which describe the principles involved in both the X Window System and the Motif widget set are listed in this preface.

You should also know how to use a window manager and **xterm** or a similar terminal emulator, and you should be able to open, resize, iconify, and close windows.

About this manual

The *XFaceMaker User's Guide* describes the procedures to follow when designing an interface with XFaceMaker. It includes examples when appropriate. Concise descriptions of the FACE language, functions used in XFaceMaker, the editing windows, menus, and the Fm file structure can be found in the *XFaceMaker Reference Manual*.

XFaceMaker documentation

The following manuals are delivered with XFaceMaker:

- The *XFaceMaker User's Guide*
- The *XFaceMaker Reference Manual*
- The *XFaceMaker Extensions Manual*
- The *Installation Guide for NSL Products*
- The *Wx User's Guide*

Also available:

- The *XFaceMaker Tutorial*

What You Need to Run XFaceMaker

To run any application using the X Window System you need:

- A Unix host computer. The Unix host will run all the applications. The application makes calls to the *Xlib* library for displaying, and for processing keyboard and mouse events. Other libraries may be needed as well; e.g. the *Xt* (X Intrinsics), or the *Xm* (Motif) libraries. The host must be set up to communicate with other machines on a network (TCP/IP). The host system must include development software: compiler, linker, etc.
- A window manager. Although XFaceMaker can run with different window managers, it is guaranteed to work with the OSF/Motif window manager *mwm*.
- A graphic display consisting of a monitor, a keyboard and a mouse.

There are two main types of graphic displays on which you can run the X Window System (and thus XFaceMaker):

- *Unix workstations*. These are Unix computers with a graphic screen. They run the X server as a Unix process.
- *X terminals*. X terminals run only the X server as a stand-alone program. The application itself runs on the remote Unix host.

Motif based applications require a screen resolution of at least 1024 x 768 pixels. If you plan to use color, make sure you have 256 colors. It is unreasonable to attempt to create a Motif-based application with only 16 colors.

The following libraries must be installed on the host computer:

- *The Xlib*. Host applications call the Xlib library when they want to use the X server. The Xlib is usually installed as `/usr/lib/libX11.a`.
- *The X Intrinsics library*. Called the *Xt* library, it is usually installed as `/usr/lib/libXt.a`.
- *The Motif widget set*. It is usually installed as `/usr/lib/libXm.a`. This version is designed for Motif 2.1 or Open Motif 2.1.
- *The XFaceMaker libraries*:

`libFm.a` -- The interpreting library for using interpreted scripts.

`libFm_c.a` -- The C code library for fully compiled applications.

`libFm_e.a` -- The XFaceMaker executable library for applications which include XFaceMaker itself, or to rebuild XFaceMaker.

The following include files must be installed on the host computer:

- The Xlib and X Intrinsics include files. These are usually installed in `/usr/include/X11`.
- The Motif include files. These are usually installed in `/usr/include/Xm`.
- The XFaceMaker include file, `FmCommon.h`. This is usually installed in `/usr/include`.

Keyboard Conventions

Key combinations are expressed as follows:

Format	Example	Meaning
Key1--Key2	Ctrl--A	Hold down the C ontrol key as you press, then release key A . The S hift key is not used.
Key1 Key2	Esc a	Press and release the E scape key, then press and release key a .
	Esc A	Press and release the E scape key, then press and release key a while pressing the S hift key.

Notice how in the case of the **ESC** modifier the two keys are pressed in succession, whereas in the case of the **Ctrl** modifier the key is pressed *while* the **Ctrl** key is held down.

Typesetting Conventions

The following typesetting conventions are used in this manual:

Type style	Used for
typewriter	Anything you should type as it appears, code, and key combinations.
<i>italic</i>	Mouse buttons and emphasis.
<i>bold slant</i>	Menu items and editing commands.

Using the Mouse

We assume you are using a three button mouse. If not, you need to know which combinations of keyboard and mouse actions correspond to *Select*, *Menu*, and *Custom*.

Mouse buttons

Mouse buttons respond differently in Build mode and in Try mode. In Build mode the mouse buttons are used as follows:

Button1: *Select*---Used to select/deselect widgets on the desktop or in the Widget List or Tree, to select menu items, open a menu, or drag.

Button 2: *Menu*---Used to select widgets on the desktop, and popup the Resources or the Resource Editor boxes.

Button3: *Custom*---Used to select the *parent* of the *current* widget.

In Try mode most button actions are determined by the widget resources and callbacks defined for the particular interface. By default, PopupMenus are popped up using the Custom button.

Mouse actions

This manual uses the following terms for mouse actions:

Press Hold down a mouse button.

Click and double-click Quickly press and release once (click) or twice (double-click) the *Select* mouse button without moving the mouse. Clicking selects. Double-clicking is for editing actions.

Shift-click Hold down the Shift key, and click the left mouse button.

Drag Move the mouse while pressing the *Select* mouse button.

Rubber band A rectangular outline displayed when the cursor is dragged. One corner is fixed at the point where the button was pressed and the opposite corner follows the pointer. Releasing the button creates a rectangle that defines either the size of a widget or a selection area.

Outline box This is the outline of a widget displayed as the widget is moved so that it can be placed correctly.

Menu Naming Conventions

Menu names in the text are indicated as follows: button *Open* of menu *File* will be indicated as *File/Open*.

Technical Support

Technical Support is available:

- Monday to Friday (except for French national holidays).
- 9.00 am 12.30 pm – 2.00 pm to 6.00 pm (MET). (5 pm on Friday)
- From French and English speaking support staff.
- By phone
- By electronic mail addressed to `support@nsl.fr`
- By fax addressed to Technical Support including your return FAX number, and a telephone number where you can be reached.
- For email addresses and telephone numbers, please check the copyright page at the beginning of this document.

Related Documents

The following manuals provide more information on the X Window System and the OSF/Motif toolkit. Items marked with an asterisk (*) are especially useful. Refer to the most recent edition that corresponds to your version of X and Motif.

From the **X Window System Series** by O'Reilly and Associates:

Adrian Nye, ed. *Volume 0: X Protocol Reference Manual*. O'Reilly and Associates, Inc.

Adrian Nye. *Volume 1: Xlib Programming Manual*. O'Reilly and Associates, Inc.

Adrian Nye. *Volume 2: Xlib Reference Manual*. O'Reilly and Associates, Inc.

Tim O'Reilly, Quercia. *Volume 3: X Window System User's Guide (OSF/Motif Edition)*, O'Reilly and Associates, Inc.

Adrian Nye, Tim O'Reilly. *Volume 4: X Toolkit Intrinsics Programming Manual (OSF/Motif Edition)*. O'Reilly and Associates, Inc.

Adrian Nye, Tim O'Reilly. *Volume 5: X Toolkit Intrinsics Reference Manual (OSF/Motif Edition)*. O'Reilly and Associates, Inc.

Dan Heller. *Volume 6: Motif Programming Manual For OSF/Motif*. O'Reilly and Associates, Inc.

Tim O'Reilly. *The X Window System in a Nutshell*. O'Reilly and Associates, Inc.
From the **Open Software Foundation**:

Antony Fountain, Jeremy Huxtable, Paula Ferguson and Dan Heller. *Motif Programming Manual for Motif 2.1 - Open Source Edition. The Definitive Guides to the X Window System, volume 6A*. O'Reilly and Associates, 2001.

Antony Fountain and Paula Ferguson. *Motif Reference Manual for Motif 2.1 - Open Source Edition. The Definitive Guides to the X Window System, volume 6B*. O'Reilly and Associates, 2001.

Open Software Foundation. *OSF/Motif User's Guide*, Prentice Hall, Englewood Cliffs, New Jersey.

Open Software Foundation, *OSF/Motif Style Guide*, Prentice Hall, Englewood Cliffs, NJ.

Open Software Foundation, *OSF/Motif Programmer's Reference*, Prentice Hall, Englewood Cliffs, NJ.

Open Software Foundation, *OSF/Motif Programmer's Guide*, Prentice Hall, Englewood Cliffs, NJ.

CHAPTER 1: Introduction

Welcome to XFaceMaker, a family of interactive software tools for *designing, prototyping and producing* OSF/Motif graphical user interfaces (GUI's). The XFaceMaker family contains several tools suitable for a variety of different user needs.

XFaceMaker increases the productivity of the software developer in all stages of the development cycle: specification, prototyping, testing, maintenance and documentation. Productivity gains with respect to straight programming can easily reach a factor of ten in the early phases of the development cycle, and average from three to five over the whole development cycle. The quality of the code generated by XFaceMaker matches that produced by a good programmer.

XFaceMaker relies on two widely used software standards: the OSF/Motif toolkit and the X Window System. The X and Motif standards have been adopted by all workstation manufacturers, including Sun Microsystems, IBM, Hewlett-Packard, Digital Equipment Corporation, Silicon Graphics, and SCO. X and Motif are available not only on workstations, but also on PCs that run Microsoft Windows or Microsoft Windows NT, and on graphics X terminals that can be connected to all of the above systems.

In a client-server environment, X and Motif are widely used as a powerful and flexible graphics interface to applications running on remote servers. The standardization and wide acceptance of the X/Motif standard guarantees your investment on X/Motif graphical user interfaces for a long time to come.

1.1 What are Widgets?

Graphical user interfaces are made of standardized objects that are used systematically and give the interface consistency, predictability and ease of use. Examples of such objects are the text fields, the pushbuttons and the scrollbars that are now commonly seen on all graphic displays. Indeed one of the main goals of the Motif toolset is to define these standardized objects. In X and Motif parlance these graphic objects are called *widgets*.

Objects of the same nature are said to belong to the same *class*. For example, all Motif buttons belong to the XmPushbutton class. Two instances of the XmPushbutton class may have different attributes; for example, a different color for the background. In X it is possible to define new classes from old ones. The system that allows such definitions is called the X Intrinsics (abbreviated Xt).

XFaceMaker helps you design graphic interfaces that use the standard Motif widgets. The standard Motif widget set includes labels, buttons, text entry fields, toggle buttons, scrollbars, as well as *manager* widgets that can contain other widgets.

You can also extend XFaceMaker with third-party or your own widgets very easily. This is very useful since the standard Motif widget set does not satisfy all user needs. Once new widget classes have been incorporated into XFaceMaker, you can use them in designing your interface.

Even more, with XFaceMaker you can create your own widget classes. We shall discuss this in more detail below.

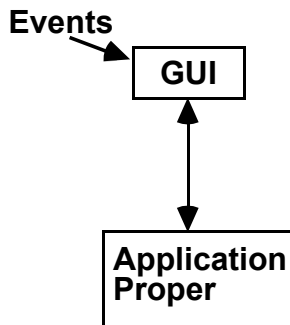
1.2 Designing GUI's with XFaceMaker

We will explain in this section how GUI builders, XFaceMaker, in particular, can simplify the design of GUIs and even of the whole application.

1.2.1 The X/Motif application in the absence of a GUI builder

The X Window System and Motif are implemented as function libraries intended to be called from a C application. These libraries implement fairly low level functions for the creation and manipulation of the objects that constitute the interface. The libraries do not further a particular design method and result in a lengthy development cycle.

In general, an application that uses the X Window System and Motif has the following structure:



An important notion of X is that of *event*. X events are significant events that occur on the interface, essentially, mouse clicks and keyboard strokes. An X application is *event driven* – it consists of a main loop that awaits an event and then triggers an action in the application.

The GUI part contains the code to create the graphic components – buttons, labels, scrollbars, windows and so on – that are part of the GUI. The application proper consists of the code that is *called back* when an interface event occurs.

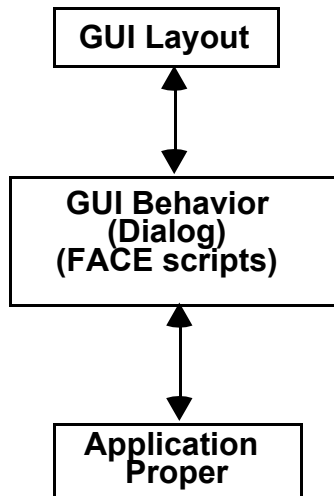
Every Motif widget has its given and unchanging set of events. For example, a Motif button has an *activate* event that is triggered when the user clicks with the mouse in the button. There are many kinds of events and we refer you to a description of the X Window System for more details.

Notice that to design the application with the X Window System and Motif, *you must program the GUI entirely*. If you want to place a button you call the button creation function to which you give as parameters the position, size and other graphic attributes (called *resources* in X) of the button. If you later want to change the GUI, you must change the program, recompile and debug – i.e. you enter the costly edit - compile - debug software development cycle.

A major characteristic of the X Window System concerns data transfer mechanisms. In X, the connection between application and GUI is only through callbacks. Nothing has been provided for the transfer of data. This shortcoming has been corrected by NSL in XFaceMaker through the active values mechanism.

1.2.2 The Seeheim model of an application and its GUI

The X Window System was not intended to prescribe the way applications with GUIs should be designed. A widely used decomposition of an application and its GUI in structural components is the so-called Seeheim model (named after the location of the technical conference where it was first formally defined). The Seeheim model is illustrated in the following figure:



The Seeheim model separates the layout and the dialog part of the GUI. This distinction is useful in guiding developers in the design of applications. The Seeheim model is independent of the graphics system used to implement the GUI, in particular of the X Window System.

The layout component consists of the graphic objects of the GUI and its geometric attributes. The behavior component is in charge of all the actions that have an effect on the interface itself. Two kinds of actions can be triggered by an event:

- *Interface actions* – actions that have an effect on the GUI itself. As a simple example, hiding a box when pressing the OK button. Often, such actions can be much more complex.
- *Application actions* – actions that are not related to the interface but are specific to the application. In the same example, pressing the OK button may also launch a computation in the application, whose results will eventually have to be displayed in the GUI

Interface actions determine the way the dialog between user and the application is executed. We call this GUI *behavior* or GUI *dialog*. Behavior is just as important as layout. The art of designing good GUIs depends as much on the layout design as on the efficient sequencing of windows and dialog boxes.

1.2.3 Classification of GUI builders

GUI builders help developers develop the graphical user interface of an application. In terms of the Seeheim model, the GUI builder assists in the design of the layout and of the behavior components of the GUI. One can distinguish two kinds of GUI builders:

- Interface Development Tools (IDTs) – tools that help only in the design of the layout component.
- User Interface Management Systems (UIMS) – tools that help design both the layout and the behavior component.

Interface development tools (IDTs)

With such a tool you only lay out the interface and generate C code for the GUI part. The developer must then provide the application proper which he inserts into *stubs*, i.e. slots that are provided for it in the code generated for the GUI. IDTs follow closely the application model provided by X and Motif. They only go halfway through in solving the GUI design problem.

A big portion of a GUI is graphical and hence an interactive CAD tool is much better for designing the GUI layout than a program library such as the Xlib and Motif.

User interface management systems (UIMS)

In the standard X/Motif programming model or when using a simple-minded IDT, GUI behavior must be handled by the application. IDTs solve only part of the GUI design problem and then not the biggest part. With an IDT it is impossible to fully test and debug the whole interface with the GUI builder.

Single toolkits versus multi-standard GUI builders

Single toolkit GUI builders use a toolkit in the native form in which it is made available by the computer manufacturer. XFaceMaker, for example, uses the X/Motif toolkit. The advantage is that they allow the full use of all the capabilities of a toolkit and of faithfully conform to the corresponding standard. The alternative is, as we shall soon see, to emulate the standard most of the time in incomplete fashion.

Multi-toolkit GUI builders intend to let you design GUIs for several graphic standards at once rather than redevelop it from scratch for each new toolkit. The main approach for multi-toolkit GUI builders is to implement a subset of widgets, common to the several graphic standards. This is often called the “least common denominator” approach. It has the drawback of leaving aside the best feature of many standards.

We believe that it is always better to respect and use standards. Thus XFaceMaker uses the full unmodified Motif toolkit and gives you access to all its resources with no exception.

1.2.4 Implementation of the Seeheim model in XFaceMaker

The XFaceMaker family supports the Seeheim model as follows:

- GUI layout is executed interactively with the tool. C code and a symbolic description of the GUI are generated by the tool.
- GUI behavior is specified and tested interactively with the tool. Behavior is specified in the form of procedural scripts in a high-level, C-like programming language called FACE. (Available in XFaceMaker/EL and the XFaceMaker product.)
- GUI behavior can be tested within the tool, that is with no need for outside compilation. XFaceMaker has a built-in interpreter and debugger for the FACE scripting language. (Available in XFaceMaker/EL and the XFaceMaker product.)

1.2.5 Advantages of separating the GUI and the application

The clear separation of GUI from the rest of the application realized in XFaceMaker has several advantages:

- The application proper is relieved of most aspects of GUI behavior.
- One can change the interface and in particular the behavior without changing the rest of the application.
- The GUI builder assists the GUI designer in all aspects of the GUI, and not just in the layout.
- In XFaceMaker, the GUI designer can test the interface together with its behavior and this speeds up considerably the development process

1.3 Examples of Interfaces Built with XFaceMaker

The graphic interface of an application can vary widely in complexity. XFaceMaker can be used to build interfaces of any complexity from the simplest to the most complex ones. Even for building the simplest interfaces, using XFaceMaker can result in substantial savings. Let us show some examples of the kind of interfaces one can build with XFaceMaker.

Some applications have a very simple interface, consisting of a single window and some buttons and text fields. The following example illustrates a simple calculator application.

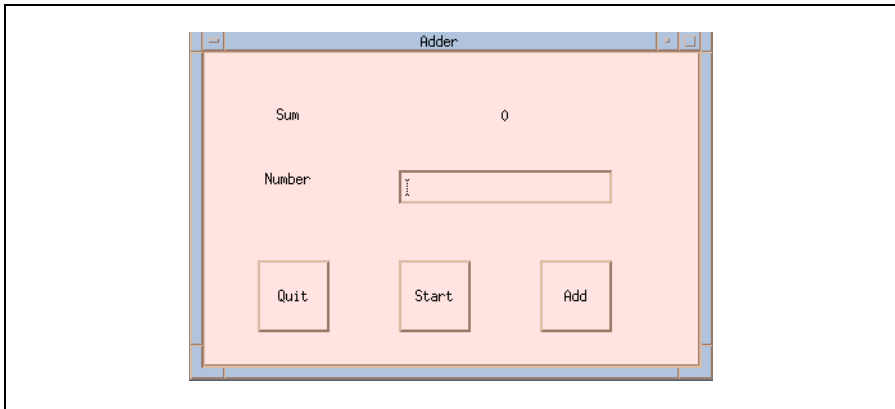


Figure 1.1: This is an example of a very simple interface created with XFaceMaker — an adder.

Other applications may have quite complex interfaces. As an example of a complex interface, XFaceMaker's own graphic interface, which involves over one thousand different widget instances. It has been designed entirely with XFaceMaker itself. There are close to one hundred different dialog windows for setting options, and for managing the numerous facilities offered by XFaceMaker.

Here are some examples of portions of XFaceMaker's graphic interface.

First there is the main window that you get when you call XFaceMaker.



Figure 1.2: XFaceMaker's command window was built with XFaceMaker.

XFaceMaker's command window is made out of standard Motif objects, mainly buttons and labels. Buttons can display pictures, the little icons shown above. There are also lists. The tree shown at the upper right is made from a proprietary widget of NSL. The tree exceeds the boundary of the window and you can show the hidden part by sliding the scrollbar. The two arrows to the right of the toolkit window can be used to scroll the contents of that window. Finally, the large text area at the bottom is used for displaying messages.

The main window includes a menu bar with pulldown menus. XFaceMaker has a special menu editor for designing such menus.

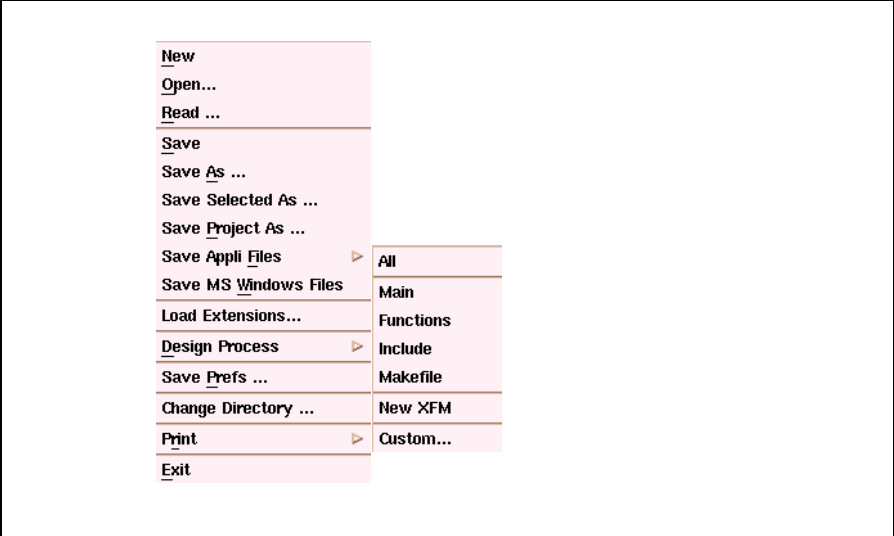


Figure 1.3: Designing menus is easy using XFaceMaker’s Menu Editor. This example combines pulldown and cascade menus.

Another example of a complex window is XFaceMaker's resource editor.

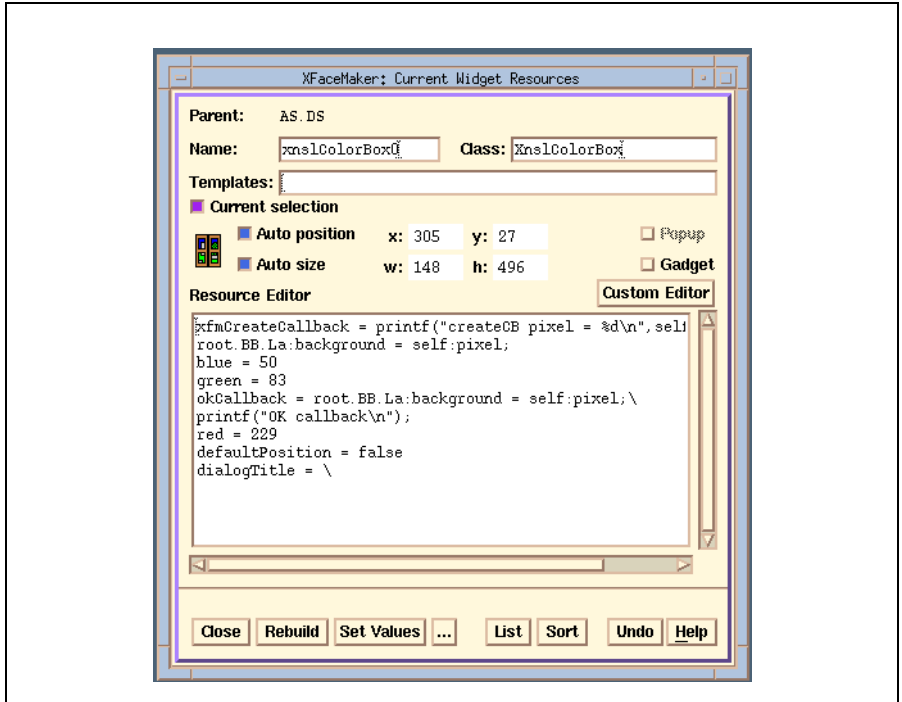


Figure 1.4: XFaceMaker's Resource Editor is built from Motif objects only.

This window is made out of Motif objects only. There are many labels with pictures in them since Motif labels or buttons can be set with pixmap backgrounds. Some windows are larger than can be shown at one time and there are then surrounding scrollbars to let you scroll through the contents. XFaceMaker will help you design such windows and layout the scrollbars and all the objects.

Dialog windows may also use widgets which are not available in Motif. As an example, consider the tree window that displays the hierarchy of widgets in your interface:

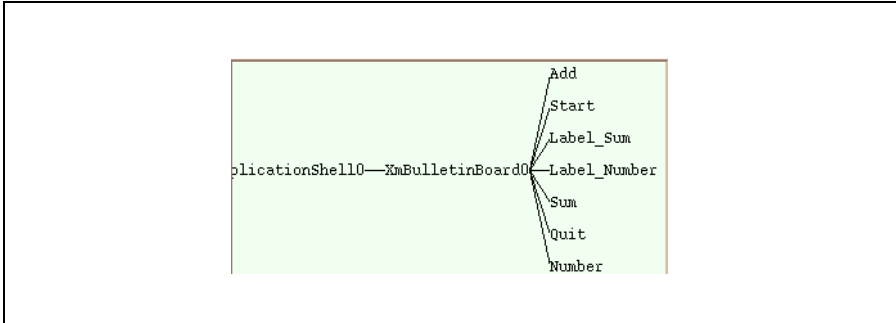


Figure 1.5: NSL’s Widget Tree extends the Motif widget set.

The tree represents widgets contained in others. To the right, lines connect the containing widget to the one contained. The leftmost widget is the root of the tree. In Motif, this is usually an `ApplicationShell` widget.

Motif does not provide a widget for representing trees, so we have designed our own and built it into the interface. In fact, the widget has been imported into `XFaceMaker` (or if you prefer, `XFaceMaker` has been extended with the new widget) and then the widget has been used in building the interface the same way as any other widget.

These examples illustrate how complex graphic interfaces can be. Interfaces may consist of numerous widgets and only some of them are shown at a given time. As you perform actions on the interface, e.g. as you press a button, scroll a window, or fill in a text box, the interface appearance will change and actions within the application will be triggered. Interfaces also include this dynamic behavior in addition to the layout of widgets. `XFaceMaker` helps the user design both layout and behavior.

1.4 The Phases of Interface Design

XFaceMaker can help you in the following development tasks:

- Interface layout
- Interface behavior
- Building the application
- Generation of reusable interface components(classes)

Interface Layout

Layout includes positioning widgets on the interface and setting their resources to the desired values.

Interface Behavior

Modern interfaces are dynamic. The behavior part of an interface comprises those dynamics that have little to do with the application proper. As a simple example, closing a window when you press on its “Close” button is behavior that strictly belongs to the interface and the application proper should not be bothered with it. In XFaceMaker/EL and in XFaceMaker you can define and test this behavior within the tool. See section 1.6 for a description of the three products of the XFaceMaker family.

Building the Application

XFaceMaker generates all the files needed for building the complete application. This includes files for creating and displaying the interface (including its behavior in the XFaceMaker/EL and XFaceMaker products), additional resource and header files, and the `make` files for building the application. The application developer must provide the files corresponding to the application proper, which will then be linked with the other files to generate the complete application.

Generating Reusable Interface Components

XFaceMaker can generate Xt and C++ classes corresponding to fragments of your interface. Typically, you can create a dialog box and ask XFaceMaker to generate the code for the Xt or C++ class. XFaceMaker also lets you define new resources for such a class. The resulting class has all the properties of a class that would be developed by a seasoned programmer. XFaceMaker generates stand-alone code for it (as long as the prototype has been built only from Motif components; otherwise, the class depends on whatever other classes may have been used to develop it).

1.5 Other NSL Products Used with XFaceMaker

NSL has developed several products that can be used with XFaceMaker.

The Widget Factory

The XFaceMaker family of products is the basis of NSL's Widget Factory. Whereas XFaceMaker is used to design interfaces using a fixed set of widgets - usually the Motif widget set, but also other third party widgets - the Widget Factory is a suite of tools for building new primitive widgets. Examples of such widgets are dials for industrial control panels, business graphics and animated diagrams. The Widget Factory comprises:

- **The Draw Library** - a library of Xt widgets for basic graphic elements such as line segments, polygonal lines, Bezier curves, text fields, images, etc. These graphic elements also have reactive capabilities and can also be used in XFaceMaker.
- **XDrawMaker** - an interactive tool for creating animated drawings composed of the graphic elements of the Draw Library. Animated drawings created with XDrawMaker can be imported into XFaceMaker and used to generate Xt or C++ widget classes.
- **XFaceMaker** itself.

These products are seamlessly integrated. In particular, XDrawMaker is called automatically from XFaceMaker when you are editing an XnsIDraw widget, the basic container widget of the Draw Library.

XFaceMaker/Db

XFaceMaker/Db is the database extension module of XFaceMaker. It gives database connectivity from FACE scripts and is similar to application generators of the more traditional kind.

FACE has been augmented with functions that allow communication with a database server through SQL commands. Full interaction with the database is implemented. At this time, Oracle, Informix, Ingres, and Sybase connections are available. Connections to other databases will become available in quick succession.

XFaceMaker/Db does not aim to compete with such sophisticated products as Oracle/Forms. Rather it allows the developer to quickly develop an application on top of an existing database. The layout facilities and the scripting facilities are superior to those available in products coming from the major database manufacturers. On the other hand, products such as Oracle Forms are mainly intended for the design of databases.

1.6 The XFaceMaker Product Family

The XFaceMaker family comprises three distinct products:

XFaceMaker/IDT is an interactive design tool (IDT) that helps you design the layout of an interface, but not its behavior. XFaceMaker/IDT is one of the most powerful tools in its category, through the provision of templates and groups.

XFaceMaker/EL is the entry level version of the full XFaceMaker product. In addition to interface layout, XFaceMaker/EL also lets you design and test interface behavior, through the built-in FACE interpreter and FACE debugger. This variant is usually classified as a user interface management system (UIMS).

XFaceMaker is the full version of the product which, in addition to the previous features also lets you design Xt (X Intrinsics) and C++ classes of interface objects.

The table below summarizes the main features of these three products.

	XFaceMaker/IDT	XFaceMaker/EL	XFaceMaker
Layout	yes	yes	yes
Groups	yes	yes	yes
Templates	yes	yes	yes
Xt Class generation	no	no	yes
C++ Class generation	no	no	yes
FACE behavior scripts	no	yes	yes

1.7 Using This Manual

The *XFaceMaker User's Guide* and the *XFaceMaker Reference Manual* contain all the information you need to use the product. If you are using **XFaceMaker/IDT** or **XFaceMaker/EL**, some information contained in these manuals does not apply.

- If you are using **XFaceMaker**, everything in these manuals is relevant.
- If you are using **XFaceMaker/EL**, everything is relevant except for information concerning classes and C++ classes.
- If you are using **XFaceMaker/IDT**, information on the following subjects is not relevant:
 - m the FACE language,
 - m active values
 - m editing scripts,
 - m classes,
 - m C++ classes

CHAPTER 2: Overview

XFaceMaker is a complex tool with many usages. In this chapter we shall give you an overview of its capabilities and pointers to the sections of this manual where you can find more details. The main topics discussed are:

- How to use XFaceMaker
- Laying out the interface
- Specifying and testing the interface
- Interface description formats
- Building the application
- Creating reusable components
- The dual-process architecture of XFaceMaker
- Internationalizing the interface

2.1 How to Use XFaceMaker

You can use XFaceMaker either as an interactive tool or as a batch process. When you start XFaceMaker you have the option of building an interface from scratch or of loading an existing interface and working on it.

Interfaces built with XFaceMaker are always saved in Unix files in a language called the “fm” interface description format or language. The fm format uses ASCII characters and a simple syntax which simply gives the list of widgets and the values of their resources. The fm format is described in Chapter 24, The Fm File Structure.

Existing interfaces described in the UIL language (supported by the Open Software Foundation) and produced by other interface design tools, can be converted to the .fm format with the `uil2fm` utility, and then loaded into XFaceMaker. XFaceMaker can generate UIL description files.

Starting XFaceMaker and the main command window are described in Chapter 3. The same chapter also describes how to open existing interface description files, how to save your work, how to save work options and how to quit the tool. The environment variables that must be set are listed in Chapter 23. Work options include, for example, the type of files that XFaceMaker will generate by default when you save your work.

Build Mode and Try Mode

XFaceMaker has two modes of operation: Build mode and Try mode.

- In Build mode you build the interface; that is, you layout the widgets and you set their resources.
- In Try mode you can test the interface. This is most useful in the UIMS products XFaceMaker/EL and XFaceMaker where you can test and debug behavior scripts. But even for the XFaceMaker/IDT product, Try mode can be useful for observing the visual effects of the widgets. Even here some of the visual effects may be non-trivial, for example the behavior of a text editing box.

Using XFaceMaker in batch mode is useful for many operations that need not be carried out interactively. For example you can use XFaceMaker to read in an fm interface description file and generate the corresponding C code. This does not require the use of the interactive layout facilities and is done by invoking XFaceMaker with the appropriate command line options. Command line options are listed in Chapter 22.

2.2 Laying Out the Interface

The first thing you can do with XFaceMaker is to lay out your interface. All three products of the XFaceMaker family have the same basic layout capabilities, except for the generation of classes which is only possible in the full XFaceMaker product.

Layout features

The interface layout phase consists of two parts: widget *positioning* and the *setting of resources*. The layout is done with real Motif widgets and you work on an interface that looks exactly the same as the real one. Most of the interaction is directly with the widgets in the interface. You can always select the widgets directly on the interface and then resize them.

The widgets of the interface are also represented symbolically in a tree hierarchy in which widgets contained in another are shown to the right. You can select any widget in the widget tree and change its name.

XFaceMaker provides many sophisticated layout features, such as:

- *Alignment boxes* which allow you to align widgets on a grid (you can adjust the pitch of the grid) or relative to one another.
- *Cutting, copying, pasting and duplicating* widgets, together with their resource settings.
- *Gadget/ungadget* - Any interface widget can be transformed into a gadget, when one is provided by Motif. Gadgets are somewhat simpler versions of a widget - basically windowless widgets - that use fewer resources than widgets.
- *Auto position and auto size commands* - These are options which allow the automatic setting of a widget's position and size by a containing widget such as the XmForm or the XmRowColumn.

Setting resources

After having put your widgets in place, you must configure them, that is alter their properties. All Xt widgets are parametrized to a large extent. For example, for all Motif widgets you can alter the width of their border. The border width is a widget attribute or, in Xt parlance, a *resource*. XFaceMaker lets you set the resources of a widget as conveniently as possible. After selecting the widget, all its resources and their values are displayed in the *resource editing window* and you can modify their values.

In XFaceMaker, changes to the widget resources are displayed immediately once you have defined a set of resource values and applied them to the selected widget. This applies to all resources of a widget and, in particular, to the position, size and class of a widget. You can thus define a widget as an XmForm widget and then change your mind and type XmBulletinBoard for the class and the widget class will change accordingly.

Motif widgets may have up to one hundred or more resources. XFaceMaker lets you order them in different ways. You can put superclass resources first, or put subclass resources first, or you can list them alphabetically. You can also put resources that you want to access often first. You can look up resources quickly by just typing one or two letters.

Once you have selected the right resource, XFaceMaker helps you as much as possible to set the right values. Some Motif resources are complex and may be quite difficult to set. When there are just a few values, you can simply select the right one from a list. For more complex ones such as fonts, more complex dialog boxes are provided.

XFaceMaker lets you view and edit the resource settings of several widgets at the same time. You can open two or more resource editing windows and cut, copy and paste resource values from one to another. You can either copy and paste the resource value as a string or just indicate visually that you want to copy by dragging and XFaceMaker will do the actual copying for you.

Specialized editors

XFaceMaker has several built-in editors for constructing special interface fragments. Some of the most useful editors available are the following:

- *Attachment editor* for editing geometric constraints in the XmForm widget.
- *Menu editor* for the quick definition of menu bars.
- *Drag and drop editor*. XFaceMaker provides advanced facilities for the easy set up of the drag-and-drop features available in Motif 1.2.
- *Text editor*. XFaceMaker allows you to edit lengthy text resources with the text editor of your choice, e.g. `vi` or `emacs`. In addition, XFaceMaker comes with the Wx text editor. Wx is NSL's text editor and is available also as an independent product. Wx is very intuitive, has a Motif graphic interface and is programmable through macros written in FACE (the same language used by behavior scripts in XFaceMaker).
- *Translation editor* for the quick editing of translation resources. Here you don't have to remember the code for an action, you simply perform it and XFaceMaker

inserts the right value for you.

- *Color editor* for editing and displaying colors. The chosen color can be either selected from a set of existing colors or new colors can be defined, displayed and added to the list of existing colors.
- *Pixmap editor* for quickly editing small pixmaps such as icons used for classes or background patterns.

Specifying application callbacks

Although an interface may consist of tens or even hundreds of different windows, not all of them are shown on the screen at the same time when you run your application. To use XFaceMaker as an example, you usually have on the screen the main window and the resource editing window, in addition to the fragment of your own interface that you are working on. Occasionally, other windows will pop up, for example a font selection box.

When you design your interface layout, you design each dialog window individually. The way the different windows interact, the way in which clicking on a button in one window brings another one on the screen is what we call interface *behavior*. Interface behavior can be quite complex as we shall see.

The XFaceMaker/IDT product lets you design only the interface layout and not the behavior. The behavior part, for example the sequencing of windows, must be handled by the application. The way this is done is the following.

All Xt widgets (and Motif widgets are a special set of Xt widgets) react to events. For example, the Motif pushbutton widget reacts to the Activate event, which is the clicking on the button with the left of your mouse. When you do this, the Motif button reacts by changing color:

At the same time, the designers of Xt have provided for the possibility of calling a function of your application when triggered by the Activate event.

The XFaceMaker/IDT product lets you specify for every widget and each of its callbacks the function that will be called when the event happens. You can pass parameters to your callback.

This is exactly the way developers who chose to program a graphical interface operate. They write code for the layout and they specify which application callbacks are to be called by each event. With XFaceMaker/IDT you will not have to write the interface layout code because XFaceMaker/IDT will do it for you. You still must provide the application callbacks.

Of course, you can work exactly this way with XFaceMaker/EL or with XFaceMaker, although there are better ways.

Groups, templates and classes

All three products of the XFaceMaker family let you define groups and templates. We shall discuss these in more detail later on. For the time being, let us just say that you can group an interface fragment, say a dialog box into a single entity. That entity can be defined as a group or a template. A group can be re-instantiated and is actually a copy of the original prototype. In contrast, a template inherits all the resources of the prototype. When the prototype resources are changed all the instances will change accordingly.

The XFaceMaker product also has the capability of generating Xt and C++ classes. More about this later.

Groups, templates and classes save you time if you must create a second set of widgets with the same resources. In XFaceMaker, you simply select the group, template or class icon in the widgetstore and drag it to the desired spot in the interface. This saves the time needed for creating all the widgets in the group and setting their resources. It is also an excellent way of defining standardized and reusable interface components such as dialog boxes, that will be used uniformly by all developers of a department or even a whole company.

Groups, templates and classes are very useful in defining standard reusable components that can be manipulated as a single entity. Groups, templates and classes can be defined at the departmental or company levels to enforce common interaction styles.

Interface layout techniques are described in detail in Chapter 4. Layout of menus and the use of the special menu editor is described in Chapter 7. Groups and templates are discussed in Chapter 9. Xt classes are discussed in Chapter 13 and C++ classes in Chapter 15. Resource editing is discussed in Chapter 5.

2.3 Specify and Test Interface Behavior

XFaceMaker can help you in specifying and testing interface behavior. This is possible only with the XFaceMaker/EL and the XFaceMaker products, not with XFaceMaker/IDT.

Behavior can vary widely in complexity

Interface behavior can vary enormously. For our calculator example it is quite simple. For the XFaceMaker interface examples that we showed earlier it can be quite complex. For example, the resource editing window that we showed in the previous chapter has quite complex behavior. Various buttons can trigger a variety of different dialog boxes. Clicking on the ellipsis button next to the font resource will bring the font selection box on the screen. Once you have selected the font with the font selection box and you click its OK button, the value of the font selected is transmitted to the resource editing box which will then display the value next to the font resource. When you press the Apply button of the resource editing box all the resource values are applied to the selection box and so on.

This complex behavior must be programmed in the application callbacks if you use XFaceMaker/IDT. In the two UIMS products, XFaceMaker/EL and XFaceMaker you specify this behavior by writing FACE scripts.

Callback Resources

X widgets have *callback resources* that are triggered by interface events. For example, the Motif pushbutton has an Activate callback resource which is triggered when the user presses the left button of the mouse within the Motif button. When programming with Motif, the value of a callback resource must be a pointer to a C function of the application. The standard way provided by X for the communication between the interface and the application is through such events: an interface event triggers a function of the application.

We have seen earlier that in the XFaceMaker/IDT product the developer must indicate for each callback resource the application function that must be called.

In XFaceMaker/EL and in XFaceMaker instead of simply indicating the application callback, the developer can write a FACE script as the value of the resource. A script is a piece of procedural code, written in the NSL C-like FACE language. The FACE script can be executed (interpreted) when switching to Try mode. XFaceMaker/EL and XFaceMaker have a built-in FACE interpreter and debugger. Execution messages are displayed in a special Messages window.

Thus, the difference between XFaceMaker/IDT and the other two products of the family is that in XFaceMaker/IDT the developer just indicates the value of the application callback that will be called when the event occurs. In contrast, in XFaceMaker/EL and in XFaceMaker the developer can give a full procedural FACE script as the value for that resource. From the script one may also call application functions. A FACE script may also perform other actions such as pop up other windows, hide windows, change the color of a text field or change other aspects of the interface in more complex ways.

The scripts can be tested in Try mode. Scripts are like any other resource. In particular, they are part and parcel of a group, template or class that may have been defined and can thus be transmitted to other widgets.

The use of scripts leads to the encoding of the whole interface behavior in the interface itself, instead of within the application as is the case with the XFaceMaker/IDT product or in standard X programming practice.

There are absolutely no drawbacks with coding behavior in scripts instead of coding them in the application proper. Scripts can get compiled into genuine C code and they do not rely on any proprietary libraries. Since FACE is so close to C there is an easy to follow, almost one-to-one correspondence between FACE statements and the C code generated.

Editing scripts is discussed in Chapter 6. For the use of the Wx editor see the accompanying Wx manual

The FACE language is briefly described below and, in more detail, in the Reference Manual. FACE scripts can call library functions. Library functions available in the standard distribution are described in section 6.5 and in the Reference Manual. XFaceMaker also has the ability of being extended with any set of functions. In particular, it is possible to call all Xlib, Xt or Motif functions, (provided their arguments can be represented in FACE), as well as the application function from scripts.

Finally, XFaceMaker scripts can use the active value mechanism described below and, in more detail, and in Chapter 8.

Specifying class behavior and resources

New widget classes are constructed as any other piece of the interface - you lay out the interface fragment and you attach behavior scripts.

The main additional feature is the definition of new resources. Defining new classes without the ability of defining new resources is of limited interest.

Class creation is only available in XFaceMaker. XFaceMaker generates complete stand-alone code for Xt classes (as long as the interface prototype from which the class has been created does not include non-Motif widgets).

C++ classes can also be generated. C++ classes are actually wrappers of Xt classes. Public and private methods for the class can be defined. C++ classes can be generated in NSL style, in Rogue Wave style (for compatibility with the Rogue Wave C++ wrappers of Motif widgets) and in customized styles.

The generation of Xt classes is described in Chapter 13. The definition of new resources is described in section 13.1.3. The generation of C++ classes is described in Chapter 15.

Testing and debugging

Testing and debugging is done in Try mode. For XFaceMaker/IDT, Try mode only provides the visual effects such as the change in color when you press on button. If XFaceMaker has been augmented with the application functions, triggering an event will also execute the associated application callback. Thus even with just the XFaceMaker/IDT product it is possible to test the application together with the interface.

XFaceMaker/EL and XFaceMaker provide the full FACE interpreter and debugger so that FACE scripts can be executed directly within the tool.

When you turn on the debugger, you can trace the execution of a script step by step or by stopping at selected breakpoints. At breakpoints you can print the value of variables, check and print the execution stack and execute any FACE statement.

The FACE language

The FACE language has been designed to meet the following objectives:

- *Ease of use and mild learning curve.* FACE resembles C and is just as easy to learn.
- *Increased expressive power for the handling of interface aspects.* FACE allows you to easily refer to widgets and their resources and makes it possible to avoid lengthy Xlib and Xt calls. FACE code is thus much shorter and easier to understand than C code.
- *Efficient compilation.* FACE code is compiled in very efficient C code. As a result XFaceMaker is just as useful as a prototyping tool as a production tool. The C code generated by the FACE compiler makes no reference to the interpreter.

The main features of the FACE language are the following:

- *Variables.* FACE supports variables. Variables may be local to a script or global to all of them. You need not declare variables as their type is determined by use. Most C types are supported.
- *Arrays.* Two types of arrays are supported: linear and hashed. Hashed arrays are indexed by a string key or by an arbitrary one-word key. Linear arrays are similar to C.
- *Structures.* Structures similar to C are supported, except that only access to structure pointers is provided.
- *Built-in functions.* A rich set of basic functions that can be called from scripts is provided. These include some C, most Xt (X Intrinsics) and all Xm (Motif) functions. In addition there are numerous convenience functions to make writing scripts even easier. The set of built-in functions can be increased. XFaceMaker can be linked with any libraries or user functions to obtain an extended version of the tool.
- *Function definition.* FACE supports function definition, including recursive definitions. In addition to functions defined within scripts, there are application function definitions to make application functions known to the FACE interpreter.
- *Widget naming scheme, widget resource and X properties reference.* One of the most useful features of FACE is its widget naming scheme. You can refer to a widget by its name, which is essentially its path in the widget tree. To refer to a resource, append its name to the widget's name.

FACE is fully described in the *XFaceMaker Reference Manual*.

Active values for sharing data

FACE uses a unique and powerful mechanism for sharing data and signals between interface scripts and the application. An active value consists of:

- an application variable
- an X resource
- a get script
- a set script

The set script transforms the application variable into the resource and the get script does the converse. The two scripts can be triggered from the application or from other scripts. As a simple example, an active value could associate an integer application variable `v` and a label in the interface. The set script would then transform the variable `v` into the label by applying the `itoa` conversion function available in the FACE library. The get script would do the converse by means of the `atoi` function.

There is total flexibility in the way variables, widgets and active values are associated. It is possible to associate several active values to the same widget or to have several widgets with the same active value. In this case the set script of the active value can be triggered simultaneously for all widgets and this will result in broadcasting to all widgets. Alternatively, the set script could be triggered for some widgets only. Active values can also be used to add resources to a widget without having to build a whole new class. An active value can be:

- Global or specific to a widget.
- Public or private.
- Automatic or immediate.

An automatic and per/object active value can be given an initial value which will be assigned to it just after the object has been created. Automatic, per object active values appear in the resource list and the resource editor of XFaceMaker.

Active values can also be used to send signals from the application to the interface, for the synchronization of data displaying different widgets and for adding new resources to widget classes. Sending signals from the application proper to the interface is very useful and is done much less elegantly without active values. Active values are described in Chapter 8.

2.4 Interface Description Formats

Once you are satisfied with the interface or the interface fragment you have designed you can save it into an fm file. As described earlier, the fm format is the standard format in which interface descriptions must be stored in XFaceMaker. An interface stored in fm format can easily be restored and you resume work on an interface by re-loading the fm file.

As we shall see, XFaceMaker can also generate C, C++, and UIL but can only read in fm files. UIL files can be translated to fm format using the **uil2fm** utility so that they can be read by XFaceMaker. Generating code for a whole interface or a fragment is done simply by selecting the top widget of the widget hierarchy one wants to save or generate code for.

Generating the code corresponding to a class is done the same way. You simply select the top widget of the widget hierarchy and order the generation of the class.

XFaceMaker allows the splitting of an interface over several files. An interface split into a main file and secondary files can be grouped in a project file. Projects can be handled as a whole and XFaceMaker knows how to trace the components to their files. Selected files are checked to ensure that they are not already loaded, although it is also possible to load multiple instances of the same module if desired.

Generation of Xt classes is described in Chapter 13, and that of C++ classes in Chapter 15.

2.5 Building the Application

There are three different ways of building the full application and using the code generated for the interface by XFaceMaker:

- *Interpreted interface.* In this version the application proper uses the fm interface description file directly. The application must call a function of the Fm library (a library provided in source form on the XFaceMaker distribution tape) that will read the fm file and will create the interface in the application from it. One says that the function *interprets* the interface from the fm file, whence the name. The final application will therefore need the fm file and will not be able to function without it.

The advantage of this approach is that the application need not be recompiled if changes are brought to the interface. Of course, any changes must be such as not to affect the rest of the application. The drawback is that the fm file must always be present. This may be a slight inconvenience if the application is moved to another directory. There is also a slight loss in performance due to the interpretation. In practice though, this is negligible and even very large interfaces can profit from the gain in flexibility provided by this method. For example, XFaceMaker's own interface is interpreted.

- *Compiled interface.* In this variant the application proper is built from C modules generated by XFaceMaker for the interface and from C modules provided by the developer for the application proper. The advantage of this method is that the application proper need not worry about the fm file. There is also a slight improvement in performance. However there is a loss in flexibility since modifications of the interface now entail the need to recompile the whole application.
- *Editable interface.* The editable version allows you to edit the user interface *while* the actual application is running. The editable version of the interface consists of XFaceMaker extended with your application functions, your new widget classes, and your widget groups. It means that your application functions are known by XFaceMaker and that they will be executed effectively whenever they are called from a FACE script.

There are two ways of using such an application:

1. You call the version of XFaceMaker augmented with your application functions, you load the fm file and then you exercise the interface in Try mode; this will launch the application functions from the scripts.
2. You launch the application (which has XFaceMaker built into it) and you then launch XFaceMaker (by a special key combination) whenever you want to edit the interface with the application running. An application with an editable interface can only work with an interpreted interface.

XFaceMaker can generate the following files corresponding to the interface or to portions of it:

- Fm files containing the interface description in fm format.
- UIL files containing the interface description in UIL format.
- X resource files.
- Internationalized message files.

XFaceMaker supports projects. Various portions of the same interface can be grouped in a project, then saved or loaded at the same time.

XFaceMaker also generates several files to help you build and structure the application proper. These include the following:

- *Main.c*. This is the main function of your application. This file contains all the required initializations of the X toolkit and of the XFaceMaker library, the attachment of application functions to the names used for them in FACE scripts (this is required only for the interpreted interface), the attachment of all active values, and the loading and mapping of interface widgets (for the interpreted interface) or the calls to the creation functions (for the compiled interface). Finally, the main function will enter a loop and wait for an interface event.
- *Functions file*. This file contains all the declarations of the active values used in the interface and the skeletons of the application functions declared in FACE scripts. The application developer must fill the body of these application functions.

- *Make file*. This is the make file necessary for building the three versions of the application: with interpreted interface, with compiled interface and with editable interface.
- *Pattern files*. These are the files that allow you to modify the pattern of the make files that are generated to accommodate different programming environments.

Building the application is described in Chapter 10. The various options for generating C code for the interface are described in Chapter 11. Extending XFaceMaker with application functions and widget classes is described in Chapter 16.

Rules to be observed by application developers are given in Chapter 10. Chapter 14 describes the development of the application proper in C++. The generation of C++ classes by XFaceMaker makes it possible to write application code that will instantiate C++ classes corresponding to interface components.

2.6 Creating Reusable Components

XFaceMaker enables you to standardize the design of your interface by building standardized reusable software components, or objects. We have briefly mentioned earlier, while discussing the layout facilities, that XFaceMaker supports three variants of reusable objects:

- groups
- templates
- classes.

You create groups, objects and classes in essentially the same way, by creating a prototype of the entity as you would create a portion of an interface:, namely by laying out the widgets, setting the resources and attaching behavior in the form of scripts. Then you select the top widget and store it as a group, template and entity. It is important to realize that the entity includes the behavior and that this is an extremely powerful feature of XFaceMaker.

We shall now explain the differences in usage and in functionality.

Groups

Groups are collection of widgets that the user can re-use as a single entity; they can be cut and pasted in a single operation; they offer a short term capability of defining new types of objects. Once a group has been created it appears with its icon in the Groups widgetstore from where you can re-instantiate it as often as you wish.

Instances of a group are independent from one another; you can modify one instance without affecting the others. A group can be modified, but instances of it created before the change will not know that the group prototype has been changed and will therefore not reflect the change.

You can think of groups as being collections of widgets and their resource assignments, including scripts, that are similar to collections that have been cut and pasted.

Templates

In contrast to groups, instances of templates inherit all their resources from the template prototype. When you change the prototype, all instances change accordingly. Furthermore, you can use one or more templates to define a new one. For example, you can build a red button template and a blue label template, use both in a form, make the form a new template that you use in a dialog box. Then changing the red button template to green, will propagate to all instances of the dialog box.

Templates can also be applied to existing widgets. For example, you could apply the red button template to a button that has already been created and the button will inherit the red value for the background color from the template. Even more you could apply the red button template not to a button but to a label. The label happens to have the same background resource as the red button template and will inherit the value of the background resource just the same. When a widget inherits from two different templates, as was the case in our example, the latest value applied prevails.

Templates can be considered as a form of medium-term persistent objects.

Classes

XFaceMaker can generate the code for an Xt or C++ class of a class prototype constructed as indicated earlier. The code is as good as that written by a good programmer. The code generated is truly compiled C (for Xt classes) or C++ code (for the C++ classes; actually the code generated is a C++ wrapper around the Xt class with the additional public and private methods specified by the user). This means that any FACE scripts contained in the prototype, whether for callback resources or for active values or new resources defined for the class, are compiled in the necessary C code, without any call to the interpreter.

The definition of new resources for a class is done by means of the active value mechanism and is of particular value and importance.

Also, XFaceMaker allows immediate use of a class after it has been defined, both for instantiation or for the definition of new classes. The class icon appears in the Classes widgetstore from where it can be used just as a template or a group.

The C code generated for an Xt class this way can be also used in an ordinary C program, just as if had been written by the developer.

Generating classes in this manner is extraordinarily productive and increases the developer's productivity by one or even two orders of magnitude over direct coding. Writing widgets is a very time-consuming exercise and requires an intimate knowl-

edge of the X Intrinsics. XFaceMaker removes this burden from the programmer and makes widget class generation as easy as that of designing an ordinary interface. Developers will be able now to make full use of widget classes.

C++ classes can be generated in either:

- *NSL style*
- *Rogue Wave style*, to ensure compatibility with the widely used Rogue Wave View.h++ wrappers of Motif widgets.
- *Custom style*.

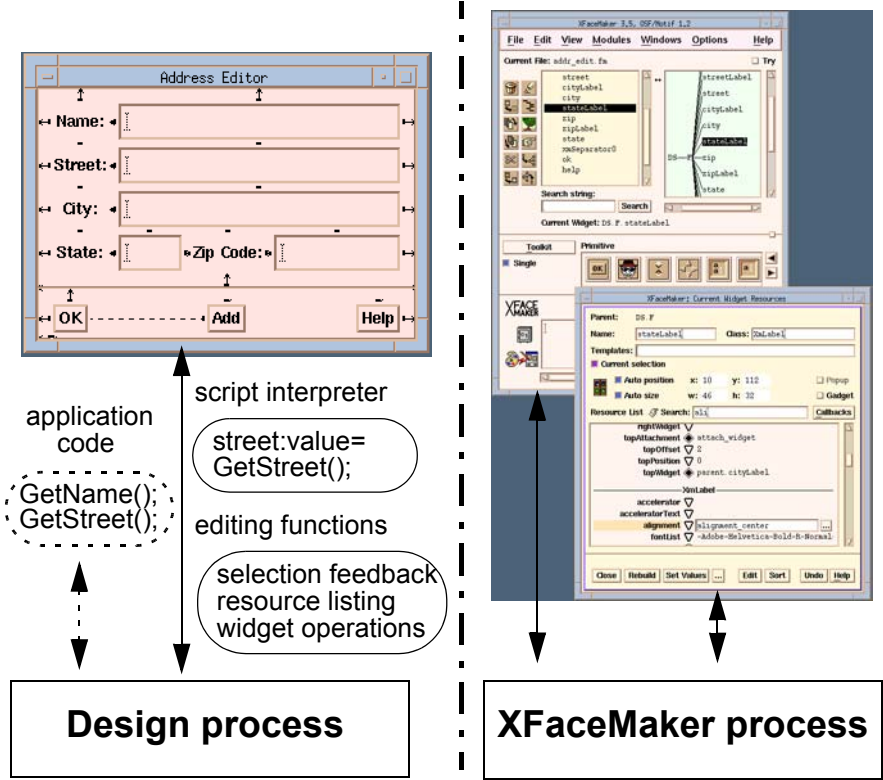
C++ classes generated by XFaceMaker can be used in an application as any C++ classes, either to instantiate or to derive a new class.

2.7 The Dual-Process Architecture

XFaceMaker uses an innovative dual--process architecture consisting of two processes communicating via a network protocol - XFaceMaker's main process (to which we shall refer simply as the XFaceMaker process) and the Design process.

The XFaceMaker process manages and displays XFaceMaker's own graphic interface, while the Design process manages the GUI being designed by the user.

Dual-Process Architecture



This architecture provides unusual advantages:

- *Automatic fault tolerance.* In case of a design error, for example an inconsistent

assignment of resources or an error in a FACE script, only the Design process will crash and it will inform the XFaceMaker process of the cause of the crash. The XFaceMaker process can then restart the Design process in a trusted state.

This solves a problem inherent to all interface building tools up to now in which an inconsistent resource assignment or a fault in a script (for the few UIMS's that provided a script interpreter) led to a crash of the whole tool. With the new dual-process architecture, XFaceMaker's own interface always stays up and only the Design process needs to be restarted. The robustness of the tool is thereby increased since the fault gets automatically reported to the main process, thus avoiding lengthy tests.

- *Dynamic extension.* Thanks to the dual-process architecture, XFaceMaker can be easily augmented with application functions or third party widget classes. Instead of doing this by re-linking all of XFaceMaker with the new functions or classes, one only has to do it for the design process and avoid the need to rebuild the main process. Since in many cases rebuilding the Design process only takes a few seconds, it appears to the user as if the new objects had been loaded dynamically into the tool. This feature in particular allows the editing of the interface while the application is running.

The dual-process architecture is described in Chapter 17. Error protection is discussed in Chapter 6.

2.8 Internationalization

XFaceMaker supports a flexible and powerful approach to the internationalization of interfaces. The handling of internationalization is based on the X/Open Global Language Support (GSL) library and is available only on systems that support it.

Essentially, internationalization means that all literal strings that are internationalized will be placed in a national catalogue selected by setting an environment variable. Every internationalized string is identified by a unique code and is searched for when needed in the appropriate catalogue.

In XFaceMaker, this approach which the GLS supports only for messages, has been generalized to resources. Every resource value, in particular all string values can be internationalized in this manner.

Internationalization is described in Chapter 21.

CHAPTER 3: Getting Started

This chapter will get you started using XFaceMaker. It explains how to:

- start XFaceMaker
- use the Command window
- open new and existing interface files
- save your work
- exit the application.

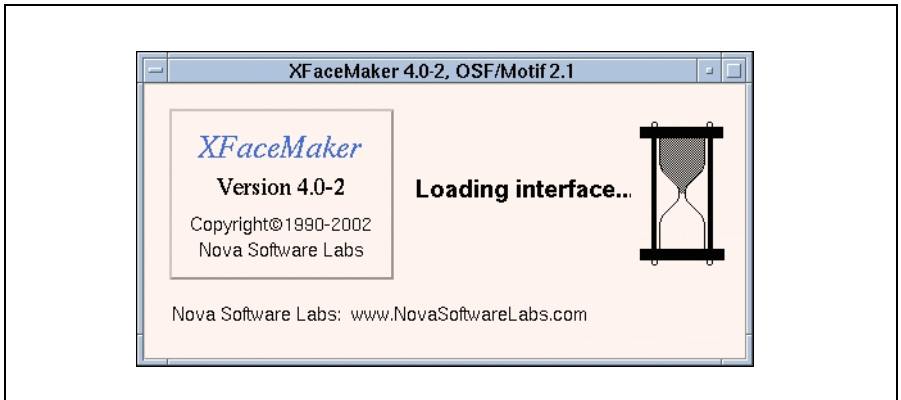


Figure 3.1: The Load Window

Before starting XFaceMaker, make sure that:

- XFaceMaker has been installed on your system. If not, consult the separate NSL document *Installation Guide for NSL Products*.
- the X Window System is up and running.
- your window manager is running. Although you may use other window managers, (e.g. twm and olwin), XFaceMaker has been developed and tested with the Motif window manager *mwm*. If problems arise when using XFaceMaker, verify if they remain when you use *mwm*.
- The NSL token server has been configured to give you access to the product and the environment variables are properly set.

3.1 Starting XFaceMaker

You can launch XFaceMaker alone, or include the NSL Widget Library. To start XFaceMaker, enter one of the following in your xterm window:

- Enter `xfm&` to start XFaceMaker alone.
- Enter `xfmwf&` to start XFaceMaker linked to the DrawLibrary.
- Enter `newxfmname&` to start your own version of XFaceMaker where `newxfmname` stands for the name you have given your new XFaceMaker. See Chapter 16 for details.

At first, the XFaceMaker Load window appears. When the program is initialized, the Command window appears on your screen. (You can also specify a `.fm` file to load, as well as other options, using the command line:

- `xfm <options> <fmfile>` to load XFaceMaker.
- `xfmwf<options> <fmfile>` to load XFaceMaker and the Draw Library.
- `newxfmname <options> <fmfile>` to load your own version of XFaceMaker.

where `fmfile` is the name of an interface file that you wish to load along with XFaceMaker. You can specify several `fmfile` names to load, but you must specify the `.fm` file containing the main window (usually the application shell) first. The command line options are fully described in the Chapter 11 of the *XFaceMaker Reference Manual*.

Note: if XFaceMaker outputs the message:

This is a demo version of XFaceMaker:

You will not be able to save your interface on disk.

you should verify your NSL token server installation and configuration, and/or check if all the tokens available for XFaceMaker on your network are already in use. Make sure also that your `XFM_LICENSE_NAME` environment variable is set for the type of license you have purchased.

If the message “Command not found” appears, the `PATH` environment variable wasn’t set at install time. If the interface icons don’t appear, then the `NSLHOME` environment variable wasn’t set.

3.2 The Command Window

XFaceMaker's Command window gives you access to the widgets and editing functions you'll need to create your interface.

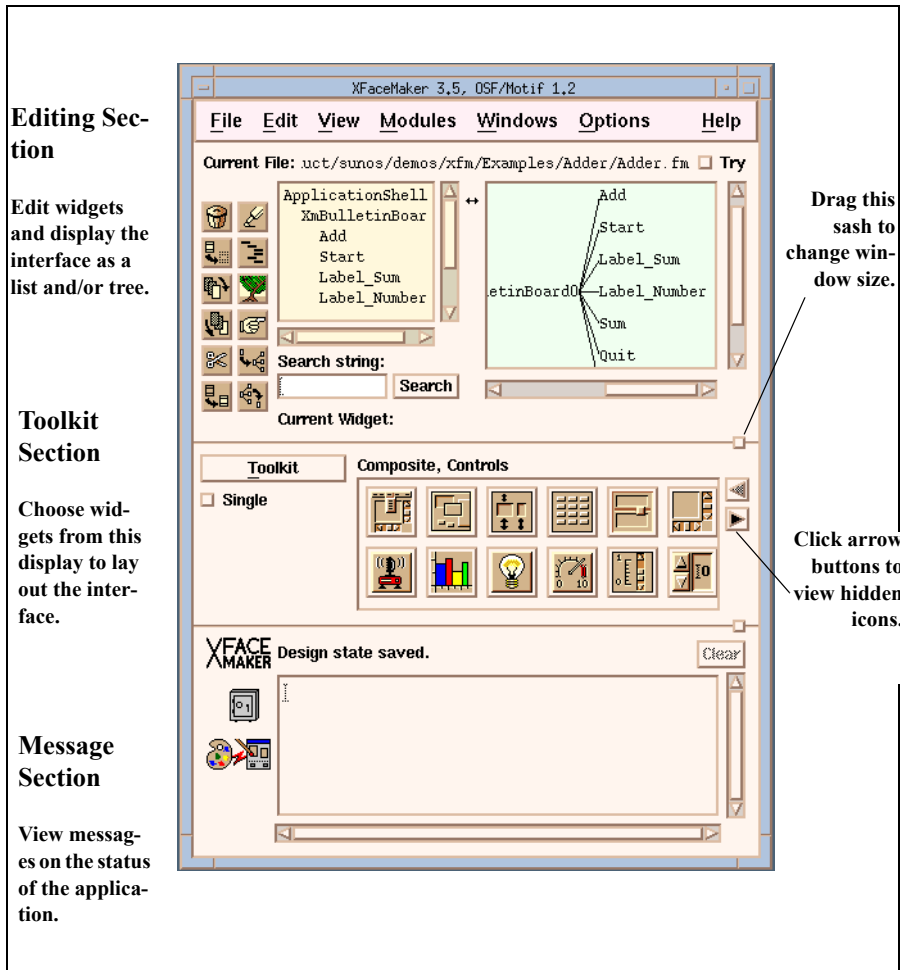


Figure 3.2: The Command Window

The Editing, Toolkit, and Message sections are described below.

3.2.1 The Editing Section

The editing section of the Command window has a number of features used to display and edit the widgets on your interface.

- Resize the window by dragging the sash.
- Adjust the space between the Widget Tree and Widget List by dragging the arrow found between them.

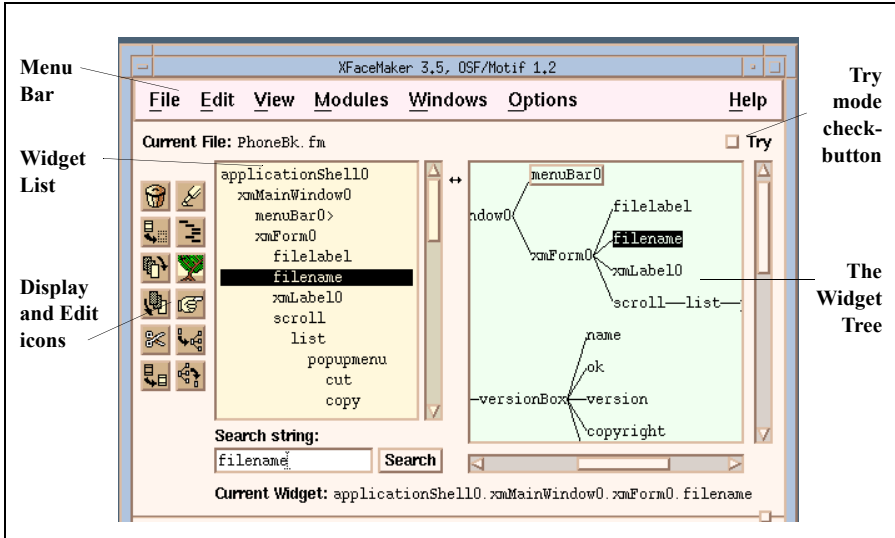


Figure 3.3: The Editing Section

- The **Menu Bar** gives you access to most of the commands and editing windows in XFaceMaker.
- The **Display and Edit Icons** give you quick access to some of the commonly used display and editing functions.
- The **Widget List** window displays your interface as a hierarchical list of widgets and can be used to edit widgets.
- The **Widget Tree** window displays your interface as a tree structure and can be used to edit widgets and widget names.
- The **Try mode** checkbutton toggles between Try and Build modes.
- **Current File** displays the path name of the current interface file.
- **Current Widget** displays the name of the most recent selection.

The Menu Bar

The menu bar contains six pulldown menus, and a Help menu. In this *User's Guide*, the menu commands are described as needed. Full descriptions of all the menu commands can be found in the *XFaceMaker Reference Manual*.

Giving commands from the menu with the mouse:

- To open a menu, click on its name.
- To select an item from the pulldown menu, click on it. This launches the selected command and closes the menu.
- To close a menu without giving a command, click outside the menu.

If the menu item is followed by an ellipsis (. . .) or an arrow, you will be asked to do some further action before the command is carried out. For example, if you click on ***File/Open...***, the File Selector box pops up for you to enter the name of the file you wish to open.

Giving commands from the keyboard:

Menu commands can also be given from the keyboard. Each menu and menu item name has one letter underlined; for instance, the F in the File menu. To open a menu and give a command from the keyboard:

1. Move the cursor into the Command window.
2. Press the Alt or Meta key plus the underlined letter in the menu name. This opens the menu.
3. Press the underlined letter in the menu item name, or the key combination listed to the right of the menu command.

For example, move the cursor into the command window, and press the key sequence **Alt - f o**. This opens the ***File*** menu and selects the ***Open...*** command.

The Display and Edit Icons

The display and edit icons give you quick access to the editing and display features of XFaceMaker. To use the display toggle icons, simply click on .the icon. To use the editing icons, you can do either of the following:

- Click on the icon, move its “shadow” to the widget or its name, and click.
- Move the widget selection over the icon, and release.

When only one widget is selected, the two operations are equivalent. However, if multiple widgets are selected, dragging an editing icon over the selection affects only one widget at a time.

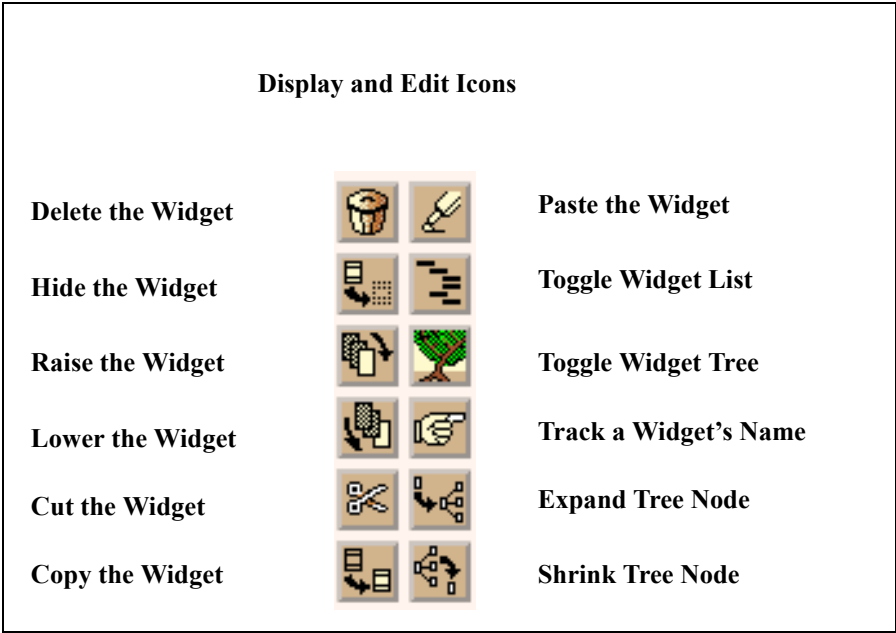


Figure 3.4: The Display and Edit Icons

Build and Try Modes

XFaceMaker has two modes of operation, Build and Try

- Use Build mode when you are building or editing the interface.
- Use Try mode when you want to test some aspect of the interface.

To switch between Build and Try modes, simply toggle the Try checkbox at the top of the Command window, or type `Ctrl-t` for Try mode or `Ctrl-B` for Build mode.

Try mode enables you test an element of the interface to see how it will work in your final application. To get a more complete idea of how your final application will work, you must link the interface and application as described in Chapter 10.

The `-try` and `-build` command line options enable you to choose different accelerators. Refer to the *XFaceMaker Reference Manual* for a complete description.

The Widget Tree and Widget List

There are two XFaceMaker windows that are useful for visualizing and editing your interface:

- The Widget Tree displays the current interface as a tree structure.
- The Widget List displays the current interface as a hierarchical list of widgets.

You can use the Widget Tree or Widget List to:

- *Select* a widget by clicking on its name.
- *Duplicate* a widget by clicking on its name, then pressing `Ctrl-d`.
- *Hide* a widget by clicking on the Hide icon and moving it over the widget name, or by dragging the name over the icon.
- *Drag* a widget from the toolkit to the parent in the List or Tree.
- *Change* the name of a widget in the Tree (but not the List) by double-clicking on its name, then entering a new name in the text field.
- *Search* the interface for a widget's name in the List (but not in the Tree).
- *Edit/Select* a widget. Choose the action you want to trigger in a Popup Menu that you open on a widget by pressing Mouse Button 3.

You can display one or both windows by toggling the List or Tree icons. The Widget List and Tree are described in Chapter 4.

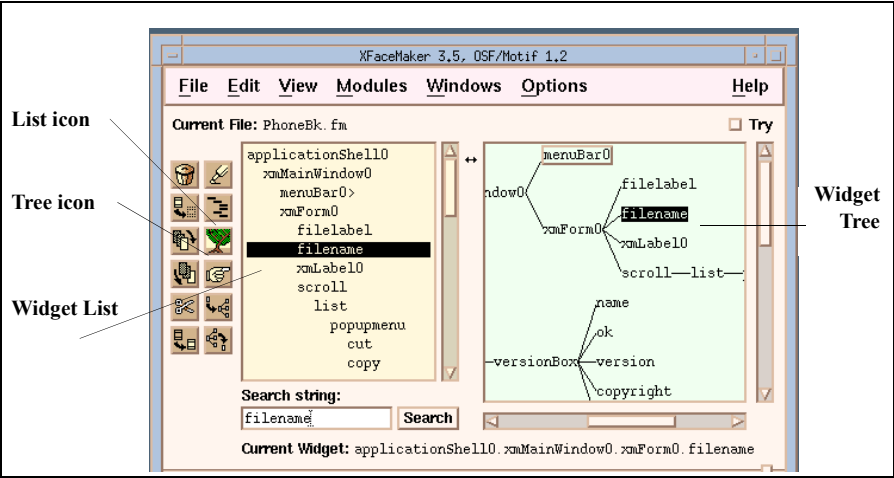


Figure 3.5: The Widget List and Tree

3.2.2 The Toolkit Section

The Toolkit gives you access to all the widgets available for building your interface, including any new widget classes or templates you may have added. Widgets are grouped by class in “widgetstores”.

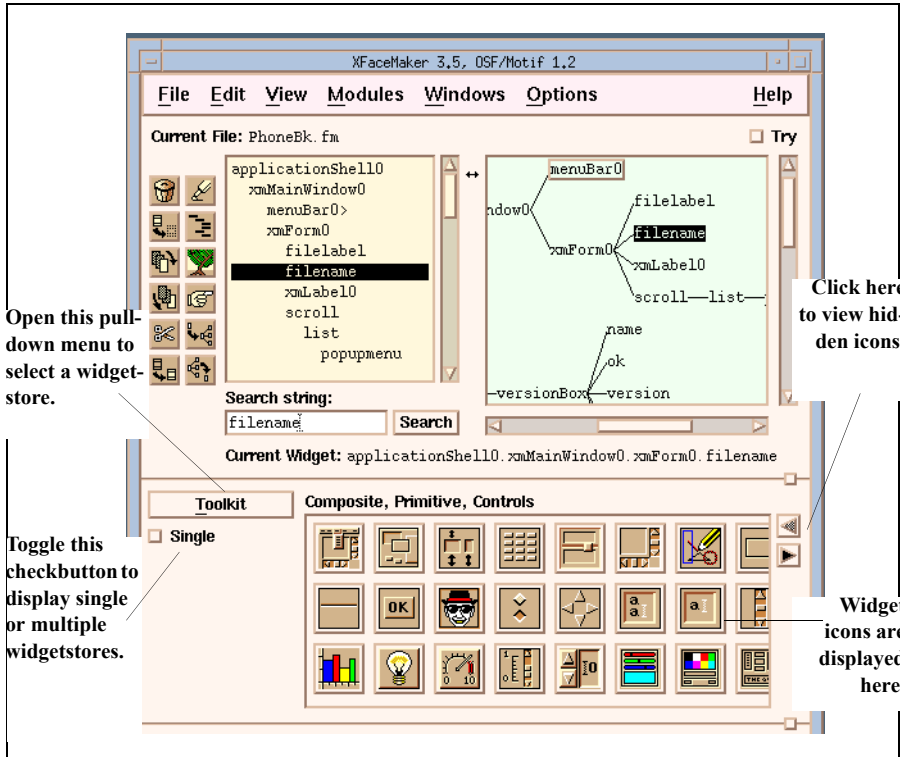


Figure 3.6: The Toolkit

The Toolkit pulldown menu contains the following widgetstore names: Shell, Composite, Primitive, Menu, Dialog, Global, and Controls. In addition, any templates, groups, or classes you’ve loaded in your XFaceMaker session, can be accessed in the Template, Group, and User-defined widgetstores. If your XFaceMaker is extended with NSL Draw widget and gadgets (e.g., you are running xfmwf), you will also have an NSL and an NSL Draw Gadgets widgetstore.

- Open a widgetstore by selecting its name in the Toolkit pulldown menu. The widget icons are displayed in the display field. When the pointer is over an icon, the widget’s class name is displayed.

- You can open more than one widgetstore at a time; for example, Composite, Primitive and Controls as shown in the figure. You can open as many widgetstores as you like.

To open multiple widgetstores:

1. De-select the “single” checkbutton.
2. Select a second widgetstore from the Toolkit pulldown menu. This displays the icons of both widgetstores.

To display only one widgetstore:

1. Select the “single” checkbutton. It is highlighted when set.
2. Open the Toolkit pulldown menu.
3. Select the name of the widgetstore to be displayed.

If several widgetstores are displayed, and you then select the “single” checkbutton, the first of the previously selected widgetstores is displayed.

Global XmDisplay and XmScreen Objects

The Motif 1.2 library defines *global* widgets which are used to store resources on a per-display or per-screen basis. These objects can be used with the same programming interface as widgets: they have resources which can be read with `XtGetValues` and modified with `XtSetValues`. Global objects are not created by the application: they are defined internally by the Motif library, and there is only one instance of each. Global objects are accessed in the Global widgetstore.

The differences between normal and global class icons are as follows:

- When the user clicks on a global object’s icon, the corresponding global object is selected directly and its resources appear in the Resource Editor’s list and can be edited normally.
- When an interface is saved, if a global object has been modified, its resources are saved in the interface file as for a normal widget.
- When the interface is loaded, the global object’s resources will be set according to the values saved.
- If an application loads several files with global object definitions, the values stored in the last file loaded override the previous ones.

See Paragraph 5.3, Editing Global Objects.

3.2.3 The Message Section

The Message section of the Command window has the following fields:

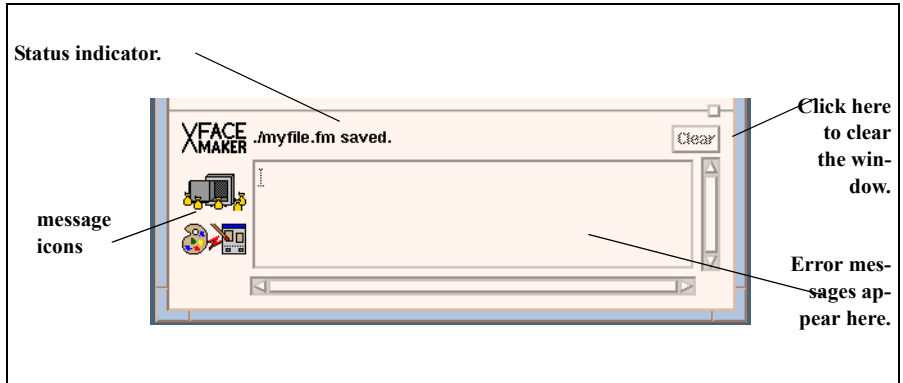


Figure 3.7: The Message Section

- **Status Indicator** -- This displays the current internal condition of XFaceMaker. For instance, after startup, it states “XFaceMaker Ready”. When saving files, the Status indicator shows whether the operation has succeeded or failed.
- **Message Window** -- Errors occurring while interpreting FACE scripts or creating widgets are reported in this box. These messages may be internal to XFaceMaker, may be produced by the Motif library or can be X Toolkit Intrinsics library warnings.
- **Clear button** -- Clicking here clears the Error Message window.
- **Icons** -- The icons symbolize the following:

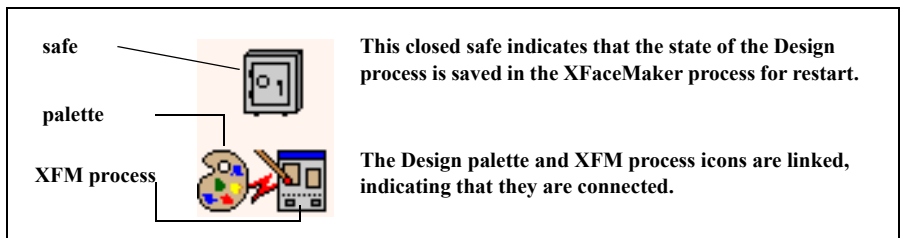


Figure 3.8: The Message Icons

3.3 Opening Files

When you launch XFaceMaker, you can specify that one or several files be loaded by using the command line: `xfm <options> <fmfile>`. If you launch XFaceMaker without specifying the name of an interface file to load, an empty Design process is launched, whose default name is `unnamed . fm`. You can then do any of the following:

- build a new interface by selecting widgets from the Toolkit and laying them out on the screen.
- open an existing interface file using the **File/Open...** command. (Change directories by selecting **File/Change Directory**).
- open an existing project (`.fmp`) file using the **File/Open...** command. This opens files that were saved using the **File/Save Project As...** command.

Once an interface is open, you can do the following:

- insert another file into the interface using **File/Read**.
- clear the existing interface to work on a new one, using the **File/New** command.

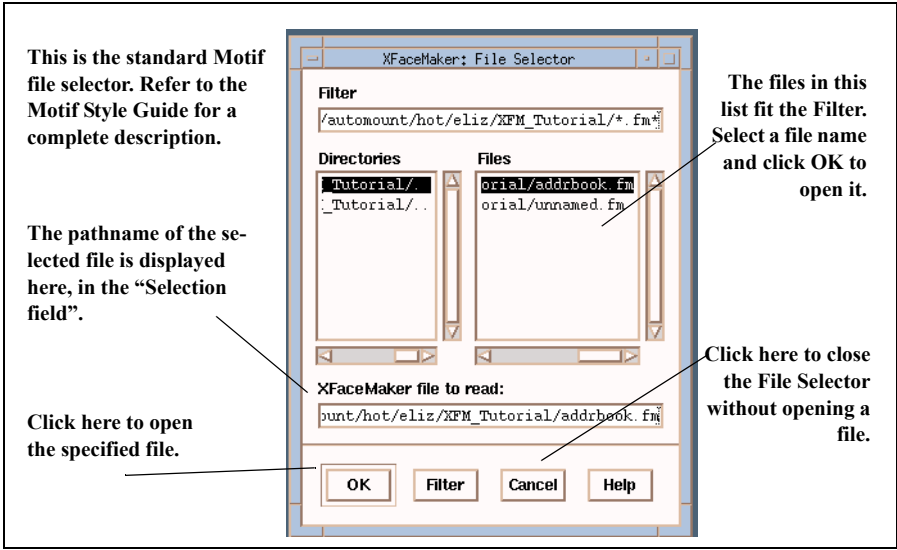


Figure 3.9: The File Selector

To open an existing file:

1. Select **File/Open...**; the File Selector pops up.
2. In the Selection field, enter the name of the file you want to load. Or click on the file's name in the "Files" list.
3. Click on **OK**, or press the Return key. XFaceMaker will attempt to load the file.

To clear the existing interface and work on a new one:

1. Select **File/New**. If you have not saved your work since the last change, an alert box will warn you.
2. Select a widget from the Toolkit to begin building a new interface.

This clears the XFaceMaker Design process (i.e. removes all widgets from the interface). You can then proceed to lay out a new interface. By default, the name of this new file is `unnamed . fm`.

To insert a file into an open interface file:

1. Select **File/Read...** from the File menu. The File Selector pops up.
2. Specify the name of the file to be read.
3. Click on **OK**, or press the Return key. XFaceMaker will attempt to load the file.

XFaceMaker checks that the selected file is not already loaded. (Otherwise, there would be two instances of the same file, which could cause undesired effects.) An error message warns if you attempt to load the same file twice.

To load a project file:

1. Select **File/Open...**; the File Selector pops up.
2. In the Selection field, enter the name of the file (with the suffix `. fmp`) you want to load. Or click on the file's name in the "Files" list.
3. Click on **OK**, or press the Return key. XFaceMaker will attempt to load the file.

3.4 Saving Your Work

There are a number of different ways to save your work. You can:

- Save *all of the current interface* as a simple .fm file. To do so, use the **File/Save** command. All widgets in the interface will be saved whether they are selected or not.
- Save *a selected part of the interface* as a separate .fm file. To do so, use the **File/Save Selected As...** command. This saves the selected top-level widget and its sub-tree.
- Save *several different interface files as a project*. To do so, use the command **File/Save Project As**. This saves the list of specified files as a single project file, given the suffix (.fmp). When you open a project file, XFaceMaker loads the main and secondary files of the interface. You can open a project file using the **File/Open...** command or by specifying the .fmp file as a command line argument when launching XFaceMaker. See section 3.4.1 for more information on project files.
- Save *groups, templates, and classes*. This is explained later in this manual in the corresponding chapters called *Groups*, *Templates*, and *Classes*.

Of course, as with any file, you can save the full interface or an interface fragment under a different name by using the **File/Save As...** command.

Also, there are a number of optional files you can tell XFaceMaker to generate at save time, such as C code, UIL, or RDB files. These options are discussed in the next section. Save options must be set *before* you give the save command.

XFaceMaker remembers which file an object was saved into. Instead of saving all current objects in a single .fm file, the **File/Save** and **File/Save As...** commands look at all the loaded files, and save the objects to their corresponding files. Groups, templates, and classes are saved into the corresponding group, template, or class file, not into the main interface file. **File/Save As...** pops up the File Selector for each separate file.

For example, if you load a main window (MainWin.fm) using **File/Open...**, and then load an alert box (Alert.fm) using **File/Read...**, XFaceMaker knows that the alert box must not be saved into MainWin.fm but into Alert.fm.

To save an interface:

1. Select **File/Save**. This saves the current interface under the name specified during

the last **File/Save** or **File/Save As...** If no name has been specified, it has the default name `unnamed . fm`. If the file has not previously been saved, an alert box will warn you. You must then use the **File/Save As...** command to save the interface and specify a name.

To save the interface and change its name:

1. Select **File/Save As...** to pop up File Selector.
2. In the Selection text input field, enter a name for the interface, with the extension `. fm`.
3. Choose **OK** or press the Return key. XFaceMaker will attempt to save the file. If a file of the same name already exists in the directory, an alert box will ask your permission to overwrite it. The success of the save is shown in the Message area and the current file name in the Command window is updated.

To save a selected part of the interface:

1. Select **File/Save Selected As...** This pops up the File Selector.
2. Specify a filename click **OK**. The selected widget hierarchy will be saved to a file under the specified name. Subsequent **File/Save** or **File/Save As...** commands also save the selected hierarchy to this file.
3. Select the **File/Save** command and save the rest of the interface. XFaceMaker will save the rest of the interface into a file that does not include the selected part of the interface.

To save your current screen configuration, including the position of the widgetstores, editing boxes, etc. select the **File/Save Prefs** command in the File menu. The information is stored in a preference file called `. xfm . startup` in your home directory. XFaceMaker loads this file automatically on startup. No user interface windows are saved.

3.4.1 Saving Project Files

A *project file* is a FACE file which contains calls to the XFaceMaker commands to load the main and secondary files of the interface. The command ***File/Save Project As*** does not save the *interface* into the `.fm` files, but the *list* of `.fm` files into the `.fmp` file.

The project files saved by the *Save Project As* command contain the options with which the files were saved: RDB mode, RDB resources and classes, C code options, UIL mode. (See section 3.5 below.) The groups, templates and classes loaded when the files were saved are also recorded as well as the environment variables set with the *Environment* box.

In particular, the project file contains all the information needed to generate the application. Once your project file is saved, you can generate your application very simply by typing:

```
% xfm -gen MyProject.fmp
% make -f MyProjectMake
```

To save the currently open interface files as single project file:

1. Select ***File/Save Project As...*** to pop up the File Selector.
2. Specify a name for the project file, giving it the suffix `.fmp`, and click ***OK***. This saves the main and secondary files of the interface individually, and into a project file. Any options that you specified for the files, such as C code, UIL, RDB, or environment options are also saved.

Note that options saved in the project file *override* the command-line options passed to XFaceMaker.

For example, suppose you save your project file with the C code option “*Use XtSetArg*”. Then, you change the option to “*Use Resource Database*”, and save your application files. In that case, the option in effect will be “*Use XtSetArg*” even though the command line used in the Makefile to generate the C file will be

```
“xfm -compile MyProject.fmp -cflags “raC””.
```

3.5 Save Options

There are a number of save options available in the Options menu. Set the options in the corresponding dialog box *before* saving the interface. Some options---***Save C Code***, ***Save RDB file***, and ***Save UIL file***---have direct consequences on each other. For example, if you set both the ***Save C Code*** and the ***Save RDB file*** options, the resources you specified in the RDB Options box will be saved in the RDB file, and not in the C Code file. For more information, refer to the chapters entitled “RDB Files” and “Generating C Code” in this User’s Guide.

- ***C Code...*** Pops up the C Code Options box which is used to set the options necessary to generate a C language version of the interface at save time. See Chapter 11.
- ***RDB...*** Pops up the RDB Options box which is used to set the options necessary to generate an RDB (Resource Data Base) version of the interface at save time, along with a version of the .fm file with no resources, whose extension is fm_rdb. See Chapter 19.
- ***Message Catalog...*** Pops up the Message Catalog Options box which is used to set the options necessary to save a message catalog source file when the interface is saved. Un-numbered messages are automatically numbered. See Chapter 21.
- ***UIL...*** Pops up the UIL Options box which is used to set the options necessary to generate a UIL language version of the interface at save time. If you do not have a UIL version of XFaceMaker this option will be dimmed and therefore not available. See Chapter 18.
- ***Preserve User Code*** When set, (as indicated by the checkbutton), XFaceMaker will save the user-supplied section of the interface code.
- ***Auto-Save State*** This submenu controls how often XFaceMaker saves the *trusted state* of the Design Process. The default is every minute. Selecting ***never*** disables the auto-save mechanism. In that case, the Design Process state is only saved when you switch to Try mode.

3.6 Clearing the Interface

The **File/New** command clears the XFaceMaker Design Process so that you can begin building a new interface. This can be useful if you have been experimenting and want to throw away your test interface, or if you have saved an interface and want to start afresh.

All widgets in the interface will be removed whether they are selected or not. An alert box will warn you if the current interface has not been saved.

With this command, you also clear the FACE function declarations that XFaceMaker has registered during the current session.

3.7 Quitting XFaceMaker

To quit XFaceMaker, select **Exit** in the File menu.

If the interface has been modified since the last time it was saved, an Alert box will warn you. This allows you to cancel the **Exit** command if you want to save the changes before exiting.

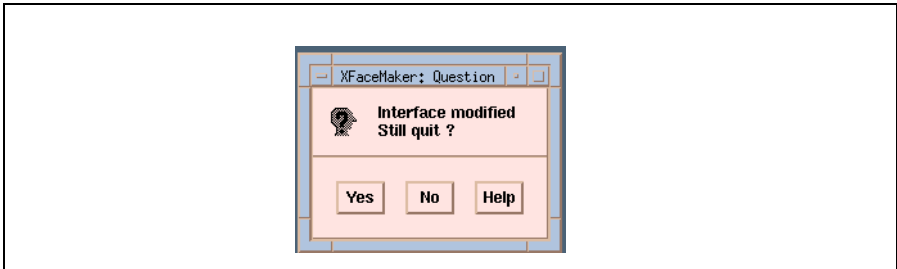


Figure 3.10: The Alert Window

CHAPTER 4: Laying out the Interface

Laying out an interface in XFaceMaker means choosing widgets from the Toolkit with the mouse, placing them on the interface, then aligning them to create the final design. This process is made easy by XFaceMaker's WYSIWYG approach and convenient editing boxes.

Each type of widget is contained in a separate “widgetstore” in the Toolkit: Shells, Composites, Primitives, Menus, etc. If you need more information on the characteristics of the Motif widgets, please refer to your Motif documentation. Some of the available manuals are listed at the beginning of this User's Guide.

As explained in Chapter 2, the “Design process” is responsible for managing the interface you are designing. It creates the interface and handles a number of editing commands such as saving and loading files. On the other hand, the “XFaceMaker process” implements the overall XFaceMaker interface. One important advantage of this dual-process architecture is that it enables you to restore the design process if an error occurs while you are designing your interface. Error recovery is explained in Chapter 6.

Each section in this chapter explains a particular layout task or technique, and describes the XFaceMaker editing boxes used to perform that task. Chapter 5 describes how to edit the resources for your interface widgets; e.g. change colors, fonts, etc.

4.1 Choosing and Placing Widgets

When you lay out an interface, XFaceMaker must be in Build mode.

Note: In this User's Guide, "click" always refers to mouse Button 1 unless indicated otherwise.

1. Open a widgetstore by clicking on its name in the Toolkit pulldown menu in the Toolkit.
2. Choose the widget type you want by clicking on its icon. The pointer changes to a cross.
3. Move the cross over the desktop and do *one* of the following:
 - m Click the mouse button. The widget will take its default size. Widgets can also be placed in the Widget Tree or List this way.
 - m Drag out the rubber band until the widget is the size you want, then release. The widget is mapped on the desktop.

You can now add other widgets by selecting them from the Toolkit and placing them on the interface. If you create a widget within another widget, the new widget will be, if possible, a child of the existing widget. For example, if you place a `PushButton` on a `BulletinBoard`, the `PushButton` is the child of the `BulletinBoard` parent. If you create the new widget outside of any existing interface widget, the new widget will be "independent" i.e. have its own toplevel Shell child of `ApplicationShell`.

- If you place a non-Shell widget directly on the desktop, XFaceMaker places a Shell widget of the same size beneath it automatically. If an `ApplicationShell` is already present in the interface, the type of shell is `TransientShell`. If there is no `ApplicationShell` in the interface, the shell is an `ApplicationShell`.
- If you place a Primitive or Composite widget directly within a Shell widget and click without dragging out the rubber band, the widget will be the same size as the Shell.
- If you place a Primitive or Composite widget directly within a Shell widget, but drag it out to a size which is smaller or larger than the Shell, the Shell will resize to the same size as the other widget.
- If you place a Primitive or Composite widget directly on a Composite widget and click without dragging out the rubber band, the widget will be given a default size.

4.2 Selecting Widgets and Gadgets

You must first *select* a widget if you want to manipulate it, or edit its resources and active values. Widgets and gadgets can be selected and manipulated singly, or in *groups*. In XFaceMaker, you can select widgets and gadgets in the same way. In this User's Guide:

- “click” always refers to mouse Button 1, unless otherwise indicated.
- “widget selection” refers to one or more selected widgets or gadgets.
- “current widget” refers to the last widget or gadget selected.

Selection/deselection procedures differ slightly for different types of widget. You can select a widget in the following ways:

- Drag a rubber band box around the widget.
- Click on the widget in the interface.
- Click on the widget's name in the Widget Tree or Widget List
- Open the edit/select popup menu with mouse button 3 press on the widget in the interface or on its name in the Widget Tree or List.

When a widget is selected, its name is highlighted in the Widget Tree and List, and *handles*, (small boxes), appear at its four corners on the interface. The three types of handle are shown below.

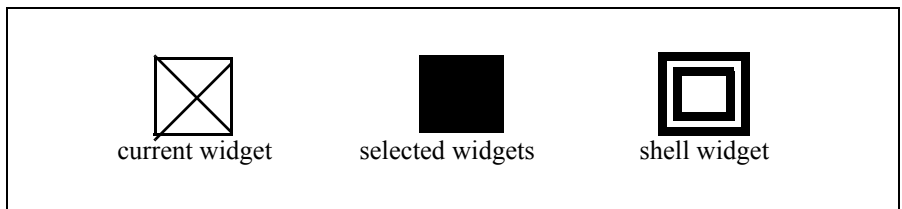


Figure 4.1: The Selection Handles

- The first type of handle indicates that the selected widget is the *current* widget. In a group of selected widgets, the current widget is the last widget selected.
- The second type of handle indicates that the widget is one of a *multiple selection*.
- The third type of handle indicates that the selected widget is a *Shell*. Shell widgets must have different handles because a Shell widget is the same size as its first child; e.g., a BulletinBoard placed within an ApplicationShell.

4.2.1 Selecting Primitive Widgets

There are several methods for selecting/deselecting primitive widgets. To *select* a single primitive widget, do one of the following:

- Click on its name in the Widget List or the Widget Tree.
- Click directly within the widget on the interface in Build mode.
- Add a widget to the selection *and* open the Resources box by clicking mouse Button 2 on the widget or its name.
- Open the *edit/select* popup menu with button 3 on the widget in the interface, or on its name in the Tree or the List and choose the *Select <name>* item in the *Cursor Object* menu.

To *deselect* a single primitive or menu widget.

- While pressing **Control**, click within the selected widget, or on its name in the Widget Tree or Widget List.

To *clear* the current selection

- Place the pointer outside any selected widget, but inside a composite, and click.

4.2.2 Selecting Composite or Shell Widgets

To select or deselect a composite or shell widget, do one of the following:

- Control-click on the widget's name in the Widget Tree or List.
- Open the *edit/select* popup menu on a widget or on its name in the Widget Tree or List and choose the *Select <name>* item in the *Cursor Object* menu>
- If the current object is a direct descendant of the desired object, open the *edit/select* popup menu with button 3 and choose the *Select <name>'s Parent* item in the *Current Object* menu. Do this repeatedly to climb the widget hierarchy.

4.2.3 Selecting Multiple Widgets

There are several ways to select multiple widgets.

Note: You can only multiple-select widgets that are children of the same parent, i.e., siblings.

- Select a single widget. While pressing **Control**, click on another widget to add it to the selection.
- Place the pointer outside the widgets you wish to select, drag out the rubber band

box to surround the widgets, and release. This selects all widgets fully contained within the rubber band box.

- Click button 2 on a widget to add it to the current selection and make it the current widget. This opens a resource editing box if one is not already open.

To *deselect* multiple widgets, place the pointer outside the selected widgets, and click. All widgets are deselected.

4.3 Naming Widgets

XFaceMaker assigns a default name to every widget in your interface, and that default name remains until you change it. The widget name is not a resource, but is used within the interface and the application to identify and access the resources of the widget. To change the default names, specify the `defaultNames` resource or use the `-defaultNames` command line option. Here is an example of specifying the `defaultNames` resource in an `.Xdefaults` file.

```
Xfm*defaultNames: ApplicationShell=AS,XmDialogShell=DS,\
TopLevelShell=TL, TransientShell=TS, \
XmBulletinBoard=BB,XmForm=F,XmFrame=FR,XmRowColumn=RC,\
XmPanedWindow=PW,XmMainWindow=MW,XmScrolledWindow=SW,\
XmDrawingArea=DA,XmLabel=La,XmText=T,XmTextField=TF,\
XmScrollBar=SB,XmArrowButton=AB,XmToggleButton=TB,\
XmPushButton=PB,XmList=Li,XmDrawnButton=DB,\
XmSelectionBox=SB, XmFileSelectionBox=FSB,\
XmMessageBox=MB
```

Any legal C language variable name is a legal widget name. XFaceMaker ensures that two sibling widgets do not have the same name by adding a numeric extension to a name if it already exists, unless you use the command line option `-nonuniqueNames`.

If you change the name of an `ApplicationShell`, the change will be displayed in the “Name” field of the Widget Resources window, in the Widget Tree, and the Widget List. However, the name of the `ApplicationShell` itself, as it appears in the title bar, is always that of the *current* application, i.e. `xfm`, unless you set the `title` resource to the new name.

When a script refers to a widget and that widget’s name is changed, the script is updated to use the new name *if* the widget whose name is changed (the widget referred to by the script) is created *before* the script is parsed. Therefore, the creation order of widgets is important.

There are two methods to name a widget:

- In Widget Tree, double-click on the name of the widget and enter the new name in the text field that is overlayed on the Tree. Use Ctrl-Delete to delete the previous name.
- In Widget Resources window, enter the new name in the “Name” field.

To retrieve a widget's identifier in a script, use its reference name, not just its name. The reference name always contains one of the FACE keywords: **self**, **parent**, **toplevel**, or **root**. For example, to hide a widget whose name is BB1, you could have:

```
Hide (toplevel.BB1);
```

To refer to a widget's resource, follow its reference name by a colon (:), then the name of the resource. You may use the resource in expressions or in assignments, just as you would use any ordinary variable.

4.4 The Widget Tree and Widget List

The Widget Tree displays your current interface as a tree structure. You can scroll through the tree or enlarge it to get a full view of the interface as you are designing it. The Widget List displays all the widgets of your current interface in a hierarchical list. Widget names are indented from the left hand edge of the window to reflect their position in the widget hierarchy. Thus, a child widget is listed below its parent and two characters to the right. Selected widgets are highlighted.

4.4.1 The Widget Tree

Open and close the window by toggling the Widget Tree icon.

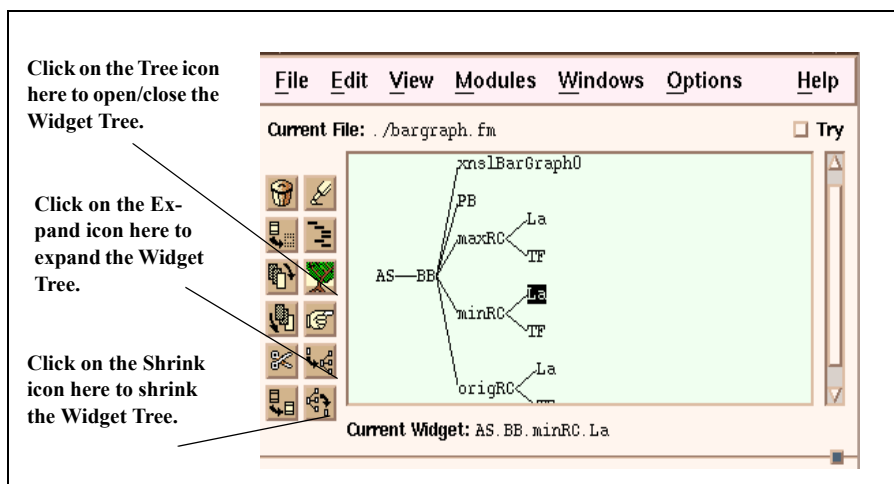


Figure 4.2: The Widget Tree

You can use the Widget Tree to do the following:

- Select a widget or gadget by clicking on its name in the Widget Tree. The name will be highlighted.
- Add a widget to the hierarchy by clicking on its icon in the Toolkit, then moving the cursor to the Tree and clicking where you want the widget to be positioned in the hierarchy.
- Change the name of a widget by double-clicking on its name in the Widget Tree and entering the new name. Use Ctrl-Delete to delete the previous name. See section 4.3 on naming widgets.
- Display the hidden sub-tree of the selected widget by toggling the “Expand” icon

in the Command window, or by choosing the ***Expand*** command in the View menu.

- Hide all the sub-tree of the selected widget by toggling the Shrink icon in the Command window, or by choosing the ***Shrink*** command in the View menu. The selected widget's name is then highlighted to indicate that it has hidden children.
- Manipulate widgets. Press the Select mouse button on the widget name and drag the *widgetname* outline over any of the Tool Icons.
- Place a widget on the interface. Select a widget type from a widgetstore, then place the cross cursor on the name of a *parent* widget in the Widget Tree and click or draw out the widget's shape.
- Re-order menu elements or widgets. Simply drag the widget to a new position in the Widget List. Widgets can only be dropped onto another widget of the same level in the widget hierarchy (a sibling).
- Open the *edit/select* popup menu. Press mouse button 3 and choose the item you want in up to three sub-menus: *Selection*, *Current Object*, *Cursor Object*.

4.4.2 The Widget List

Open or close the window by toggling the Widget List icon in the Command window. You can scroll or resize the window to view the entire interface.

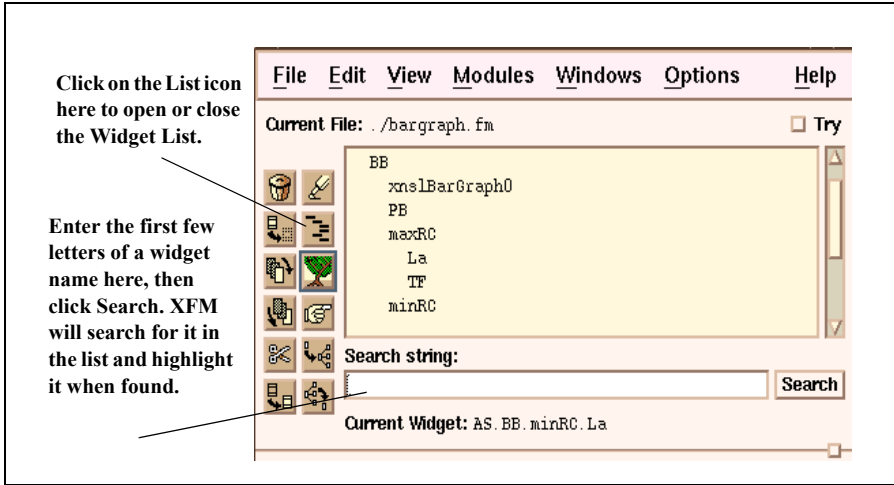


Figure 4.3: The Widget List

You can use the Widget List to do the following:

- Select a widget by clicking on its name in the List.
- Search for a widget name in a large interface, and select it. To do this, type the first few characters of the widget's name in the "Search filter" box, then click on the "Search" button. The search is case sensitive.
- Manipulate widgets. Press the Select mouse button on the widget name and drag the *widgetname* outline over any of the Tool Icons.
- Place a widget on the interface. Select a widget type from a widgetstore, then place the cross cursor on the name of a *parent* widget in the Widget List and click or draw out the widget's shape.
- Re-order menu elements or widgets. Simply drag the widget to a new position in the Widget List. Widgets can only be dropped onto another widget of the same level in the widget hierarchy (a sibling).
- Open the *edit/select* popup menu. Press mouse button 3 and choose the item you want in up to three sub-menus: *Selection*, *Current Object*, *Cursor Object*.

4.5 Moving Widgets

This section refers to *non-shell* widgets. A *shell* widget is under the control of the window manager and is moved in the same way that any window is moved. Moving a *non-shell* widget has one of two meanings:

- It changes a widget's geometry (its x,y position in the interface).
- It changes a widget's position in the interface hierarchy.

You may wish to change a widget's place in the hierarchy in order to change the creation order of sibling widgets. (See Chapter 6.) You can move a selected, non-shell widget using the mouse or by changing its parameters in the Resources box.

To move (change the x,y position) a non-shell widget:

1. Press the Select mouse button on a widget in the interface. The pointer changes to the move cursor.
2. Drag the widget to a new position and release.

To re-order a widget's place in the hierarchy:

1. Press the Select mouse button on a widget's name in the Widget Tree, or the Widget List. The pointer changes to the move cursor.
2. Drag the widget to a new position and release. The move cursor *must* be over another widget when you release.

If you move a selected widget in the Widget List or Widget Tree, the widget's place in the hierarchy is changed, but not its x,y coordinates. You cannot re-parent a widget by moving it in the Widget List or Tree.

4.6 Cut, Copy, Paste, Duplicate, Delete

You can cut, copy, paste, duplicate, and delete widgets using the editing icons or the commands in the Edit menu, or the edit/select popup menu. Use Cut and Paste to re-parent widgets in your interface.

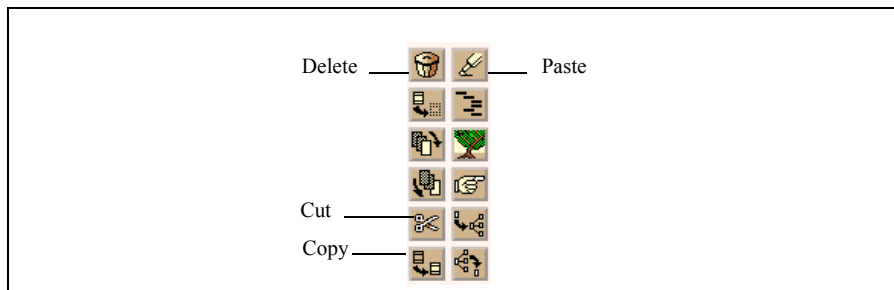


Figure 4.4: The Cut, Copy, Paste, Delete Icons

4.6.1 Cutting Widgets

When you *cut* a widget, it is removed from the interface, and a copy of it is placed in the widget cut buffer for subsequent pasting. There are several ways to cut widgets. Start by selecting the widget or widgets. Then do one of the following:

- Enter **Ctrl-X** to cut the selection.
- Drag the widget selection over the Cut or the Copy icon in the Main Command window. Release the mouse button.
- Select the widget(s), then choose **Edit/Cut**.
- Click on the Cut icon, move the icon shadow over the widget in the interface (not on a gadget - you would operate on its manager), or in the Widget Tree or List, then click the *Select* mouse button. This cuts a widget whether it is selected or not.
- Open the edit/select popup menu and choose the *Cut* item in the *Selection* menu.

4.6.2 Copying Widgets

When you *copy* a widget, it remains on the interface, and a copy of it is placed in the widget cut buffer for subsequent pasting. There are several ways to copy widgets. Start by selecting the widget or widgets. Then do one of the following:

- Enter **Ctrl-C** to copy the selection.
- Drag the widget selection over the Copy icon in the Main Command window. Re-

lease the mouse button.

- Select the widget(s), then choose *Edit/Copy*.
- Click on the Copy icon, move the icon shadow over the widget in the interface (not on a gadget - you would operate on its manager), or in the Widget Tree or List,, then click the *Select* mouse button. This copies a widget whether it is selected or not.
- Open the edit/select popup menu and choose the *Copy* item in the *Selection* menu.

4.6.3 Pasting Widgets

To paste, there must first be a selection in the widget cut buffer; that is, you must have previously cut or copied widgets, although several other actions may have occurred in the meantime. The *last* widget selection cut or copied from the interface will be pasted. There are several ways to paste widgets.

- Enter `Ctrl-P`. Move the widget outline over the interface and click.
- Click on the Paste icon, drag its outline to the position on the interface, or in the Widget Tree or List, where you want the widget selection to be pasted, and click.
- Choose the *Edit/Paste* command, drag the outline of the widget to the interface, and click.
- Open the edit/select popup menu and choose the *Paste* item in the *Selection* menu. The item is insensitive if the widget cut buffer is empty.

A copy of the *last* selection cut, copied, or pasted remains in the widget cut buffer until a new selection is cut or copied. You can re-paste as many times as you wish.

4.6.4 Duplicating Widgets

When you duplicate a widget, a copy of it is created just below and to the right of the original. To duplicate, select the widget(s), then:

- Enter `Ctrl-D`. The duplicate widget appears just below the original.
- Choose *Edit/Duplicate*.
- Open the edit/select popup menu and choose the *Duplicate* item in the *Selection* menu.

4.6.5 Deleting Widgets

Deleted widgets are removed from the interface, but are not placed in the widget cut buffer. There are several ways to delete widgets.

- Select the widget(s) and enter **Ct r l - K**.
- Drag the widget over the Delete icon in the Command window, then release the *Select* button.
- Select a widget, then click on **Edit/Delete**.
- Open the edit/select popup menu and choose the *Delete* item in the *Selection* menu.
- Click on the Delete icon in the Command window, move the icon shadow over a widget in the interface (not on a gadget - you would operate on its manager), or in the Widget Tree or List, then click the Select mouse button. This deletes the widget from the interface whether it is selected or not.

4.6.6 Gadgets

Gadgets can be copied, cut, and deleted just like widgets, by selecting them and dragging them to the icon in the Main Command Window, or by bringing the icon over the widget name in the List or Tree.

Do not drag the Cut, Copy or Delete icons over a gadget in the interface because the icon will act on the composite widget above the gadget, since it has an X Window and the gadget does not.

4.7 Resizing a Widget

You can resize a selected widget or gadget in two ways:

To resize a widget using the mouse:

1. Place the pointer on one of the widget handles, and press *Select*.
2. Drag the corner until the widget is the size you want, and release.

To resize a widget using the Resources box, enter the values directly in the *w* and *h* fields of the Resources box.

4.8 Tracking a Widget Name

XFaceMaker's Track feature enables you to fetch the name of a widget and store it in the X server's cut buffer. You can then paste the widget name somewhere else; for example, in a script you are writing. This saves time and helps you to avoid errors when referring to widgets in your interface.



Figure 4.5: The Track Icon

To track a widget name:

1. Click on the "Track" icon. This pops up the track cursor ('?').
2. Move the cursor over the widget in the interface, or the widget name in the Widget Tree or Widget List, and click.

This stores the widget reference name in the X buffer for subsequent pasting. The name is also displayed in the status field.

- If no widget is selected, it is the tracked widget's absolute reference name.
- If there is a currently selected widget, the name displayed and stored is the tracked widget's reference name relative to the selected widget.

4.9 Raising and Lowering Widgets

You can raise or lower the widgets on your interface to make them easier to see or manipulate. Raising or lowering a widget has no effect on the final interface. The relative placing of widgets or windows is dependent solely on their order of creation, i.e., the last widget as shown in the Widget List is the last created and therefore the topmost.

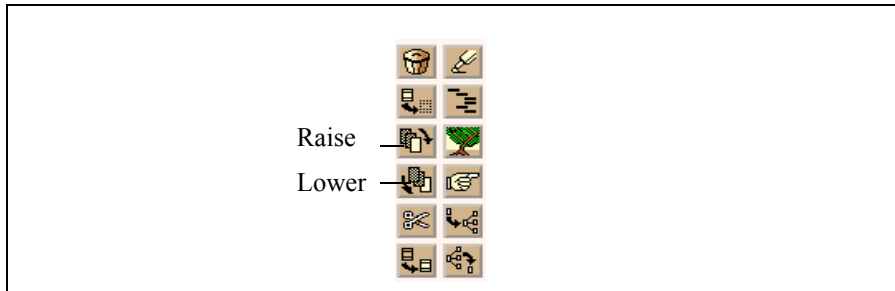


Figure 4.6: The Raise and Lower Icons

There are three ways to raise or lower a child widget:

- Drag the widget selection over the Raise or Lower icon in the Command window and release the button. The selection will be raised or lowered. Note: When a widget selection is moved to the Raise or Lower icon, *all* the currently selected widgets are raised or lowered.
- Select the Raise or Lower icon, drag its shadow over the widget and click. The widget does not need to be selected.
- Select a widget, then choose **View/Raise** or **View/Lower**.

Note:

- Gadgets cannot be raised or lowered.

4.10 Hiding and Showing Widgets

You can display or hide your interface, or any selected portion of it, using the Hide and Show icons or the commands in the View menu. This affects the final interface because a hidden widget will be created unmapped.

To hide or display the entire interface, whether or not any widgets are currently selected:

- Select **Hide/All** from the View menu to hide the interface.
- Select **Show/All** from the View menu to display the interface.

There are two ways to hide a *part* of the interface. First, select the widgets, then:

- Drag the selection over the Hide icon.
- Choose **Hide/Selection** from the View menu.

If you selected a parent widget, the parent and all its children will be hidden, even if the children were not selected.

There are two ways to display part of the interface:

- Select **Show/Selection** from the View menu to re-display.
- Drag the Show icon over the selection.

To hide or display a Shell widget and its children:

- Drag the Hide or the Show icon over the Shell widget and release.
- Select **Show/Shells** or **Hide/Shells** from the View menu.

Note: When using a DialogShell, the child should be hidden and not the DialogShell itself, because of the way Motif implements DialogShells. Do this by selecting the child, then the **View/Hide** command.

4.11 Aligning Widgets

When laying out an interface it is often useful to align widgets relative to each other, to a parent widget, or on a grid. The Alignment window helps you to do this. Open it by clicking on *Edit/Align....*

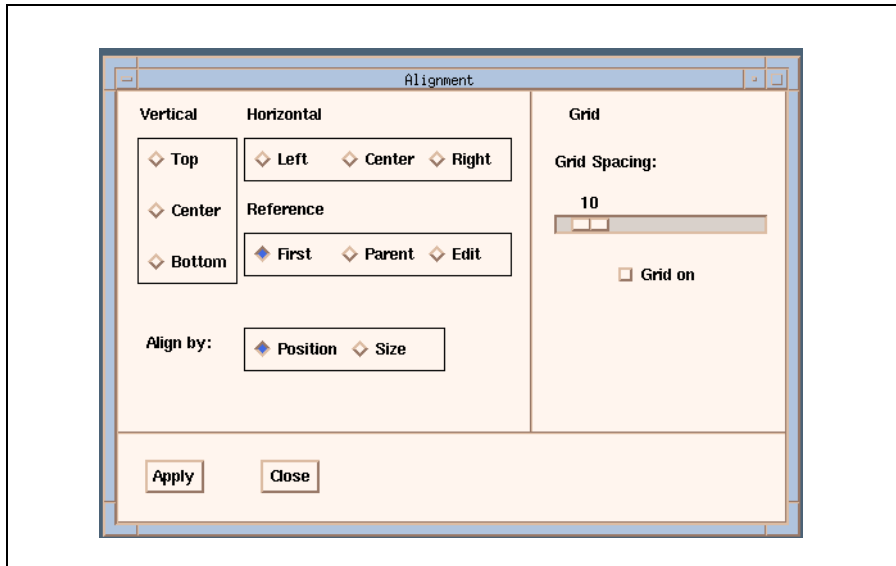


Figure 4.7: The Alignment Box

To align widgets on your interface:

1. Select the widgets to be aligned.
2. Specify the alignment choices in the Alignment window.
3. Click on the *Apply* button in the Alignment window.

There are four groups of radio buttons in the Alignment window which are used to define widget alignment.

- The **Vertical** and **Horizontal** radio buttons define the orientation of the alignment.
- The **Reference** radio buttons define the referent or model; i.e. the widget with respect to which widgets are aligned.
- The **Align by** radio buttons specify whether the alignment changes the widget's position or its size.

4.11.1 Aligning Widgets Relative to Other Widgets

A widget can be aligned with its parent or to a sibling widget; that is, a child of the same parent. When a widget is aligned to a sibling, the reference widget can either be the first widget that was selected, or the last.

The “Reference” radio buttons work as follows.

First The widget is aligned with the first widget that was selected, which must be a sibling. The first widget has solid handles.

Parent The widget is aligned with the parent widget.

Edit The widget is aligned with the last widget selected, called the *Edit* widget, or the *current* widget. The *Edit* widget has X handles.

4.11.2 Aligning Widgets by Size

Aligning primitive widgets horizontally or vertically by *size* is a convenient way to ensure that siblings have equal widths or heights. To do so:

1. Select the primitive widgets to be aligned.
2. Specify the reference widget—“First” or “Edit”.
3. Specify a vertical value (for height) and/or a horizontal value (for width).
4. Click ***Apply***.

Notes: Bottom and right alignments do not operate in the same way as top, left or center alignments, due to Motif. For instance, if you try to align a primitive widget to “bottom of parent”, the widget will not move unless you set the composite widget’s `marginHeight` resource to zero.

Similarly, you must specify a zero `marginWidth` to obtain correct alignment to the right. Notice that if you set zero margins, all the border alignments cause the primitive widget to touch the inside border of the composite while, in the default situation, a gap, dependent on the size of the margins, is left between the edge of the composite and the primitive.

4.11.3 Aligning Widgets on a Grid

You may find it helpful to apply a grid to a *composite* widget before laying out child widgets. The grid will not exist in your interface, it is only a visual aid placed on the screen by XFaceMaker.

When the grid is used, a child widget placed on the composite will *snap* to the nearest grid lines, both horizontally and vertically. This allows you to align a collection of child widgets on the interface neatly. Use the slider to set the spacing between lines of the grid.

If you simply move or resize a widget on a grid, the corner of the widget nearest to a grid point will snap to that grid point---the other corners may not, depending on the size of the widget.

To apply a grid to your interface:

1. Select a single composite widget
2. Set the Grid Spacing slider
3. Set the ***Grid on*** button.

4.12 Printing an Interface or Widget Tree

You can create a postscript file of your interface or of the Widget Tree for subsequent printing, using the **Print** command in the File menu, or the Print icon in the Command window. Use the Print Options box to specify the printing options you wish. Once the .ps file is generated, you can print it as you would any other .ps file.

To create a .ps file of an interface window using the **Print** command:

1. Select **File/Print...**
2. Select **Window...** This calls the `xwd` command. A cross-hair cursor appears.
3. Click on the interface window you wish to print. XFaceMaker generates a postscript file of the window. By default, the .ps file name is that of the top level widget of the window.

You can also create a .ps file of the Widget Tree for your interface, descending from the currently selected widget. If no widget is selected, or if the root widget is selected, the complete tree will be described.

To create a .ps file of the Widget Tree using the **Print** command:

1. Select **File/Print...**
2. Select **Tree...** The File Selector pops up for specifying the file.
3. Enter the file name and click OK. XFaceMaker will generate a .ps file of the Widget Tree for the specified interface file.

4.13 Setting Print Options

You can set options for printing interface windows in the Print Options box. The options set here do not affect the File/Print/Tree command, only the File/Print/Window command. To open the Print Options window, select **Print** from the Options menu.

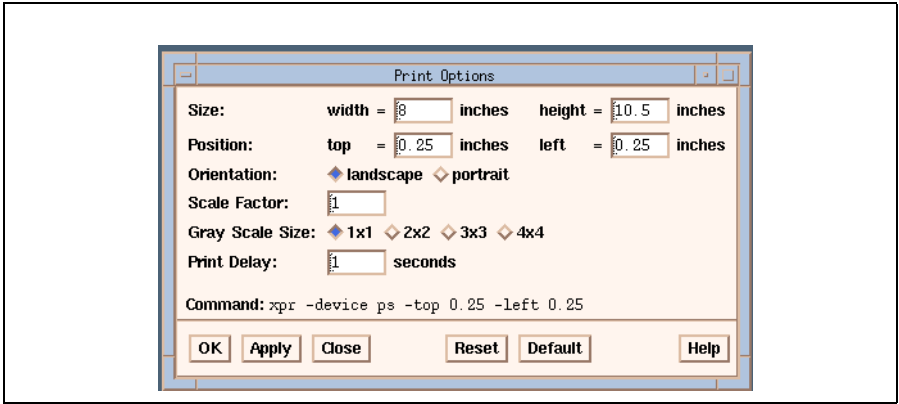


Figure 4.8: The Print Options Window

You can set the following options by entering values, and clicking the Apply button:

- width** The page width in inches; default value 8.
- height** The page height in inches; default value 10.5.
- top** The page top offset in inches; default value 0.
- left** The page left-hand side offset in inches; default value 0.
- landscape** Prints the window in landscape mode; default. “Landscape” orientation is a page that is wider than it is tall.
- portrait** Prints the window in portrait mode. “Portrait” orientation is a page that is taller than it is wide.
- Scale factor** Specifies the size of the window on the page by adjusting its size proportionately. The value can be between one and six, inclusive. For example, setting a scale factor of three enlarges the window by a factor of three. By default, a window is printed with the largest scale that fits onto the page for the specified orientation.
- Gray Scale Size** Gray scale conversion of color image. Value can be between one and four.

Print delay Number of seconds to wait before storing the window; default one second.

Command Shows the specified `xpr` command parameters.

For further information on the print parameters, please refer to the `xpr` documentation.

The pushbuttons at the bottom of the window are used as follows:

OK Sets the print options for future printing and closes the box.

Apply Applies the print options you have entered in the window to any future **Print** commands, but does not close the Print Options box.

Close Closes the window without applying the options.

Reset Resets the fields of the window to the previous values; i.e. the values specified at the last **Apply**.

Default Resets all the fields of the box to their default values.

Help Pops up a Help box on using the Print Options window.

Once the options are set and applied, select the **Save Prefs** item in the File menu to save the print options for subsequent XFaceMaker sessions.

CHAPTER 5: Resource Editing

Resources are properties that define such things as a widget's position on the interface, its size, its color, or the type of font used for text. There are also specialized resources called *callbacks*, which determine how the widget behaves in response to events such as mouse clicks. In addition, XFaceMaker's unique "active values" can act as user-defined resources. Active values are described in Chapter 8.

XFaceMaker provides a number of resource editing windows and dialog boxes.

- The Widget Resources window displays all the resources available for the selected widget, and enables you to edit them in a consistent way. It also gives you access to the specialized resource editing windows. You can display two different versions of this window:
 - m You can display its default version, which includes icons that represent resource value types.
 - m You can display the "Fast" version, which does not include the icons, but which has a faster reaction time. To display the "Fast" version of the window, you can launch XFaceMaker with the `-fast list` option, or set the ***Use Fast Resource List*** item in the Options menu.
- Specialized editing boxes can be popped up when you need to specify values such as booleans, integers, or strings, to select fonts or pixmaps, or to edit scripts for a callback resource.

This chapter explains how to edit widget resources, and how to use the resource editing windows provided by XFaceMaker.

5.1 General Procedures

This section describes the general procedure for editing a resource. Callback resources are discussed in Chapter 6, active values in Chapter 8.

To edit the resources of a selected widget:

1. Open a Widget Resources window.

By default, the window displays the resources of the currently selected widget. (Select a widget *and* open a Resources window by clicking on the widget with mouse Button 2, or by opening the edit/select popup menu with mouse Button 3).

2. Click to the right of the resource you wish to edit.

This opens a text field to the right of the resource name. If you click on the resource name, the resource is selected---its name is highlighted. You can select one or more resources, edit them, then apply only the selected resource changes to the widget.

You can search for a resource in the list in two ways.

m Enter a few letters of its name in the Search field and press Return.

m Place the pointer in the resource list and type the first letter of the resource name. If your keyboard focus policy is “pointer”, place the pointer in the list outside of the text field. If your keyboard focus policy is “explicit”, the focus must be on the icon to the left of the text field. You can also use the arrow keys to move up and down the list.

3. Enter a value in the text input field next to the resource name.

Or click on the ellipsis to pop up an editing window for that type of resource. For example, when you are editing the `borderPixmap` resource for a widget, clicking on the ellipsis opens the Pixmaps box in which you can specify a pixmap.

4. Repeat steps 2 and 3 for all the resources you wish to modify.
5. Click on the **Rebuild** or the **Set Values** button, or press **Return**.

This registers the changes to be written to the interface description file. The window icon, which was “open” while you were editing the resource, is now displayed in a “closed” state to indicate that the resource list has been applied. The apply options are described in Section 5.2.6.

- m The **Rebuild** option applies the resource list by first destroying the object being edited, then re-creating it.
- m The **Set Values** option applies the resources list to the edited object by setting the values. The changes may not be immediately displayed.
- m Pressing **Return** applies the resource list, including the **x**, **y**, **w**, and **h** fields, by setting the values; i.e. as if you had pressed the **Set Values** button. However, if you simply clear the text field and press **Return**, the resource is reset to its default value by *rebuilding* the object.

If you do not apply the resource changes before closing the window or selecting another widget, an alert box will warn you that the changes will be lost. The resource changes may not be immediately visible, depending on whether you applied the changes by rebuilding or setting values.

You can reset a resource to its default value in two ways:

- Clear the value in the text field, then press **Rebuild**, **Set Values**, or **Return**. Pressing **Return** resets the resource to its default value by rebuilding the object.
- Press mouse Button 3 while the pointer is over the resource name or icon you want to change. Select the **default** item in the “Value” popup menu. Then click **Rebuild** or **Set Values**. (This action is not possible when using the “fast” resource list.)

5.2 The Widget Resources Window

The Widget Resources window is structured so that you can easily identify and access all the resources for the widget you are editing. You can open multiple Resource windows, and iconify them. You can edit selected or unselected widgets. To open a Resources window, select *Windows/New Resource Window*, or select a widget with mouse button 2.

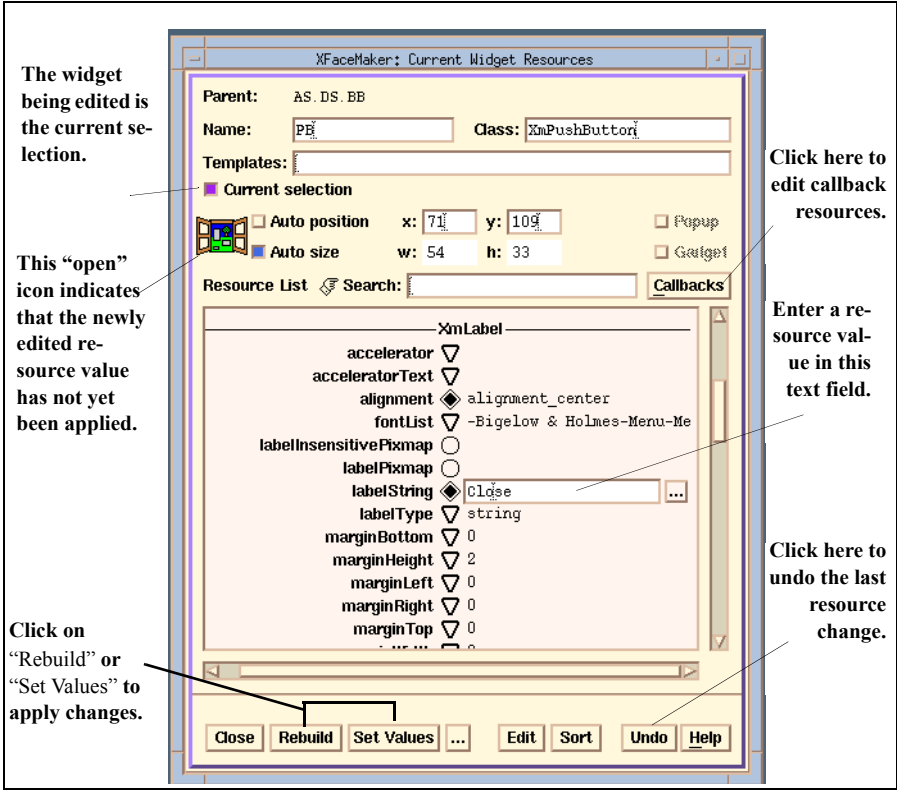


Figure 5.1: The Widget Resources Window

You can edit several different widgets at once by opening multiple Resource windows. To do so, open one Resource window, deselect the "Current Selection" check-button, then open another Resources window.

You can also drag and dropt an object to a Resource window to display its list of resources for editing. Use mouse Button 2.

5.2.1 The Widget Identification Fields

The fields at the top of the Widget Resources window display information on the widget you are editing. You can enter new values directly in several of these fields. The field is dimmed if the value does not apply to the widget being edited.

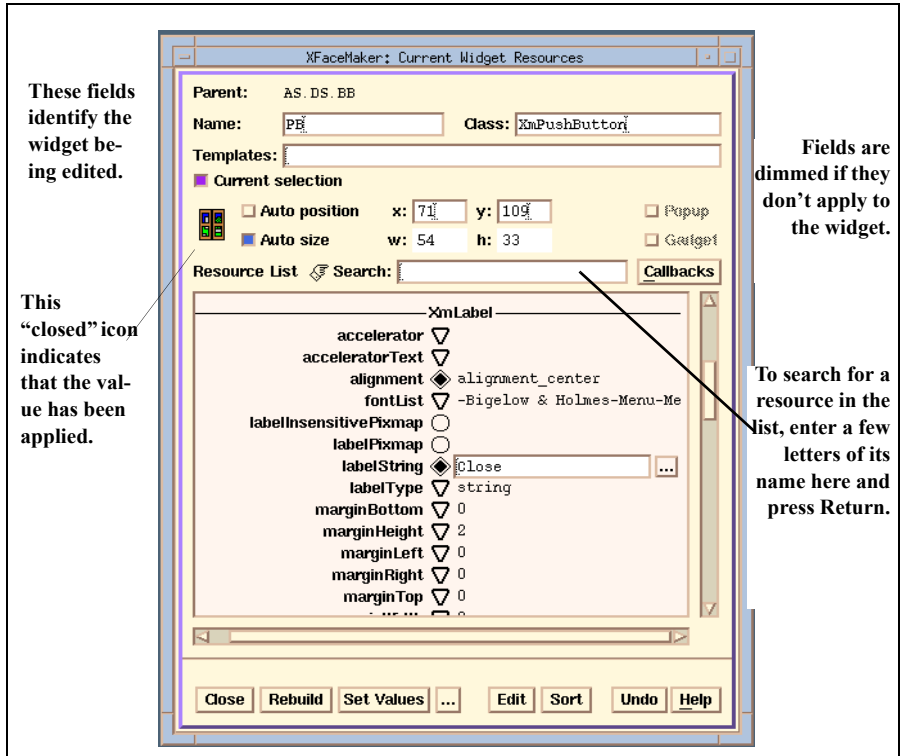


Figure 5.2: The Identification Fields

The Resources window has the following identification fields:

Parent Displays the name of the parent of the widget being edited.

Name Displays the name of the widget being edited.

XFaceMaker assigns a default name to every widget in your interface, and that default name is displayed in this field until you change it. The widget name is not a resource, but is used within the interface and the application to identify and access the resources of the widget. Specify the XFaceMaker defaultNames resource to change default name assignments..

To assign a different name to the widget you are editing:

1. Modify the name displayed in the “Name” field.

Any legal C language variable name is a legal widget name. XFaceMaker ensures that two sibling widgets do not have the same name by adding a numeric extension to a name if it already exists, unless you use the command line option `-nonuniquenames`. If you change the name of your ApplicationShell, the change will be displayed in the “Name” field, the Widget Tree, and the Widget List. However, the name of the ApplicationShell itself, as it appears in the title bar, is always that of the *current* application, i.e. `x_fm`, unless you set the `title` resource to the new name.

2. Click on the **Rebuild** button or the **Set Values** button.

Class identifies the widget’s class. This can be edited. You can drag an icon from the Toolkit to this field. When you change the class, you may need to remove resources that you had set and that do not exist for the new class. This is true also of constraint resources on children.

Templates Identifies the template associated with the widget, if one is defined. You can apply or delete a template. There can be more than one template listed here, separated by commas. You can apply a template using the **Apply/Unapply template** commands in the Edit menu, or in the Resources window as described below.

To apply a template to the widget you are editing:

1. Enter a template name in this field.
2. Click on the **Rebuild** button or the **Set Values** button.

To remove a template from the widget you are editing:

1. Delete the name of the template from the **Templates** field.
2. Click on the **Rebuild** or the **Set Values** button.

Current Selection Indicates if the widget is the current selection. Deselect this checkbox to continue editing the widget after deselecting it.

Auto position When set and applied to a widget, XFaceMaker will not specify `x` and `y` settings.

Any previous settings for the `x` and `y` resources in the Resource Editor box are removed. For example, you might use this option for a button whose position is defined by its attachment resources and whose `x` and `y` resources are therefore not required. This also reduces the size of the `.fm` or C-code file generated. Note that *Auto Pos* and *Auto Size* may be applied to several widgets at once, or to the whole interface, by using the menu item *Auto Pos/Size* in the Edit menu.

x and y Indicate the position of the upper left corner of the widget relative to the composite or shell widget that contains it; or, if it is a shell widget, to the upper left corner of the display.

The x axis increases from left to right, and the y axis downwards. All measurements are in pixels. The previously described *Auto position* option may override these fields. There is no “3-d” shadowing in these fields when “Auto pos” is set.

Auto size When this checkbox is set and applied, XFaceMaker will not specify *w* and *h* settings.

For example, you might use this option for a label that should default to the size of its `label-String` resource, or for an `XmlList` contained within an `XmlScrolled Window`. Any previous settings for the width and height resources in the Resource Editor box are removed when Auto Size is set. This also reduces the size of the .fm or C-code file generated. Note that *Auto Pos* and *Auto Size* may be applied to several widgets at once, or to the whole interface, by using the *Edit* menu item *Auto Pos/Size*.

w and h Indicate the size of the current widget in pixels. The *w* parameter gives the width of a widget along the x axis and the *h* parameter its height along the y axis.

Note that the previously described *Auto size* option may override these fields. Measurements always reflect the actual values on the screen. Thus if you select a widget and move or resize it with the mouse, these values are updated automatically. You can also type in new values and *Apply* them to the widget. It may help to remember that monitors vary in size. Common monitor dimensions are about 1200 pixels in width and 1000 in height. A Sun 19-inch monitor is 1152 by 900 pixels, while a PC screen may only have a resolution of 1024 by 768 pixels. There is no “3-d” shadowing in these fields when “Auto size” is set.

Popup Indicates if the widget is a `PopupShell`.

In most interfaces, any `Shell` widget which is not the interface’s `ApplicationShell` is a `PopupShell` and XFaceMaker will set this flag by default. Because the Xt Toolkit allows the creation of toplevel shells that are not `Popup`, this possibility has been implemented in XFaceMaker. However, we recommend that this flag is never changed.

Gadget When this checkbox is set and applied, XFaceMaker uses the equivalent gadget version of the current widget, if available.

Some widget resources are not accessible to gadgets. XFaceMaker refuses to change a widget which uses such a resource to a gadget. Therefore, you must not use such resources for any widget you wish to change to a gadget. Note that several, or all widgets in the interface can be changed to their gadget equivalent if it exists by using the *Edit* menu item *Gadget*.

5.2.2 The Resource List

The Resources List displays the resources of the widget you are editing.

- An icon next to the resource name that indicates its value type: default, template, RDB, or XFM, or unset.
- You can edit the resource in the list or open a specialized editing box.
- The resources can be sorted in various ways; for example, so that the ones you use frequently are displayed first.
- You can use the Motif drag and drop feature to drag resource values between the interface and the list, or between resources.

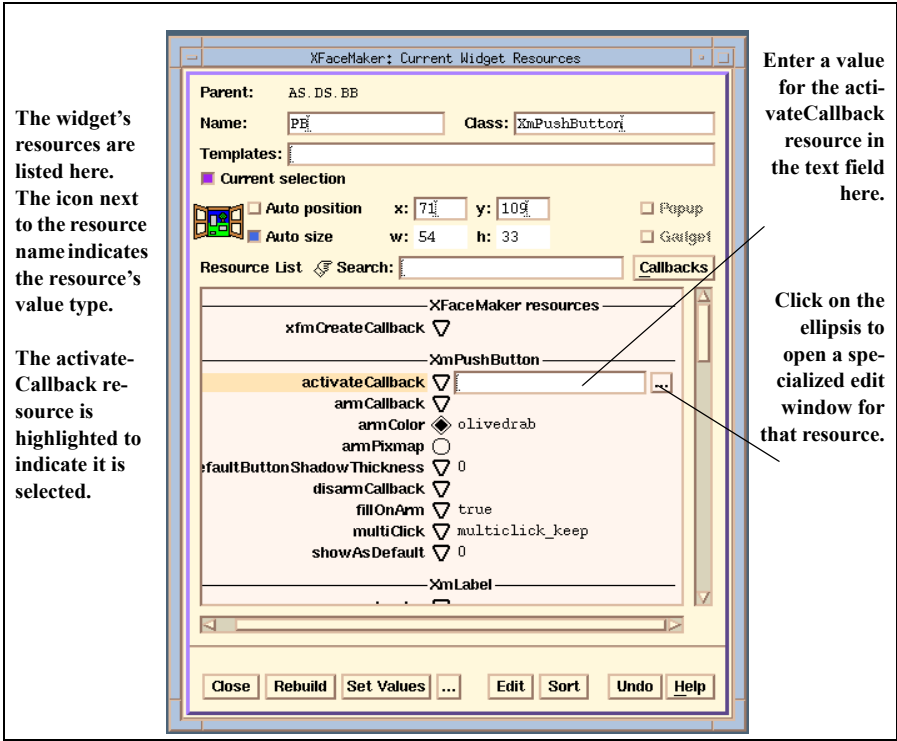


Figure 5.3: The Resource List

To edit a resource, click next to its name. This opens a text field for the value. Clicking *on* the resource name also *selects* the resource. You can edit selected or unselected resources. To apply the changes, click on **Rebuild** or **Set Values**, or press **Return**. See Section 5.1 which explains more on resource editing.

- To search for a resource, scroll the list or enter its name (or a few letters of its name), in the “Search” field and press **Return**. Or, with the keyboard focus in the resource list, type the first letter of the resource’s name. The list will advance to the first resource beginning with that letter.

5.2.3 Using Drag and Drop in the Resource List

You can drag and drop widgets and resource values between the interface and the list, using mouse Button 2. You can do the following operations:

- Drag a widget to a Resources window for editing.
Press mouse Button 2 on the widget, or on its name in the Widget Tree or List. The cursor changes to a drag icon. Drag the icon to a Widget Resources window, and drop it anywhere in the window.
- Drag a resource value to a widget.
Press mouse Button 2 on a text field in the list. Drag the icon over a widget or its name in the Tree or List and release. If the widget has a resource of that type, the value will be applied. If not, of course, no change is made, and an error message will appear. For instance, you cannot set an `arrowDirection` resource value for a `PushButton`.
- Drag a value from one resource to another resource in the list.
Click to open the text field next to a resource. Press mouse Button 2 on the value listed. Drag the icon over another resource and release. You must then apply the new value by clicking on **Rebuild** or **Set Values**. (This is unavailable in the “Fast” Resource list.)
- Drag a list of selected resources to a widget.
Select the resources using the standard Motif extended selection process, then drag them to the widget using Button 2. This sets the resources for that widget. If you drag to a resource window, it updates the window, but you must click **Rebuild** or **Set Values**. If you drag to a widget (or its name in the Widget List or Tree), it applies the list to the object, by rebuilding or setting the values, on the window where the list came from.

By default, XFaceMaker displays the default values of unset resources in the list. You can disable this option by toggling the item in the Options menu.

5.2.4 The Resource List Icons

The icons in the Resource list indicate the following:

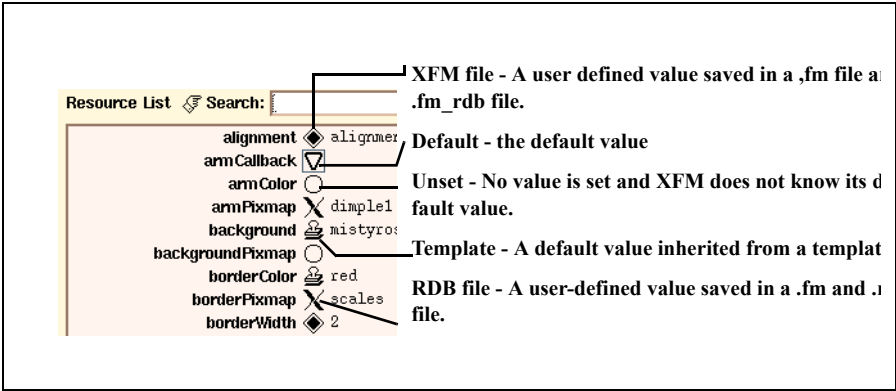


Figure 5.4: The Resource List Icons

5.2.5 The “Fast” Resource List

This version of the Resource list is displayed if you launched XFaceMaker with the `-fastlist` command line option, or if you set the *Option/Use Fast Resource List* item.

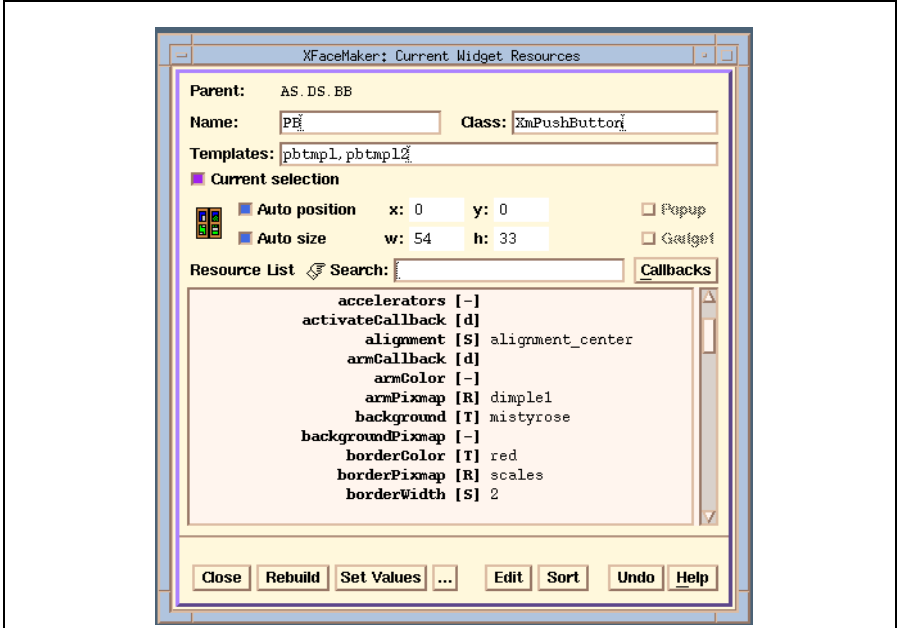


Figure 5.5: The “Fast” Resource List

The letters in the fast Resource list indicate the following:

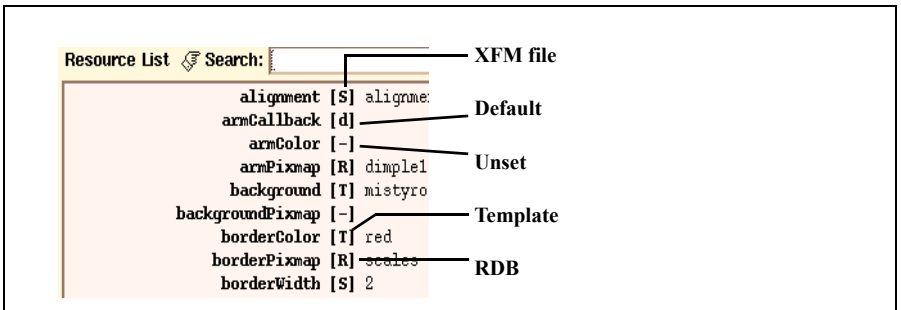


Figure 5.6: The Fast List Letters

5.2.6 Overriding the Default Apply Options

By default, when you edit resource values and click **Rebuild** or **Set Values**, XFace-Maker applies:

- m **all resources** (all the resource modifications)
- m to **self** (the widget being edited in that window).

You can override the default apply options. To do so:

1. Enter the new resource values in the Resources list, but do not click **Rebuild** or **Set Values**.
2. Open the Apply Options box by clicking on the ellipsis next to the **Set Values** button in the Resources window.
3. Set the options in the Apply Options box and click on **OK**. This applies the new resource values and closes the Apply Options box.

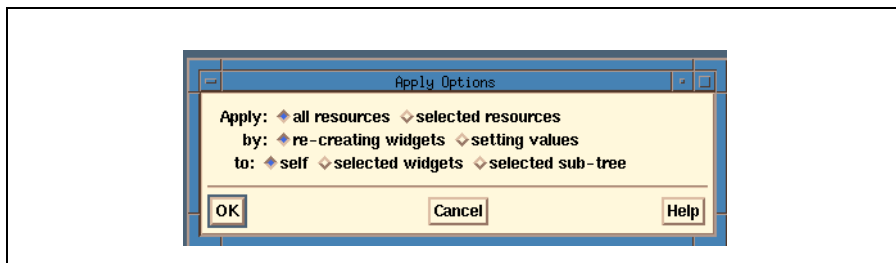


Figure 5.7: The Apply Options Box

Note: The options you set are only in effect for the current edit. The next time you edit a resource, the default apply options will again be in effect. If you want to keep the default options, simply click on **Rebuild** or **Set Values**. Otherwise, re-open the Apply Options box, change the options, and click **OK**.

The choices in the Apply Options box are as follows. You can apply:

- **all resources** -- Applies all changes made since the last *Apply*. This is the default mode.
- **selected resources** -- Applies only the selected resources in the list.

The new resource values can be applied by:

- **re-creating widgets** -- A new instance of each object implied in the resource change will be created for each resource modification. The effects of the change are immediately visible. This is the default mode.
- **setting values** -- Choosing this option applies the new resource values to existing objects. Most resource changes are directly visible, although some are not. Some resources cannot be changed this way, and an error message will be generated.

The new resource values can be applied to:

- **self** -- Applies the values to the widget being edited in that window. This is the default mode.
- **selected widgets** -- Applies the values to all selected widgets.
- **sub-tree** -- Applies the values to the current edit widget and all of its descendants.

You can undo the last resource change by clicking on the *Undo* button at the bottom of the Resources box. Doing so replaces the change with the previous values. Each time you click on *Undo* you will undo an earlier change until no more can be undone. If there is nothing more to undo, this button is not sensitive.

5.2.7 Selecting Sort Order and Display Options

To specify sort order and display options, open the window shown below by clicking on the *Sort* button at the bottom of the Resources window.

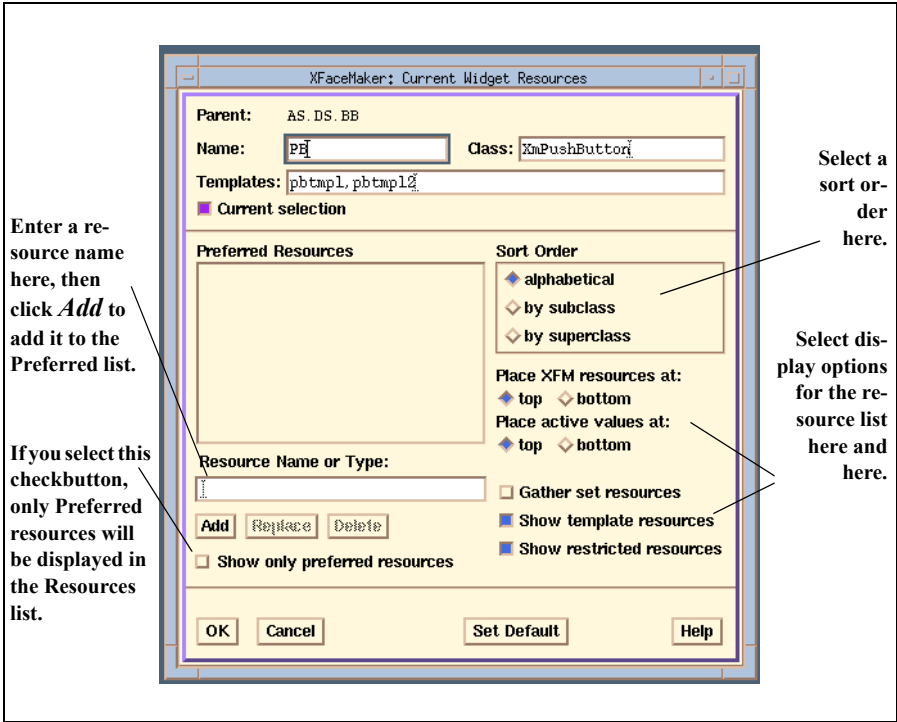


Figure 5.8: The Sort Order Options

To change the sort order and display options of the Resources List, specify your choices, then select one of the following:

- Select **OK** to apply the choices and re-open the Resource List.
- Select **Cancel** to return to the List without applying a choice.

The sort order options are:

alphabetical -- Present resources in alphabetical order.

by subclass -- Group resources by subclass.

by superclass -- Group resources by superclass

XFaceMaker gives you the following options for displaying resources:

Gather set resources -- Setting this option groups resources that have user-defined values, and displays them at the top of the Resources list---or after the “Preferred resources” if you have established that list.

Show template resources -- Setting this option displays any template resources that have been set.

Show restricted resources -- Setting this option displays any restricted resources. These are resources that cannot be changed in XFaceMaker, such as colormap, num-Children, screen, or template resources.

It may be useful to establish a list of the resources you use most frequently, and then display the list at the top of the Resource List.

To create a list of preferred resources:

1. Click on the **Sort** button at the bottom of the Widget Resources window. This opens a window displaying Sort Order options.
2. Enter a Resource Name or Type in the text input field.
3. Click the **Add** button. The resource or resource type is listed in the Preferred Resources box above the text field.
4. If you wish to add more resources or types, enter their names in the text field, and click the **Add** button. Or, you can drag selected resources from one resource window to the preferred list in another.
5. Click on **OK** when you are finished adding resources. This displays the Preferred Resources at the top of the Widget Resources list.

To delete a resource from the Preferred Resources list, select its name in the Preferred Resources box and click on the **Delete** button.

To replace a resource in the Preferred list with another, select its name in the Preferred Resources list, enter the name of the replacement in the “Resource name or type” text field, then click on the **Replace** button.

Once you have organized your Resource options, you can make them the default case for any new Resource windows you open. Select the options in the sort order editing window, then click on the **Set Default** button.

Note: Sort options are saved when you select the **Save Prefs** command in the File menu.

5.2.8 The Resource Editor

The Resource Editor lists the edits carried out so far. The box only lists the resources that are explicitly specified, which makes it easier to see the changes you have made. You can edit resources directly in this box, and apply them to a widget. Note that callback resources you have defined use FACE language syntax, while other resources do not.

Resources can consist of multiple lines. A multi-line resource is continued by ending a line with a \ (backslash) or beginning the next line with a Tab, or both. The last line is one not ending with a \ and not followed by a Tab. These characters alert XFaceMaker not to treat each line as a separate resource. A \ followed by n as in \n is treated as a character constant.

To open the Resource Editor you can do either of the following:

- Click the *Edit* button at the bottom of the Widget Resources window.
- Control-click Button 2 on a widget, or on its name in the Widget List or the Widget Tree.

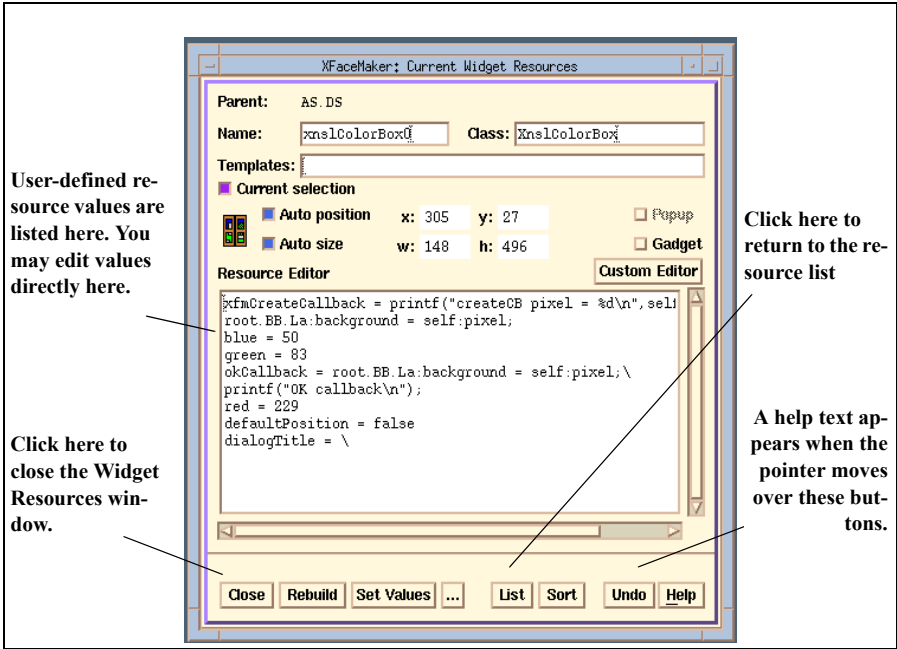


Figure 5.9: The Resource Editor

You can use the Resource Editor to remove resource settings for resources that do not appear in the resources list, e.g. when you change a widget's class and you had edited resources that were in the original class but are not in the new class. You will need to remove resources using the Resource Editor also when you change the class of a widget's parent and need to remove constraint resources that you had specified, e.g. in a Form.

5.2.9 The Resource Dialog Boxes

When you select a resource from the list, you can either enter a value directly in the text input field, or click on the ellipsis button to open a specialized edit box for that type of resource. Some edit boxes are fairly simple, while others allow you to edit more complex resources, such as fonts, or translations. All boxes are described in the Reference Manual.

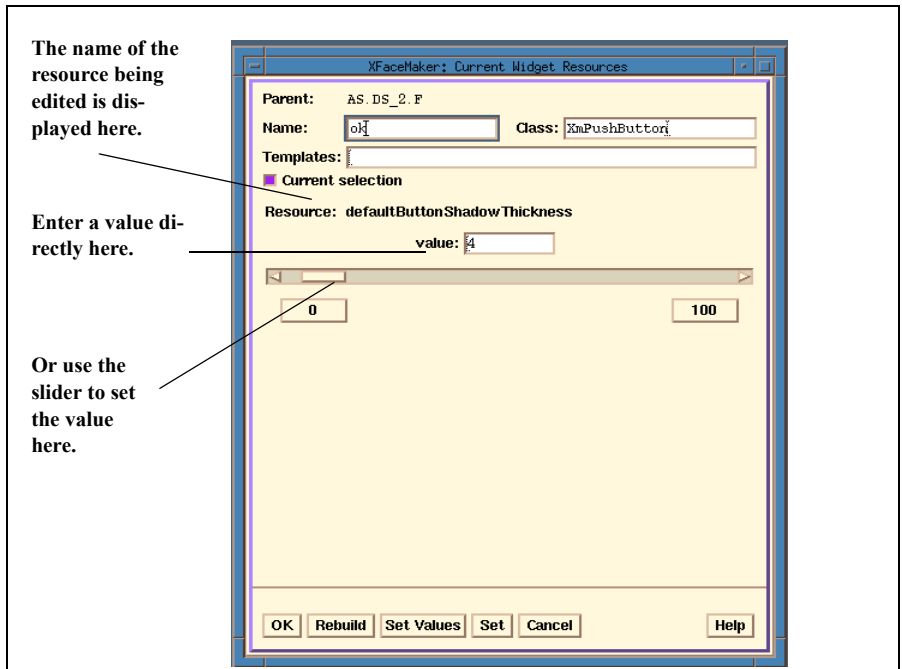


Figure 5.10: The Integer Edit Box

Edit boxes differ but they all have the following pushbuttons:

OK Applies the new value to the widget, and returns to the List.

Rebuild Applies the new value by rebuilding (destroying the widget and creating a

new one), and remains in the editor.

Set Values Sets the new value, and remains in this edit window.

Set Updates the value in the Resource List without applying it, and returns to the Resource List.

Cancel Re-opens the Resource List without changing any values.

The simple resource edit boxes are described below. The more complex windows for editing fonts, pixmaps, translations, and constraints are described later in this chapter.

Boolean For boolean values. To set the value, click on “True” or “False”.

Enumeration Displays a list of resource options, with the current setting highlighted. Clicking on an item makes it the current value. An item may be blank where the enumeration items are noncontiguous. Selecting a blank item has no effect.

Float For resources that require a single-precision floating-point value. You can enter the value directly or use the scale. The *maximum* and *minimum* values may be changed by clicking on the *stepper* near each side of the scale. Clicking in the right arrow button of a stepper increases the value by an order of magnitude, and clicking in the left arrow button decreases it. None of the standard OSF/Motif widgets have floating-point resources, so the Float box is used only for user-supplied or NSL widgets.

Integer For resources that require an integer value. The value may be entered directly or set by using the scrollbar. You can set the *maximum* and *minimum* values by clicking on the buttons beneath the scrollbar. Clicking with the *Select* button increases the value by an order of magnitude and clicking with the *Custom* button decreases it.

String Used to enter text for a resource. Clicking on the **GLS** button marks the string as an internationalized string, and gives it the default set number. When saving a message file, XFaceMaker automatically assigns message numbers to messages that do not have one.

Widget Used to specify the reference of a widget used as a resource value. Enter the name directly in the box, or fetch it by clicking the **Select** button on the right of the edit box. The question mark cursor appears. Move it over the widget or widget name you want, and click. The widget’s full name is written in the text input field.

A widget must exist if it is referred to as a widget resource. XFaceMaker provides a special callback--`xfmCreateCallback`--which is called *after* a wid-

get and all its children are created. The create callback is a good place to set widget resources.

5.3 Editing Global Objects

XmDisplay and XmScreen Objects

The Motif 1.2 library defines *global* widgets which are used to store resources on a per-display or per-screen basis. These objects can be used with the same programming interface as widgets: they have resources which can be read with `XtGetValues` and modified with `XtSetValues`. Global objects are not created by the application: they are defined internally by the Motif library, and there is only one instance of each. Global objects are accessed using the *Global* button in the Toolkit.

The difference with regular class icons is that, when the user clicks on a global object's icon, the corresponding global object is selected directly and its resources appear in the Resource Editor's list and can be edited normally. When an interface is saved, if a global object has been modified, its resources are saved in the interface file as for a regular widget. When the interface is loaded, the global object's resources will be set according to the values saved. If an application loads several files with global object definitions, the values stored in the last file loaded override the previous ones.

Global objects can also be modified dynamically in FACE scripts. They must first be fetched using the functions `FmGetXmDisplay` and `FmGetXmScreen` and then used like normal widgets to read or change their resources, for example:

```
display = FmGetXmDisplay(self);
display:dragInitiatorProtocolStyle = XmDRAG_DYNAMIC;
screen = FmGetXmScreen(self);
screen:font = "9x15";
```

If a script is specified for the `xfmCreateCallback` resource, it is executed when the interface is loaded.

5.4 Editing Color Resources

This section describes how to edit color resources, whether you work on a monochrome or a color display. It also describes XFaceMaker's color description file, `xfm.color`.

To edit a color resource, you can do either of the following:

- Enter the color value directly in the input field.
- Open the special dialog box and specify the color value.

5.4.1 Using the Color Editor

Open the dialog box by clicking on the ellipsis next to a resource that requires a color or pixel value. The box is shown below as it would appear on a monochrome display.

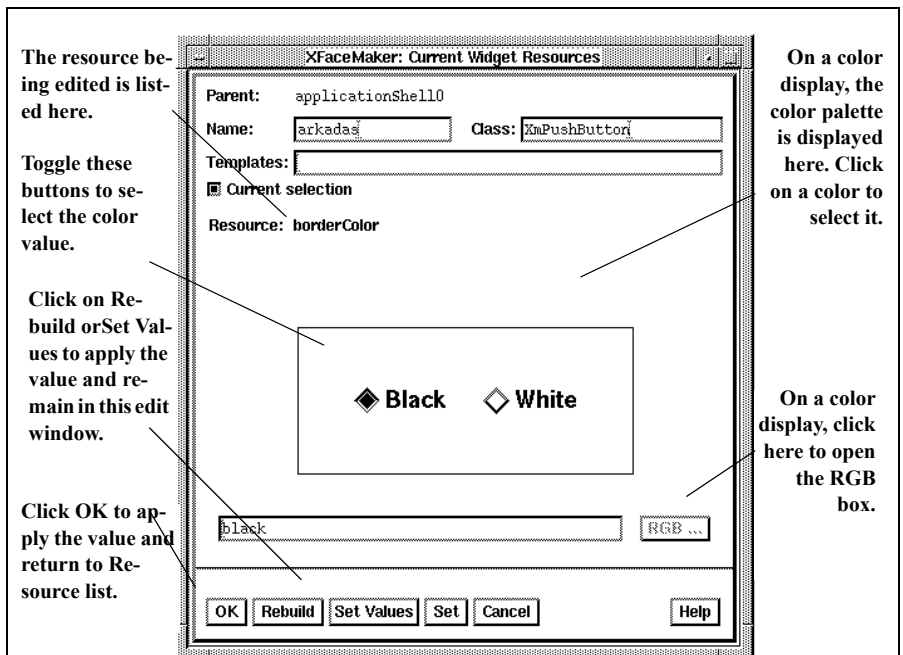


Figure 5.11: The Color Editor (Monochrome Screens)

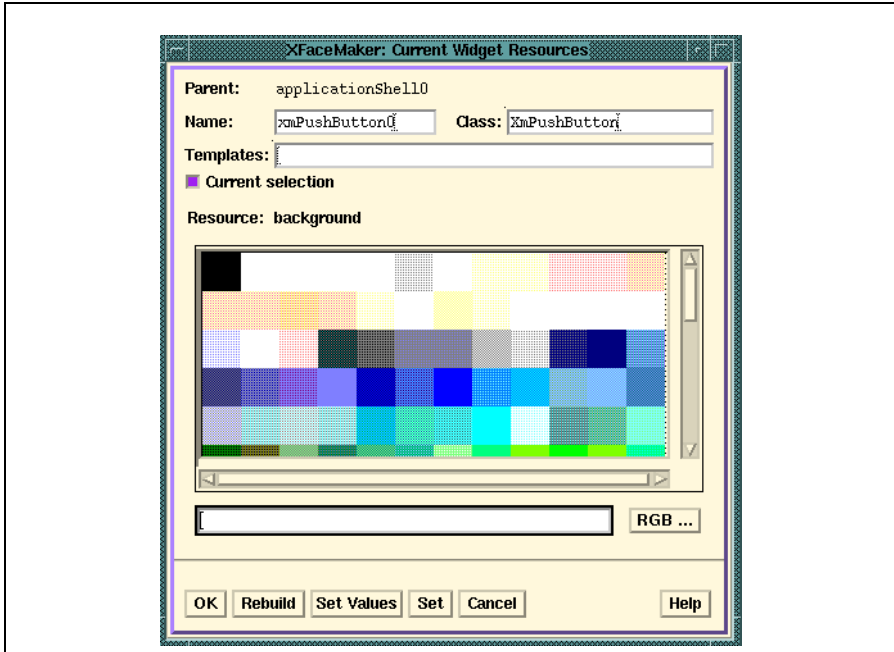


Figure 5.12: The Colors Box (Color Screens)

5.4.2 Using the RGB Box

On color displays, you can also select a color by its RGB (Red, Green, Blue) value or HSV (Hue, Saturation, Value) value. RGB and HSV definition are only available if you use XFaceMaker on a color display with a read/write color table, i.e., with PseudoColor or DirectColor visuals.

To set RGB and HSV values:

1. Pop up the RGB box by clicking on the **RGB...** button in the color editing box.
2. Use the sliders to set the values. The actual color is shown in a display below, and its hexadecimal representation is listed in the text field. The text field may also be edited directly.
3. Click on **OK**. If you have defined a new color, you will be offered the chance to save its definition in the *color description* file.

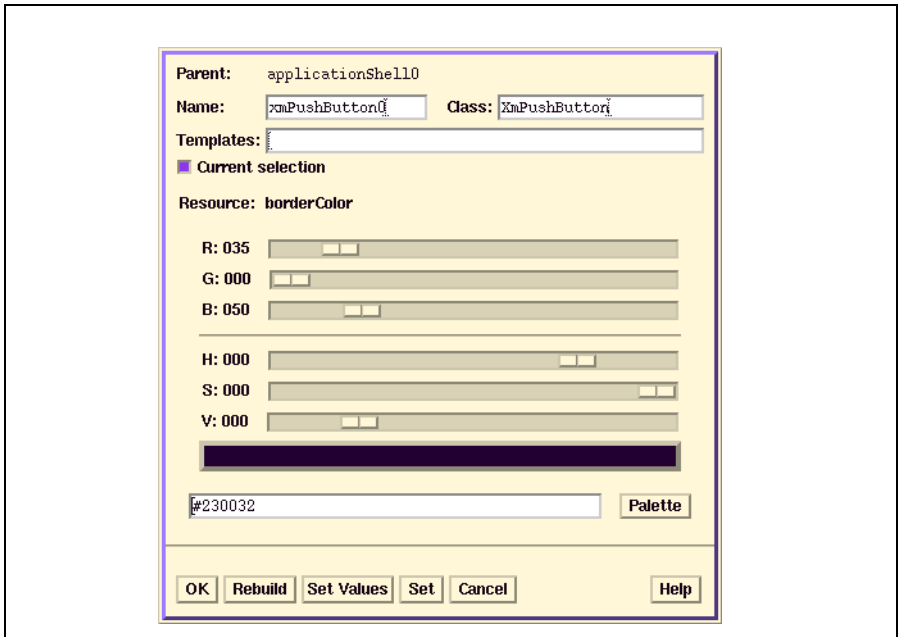


Figure 5.13: The RGB Box

5.4.3 The Color Description File

XFaceMaker uses a color description file to know which colors should be allocated in the color editing box. This file, `xfm.color`, is based on the standard X Window System file `rgb.txt`, with duplicate and redundant definitions removed.

The colors are *approximated* using dither patterns, so as not to consume too many color entries in the colormap. For example, with a colormap of 256 entries, approximately 200 colors are displayed using only 27 colormap entries. Note that the RGB editing box displays the *exact* color, not the approximated color. The standard entries can be removed or added to as required, in the system default file or in your own copy of the file.

For a particular display you may wish to use a special database file, such as those found in the X11R5 distribution `mit/rgb/others` directory. User-defined colors can be saved when quitting the RGB box as described above. By default this is in the

user's \$HOME directory as `.xfm.color`. There are various versions of the file for different colormaps. Therefore, XFaceMaker searches for the following file names until a file is found:

1. `.xfm.color.N` where N is the size of the colormap, e.g., 16, 256, etc.
2. `.xfm.color`
3. `xfm.color.N` where N is the size of the colormap,
4. `xfm.color`

Normally the dot files `.xfm.color.N` and `.xfm.color` are user-defined, and `xfm.color.N` and `xfm.color` are the system-wide default files. This directory search sequence is used to find the color description file.

1. The current directory,
2. The \$HOME directory,
3. The \$FMLIBDIR directory,
4. The directory `/usr/lib/X11/xfm`.

If the colormap has only sixteen entries they may be already used so that you cannot define extra entries. For instance, if `mwm` uses four colors in your `.Xdefaults`, or `system.mwmrc` file, sixteen colors will be allocated. `topShadowColor`, `bottomShadowColor`, and `highlightColor` will be calculated and allocated automatically from the base color.

5.5 Selecting Fonts

The dialog box for editing fonts can be opened from the Resources List, or from inside another editing box such as the Menu Editor.

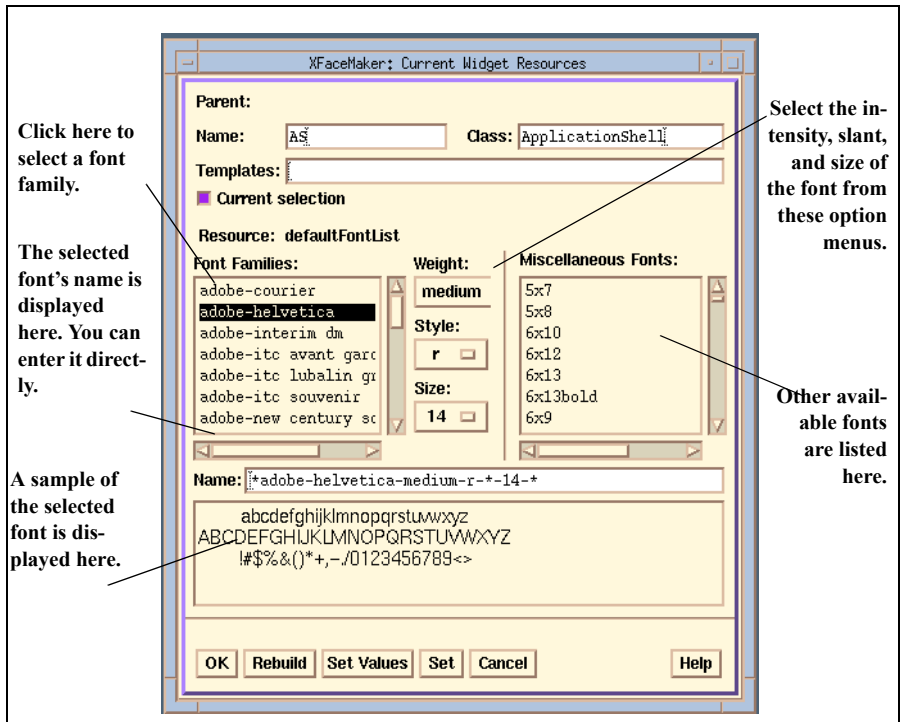


Figure 5.14: The Fonts Editor

To specify a font:

1. Select its name from a list or enter it directly in the Name field.
2. Choose options from the option menus: Weight, Style, Size.
3. Click on **OK**, **Rebuild**, or **Set Values**.

The **Set** button at the bottom of the window updates the value in the Resource list but does not apply it. The **Cancel** button returns to the list without changing the resource.

5.6 Creating and Editing Pixmaps

XFaceMaker provides a specialized box to help you edit pixmap resources. You can display it by clicking on the ellipsis button next to the pixmap resource you are editing.

When you edit a pixmap resource, you have three options. You can:

- load an existing pixmap file.
- select (and perhaps edit) a pre-defined pixmap.
- create a new pixmap in the editing box.

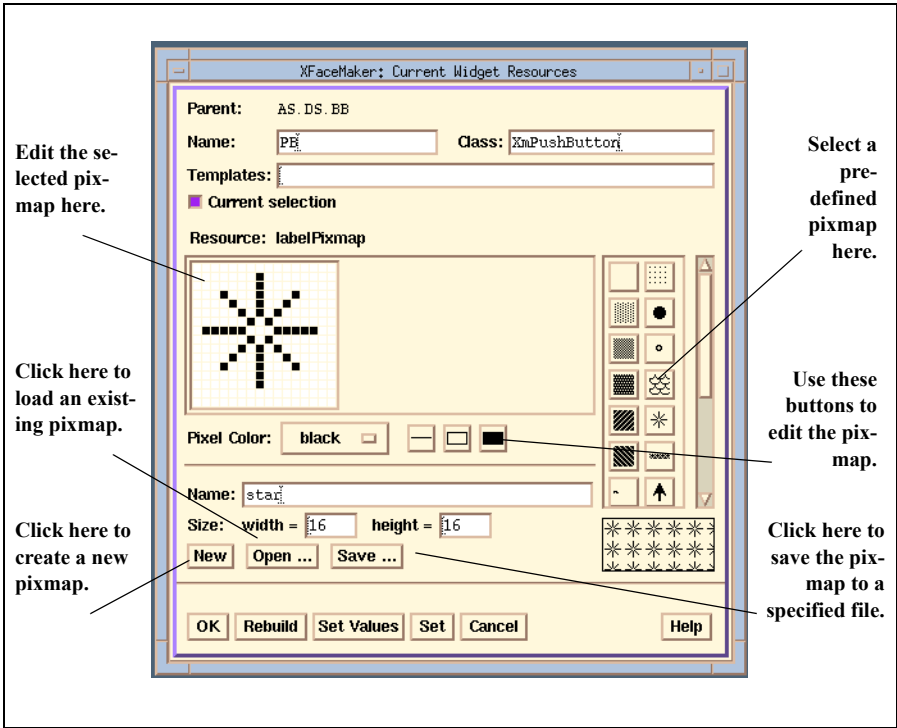


Figure 5.15: The Pixmap Editor

5.6.1 Using the Pixmap Editor

The pixmap editing box enables you to modify pixmaps on a pixel-by-pixel basis, draw lines and rectangles, and select pixel colors. You can thus create and edit simple colored or black and white pixmaps directly and save them in XWD or bitmap format. Open it by clicking on the ellipsis button next to a pixmap resource.

The edit window, on the left side of the box displays the currently selected pixmap. The label below indicates the bitmap or XWD filename and path. The default bitmap is **background**.

The pixmap selector on the right side of the box displays some of the most useful bitmap files in the standard X11 directory `/usr/include/X11/bitmaps`. Clicking on one of these makes it the current pixmap resource. A display field below shows you what the current pixmap looks like when tiled.

To load an existing pixmap file and apply it to the widget:

1. Open the pixmap editing box. (Click on the ellipsis next to a pixmap resource.)
2. Click on the **Open...** button. This pops up the File Selector.
3. Select a bitmap or XWD file from the list, and click on **OK**. This loads the file and displays it as the current pixmap in the edit window. The pixmap may now be edited or kept as is.
4. Click on the **OK** button. This registers any changes made to the current pixmap, applies them to the current resource, and re-opens the Widget Resources window.

To select a pre-defined pixmap from the pixmap editing box:

1. Open the pixmap editing box.
2. Click on one of the pixmaps displayed in the selection field of the Images box. It will be loaded and displayed as the current pixmap in the Images box. The pixmap may be used directly or edited first.
3. Click on the **OK** button. This registers any changes made to the current pixmap and applies them to the current resource, then re-opens the Widget Resources window.

To create a new pixmap using the editing window:

1. Enter the width and height of the desired pixmap, in pixels. The default is the size of the previous pixmap or 16 x 16.
2. Click on **New...** This clears the pixmap display area
3. Create a new pixmap using the mouse and the editing buttons.
4. Click on the **Save...** button. This pops up the File Selector box.
5. Enter a name for the pixmap file. If the pixmap currently loaded contains only white and black pixels, then a bitmap file is saved. If the pixmap contains other colors, then an XWD file is saved.
6. Click on the **OK** button in the File Selector. This registers the current pixmap in a file and closes the File Selector.
7. Click on **OK** in the Pixmap editor. This applies the pixmap to the current resource and returns to the Resource editor.

The pixmap editor has the following editing features:

Pixel Color This option menu allows you to select the pixel color. You can select an existing color from the menu, or add one (on color screens only) by selecting **New** which pops up the Colors box.

Line Clicking on this button changes the cursor to a cross, which you then use to draw a line. Define the end-points of the line by clicking twice in the edit window.

Rectangle Clicking on this button changes the cursor to a cross, which you then use to draw a rectangle. Define the opposite corners of the rectangle by clicking twice in the edit window.

Filled Rect. Clicking on this button changes the cursor to a cross, which you then use to draw a filled rectangle. Define the opposite corners of the filled rectangle by clicking twice in the edit window.

5.6.2 Pixmap Tables

Some of the widgets have resources that imply several pixmaps be defined for the same resource. To specify such resources, you can enter a comma separated list of pixmap names, or you can open the pixmap table resource editor shown below.

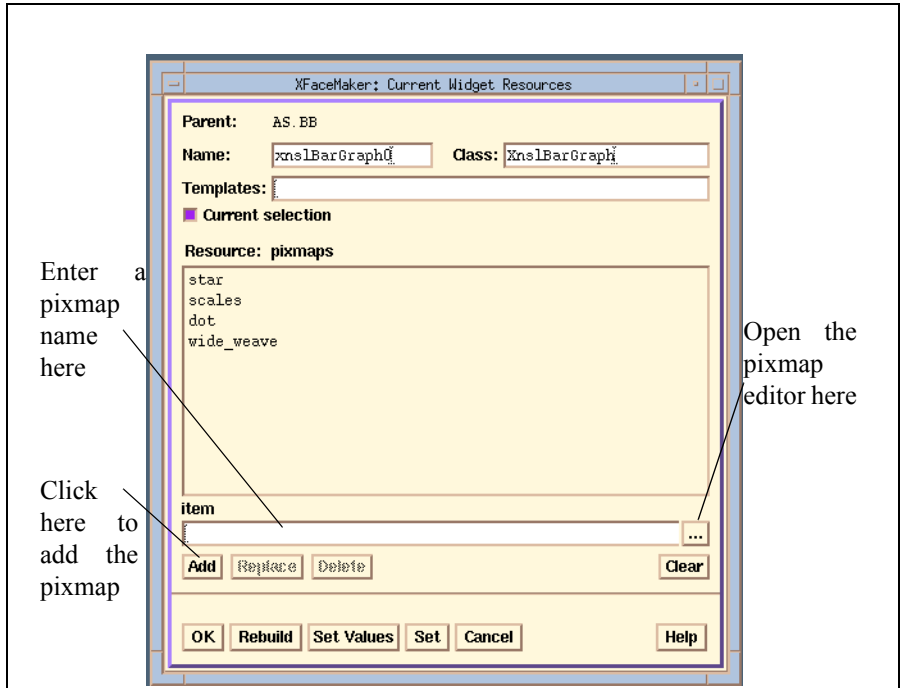


Figure 5.16: The Pixmap Table Editor

The Replace and Delete buttons are sensitive if you have selected a pixmap in the table. You can then either Replace or Delete it, or you can Add a pixmap in the table just after the one that is selected.

Once you have finished specifying all the pixmaps, click *OK*, *Rebuild*, *SetValues* or *Set*, to apply your resource setting, or *Cancel* if you want to close the editor without applying your changes.

5.6.3 Pixmaps and Bitmaps in XFaceMaker

Pixmaps describe rectangular areas of pixels, or *rasters*. They can have multiple *planes*, up to a number of planes equal to the *depth* of the display. Each plane is used to construct a color value which helps define the final color on the display. A monochrome display has a depth of one; therefore, all the standard pixmaps used by X are one plane deep, for compatibility across all display types.

A bitmap is a pixmap with one plane; that is, the pixel values are limited to one bit, set to either one or zero. Bitmap files are text files for loading into bitmap-type Pixmaps. The standard X bitmaps are found in the `/usr/include/X11/bitmaps` directory. By convention, bitmaps created by users are postfixed with `.bm` to distinguish them from other files. Bitmaps can be created and edited in the Images Box, or with stand-alone editors such as `bitmap` which offer more specialized facilities.

XFaceMaker uses a special resource converter for pixmaps that can also load files in the `XWD` format. This format is defined by the X Window System and used by the `xwd` and `xwud` commands. This means that you can assign a full-color image to any pixmap resource. The converter first checks if the specified file is a bitmap file, and if so loads it. Otherwise, it tries to load it as an `XWD` file.

To locate `XWD` files, XFaceMaker uses the environment variable `FMPIXMAPSPATH` with `%P` as the substitution character for `XWD` files. If `FMPIXMAPSPATH` is not set, `XBMLANGPATH` is used instead with the `%B` substitution character. By convention, bitmap files have the suffix `.bm` and `XWD` files `.xwd`. Programs exist in the X Contributed Software that convert `XWD` files to and from all other color image file formats (GIF, TIF, etc.). One such example is the `pbmp` package. There is also a pixmap equivalent of `bitmap`, called `pixmap`.

5.7 Translations

We suggest that, wherever possible, you should use callbacks and not translations as they are the standard way of specifying widget behavior. However, when you do edit the `translations` or `accelerators` resources of a widget, you can use the Translations editor to select and create elements for the widget's translation table.

When you use the Translations editor, XFaceMaker automatically inserts the required syntax elements, and makes sure that they occur in the right order. For more information on writing scripts, refer to Chapter 6.

Neither the concepts involved in translations, nor the syntax of translation tables will be discussed fully here. However, at the simplest level, *events* such as clicking a particular mouse button, or pressing a key, are linked to *actions* through translations. These are functions predefined for each widget. For example one of the translations for the `XmPushButton` widget is the following:

```
<Btn1Down> : Arm()
```

which specifies that pressing the Select mouse button should invoke the action `Arm()`, which changes the pushbutton background to `armColor` and calls the `arm-Callback`.

The Translations editor can also be used to define accelerators, which are a particular case of widget event. An accelerator is a key combination that invokes the action of a component without requiring that the mouse pointer be within the component when the key combination is typed. Giving a keyboard command is often quicker than using a pointing device, and so keyboard commands are said to *accelerate* the operation.

For a full description of translations, refer to your X documentation.

5.7.1 Defining a Translation

XFaceMaker provides a convenient editing window which you can use to define a translation resource. Open the Translations editor by clicking on the ellipsis button next to a translations resource.

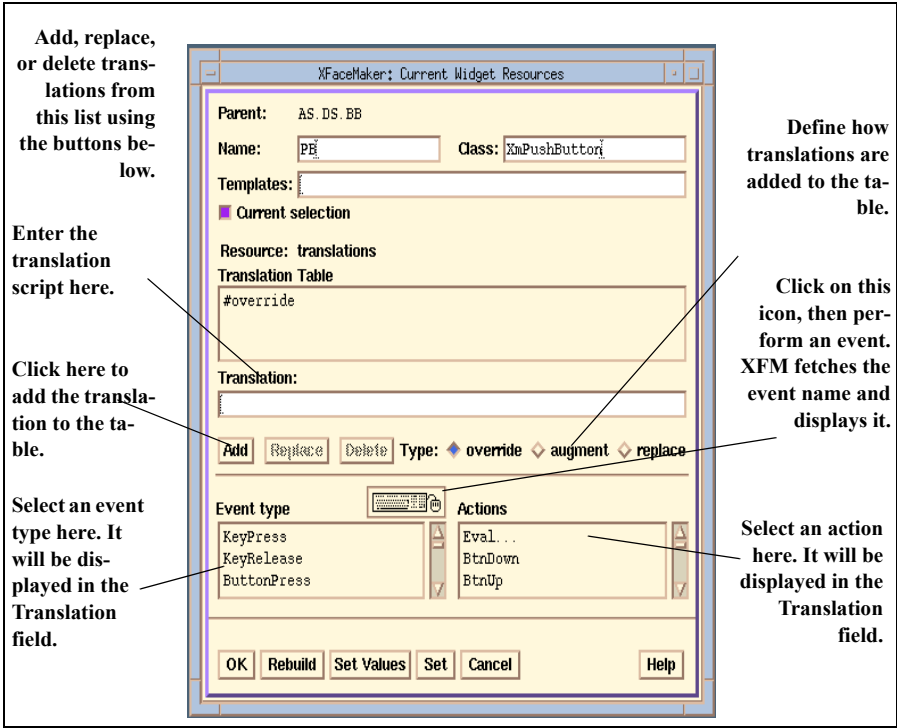


Figure 5.17: The Translations Editor

To define a translation:

1. Enter the translation script directly in the Translation field, or select an Event type and/or Action from the scrolled lists.
2. Choose a mode from the Type field. (See below.)
3. Click on the **Add** button. This adds the translation to the table.
4. Click on the **OK** button. This applies the value to the translation resource in the Resource list. Or click on **Rebuild** or **Set Values** to apply the change and remain in the Translations editor.

To define how new translations are added to the table, choose one of the following from the Type field:

augment merges new translations into the existing (default) translation table, ignoring new translations that conflict with existing ones.

override (the default mode), merges new translations into the existing (default) translation table, overriding translations which conflict with the new ones.

replace replaces the existing translations table with the new translations specified.

Note: You should check with the local widget documentation that the above modes are valid. As an example, for the `XmPushButton`, the OSF/Motif 1.1 Reference Guide states that altering translations in `#override` or `#augment` mode is undefined, i.e., they are ignored, but this is not true for all implementations.

If you do not know the official name of an event, you can fetch it by doing the following.

1. Place the pointer over the mouse-and-keyboard icon. (Or use the Tab key to traverse to the icon in explicit mode.)
2. Simulate the event you are interested in; for instance, click a mouse button or press a key. The name of this event will be inserted into the Translation text field. However, if a key is defined in the `XKeysymDB` file, you may prefer to use the virtual keysym, e.g., `osfUp` for the up arrow key name as in:

```
<KeyPress> osfUp: PrimitiveTraversePrev()
```

Note: The button event names `<ButtonPress>Button1` and `<ButtonRelease>Button1` do *not* work (X11R5). The correct name for these events is generated by `XFaceMaker` if you use the mouse-and-keyboard icon:

For `<ButtonPress>Button1` `XFaceMaker` generates:

```
<Btn1Down>
```

And, for `<ButtonRelease>Button1`:

```
Button1<Btn1Up>
```

5.7.2 Using Eval in Translations

The `Eval` action is a special case because it invokes a user-defined script instead of a built-in function. In this context, `$` is the address of the X event structure. Selecting the `Eval` action pops up a dialog box in which you can enter a standard FACE script.

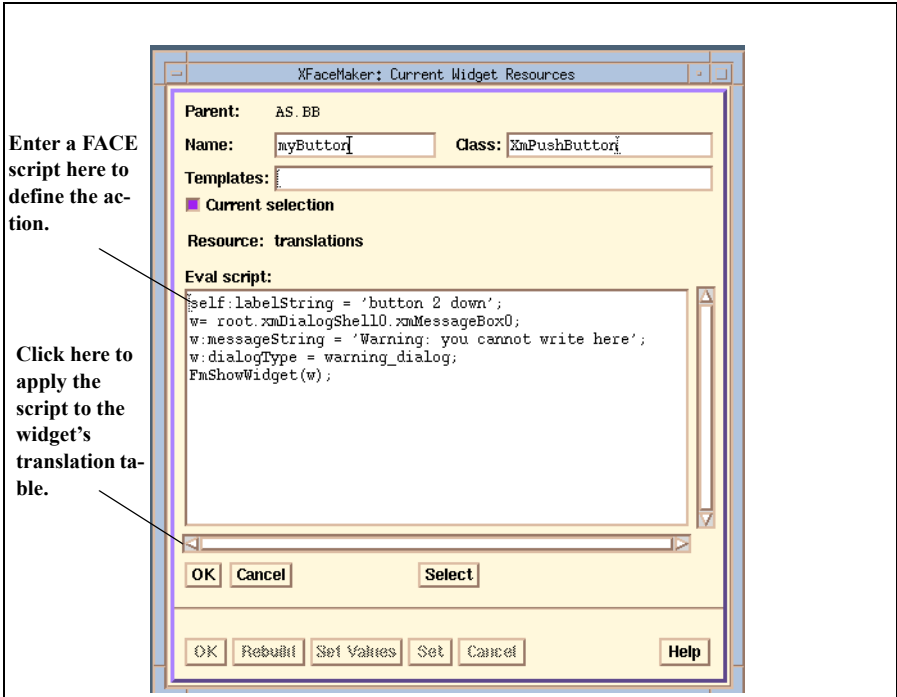


Figure 5.18: The Eval Script Editor

The script can call all the usual FACE functions and any application functions needed. In particular, it can call the function `XtCallCallbacks` and thus associate a callback to a particular key combination in addition to the standard one.

Click the “Select” button to fetch a widget name. When the cursor changes, move it over a widget in the interface, Tree, or List and click.

Once you have entered a script in the “Eval script” box, click on one of the following buttons:

Done This registers the script and applies it to the widget.

Ignore Clicking here returns to the translations editing window without applying a script.

Select The cursor changes to a “?”. Move it over an interface object, or its name in the Widget Tree or Widget List and click. This fetches the widget name.

In the example below, typing a “?” in a Text widget causes the `armCallback` of a sibling `PushButton` to be executed.

```
Shift<KeyPress>slash:  
eval("XtCallCallbacks(parent.PushButton, 'armCallback', 0);")
```

Because this script is parsed, memory is allocated for its storage. The `script` keyword, described in Section 6.5, can be used to name the script so that it does not need to be reparsed, and hence allocate memory, each time it is executed. Thus:

```
Shift<KeyPress>slash:  
eval("script bb:callback(parent.PushButton, \  
armCallback, 0);")
```

5.8 Attachments

Widgets within a Form can be *attached* to other widgets or to particular x and y positions. When attachment resources are set and the interface is resized, the widgets within it will either be resized, or keep their position relative to the Form composite widget. XFaceMaker provides a special editing window for specifying a widget's attachments. Of course, these constraint resources can also be set directly in the Widget Resources box.

Widgets placed within a Form composite widget receive a set of attachment resources. The following resources define the attachment for each side of a widget:

```
topAttachment    bottomAttachment
leftAttachment   rightAttachment
```

The enumerations available are:

- `attach_none` do not attach current side.
- `attach_form` attach current side to the corresponding side of the Form widget.
- `attach_opposite_form` attach current side to the opposite side of the Form widget.
- `attach_widget` attach current side to the side of another widget as specified in the *sideWidget* resource. e.g., `leftWidget = parent.PushButton0`
- `attach_opposite_widget` attach current side to the opposite side of another widget as specified in the *sideWidget* resource.
- `attach_position` attach current side to a relative position within the Form widget, specifying the relative position in the *sidePosition* resource as a percentage of the corresponding Form's size. e.g., `leftPosition = 50` attaches the left side to a point which is 50 percent of the Form widget's size.
- `attach_self` attach current side of the widget to its original position.

See your OSF/Motif documentation for details.

5.8.1 Using the Attachments Editor

To open the Attachments Editor, click on the ellipsis button next to an attachment resource in the Resource list, or activate the ***Attachments ...*** button in the ***Edit*** Menu.

There are two ways to set attachment resources for the widget you are editing. Both methods are explained in greater detail on the following pages. The methods are:

- Use the attachment arrows. You can simply click on the arrow which corresponds to the side of the widget you'd like to attach, move the cursor over the interface, and click.
- Set values directly. Instead of using the arrows, you can enter the values directly in the fields provided.

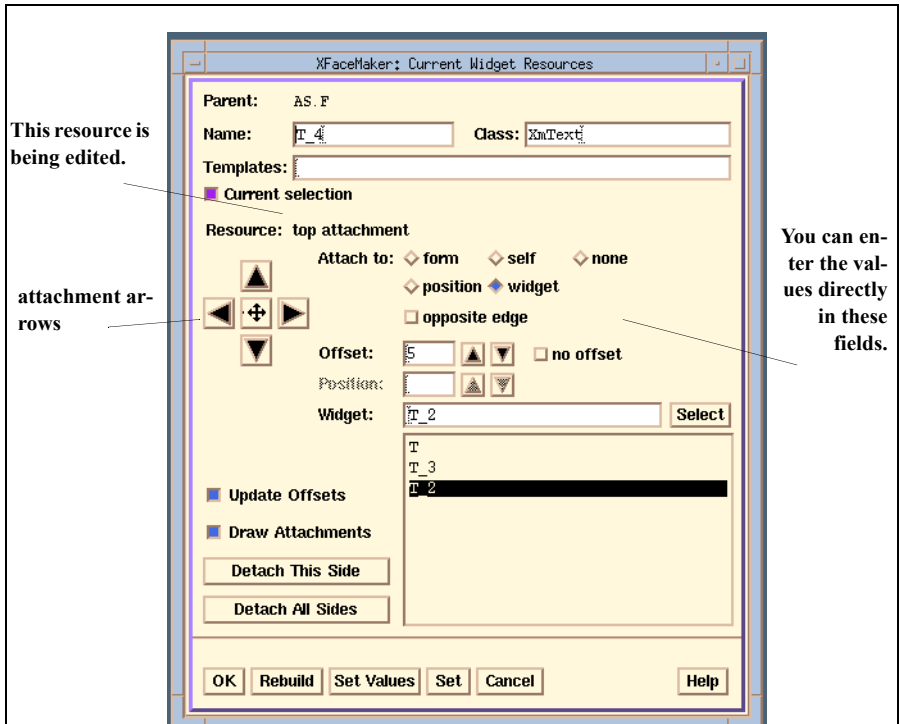


Figure 5.19: The Attachments Editor

Before you set attachments using either method, you should first make the following choices in the Attachment Editor:

- “Update Offsets” Set this checkbox if you want the object’s offsets to be updated when you move the object to a new position. If you leave this button unset, the offsets you specified for the object will remain fixed, even if you attempt to move the object. This can also be set by selecting **Options/Update Offsets on Move** in the menu.
- “no offset” If you set this checkbox, the attachment is done without offsets; that is, the object will be attached flush to the Form, or to the sibling you specify. If this checkbox is *unset*, the offset will be the value displayed in the “Offset” field of the editor. You can change this by entering a new value directly, or by dragging the object on the interface (if “Update Offsets” is set).
- To help you visualize attachments, you can display them by setting the **Draw Attachments** item in the View menu, or by selecting the “Draw Attachments” checkbox in the editing box. The vectors originate from the selected widget edge and point to the reference widget’s edge. Attachment vectors for “position” are displayed as broken lines while attachment vectors to another widget are drawn with solid lines.

To set attachments using the arrow buttons:

1. Click on the arrow that corresponds to the attachment resource you want to set for the selected widget. For example, click on the “up” arrow for `topAttachment`, the “left” arrow for `leftAttachment`, etc.
2. Move the cursor over the interface and click.
 - m If you click in another object, the widget is attached to that object.
 - m If you click in the Form widget, but not in another object, the widget is attached to the Form.
 - m If you click within the widget itself, the widget’s attachment is to “self”.

If “position” is selected, when you click on an arrow the selected widget is attached to the Form---the “?” cursor does not appear. Note that setting an offset and a position is valid. For instance, if you want a PushButton in the middle of a Form you might give a position of 50 percent with an offset of minus half the button width.

You can also set the attachment resource values directly. To specify which attachment resource you edit, click on the cross in the middle of the attachment arrows, then click on the arrow corresponding to the attachment resource; i.e. the “up” arrow for `topAttachment`, etc.

1. Specify what the selected widget is “attached to”.

The selected widget can be attached to a *reference widget*: the Form, a sibling, or “self”. If you want to attach the widget to a sibling, select “Attach to widget” and specify its name in the “Widget” text field.

Or, the selected widget can be attached to a position. In this case, the selected widget’s attachment is always to the Form parent. In resizing operations, a widget thus attached keeps its relative position with respect to the Form border.

“none” specifies that the selected widget is not attached on that side.

“opposite side” specifies that the widget’s side is attached to the opposite side of the widget your are attaching to. See your Motif documentation for details.

2. Enter the specific widget or position the selected widget is attached to. If you specified “widget” in the “Attach to” field, then you can:

Enter a widget name directly in the “Widget” input field.

Or, click on a widget name in the list below the field. The name is displayed in the “Widget” field.

Or, click on the “Select” pushbutton. When the cursor changes to a “?”, move it over a widget or widget name and click. The name of that widget will be displayed in the field.

3. Accept the current offset or enter a new value. It specifies the distance in pixels between the side being attached and the position or side to which it is being attached.
4. Click **OK**, **Rebuild**, or **Set Values**.

You also have the following options:

Detach This Side When clicked, this button removes all attachment resources for the corresponding side of the widget currently being edited.

Detach All Sides When clicked, this button removes all attachment resources for all sides of the current widget.

Note 1: If you want to modify attachments, it is safer to first **detach** the widget (or the side whose attachment you want to change), then specify a new attachment. This way, you are sure you will not end up with conflicting attachment resource

settings. This is because an attachment specification usually involves the setting of several resources.

Note 2: When attaching to *widget*, beware of the order in which the widgets are created. The widget you attach *to* must be created *before* the widget you are attaching. XFaceMaker will switch the creation order of widgets for you, on a one to one basis. But if you have a cascade of widgets that are attached sequentially, you should first decide what the creation order should be and re-order the widgets before you start specifying the attachments.

5.9 Setting Default Resources for Your Interface

When you create an application shell using XFaceMaker as it was delivered, the application class of your shell is “**XfmApplication**”. If your interface (or module) does not contain an application shell, XFaceMaker creates an implicit application shell which also has the class “XfmApplication”.

This means that you can define -- in your `.Xdefault` or in a resource database file loaded with `xrdb` -- default resources that will apply to the widgets you will edit using XFaceMaker. To do this, specify the class

`XfmApplication` at the beginning of your resource names, for example:

```
XfmApplication*fontList: *helvetica*-12-*
```

If you are using a “home-made” XFaceMaker, the application class used to create user-defined application shells is the class you pass to `FmInitialize`. In that case, you must use the same class name in your `.Xdefaults` file to specify default resources. For example, if your main contains:

```
FmInitialize(“myxfm”, “MyXfm”, 0, 0, &argc, argv);
```

then you must specify resources such as :

```
MyXfm*fontList: *helvetica*-12-*
```

Note, however, that if you pass the class `Xfm` to `FmInitialize`, it will be replaced by `XfmApplication`.

CHAPTER 6: Editing and Debugging Scripts

This chapter describes how to write and edit scripts. It includes:

- general rules for writing FACE scripts
- using a custom text editor
- writing callback scripts
- writing creation scripts
- writing active value scripts
- writing named scripts
- de-bugging scripts
- error recovery

6.1 Writing FACE Scripts

XFaceMaker scripts are written in FACE, a simple, X-toolkit-oriented language similar to C. A brief description follows. See the *XFaceMaker Reference Manual* for complete details on the FACE language.

FACE supports the following statements as in C:

assignment statement

function call

if ... else

while

for

FACE supports the following operators:

Assignment operator =

Unary operator -

Arithmetic operators + - * /

Logical operators && || !

Relational operators == != <= >= < >

All arithmetic operations work on integer or floating-point operands. If any of the operands is a floating-point value, then the operator performs a floating-point operation, but the other operands are not converted automatically to a floating-point type (you must use the function `citof`). Expressions can be parenthesized and operator precedence is as in C.

Comments are, as in C, enclosed between `/*` and `*/`.

You can define hash arrays, linear arrays and structures in FACE scripts

Some important differences between FACE and C:

- Assignment statements may not be used as expressions.
- Values are not automatically converted from int to floating point.
- Integer constants must be decimal.
- There are no character constants.
- Both single quotes (') and double quotes (") delimit strings.
- In strings, a backslash (\) may escape only the quote character or another back-

lash. Furthermore, strings may contain newlines.

- An unknown identifier is treated as a string.
- FACE does not implement pointer arithmetic, except through arrays.
- All FACE variables have the same size, the size of XtArgVal.

FACE automatically creates a variable when assigning it a value. The variable has the same type as the expression to which it is assigned. You may also use the **local** and **global** keywords to declare variables and assign them a specific type, for example:

```
local Boolean isbusy;  
global String ApplicationName;  
global Widget AlertBox;
```

Note: **local** may be omitted.

```
Boolean isbusy;
```

is the same as

```
local Boolean isbusy;
```

FACE recognizes most types in the Xt Intrinsics and OSF/Motif libraries. New types can be specified with the keyword type, for example:

```
type MyType
```

To refer to a widget, use one of the following special widget references:

self The widget containing the script

parent The widget's parent.

root The application shell at the top of the widget tree.

toplevel The shell that contains the widget and that is a direct descendent of the root widget.

You may descend the widget tree from any of these special names, or from a variable that contains a widget, by providing the names of descendents separated by either periods (.) or asterisks (*). Note that, when you refer to a widget in a script using its name, the presence of one of these special names is mandatory.

A period indicates an immediate descendent, and an asterisk indicates any descendent. You may also connect several **parent** references separated by periods to start from an ancestor other than the widget's immediate parent. Here are some examples:

```
root.alertBox
toplevel.form.rowColumn.toggleA
self.answer
parent.parent.questionForm
```

To refer to a widget's resource, follow its reference name by a colon (:), then the name of the resource. You may use the resource in expressions or in assignments, just as you would use any ordinary variable.

For example:

```
self:leftOffset = -(self:width / 2);
self:indicatorSize = self:indicatorSize - 2;
parent:spacing = self:height / 4;
toplevel:title = "Warning";
root:iconPixmap = 'AppIcon';
```

When assigning a string to a resource, you may need to convert the string to the appropriate type. This is true in particular of `XmString` resources.

The characters `@` and `$` introduce special variables in FACE. These variables are initialized before a script is launched. The value to which they are initialized differs according to the context of the script. For example, in a widget's callback script, `$` is initialized to the `call_data` (value of the pointer to the callback structure). See the *XFaceMaker Reference Manual* for details.

In FACE scripts, you can define FACE functions, using the keyword *function*, followed by the function synopsis, then the function body. FACE functions are global to an entire interface. A FACE script can call also call application functions; if these are not declared with the keyword *function*, a warning is generated, stating that an unknown function is being called. A number of functions have been registered with XFaceMaker for your convenience. See the *XFaceMaker Reference Manual* for a list of these functions.

In callbacks, you will need to access the callback structures. All the standard Motif callback structures are registered in XFaceMaker. Refer to *XFaceMaker Reference Manual* for the name under which the Motif structures are registered. In general, it is the same as the Motif name, but with the Struct part removed. For example, `XmListCallbackStruct` is registered in XFaceMaker as `XmListCallback`.

The FACE language implements data sharing between the application and the interface with *active values*. See Chapter 8, *Active Values* for details.

6.2 Using a Custom Text Editor

When you need to edit text in one of XFaceMaker's editing windows you can connect to an external text editor of your choice. To do so, set the **FMEDITOR** environment variable, and specify your choice of editor. Then, whenever you click on the **Custom Editor** pushbutton in one of the XFaceMaker editing windows, the editor you specified will be called.

The editor you specified, or 'vi' by default, contains the current contents of the corresponding XFaceMaker text area. The edited text is saved in a temporary file which is examined periodically. When the file is modified, the new text is sent back to XFaceMaker just as if it had been typed in the XFaceMaker interface. While a custom editor is running, the corresponding XFaceMaker text area is not editable.

NSL's multi-window text editor *Wx Version 2.4.2* fully supports the XFaceMaker custom editor protocol. To use it, set FMEDITOR to wx242. Wx provides full custom editor functionalities:

- the editor text is automatically updated when the XFaceMaker text area changes, e.g. if you select another widget,
- the XFaceMaker text area is updated immediately when the editor text is saved,
- the editor window is mapped and unmapped together with the XFaceMaker text area,
- The title of the Wx window indicates what is being edited, e.g. "resources..."

If the custom editor is 'vi' or another "alphanumeric" editor, it is run in an xterm window. In this case, the xterm window is iconified whenever the XFaceMaker text area is unmapped (for example if the 'List' button is pressed in the resource window). When the text area becomes visible again, the xterm window is de-iconified. In addition, when the XFaceMaker text area changes (for example when a new widget is selected), XFaceMaker updates the editor automatically by killing and restarting the editor with the new text--the xterm window is kept alive.

Other external "windowed" editors, such as xedit, can also be used as custom editors. However, in that case the contents of the editor are not updated automatically when the XFaceMaker text area changes: the file must be re-loaded manually from the editor. Also, the editor window is not mapped and unmapped together with the XFaceMaker text area.

6.3 Callback Scripts

In the X window system, widgets have *callback resources*---functions which are called by a widget in response to external events such as mouse clicks. Callback resources do not specify visible properties of a widget such as a `labelString` or the width of the border. Instead, they define the name of a function which is called by the widget itself under certain circumstances.

Consider a `PushButton` widget. It has the following callback resources:

- An `armCallback` resource. You can specify a function as the value of this resource, and it will be *called back* whenever the pushbutton has been selected; i.e., whenever the Select mouse button is pressed on it.
- A `disarmCallback` resource. This specifies the function that must be called back when the button has been released or the cursor has been moved outside the push button. This callback is normally used only when two different actions are to take place on arm and disarm. Otherwise, the `activateCallback` is the one used.
- An `activateCallback` resource. This specifies the function that must be called back when the button has been released. This is the callback you should use in pushbuttons, when you don't need to specify actions that are different on arm and disarm. It differs from the succession of arm - disarm in that it is not invoked when the button is released outside the perimeter of the pushbutton.
- A `destroyCallback` resource. This is invoked when the button is destroyed by the application. This is the only callback every widget has. The number of other callbacks and their naming depends on the specific widget.

Callback resources can be used to invoke one or several functions in the application program from the interface, as well as to change the state of interface elements using FACE statements.

To edit a callback resource:

1. Select a callback resource from the Resources list. Or select a callback resource from the ***Callbacks*** pulldown menu above the list. Selecting a callback in this menu opens a special edit box for editing multi-line callback scripts.
2. Enter a callback script in the text input field using the FACE language. (For multi-line scripts, click on the ellipsis button to open a larger editing window.)
3. Apply the script to the widget by selecting ***Rebuild*** or ***Set Values***. The script will be interpreted when the callback is called.

When the callback editing window is open, you can click on the ***Custom Editor*** button to open the editor you specified.

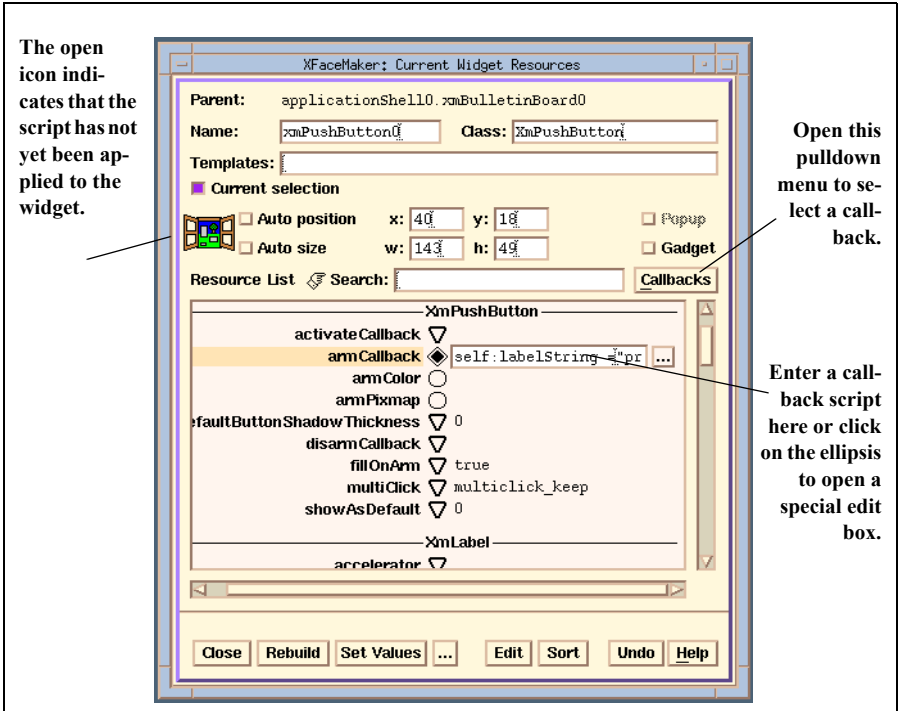


Figure 6.1: Editing a Callback Resource

Example

The following example illustrates how to edit callbacks for a PushButton widget so that it will change its own resources in response to a click on it.

1. Create an instance of a PushButton widget.
2. Select the PushButton with mouse Button 2 to open the Widget Resources box.
3. Click on `armCallback` in the list of resources or in the Callbacks pulldown menu.
4. Enter the following text into the text field.

```
self:labelString = "pressed";
```

This text is a short FACE language script which will be interpreted when the callback is called. `self` is a FACE name for the widget which called the callback—in this example, the PushButton

widget we have selected. After the `:` comes the name of a `PushButton` resource we want to change. Following the `=` sign comes the new data for the resource, consisting of the string `pressed`. The text is placed between two double quotes though `FACE` would have interpreted it as a string even without the quotes. Quotes are only mandatory when a string includes a blank. It is wise, however, to always place strings within quotes in `FACE` in order to avoid the conflicts that would occur if a variable had the same name as the contents of a string. In such cases, `FACE` would interpret a non-quoted string as the variable name. The semicolon indicates the end of the line.

5. Click on the ***Rebuild*** button.
6. Click on `disarmCallback` in the list of resources
7. Enter the text

```
self:labelString = "not pressed";
```
8. Click on the ***Rebuild*** button.

To test the callback, change to Try mode, place the pointer on the `PushButton`, and press the **Select** mouse button. The `PushButton` label will change to `pressed`. On releasing the **Select** button the label will change to `not pressed`.

Unlike resources that are to take effect immediately, callbacks are invoked in response to external events. In Try mode, `XFaceMaker` passes mouse events, etc., to the interface, instead of interpreting them as editing commands, as it does in Build mode.

Return to Build mode, select the `PushButton` widget, then click the ***Edit*** button in the Widget Resources box. This opens the Resource Editor window listing the callback functions you have defined for this widget, and any other user-defined resources you have set. Click on ***List*** to get back to the Resources List.

6.4 Creation Scripts

A creation script written in FACE can be attached to any widget. The script is called when a child widget is created, or for a parent, once all its children have been created.

Creation scripts are used:

- For widgets which need to reference their children in order to initialize their own resources
- To set those widget resources whose value can be specified only *after* the widget has been created.

To edit a creation script, select `xfmCreateCallback` from the Resource list. You can enter a script in the text field next to its name, or open the special edit box by clicking on the ellipsis button. `xfmCreateCallback` is not a genuine X resource, but XFaceMaker considers it as such to make it accessible for script editing.

As an example of a parent needing to reference a child to set its own resources, consider an `XmMainWindow` containing an `XmScrollBar SB`. The `XmMainWindow` is created first, then its child `SB`. The creation script for `SB` is called, then that for the `XmMainWindow`, so that the parent can access its child. This order is required because widgets and resources *exist* only after creation. The creation script for the `XmMainWindow` would be:

```
self.horizontalScrollBar = self.SB;
```

A creation script is executed any time the associated widget is created. Therefore the script will be called by XFaceMaker as you construct an interface as well as in the final application when that interface is used. A widget's `createCallback` is called after all the children of the widget have been created and managed.

One particular point should be noted when referring to `root` within a creation script. Consider the following application code:

```
Widget DefaultAppShell = FmInitialize("Example",  
                                     "example", NULL, 0, &argv, argc);  
Widget NewAppShell = FmLoad("Example.fm");
```

A default widget identifier `DefaultAppShell` is set by XFaceMaker in the call to `FmInitialize` which is then replaced by the *actual* `ApplicationShell` widget identifier if `Example.fm` contains an `ApplicationShell` widget. If `Example.fm` references `root` within a creation script, then it will refer to `DefaultAppShell` and not `NewAppShell` which is only set after the interface has been created and the creation script called. You should also be careful about using active values or application functions before they are *attached*.

Be particularly careful with the syntax when writing creation scripts: they are executed when the interface is being loaded in XFaceMaker. If a syntax error occurs in a creation callback, you may find yourself in the situation where you can't load the interface to edit the callback! You have to use a general purpose editor to edit the creation callback. You can load the interface with the debugger on (see "Debugging Scripts" on page 136) to help find the error.

6.5 FACE Functions and Named Scripts

There are instances where the same piece of code needs to be executed in different callbacks. With FACE, you can implement this in two ways:

- define a FACE function
- define a named script

Another reason to use named scripts is to avoid parsing a script each time it is encountered: the first time the FACE interpreter encounters a named script, it parses it and caches it, subsequently referring to it through an index. This is important if you plan to have your final application use *interpreted* mode to execute your interface scripts.

To define a FACE function, use the `function` keyword and specify the return type and the argument types, then write the text of the function:

```
function None foo (string s)
{
    t = CreateXmString (s);
    self : value = t;
    StringFree (t);
}
```

Such a function can be defined anywhere in a FACE script. The scope of a FACE function is the entire interface. If a function is defined twice, a warning is issued and the second definition overrides the first. FACE functions can be called from within any script, provided the script where it is defined has already been parsed. If this is not the case, the FACE function's prototype has to be declared:

```
function None foo(string s);
```

FACE functions can also be declared as "global":

```
global function None Hello(String message)
{
    printf("Hello, %s\n", message);
}
```


Global FACE functions differ from non-global functions only in the way they are output in the generated C code. Local FACE functions (i.e. FACE functions not declared with the “global” keyword) are output as static C functions, and are output in a C file only if they are actually called in the corresponding .fm file.

When a local function is called in several .fm files, the definition of the function will be replicated in all corresponding .c files. On the other hand, global FACE functions, are output only in the C file corresponding to the .fm file where they are defined, as extern C functions. When a global function is called from another .fm file, it must be declared like this:

```
global function None Hello(String message);
```

As for local functions, it is sufficient to declare a global FACE function once in a .fm file provided that the declaration appears before all the calls. When a global function is called from a .fm file but not defined in this file, XFaceMaker outputs an “extern” declaration for the function.

To define a named script, use the `script` keyword, followed by a colon as below:

```
script help:
```

Then enter the script body. To invoke a named script, use the `script` keyword, followed by a semi-colon as below:

```
script help;
```

The keyword `script` must always be followed by an identifying *name*, and has two uses:

- to name scripts,
- to share scripts.

The *name* must begin with an alphabetic character. The keyword `script` labels and executes a named script if one is defined immediately after a `:` (colon), or executes the named script if no colon announcing a script definition follows (i.e., it is followed by an end of statement `;` (semicolon)).

The first time FACE encounters a script, it caches the script, using its name as an index. Therefore two different scripts should never have the same name or the second one will never be executed. Also, the local variable of all instances of a named script have the same storage.

`Eval` scripts, should be named systematically even if they are only used in one place, to make use of this caching mechanism. A *named Eval* script is parsed only the first time it is encountered whereas *un-named Eval* scripts are parsed each time

they are executed, thus increasing memory allocation when the interface is executed in interpreted mode. The second usage is to share a script. For example, if two buttons in an interface are to have the same help facility, the first defines the script named `help`:

```
helpCallback =
  script help:
    root.HelpBox.dialogMessage = "help!";
    ret = PopupAndWait(root.HelpBox);
```

while the second simply calls it causing it to be retrieved from the cache:

```
helpCallback =
  script help;
```

(The above syntax is the syntax for defining callbacks.)

The effect will be the same when either callback is called; the values of `self` and `parent` are those of the widget which called the callback. Note however a script definition or call can only appear on its own within a FACE script. The following is *not* legal:

```
helpCallback =
  self:insensitive = true;           /* wrong */
  script help;
```

6.6 Active Value Scripts

Active values are used to share data between the application and the interface, or to define pseudo-resources for a widget instance. Active value *get* and *set* scripts are written in the FACE language. Two special variables refer to the data of the active value depending on whether the active value has been defined with immediate access. The variable `@` refers to the active value's contents, and the variable `$` refers to its address for active values that do not use immediate access. If the active value has immediate access, you should use `$` to refer to its content; `@` is undefined.

You can use active values as pseudo-resources of a widget instance. For example, assuming that you have defined an active value *my_res* in a widget, you can set or read its value, just like any resource:

```
my_res = 3;
```

will activate the set script, and

```
x = my_res;
```

will activate the get script. In these scripts you may call FACE and application functions.

Using active values, you can call FACE scripts directly from application code, as opposed to using callbacks, which are only called in response to external events. The XFaceMaker library includes functions to invoke the active value set and get scripts from the application.

In the Active Value box, you can view and edit multi-line scripts by clicking on the ellipsis button to open the special editing box. After editing the scripts, select **Re-build** or **Set Values** to register the changes. When an active value is defined for a widget, the *set* and *get* scripts are added to the list of the widget's callbacks in the Resource Editor box. These *set* and *get* scripts, called `set_name` and `get_name`, can be edited in a non-modal manner in this box (where *name* is the name of the active value).

For more information on active value scripts, and examples, refer to Chapter 8, and to the *XFaceMaker Reference Manual*.

6.7 Debugging Scripts

This section describes how to debug FACE scripts using the ‘FACE Debugger’ window. This window enables you to:

- trace the execution of FACE scripts.
- execute instructions step-by-step.
- print the stack of script and function calls.
- set named breakpoints.
- interrupt and abort execution of scripts.
- execute FACE statements to print variables, etc.

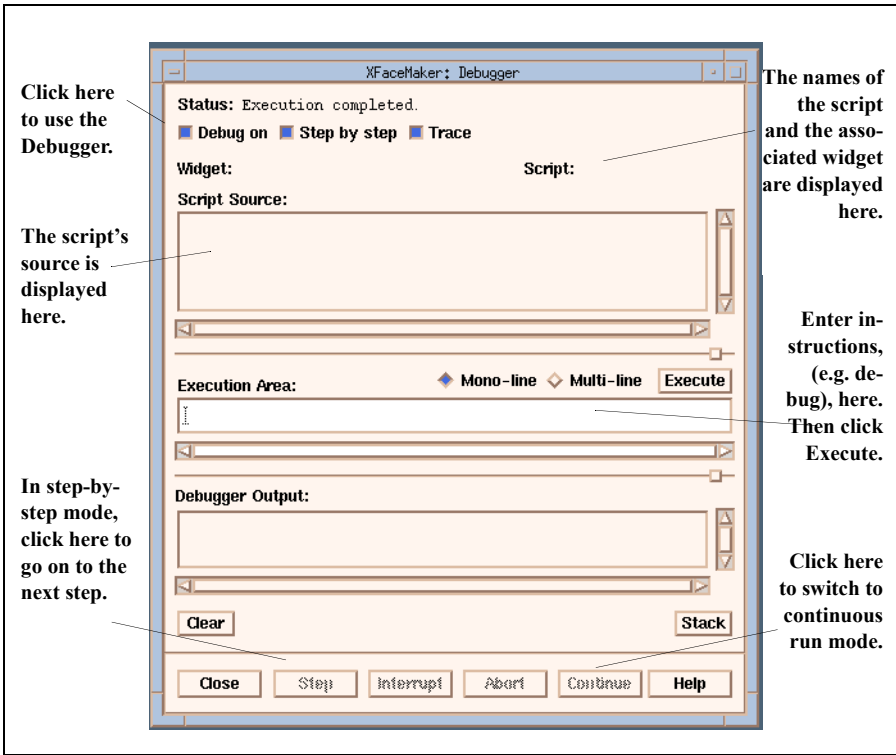


Figure 6.2: The Script Debugger

Debugging slows down the execution of scripts. Therefore, when you are not debugging scripts, you should deselect the *Debug On* checkbox.

The **Script source** and **Debugger output** areas are read-only. If you want to modify a script, you have to switch back to Build mode, if necessary, and edit the script in the usual XFaceMaker editing window.

To debug a script, do the following:

1. Set the ***Debug On*** toggle button to activate the FACE debugger.

This button is automatically unset when the FACE debugger window is hidden.

2. If desired, set the ***Step by step*** toggle button.

It controls whether the FACE debugger stops before it executes each instruction or not. In step-by-step mode, you can only use the FACE debugger window and control the execution with the ‘Step’, ‘Abort’ and ‘Continue’ buttons. The rest of the XFaceMaker interface as well as your user interface is locked.

3. If desired, set the ***Trace*** toggle button.

When the Trace toggle is set, the FACE debugger traces the execution of all instructions by showing the source of the corresponding script, and by highlighting the current instruction. If this button is not set, the debugger only traces errors, warnings and breakpoints by showing the corresponding source and instruction, and by entering step-by-step mode. Note that the FACE debugger can trace the execution of all scripts---not only widget callbacks in Try mode, but also create callbacks in Build mode or active value scripts for a user-defined widget.

4. Test your interface by putting XFaceMaker into Try mode and executing the script; i.e., perform the action that launches the script.

If an error is detected, it will be displayed in the Script source field and an error message will appear in the Debugger output field.

Errors or warnings encountered by the FACE parser or interpreter stop the debugger at the point of the error and start step-by-step mode. Stepping, aborting or continuing after an execution error terminates the Design process. The explanatory text of the error is printed in the Debugger output area as well as in the XFaceMaker message window.

You can set breakpoints in scripts by inserting calls to the ‘breakpoint’ function, which takes two arguments: a widget (usually ‘self’) and the breakpoint name (a character string). When the breakpoint is reached, the debugger enters step-by-step mode and a message containing the breakpoint name is printed in the status area.

You can enter instructions in the Execution area, such as `debug`, to print the values of variables, resources, etc. The `debug` function is called exactly like `printf`, but the output is directed to the debugger’s output area instead of the standard output.

Enter the instruction, then press the **Execute** button. (Or press **Return** if the **Mono-line** is set. If the **Multi-line** toggle is set, you can type several lines of debugging instructions, but you have to click the **Execute** button.)

The **Execute** button executes the FACE statements that you have entered in the Execution area. In step-by-step mode, the FACE statements are executed as if they were part of the currently debugged script. The local variables of the debugged scripts are accessible as well as all the global variables defined so far (you do not need to use the **global** keyword before referencing global variables in this case).

The pushbuttons at the bottom of the Debugger are used as follows:

- **Stack** prints the stack of scripts and functions currently called, with the argument types and values, in the output text area.
- **Clear** clears the debugger output area.
- **Close** hides the debugger window (and unsets ‘Tracing On’).
- **Step** advances to the next instruction in step-by-step mode.
- **Interrupt** is used to stop the execution of a script: a SIGINT signal is sent to the design process, and the debugger switches to step-by-step mode.
- **Abort** is used to terminate execution of the current script in step-by-step mode.
- **Continue** is used to switch to normal run mode when in step-by-step mode. Execution continues without stopping at each instruction.
- **Help** pops up a help text box.

6.8 Error Recovery

Error protection is a major functionality offered by XFaceMaker's dual-process mode: the Design process may crash, but the XFaceMaker process stays alive.

6.8.1 Design Process Termination

When a fatal error occurs in an interface, the Design process terminates. A fatal error can result from:

- a call to `XtError`, usually caused by a widget used in an abnormal way, or an X Toolkit or Motif function called with wrong arguments;
- a fatal UNIX signal, typically 'Bus error', 'Segmentation violation', or 'Illegal instruction', which has usually the same origin as an `XtError` but has not been checked by the toolkit programmer;
- an error occurring during the execution of a FACE script, if the option ***Restart on FACE Errors*** was set.

The first two kinds of errors can't be disabled: they always terminate the Design process because they indicate that some data may be corrupted.

In the third case, you can choose whether or not the Design process is terminated when a FACE error occurs. By default, the Design Process is *not* terminated. However, you can change this by selecting the ***Restart on FACE Errors*** item in the Options menu. If you do so, when a FACE error occurs, an error message will be displayed but the FACE script will only be aborted. Please note that such errors may also corrupt data in some cases, so it is wiser to set the ***Restart on FACE Errors*** option.

When the Design process terminates, an alert box pops up to inform you and the nature of the error is displayed in the XFaceMaker message area. Sometimes, the Design process may be so corrupted that it cannot display the error before exiting. In this case, only the alert box will pop up.

Once the alert box is acknowledged, the Command Prompt dialog box will prompt you for a command line which will be executed to restart the Design process. In most cases, one just needs to click ***OK*** to restart the Design process. The following sections describe how to restart the Design process, and which options are available when you do so.

6.8.2 Restarting the Design Process

When the Design process terminates, the Restart Design Process window pops up, displaying the default command line. The default command line launches the Design process which just terminated, usually `xfm` (or a new user-built instance of XFaceMaker), with all the command-line options specified initially, plus the `-client` option and the `-restore` option. These options are described in the following sections.

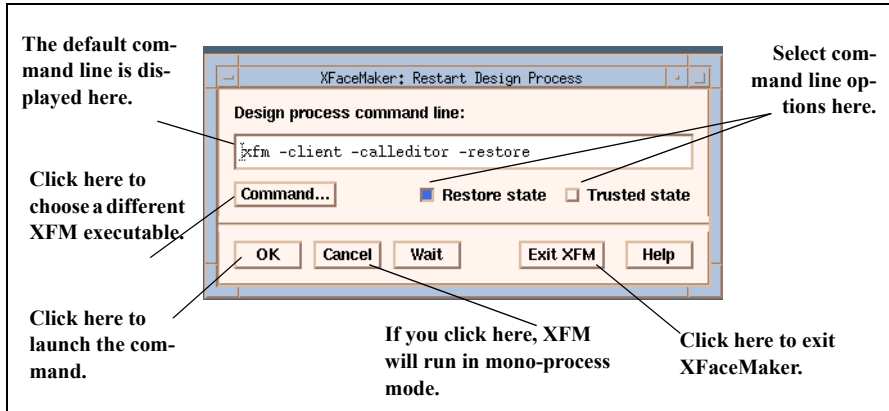


Figure 6.3: The Restart Design Process Box

- The **Command...** button pops the File Selector to choose a different XFaceMaker executable.
- The **Restore state** and **Trusted state** buttons toggle the `-restore` and `-trusted` options respectively in the command line.
- The **OK** button launches the specified command line. XFaceMaker waits until the new Design process connects to it.
- **Cancel** will not launch any Design process. An alert box pops up to warn you that XFaceMaker is running in mono-process mode. Clicking **No** in the alert box returns to the Restart Design Process dialog box.
- The **Wait** button does not launch any Design process, but XFaceMaker will still wait for a connection. The interface is disabled until the connection occurs. This is useful for launching the Design process manually from a shell, or by any other external means. Don't forget to set the `-client` option when launching the Design process.

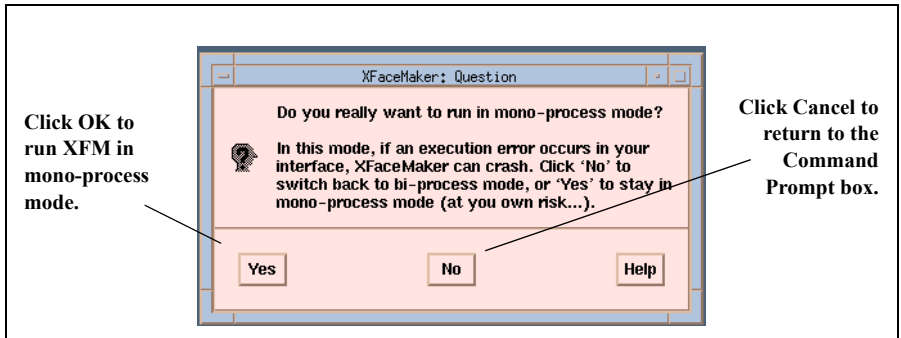


Figure 6.4: The Mono-process Alert Box

6.8.3 The -restore Option

The XFaceMaker process keeps a copy of the state of the Design process. When you re-launch the Design process, the `-restore` option tells it to fetch this state from the XFaceMaker process, and modify its own state accordingly. In short, the new Design process will restart where the old one had stopped. The Design process state includes:

- the whole hierarchy of widgets,
- the sets of loaded groups, templates and classes,
- the current editing mode (Try or Build),
- various internal information: the current directory, current file name, the current edited object, template or class model objects.

If you restart the Design process, or if it was terminated due to an execution error in Try mode, the Design process state is saved just before the Design process exits.

If the Design process crashed during a widget modification (i.e. when you clicked **Rebuild** or **Set Values** in an editing box after modifying a resource or another attribute), then XFaceMaker restores the state of the Design process “just before” the widget was re-built, and things will be as if you had just made the modifications, but not yet applied them. In this case, the modifications should be corrected and another **Rebuild** or **Set Values** should be attempted, or you should discard the modifications by selecting another object. The “Resource modified/ignore changes?” warning will be issued.

It may happen that the Design process state cannot be saved before it exits; for example, if its data is too corrupted. For this reason, XFaceMaker keeps a second security state, called the "trusted state", which is saved periodically (every minute or with the periodicity you specify - *Options* menu, *Auto Save State*), and every time you switch to Try mode. This state can be restored by using the `-restore` and `-trusted` flags simultaneously. So, the typical scenario is to try first `-restore` alone (the default), and if this fails try `-restore -trusted`.

You can also save the state of the Design process to disk. This can be useful, for example, to protect against a power failure without saving a `.fm` file. The *Save State* command in the File menu saves the current Design process state into a set of files starting with `.xfm_` in the current working directory of XFaceMaker. Once saved, this state can be restored by passing the `-restore` (and perhaps `-trusted`) option(s) to the XFaceMaker process. The state will first be loaded from disk files to the XFaceMaker process, and then restored into the initial Design process.

6.8.4 Launching a Different Design Process

You can start a new and different Design process by selecting the *Restart Design Process* command in the File menu. Launching this command terminates the current Design process (with an alert if the current interface was modified). The Command Prompt dialog box appears, enabling you to choose a different Design process and its options.

When XFaceMaker is running in mono-process mode (without any Design process connected), the *Restart Design Process* command switches back to dual-process mode when you launch a new Design process.

The *Restart Design Process* command also gives you access to XFaceMaker's 'Dynamic Extension' functionality. For example, suppose you want to define a new C function and add it to XFaceMaker so that it can be called in a callback. You will have to write your function, compile it, and link it with a main program containing calls to `FmInitialize`, `FmAttachFunction` and `FmCallEditor`. In fact, XFaceMaker will do everything – except writing the function – automatically, as explained in Chapter 10. You don't have to exit your current XFaceMaker and start the new one from scratch. Instead, just call the *Restart Design Process* command and specify the new program as the Design process. Your new function is now available.

CHAPTER 7: Menus

Menus are a convenient and standard method for the user to communicate with your application. XFaceMaker gives you two options for creating menus:

- Use the Menu Editor. It provides a simplified interface, access to the most commonly used resources, and a representation field that lets you envision the menu as you build it.
- Build a menu from scratch. XFaceMaker provides everything you need to build a menu from scratch, without the Menu Editor.

This chapter describes the two methods for building a menu in XFaceMaker, and includes step-by-step examples.

7.1 Menu Types

There are four types of menu:

- A *menu bar* is the familiar set of menus available at the top of an application's main window, or other primary windows.
- A *popup menu* “pops up” when the user presses a mouse button, usually the *Custom* button. It provides convenient access to frequently used commands.
- An *option menu* offers the user a choice among several options. It appears as a normal button with a special graphic. When the user presses the button, all of the available options appear in a menu. The selected option becomes the label of the button.
- A *pulldown menu* is actually a sub-menu, or a cascade menu that is “pulled down” from one of the three other types of menus. The OSF/Motif style guide requires that all items in a menu bar be pulldown menus.

7.2 Accelerators and Mnemonics

Accelerators, which are a type of translation, allow an *event* received by one widget to cause an *action* of another. This can be implemented as a general mechanism, but the accelerator resource defined for Motif widgets applies specifically to menus, where the accelerator allows the user to activate menu items quickly¹.

As an illustration, we'll use `Popdown.fm` from the standard XFaceMaker examples directory. The menu item *Save*, which is a `PushButton` widget, has the following resources:

```
accelerator = <KeyPress>F4
acceleratorText = F4
mnemonic = S
```

The accelerator `F4` is recognized by widgets higher in the widget tree. Pressing this key when the mouse pointer is on the `BulletinBoard` or `MenuBar` will have the same effect as if the *Save* item itself had been selected – in this instance, the `activate-Callback` script is called. The `acceleratorText` resource defines the string which is placed to the right of the item `labelString` text.

The mnemonic has a similar effect, but it is more limited. Pressing `S` will activate the menu item, but only when the menu has previously been opened either with the pointer, or by its mnemonic (`Alt - F` in this case for the File menu). It thus takes two key strokes to achieve the same effect as using the mouse, but it is still a handy alternative. It's not hard to remember which key to type, since this is indicated by automatic underlining of the mnemonic letter in each label. Further discussion of these points can be found in the OSF/Motif Style Guide.

-
1. There are actually two different accelerator resources. One is general, it is implemented at the Intrinsic level, it is defined in the `Core` class, the resource name is **accelerators**. You have to explicitly install this type of keyboard shortcut. The other is implemented by Motif, it is defined in the `XmLabel` class, the resource name is **accelerator** (no s); you can use this resource directly in pushbuttons and togglebuttons, in pulldown or popup menu panes to invoke the button's `activate-Callback` with one keystroke. But this works *only* on push or toggle buttons, and *only* when they are in a menu.

7.3 Using the Menu Editor

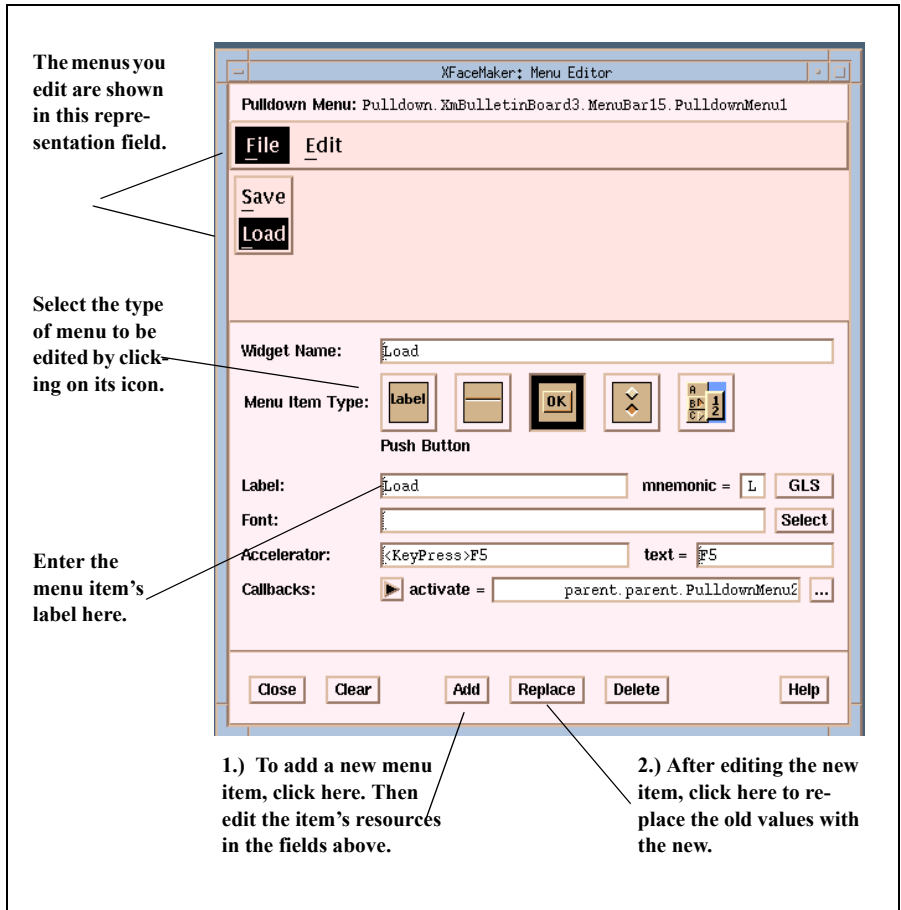


Figure 7.1: The Menu Editor

- To open this window, select *Menu Editor* from the Windows menu.
- To delete an item, select it and click on *Delete*.
- To close the window, click on *Close*, or toggle the *Menu Editor* item in the Windows menu.

The Menu Editor provides a simplified interface for building menus. It contains:

- a representation field to display the menu during editing.
- a dialog section to define resources.
- a set of pushbuttons to apply your choices.

The editing fields displayed in this window change according to the type of menu or menu item being edited. A complete description of these fields can be found in the *XFaceMaker Reference Manual*.

7.3.1 The Representation Field

The Representation Field displays menus as you build them. This field is useful because menu panes do not appear on the interface display while the menu is being built, but only when you switch to “Try” mode. This is due to the way menus are implemented in the Motif toolkit. The representation field works as follows:

- Each menu item is displayed with the correct label, mnemonic, and font. However, no additional graphics such as toggle button indicators, cascade indicators, or accelerators are displayed.
- The item currently being edited is highlighted, as well as the item in each menu that selects the next cascade.
- You can reorder items within a menu by dragging their menu representation, just as you would in the Widget List or Widget Tree.
- You can double-click on an item in the menu representation to open its corresponding pulldown menu. (This does not work in the Widget Tree or List, but you can still select the pulldown menu there.)

7.3.2 The Dialog Section

The dialog section of the Menu Editor is used for editing. Only the most commonly used resources are available in the Menu Editor, but you may still use the Resources box for complete control of an object’s resources. The resource settings of the currently selected menu item appear in the editing fields. You can change their values, then click the **Add** or **Replace** button to apply the changes.

You can select the menus or menu items to be edited on the interface itself, in the Widget List, or in the Widget Tree. When you select an option menu, you automatically select its accompanying pulldown menu. Furthermore, you can select any item visible in the menu representation by clicking on it. You can select a pulldown menu for editing by double-clicking on its cascade button in the menu representation.

7.3.3 The Pushbuttons

The buttons on the bottom of the window provide overall control of the dialog box, and work as follows:

Close Closes the window.

Clear Replaces the values you are editing with those of the previous menu item edited.

Add Adds a menu item, according to the contents of the dialog portion of the Menu Editor, to the menu shown in the menu representation.

Replace Replaces the selected menu item with a new one according to the contents of the dialog portion of the Menu Editor. Applies to the menu shown in the menu representation.

Delete Deletes the selected menu item.

Help Brings up a help box for the Menu Editor.

The Menu Editor is described in more detail in the *XFaceMaker Reference* manual.

7.4 Example—Editing a Menu

The following example, illustrated below, describes how to create a menu bar and add several pulldown menus and items, using the Menu Editor.

First create a simple interface on which to place the menus.

1. Create an `ApplicationShell` with an `XmMainWindow` composite widget as its child.
2. Open the Menu widgetstore, and place a `MenuBar` inside the `XmMainWindow` on the interface. This creates an empty menu bar on the interface.
3. Select the `MenuBar` by clicking on it with the Custom mouse button, or by selecting it in the Widget List or Widget Tree.

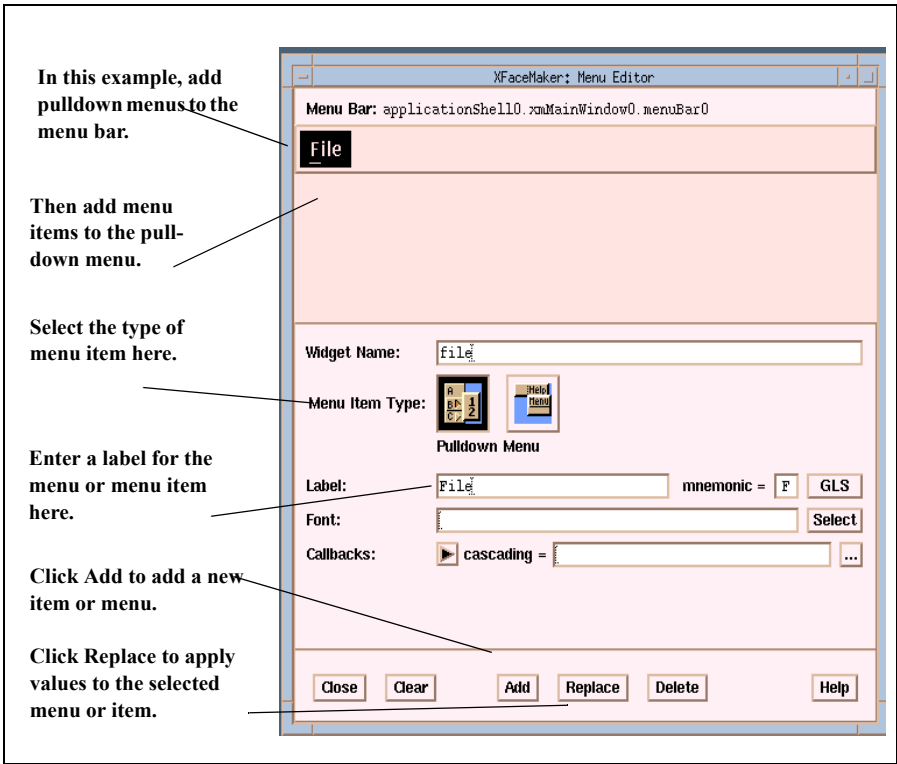


Figure 7.2: Example: Using the Menu Editor

Now add a pulldown menu---to be called “File”.

1. Open the Menu Editor by selecting that item in the Windows menu.
2. Select the **PulldownMenu** icon in the Menu Item Type field.
3. Move the cursor over the Label field and type the word **File**.
4. Move the cursor over the mnemonic field and type the letter F. If there was already a letter in the field, it is replaced by the one you type.
5. Click the **Add** button. This adds the pulldown menu labelled “File” to the interface, and to the representation field in the Menu Editor.

Note that the Menu Editor automatically creates a widget name from the menu item’s label unless you enter a name explicitly. The values in the editing portion of the dialog box remain, and may be reused for other menu items.

So far, the File menu is empty. Now add the menu item **New** to the File menu, and specify an accelerator and a callback script for **New**.

1. Double-click on File in the menu representation. An empty menu item appears below the menu bar in the representation field.
2. Select the Push Button icon in the Menu Item Type field.
3. Enter "New" in the Label field.
4. Change the mnemonic by typing the letter N in the mnemonics field.
5. Move the cursor over the Accelerator field and press the F2 function key. The key code name automatically appears in the input field, and a standard description appears in the text field next to it.
6. Toggle the arrow button after the Callbacks label to view the types of callbacks available for this menu item type. When the button is set to "activate", click the . . . (ellipsis) button to open the callback editor.
7. Enter this script into the text editing field of the callback editing box:

```
function None newbutton();
newbutton();
printf ("The New button has been pressed.\n");
```

The effect of this callback script can be seen when you put XFaceMaker in Try mode, as described later in the example.

8. Click on the **Confirm** button. This registers the callback script and returns to the Menu Editor window.
9. Click the **Add** button. This adds the menu item **New** to the File menu.

You can now easily add a second menu item to the File menu.

1. With the **New** item still selected, click on the **Add** button. This adds a second menu item below **New**. The second item becomes the currently selected item. The dialog area of the Menu Editor still contains the values of the previous edit.
2. Edit the second item; for example, change its label to “Open”.
3. Click on the **Replace** button. This applies the new values to the second menu item.

To add more menu items, repeat the three steps above.

To add another pulldown menu to the menu bar:

1. Select the File pulldown menu. The new pulldown menu will be added to the right of the File menu.
2. Select the Pulldown Menu icon in the Menu Item Type field.
3. Click the **Add** button. The new pulldown menu is added to the right of the selected pulldown menu in the representation field.
4. Edit the values for the new pulldown menu; for example, enter “Edit” in the label field.
5. Click the **Replace** button to apply the new values.

Now add a Help menu. When you add a Help pulldown menu, XFaceMaker automatically places it on the far right side of the menu bar.

1. Select the “Edit” pulldown menu (or whichever pulldown menu is farthest to the right on the menu bar.)
2. Select the Help Menu icon.
3. Click the **Add** button. XFaceMaker places a new pulldown menu on the far right side of the menu bar.
4. Select the Help pulldown menu.
5. Edit its values; for example, type “Help” in the Label field.
6. Click on **Replace** to apply the new values to the Help menu.

To see how your newly created menu works, put XFaceMaker in Try mode and click on the newly created File menu in the interface. The pulldown menu will open, displaying the menu items you defined. The text you entered in the callback script (in step 7 above), appears in your xterm window when you select the **New** menu item.

You can edit a menu whenever you wish by selecting a menu item and displaying its resource settings in the Menu Editor box. You may then change its values, and click on the ***Replace*** button to apply the changes.

- To reorder items within a menu, drag their menu representation, just as you would in the Widget List or Widget Tree. However, double-clicking an item to open its corresponding pulldown menu only works in the menu representation, not the other windows. You can still explicitly select the pulldown menu there.
- If you want to add internationalization prefixes to both the label and mnemonics fields, click the ***GLS*** button.
- To specify a font for the item, click the ***Select*** button located on the far right of the Font input field. This opens the Fonts selection box. Select a font family from the list, then click the ***Confirm*** button in the font selection box. The name appears in the "Font" field of the Menu Editor.
- To view the callbacks available for a menu item type, toggle the arrow button next to the Callbacks label. Most applications do not need callbacks for pulldown menus.

7.5 Creating Menus without the Menu Editor

Menus created without the Menu Editor are different from other objects in that menu panes do not appear on the display as a menu is being built, but only when you are in Try mode. This restriction is due to the way menus are implemented in the Motif toolkit.

- You can *select* any menu widget in the Widget List or the Widget Tree and then drag it to a tool icon, or edit it using the commands in the Edit menu.
- To *place* a menu widget, select it, move it over another widget, (or over a widget name in the Tree or List), and click. Or drag out the widget on the interface.
- To *change the order* of items in a menu, select an item and drag it to a new position.

The only thing that cannot be done in the Widget List or the Widget Tree is to resize a widget within a menu pane.

7.5.1 Creating a Pulldown Menu

This example describes how to create a MenuBar and add several Pulldown menus. The menu created here is the same as the one illustrated in Figure 7.2 which was created using the Menu Editor.

First, create an interface for the menu by placing a MainWindow composite widget in an ApplicationShell. Next, open the Menu widgetstore in the Toolkit.

1. Select a MenuBar and place it on the MainWindow.
2. Select a PulldownMenu and place it on the MenuBar. This is a short cut---XFace-Maker automatically adds the required CascadeButton along with the Pulldown-Menu. Note how the MenuBar resizes to the CascadeButton's default size.
3. Give the title "File" to this first menu. To do so, open the Widget Resources window and specify "File" as the labelString resource for the CascadeButton. Change the CascadeButton's name to "file"

4. Open the Primitive widgetstore and select a PushButton. Place it on the Pull-downMenu. This creates a menu item in the “File” menu. To name it and change its label text, select the PushButton, and specify its name and its labelString resource in the Widget Resources window. Repeat this step so that there are two items in the File menu.
5. Select another PulldownMenu in the Menu widgetstore and place it on the MenuBar. A second CascadeButton is added automatically by XFaceMaker.
6. Select the second CascadeButton and change its labelString resource to “Edit” in the Widget Resources window. Change its name to “edit”.

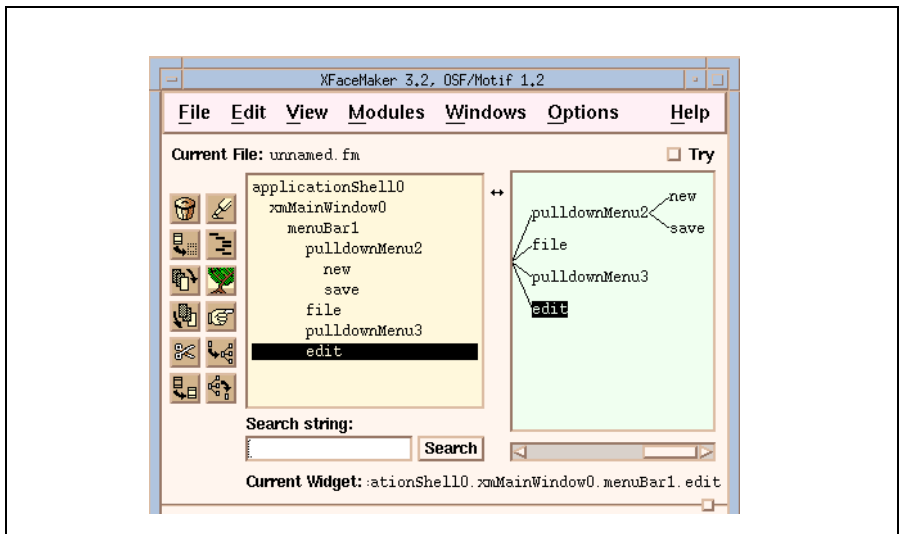


Figure 7.3: The Widget List and Tree for the Example

By convention, Help menus are placed on the far right edge of the MenuBar. This makes it easier for a user to find the Help menu. Motif Pulldown menus have an automatic mechanism for placing Help menus right edge of a menubar.

To add a Help menu to the MenuBar.

1. Add a PulldownMenu to the MenuBar. As usual, a CascadeButton is added automatically by XFaceMaker.
2. Select the PulldownMenu, and change its labelString resource to “Help”.
3. Select the MenuBar, and enter the following script in the xfmCreateCallback field:


```
self:menuHelpWidget = self.XmCascadeButton2;
```

where `XmCascadeButton2` is the name of the Help cascade button.

4. Click ***Rebuild*** or ***Set Values***, or press **Return**.

To create cascading Pulldown menus, add a `CascadeButton` to the `PulldownMenu` then proceed exactly as if this `CascadeButton` were a title in a `MenuBar`, placing within it first a `PulldownMenu`, then buttons and separators as desired.

To check the appearance of the menu, switch to Try mode, and click one of the title buttons; the corresponding Pulldown menu will appear.

7.5.2 Creating a Popup menu

Unlike a Pulldown menu, a *Popup menu* is not attached to a MenuBar. Popup menus have two main advantages over the MenuBar:

- They take up less space in the interface, since there is no associated menu bar.
- They can be brought up without moving the mouse to a specific location. You need only click on the background with the *Custom* button.

Their disadvantage is that, since they have no visible cue unless they are active, the user may not be aware that a particular background has a Popup menu until the *Custom* button is pressed.

To create an example of a Popup menu, do the following:

1. Place a BulletinBoard within a TopLevelshell, (or ApplicationShell if this is the only shell in your interface).
2. Select a PopupMenu, place it on the BulletinBoard and release without dragging.
3. You can now add PushButtons, ToggleButtons, and Separators to the menu by placing them over PopupMenu in the Widget Tree or List.
4. Edit the menu's resources in the Widget Resources window.

To check the appearance of the menu, switch to Try mode, and press on the background of the BulletinBoard with the Custom button to pop up the menu. This default is automatically set by XFaceMaker and can be changed by editing the *translations* resource of the BulletinBoard.

7.5.3 Creating an Option Menu

Option menus are handy when you don't want to use a menu bar and when space in a window is limited. An Option menu is associated with a set of buttons, only one of which is visible at a time. This has the advantage of allowing you to choose a default value as the one to be shown.

To create an example of an option menu, do the following:

1. Place a `BulletinBoard` in a `Toplevel` shell, select an `OptionMenu` widget from the Menu widgetstore and place it on the `BulletinBoard`.
2. Select a `PulldownMenu` and place it on the `OptionMenu` in the Widget List.
3. Now add a few `PushButtons` to the `PulldownMenu` in the Widget Tree or Widget List.

To check the appearance of the menu, switch to *Try* mode, and press on the button you see with the `Select` button. This will display the whole menu just created, and when you select a new item and release, that item will be the one visible on the `BulletinBoard`.

You can add a label to this menu by selecting the `OptionMenu` widget in the Widget List or Tree and editing its `labelString` resource. This label will appear to the left of the button. You can also add a label to any `OptionMenu` item by selecting it and editing its `labelString` resource in the Widget Resources window.

CHAPTER 8: Active Values

Active values represent a unique feature of XFaceMaker. They are the means of communication between your application and its interface. Active values can be used to do the following:

- Share data between the interface and the application.
- Define OSF/Motif drag targets and drop sites.
- Define resources for a new Xt widget class.
- Define members of a C++ class generated by XFaceMaker.
- Add pseudo-resources to an existing widget instance.
- Define scripts that can be launched independently of X events, either from the application or the interface.

Active values may store information for the interface, such as a file name, or they can allow the application to access interface data. They may also be used just to define scripts. In this case, there is no need to associate an application variable to the active value.

Active values make it possible for the interface and the application to share data while keeping a cleanly defined separation between the two, and a high degree of independence from the toolkit. Such separation is especially desirable when you are designing an application for use with multiple toolkits, and for compatibility with future revisions of the same toolkit.

Moreover, a clear separation between the interface and the application ensures that, if the interface is modified, no modification need be done in the application, and vice versa.

For example, if the application has an integer value that it expects the interface to display (i.e. that it wants to make known to the user), the application should not need to know if the value will be displayed as a String in a TextField, as an XmString in a Label, as an integer in a Scale, as a Pixel in a color-coded widget, etc. Perhaps the value is to be displayed in several widgets in the interface. Here, again, the application should not need to know how many copies of the value are displayed, or within which hierarchy.

Conversely, when the user enters a value in the interface, the application should not need to know if the value was typed in a TextField, if it was specified using the slider of a Scroll Bar, or Scale, etc. When the application needs the value that was provided by the user, it should just fetch it, and not worry about how it was acquired. If the

user decides that he is more comfortable using a slider than a text input widget, the interface designer should be able to change it without having to modify the application in any way.

Communication with active values takes place through specially designated data in the application and through FACE scripts in the interface. An active value's data has the following properties:

- A **name**. The name must begin with an alphabetic character.
- A **type**, indicating what (and how) the data represents. Active value types are FACE types, e.g. Int, String, Widget.
- Its **storage** characteristics. There are three kinds of storage.

m *global* The entire interface uses the same data for the active value regardless of the widget using the active value.

m *per widget* Separate storage locations are specified for each widget that uses the active value.

m *per call* Use this if you want your set script to be called with arguments. XFaceMaker allocates storage each time it calls the active value's FACE script (then de-allocates it after). See FmCallValue.

In “global” or “per widget” storage, the application can provide the storage, or it can let XFaceMaker allocate the storage automatically. In “per call” storage, XFaceMaker *always* allocates storage.

- Its **access**:
 - m interface type active values -- There are two ways to access the data in a FACE script, depending on whether the script accesses the *contents* of the storage with the @ special variable (\$ then refers to the address of the storage) or directly with the \$ special variable. The latter case is called *immediate access*, in which the @ variable is meaningless and must not be used.
 - m C++ class active values -- XFaceMaker uses the access mode field of active values when it generates C++ code. The access mode field does not affect C code. The access modes "private", "protected", and "public" indicate the access mode of the member functions of the C++ class corresponding to the XFaceMaker interface. These functions call the FACE scripts for the active values in the interface.

Also for C++ class active values, the *Generate access functions* toggle specifies, for an active value whose storage is per-object, i.e., a data member, whether access member functions (for read and write access) should be automatically generated by XFaceMaker.

The default settings of the active value attributes correspond to the active values in earlier versions of XFaceMaker; i.e. the value has no type, its storage is global and not automatically allocated, and it has public access by its address. (It does not have immediate access.)

If an active value has *per widget* storage allocated automatically by XFaceMaker, you can give it an initial value which XFaceMaker assigns to it just after creating the widget. You can edit initial values in the same manner as standard widget resources. This lets you add pseudo-resources to instances of pre-defined widget classes.

You may associate more than one active value with a widget, and you may associate more than one widget with an active value. You can associate two kinds of FACE scripts to each active value in each widget: a get script and a set script.

By convention

the *get* script updates the active value's data to reflect the interface's current setting: it is responsible for converting the piece of information residing in the interface to the format expected by the application, and for placing the value in the *@* special variable (or *\$* for immediate access active values).

the *set* script is responsible for converting the piece of information stored in the application to the various formats in which it is to be used in the interface and updating the corresponding widget resources so as to update the display as required. It receives the data in the *@* special variable (or *\$* for immediate access active values).

XFaceMaker does not enforce this convention, but it is a good idea to adhere to it. What XFaceMaker does is initialize *@* and *\$* to the current content and address of the active value before entering the set/get script.

Note:

- accessing an active value produces a call to its get or set script. For example, from the application *FmReadValue()*, or from a FACE script, *value=widget:@name;* will activate the get script.
- when you update an active value, the set script is called after the value is modified, i.e. *@* contains the new value on entering the script..
- you can access an active value within its get or set script without causing a recursive call.

8.1 Writing Active Value Scripts

You write the *get* and *set* scripts in the FACE language. Within a *get* or *set* script, two special variables refer to the data of the active value.

- When you are not using immediate access, the variable `@` refers to the active value's contents, and the variable `$` refers to its address.
- When you are using immediate access, `$` refers to the active value's contents; `@` is meaningless.

You may call FACE and application functions as required.

Using active values, you can call FACE scripts directly from application code, as opposed to using callbacks, which are only called in response to external events. You do not need to use the variables `@` and `$` if no data transfer is involved.

The `@` and `$` special variables can also be *named* by combining the variable with a widget and an active value name. This enables you to refer to an active value which is not associated to the current script. That is:

`[widget:]@name` refers to the contents of the active value *name* for the specified *widget* (no immediate access).

`[widget:]$name` refers to the address of the active value *name* for the specified *widget* (no immediate access).

`[widget:]$name` refers to contents of the active value *name* for the specified widget (immediate access),

If you do not specify a widget, `self` is used, the value is updated but the *set* or *get* script is not triggered. In creation scripts, you have to specify the widget explicitly, e.g. with `self`.

To access active values whose storage is *per call* use the following syntax:

widget:<avname>(arg1,arg2,...) where arg1, arg2, etc are the arguments that will be passed to the active value's *set* script. This is equivalent to a function call, where the function is the active value's *set* script. All the arguments have to be the size of a FACE word, i.e. `sizeof(XtArgVal)`.

Named active value variables can be used in all forms of FACE scripts, not just *get* and *set* scripts.

8.2 Allocating Storage

You specify storage for an active value by calling the XFaceMaker library function:

```
FmAttachValue(Widget widget, char *name, XtPointer address)
```

where:

widget is the widget to attach the active value to.

name is the name of the active value.

address is the address of the active value's storage. If immediate storage is used, this parameter is the contents of the active value. You will need to cast to `XtPointer` to satisfy function prototyping.

If the widget has *global* storage, specify `NULL` as the widget. If it has *per widget* storage, you must call `FmAttachValue` once for each widget you want to attach the active value to. You should not call this function for widgets with *per call* storage.

If the active value has global storage, it is common to attach the active value to the address of an application variable with a similar or related name, but not necessarily the same name.

In Try mode, if the active value has no storage attached to it, i.e. `FmAttachValue` has not been executed for that active value, calling *get* and *set* scripts may produce error messages. This is not true for active values whose storage is allocated automatically by `XFaceMaker`.

Before allocating storage, `XFaceMaker` checks if the active value has been attached to an application variable. If the active value has been attached, `XFaceMaker` uses the storage specified; otherwise, it allocates storage. Thus, it is not dangerous to specify automatic allocation for *global* or *per-widget* active values.

There is one exception to this: active values whose type is *String*, for which a default value has been specified. In that case, `XFaceMaker` always allocates storage. Use `FmSetValue` to change the setting of such active values.

Example 1: For an active value of type `Int`:

If, in the application, you have:

```
int i;
Widget w;
FmAttachValue(w, "myav", &i);
```

in a FACE script, you access the active value through `mywidget:@myav` in scripts in general, or `@` in the active value's set and get scripts. You are not using immediate access.

If, in the application, you have:

```
int i;
Widget w;
FmAttachValue(w, "myav", (XtPointer)i);
```

in a FACE script, you access the active value through `mywidget:$myav` in scripts in general, or `$` in the active value's set and get scripts. You are using immediate access.

Example 2: An active value of type `String`.

If, in the application, you have:

```
char* s;
Widget w;
FmAttachValue(w, "myav", &s);
```

In a FACE script, you access the active value through `mywidget:@myav` in scripts in general, or `@` in the active value's set and get scripts. You are not using immediate access.

If, in the application, you have:

```
char buf[100];
Widget w;
FmAttachValue(w, "myav", buf);
```

in a FACE script, you access the active value through `mywidget:$myav` in scripts in general, or `$` in the active value's set and get scripts. You are using immediate access.

8.3 Defining a New Active Value

XFaceMaker provides an editing box for defining and editing active values for the currently selected widget. Open it by selecting **Active Values** in the Windows menu. The options available change according to the “Usage” you select for the active value: interface, drag source, drop site, widget class, or C++ class. The general steps for defining an active value for a selected widget are:

1. Select a widget. You can only edit an active value for the currently selected widget. For **interface** active values, select a high level widget in the interface, so the application will get the widget ID when it instantiates the interface fragment (Return value of `FmCreatexxx` or `FmLoadCreate` function calls).
2. Select an item in the **Usage** field at the top of the box. This indicates the type of data the active value defines for the widget, and also the way in which that data is represented. You have the following choices:

interface The active value is intended to share data with the application or to define a script, or to define a pseudo-resource, or a method.

drag source The active value names a target that the widget can drag, i.e. the type of data the widget can provide

drop site The active value names a target accepted by the widget when used as a drop site.

widget class The active value is a resource in a widget class described by the widget.

C++ class The active value is a member of the C++ class generated in connection with the widget.

When you select a usage for the active value, the Active Value box will display the options available for that sort of active value. These options are described in the following sections. Only those active values with the same usage will appear in the “Active values” field of the editing box if you are using “new-style” active values (XFaceMaker v.3.0 and later). However, if you are using an older .fm file with “old-style” (pre-version 3.0) active values, all the active values defined for the widget will be displayed, regardless of their usage.

Once you define a *per widget* active value, it is handled as a pseudo resource of the widget and it appears in the Resources window when you edit the associated widget. It appears as a resource whose name is that of the active value prefixed by @ or \$, depending on whether it has immediate access, with the default value set to the initial value you specified. The *get* script and the *set* script appear in the resource window as `set_name` and `get_name`, where *name* is the name of the active value.

8.3.1 Interface

An active value whose usage is set as *interface* is used:

- to share data between the interface and the application.
- to specify pseudo-resources of an instance of a pre-defined class.
- to specify get and set scripts without associated data storage.

To edit an active value of this kind, specify “interface” in the *Usage* field.

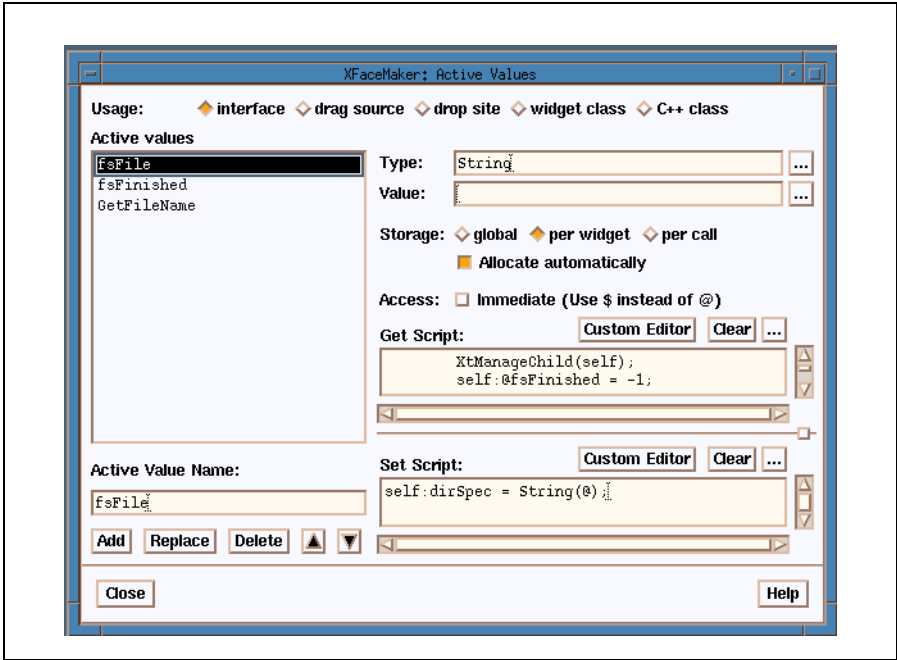


Figure 8.1: Editing an “Interface” Active Value

1. Enter a *name* in the *Active Value Name* field to identify it. The name must begin with an alphabetic character.
2. Specify the type of variable for the active value by entering it directly in the text field, or by selecting it from the box popped up when you click on the ellipsis. If no type is defined, the type is *Any*.
3. In the Storage field, select the type of storage for the active value, and how it is allocated. You have three choices:

m *global* if the entire interface may always use the same data.

m *per widget* if you want separate storage locations for each widget in the interface that uses the active value.

m *per call* if you want to define interface set scripts that can be called with arguments. XFaceMaker allocates storage each time it calls the active value's set script (and then deallocates it after the script is executed).

4. If you want XFaceMaker to allocate storage, select the checkbox marked “allocate automatically”. If you do not choose this option, make sure your application code allocates storage explicitly using `FmAttachValue`.
5. Decide whether your FACE script will refer to the contents of the active value via the `@` special variable or by its immediate value using `$`. If by immediate value, select the checkbox marked “immediate”.

XFaceMaker is able to allocate storage for immediate values of type `String` only. If you want XFaceMaker to allocate storage for other types, de-select “immediate” storage.

6. If storage is *global* or *per widget* and allocated automatically, you may enter an initial value in the “Value” text field.
7. Define the *get* and *set* scripts for the active value in the corresponding text fields. Clicking on the ellipsis opens an edit box for multi-line scripts. Clicking on the **Custom Editor** pushbutton opens the editor that you specified in your `FMEDITOR` environment variable.
8. Click on **Add** to register the changes and apply them to the current widget. The name of the newly defined active value for the selected widget will be listed in the “Active Values” display area, along with the set/get script if any have been defined.

8.3.2 Drag Source

An active value whose usage is set as *drag source* specifies a target that the widget can drag. To edit an active value of this kind, select “drag source” in the *Usage* field.

Refer to Chapter 20 for a complete description of using Motif drag and drop in XFaceMaker.

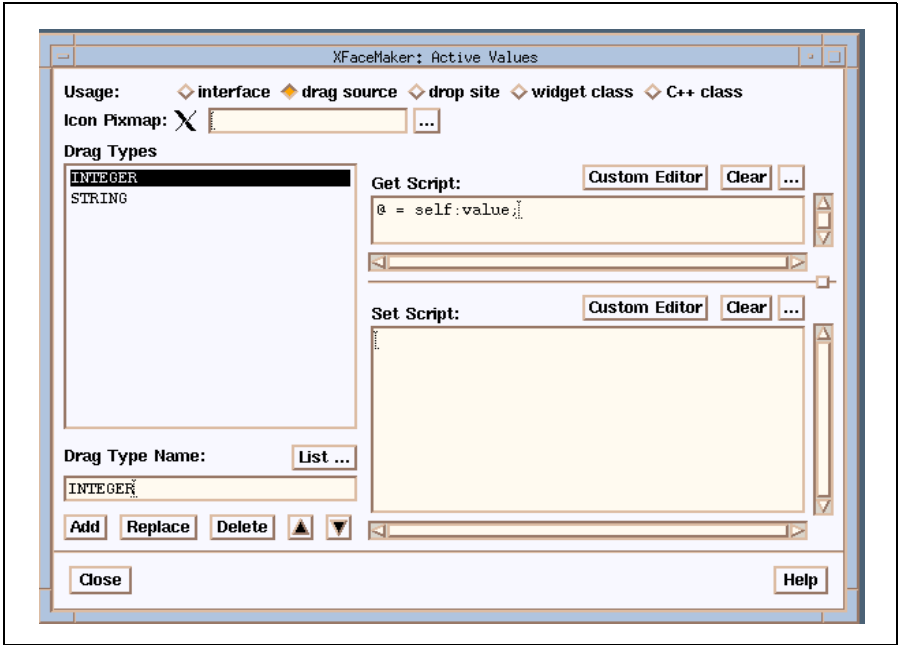


Figure 8.2: Drag Source

1. Enter a drag type (the targets exported by the drag source) in the "Drag Type Name" field. Or click on the **List...** button to pop up the Drag Target Selector which contains a list of commonly accepted selection targets defined in the ICCM. Select a drag target from the list, then click on the **OK** button. A widget can have more than one target.
2. Enter the file name of the pixmap that the pointer displays during a drag operation. The ellipsis pops up the Pixmap editor for choosing or defining a pixmap.

3. Define the *get* script for the active value in the corresponding fields. The *get* script should store the data to be transferred into the *@* argument. Clicking on the ellipsis opens an edit box for multi-line scripts. The *get* script will be added to the Resource Editor.
4. Click on **Add** to register the changes and apply them to the current widget. The name of the newly defined active value will be listed as an active value in the resource list.

You can also remove a target name from the list by selecting it in the “Drag Types” list (or entering its name directly), then clicking on the **Delete** button.

8.3.3 Drop Site

An active value whose usage is set as *drop site* specifies a target accepted by the selected widget as a drop site. To edit an active value of this kind, select “drop site” in the *Usage* field. For a complete description of using Motif drag and drop in XFace-Maker, see Chapter 20.

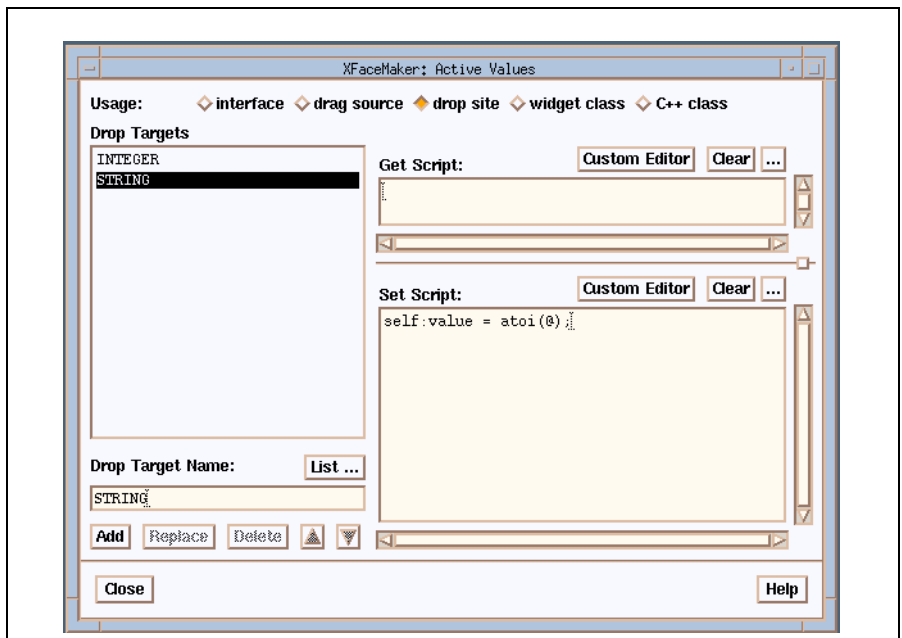


Figure 8.3: Drop site

1. Enter a drop target (the targets imported by the drop site) in the "Drop Target Name" field. Or click on the **List...** button to pop up the Drop Target Selec-

tor which contains a list of commonly accepted targets defined in the ICCCM. Select a drop target from the list, then click on the **OK** button. A widget can have more than one drop target.

2. Define the *set* script for the active value in the corresponding fields. The *set* script should read the data received in the @ argument, and use it as needed. Clicking on the ellipsis opens an edit box for multi-line scripts. The *set* script will be added to the Resource Editor. Clicking on **Custom Editor** opens the editor that you specified in your FMEDITOR environment variable.
3. Click on **Add** to register the changes and apply them to the widget.
4. You can remove a target by selecting it in the “Drop Targets” list (or entering its name directly), then clicking on the **Delete** button.

8.3.4 Widget Class

The Active Values box can be used to define new resources and behavior for a widget class. For a complete description of new widget classes, see Chapter 13.

An active value whose usage is set as *widget class* defines a new resource for a widget class. To edit an active value of this kind, select the top level widget of the class, then select “widget class” in the *Usage* field.

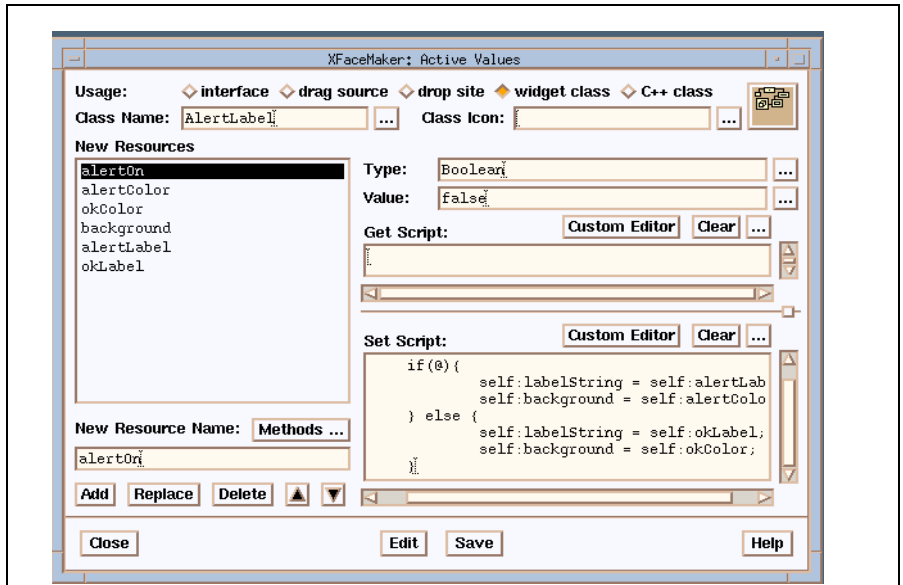


Figure 8.4: Widget Class

First define the class name, and class icon--they are associated to the widget. Then define as many resources as you like for this class, and do one overall **Save**. (The class name and icon can also be specified using the commands in the Classes submenu of the Modules menu.)

1. Enter the name of the class being defined or edited in the Class Name field, or click on the ellipsis to open a list of classes, and choose one. (The **Edit** button pops up the Class Name box in which you may select the class name that you want to edit.)
2. Enter the file name of the bitmap, or click on the ellipsis to pop up the Pixmap editor for choosing or defining a pixmap. This icon will represent the class in the User-defined widgetstore.

3. Resources:

- m To define a new resource for your class, enter a name for it in the “New Resource Name” text field.
 - m To edit an already defined resource, select it in the “New Resources” list.
 - m To specify a method for your widget class, click on **Methods...** to pop up the Method Selector. You can select a method name from the list or enter a name directly in the text field of the Method Selector box. Then click the **OK** button.
 - m To modify an already defined resource, select its name in the “New Resources” list or enter it directly, make the changes, then click on **Replace**.
 - m To remove a resource, select its name in the “New Resources” list or enter it directly, then click on **Delete**.
4. Enter the active value’s *type* in the “Type” field, or click the ellipsis to pop up the Type Selector box. Choose a type from the list or enter one in the “Resource type” text field, then click **OK**.
 5. Enter a default value for the new resource being defined in the “Value” field. Or click on the ellipsis to open an editing window for that type of resource. Enter a value, then click on the **Add** button. Or click on the **List** button to return to the Active Values box without specifying a value.
 6. Define the *get* and *set* scripts for the active value in the “Set Script” and “Get Script” fields. Clicking on the ellipsis opens an edit box for multi-line scripts. Clicking on the **Custom Editor** pushbutton opens the editor that you specified in your FMEDITOR environment variable.
 7. Click on **Add** to register a resource you have just specified. You can then specify further resources for the widget class, or save the class.
 8. Once you have finished adding new active value resources to the widget class, click on **Save** to save the selected object as a widget class.

8.3.5 C++ Class

An active value whose usage is set as *C++ class* is a member of the C++ class generated in connection with the widget. To edit an active value of this kind, select *C++ class* in the **Usage** field, select the options and enter the necessary data, then click the **Add** button.

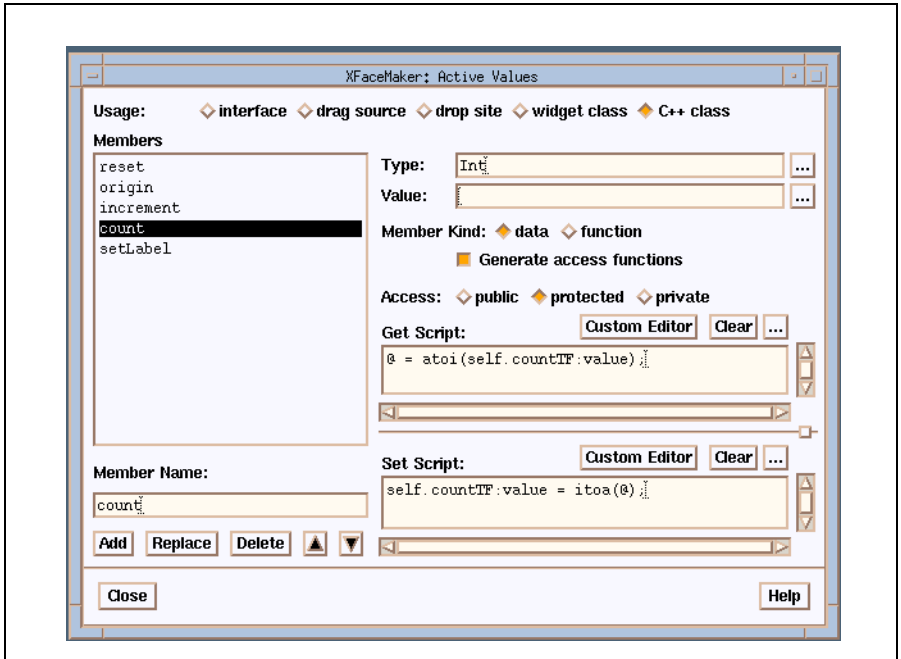


Figure 8.5: Editing a C++ Class Type Active Value

See Chapter 15 for a complete description of this box.

8.4 Active Value Functions

The following functions can be used from the application to activate the script of one or more active values. A full list of functions can be found in the *XFaceMaker Reference Manual*. The new active value functions are available to application programmers to interact with new-style active values. Of course, the old (version 2.1) functions are still available and work with “old-style” active values.

8.4.1 New Active Value Functions

```
int FmAttachValue(  
    widget    widget,  
    char      *name,  
    XtPointer address)
```

This function attaches an address to an active value not using immediate storage for a given widget. If immediate storage is used, the address is the contents of the active value. If *widget* is null, this is equivalent to **FmAttachAv** and the function returns 1. Otherwise, if the active value has per widget storage, the address is attached to the active value for the given widget only. If the active value has global storage, the address is attached globally as with **FmAttachAv**. In both cases, the return value is 2. If the active value does not exist, the function returns 0. This function is the new version of **FmAttachAv**, and should be used instead of it for per widget active values.

```
int FmCallValue(  
    widget    widget,  
    char      *name,  
    int       count,  
    ...)
```

This function stores its *count* last arguments in a structure (starting at field 1), and calls the *set* script of the specified active value with the address of the structure passed as the \$ argument for an immediate active value, and as @ otherwise. The function returns the first field of the structure, which is initialized to 0. Calling this function is similar to executing an instruction of the form:

```
<widget>:<name>( ... );
```

in a FACE script.


```
int  FmChangeValueAddress(  
    Widget    widget,  
    char      *name,  
    XtArgVal  value,  
    String    type)
```

This function changes the address attached to the specified active value and associated with the specified widget, and calls the *set* script of the active value with the new address passed as the \$ argument. If it succeeds, the function returns 1. If the active value does not exist, the function returns 0. If *widget* is null, the function returns -1. Calling this function is similar to executing an instruction of the form:

```
<widget>:$<name> = value;
```

in a FACE script. See **FmWriteValue** for the meaning of *type*.

```
Boolean FmFetchValue(  
    Widget    widget,  
    char      *name,  
    XtPointer *address_return,  
    XrmQuark  *type_return,  
    int       *storage_return,  
    int       *scope_return,  
    int       *immediate_return,  
    int       *automatic_return,  
    int       *genfun_return)
```

This function returns the attributes of a given active value for a given object. If the active value is automatic, not immediate and it is not yet attached, the active value storage is allocated. The contents of *storage_return* is one of the constants **FM_AV_GLOBAL**, **FM_AV_OBJECT** or **FM_AV_NONE** defined in **Fm.h**. The contents of *scope_return* is one of the constants **FM_AV_PUBLIC**, **FM_AV_PRIVATE**, or **FM_AV_PROTECTED** defined in **Fm.h**. *Immediate_return*, *automatic_return*, and *genfun_return* contain 0 or 1. The function returns True if the active value exists, False otherwise. This is the new version of **FmGetActiveValueAddr**, and should be used instead of it.

```
int  FmFetchValueAddress(  
    Widget    widget,  
    char      *name,  
    XtArgVal  *value_return,  
    String    *type_return)
```

This function fetches the address attached to the specified active value and associated with the specified widget, calls the *get* script of the active value with the address passed as the \$ argument, and returns the address. The type returned is the type of the active value if it is immediate, or **Pointer** otherwise. If it succeeds, the function returns 1. If the active value does not exist, the function returns 0. If *widget* is null, the function returns -1. Calling this function is similar to executing an instruction of the form:

```
value = <widget>:$<name>;
```

in a FACE script.

```
Boolean FmGetValue(  
    Widget    widget,  
    char      *name,  
    XtPointer address)
```

This function calls the *get* script of the specified active value for the specified widget, passing it the specified address as the \$ argument. This function is similar to **FmGetActiveValue**, but allows you to specify the attached address for each call. The original address remains valid after the call.

```
int  FmReadValue(  
    Widget    widget,  
    char      *name,  
    XtArgVal  *value_return,  
    String    *type_return)
```

This function fetches the address attached to the specified active value and associated with the specified widget (possibly allocating a memory location of size `sizeof(XtArgVal)` if the active value is automatic), calls the *get* script of the active value with the address passed as the \$ argument, and returns the contents of the address. The type returned is the type of the active value. If it succeeds, the function returns 1. If the active value does not exist, or if it is not attached and it is not automatic, the function returns 0. If it is immediate, or if *widget* is null, the function returns -1. Calling this function is similar to executing an instruction of the form:

```
value = <widget>:@<name>;
```

in a FACE script.

```
Boolean  FmSetValue(  
Widget   widget,  
char     *name,  
XtPointer address)
```

This function calls the *set* script of the specified active value for the specified widget, passing it the specified address as the \$ argument.

This function is similar to **FmSetActiveValue**, but allows you to specify the attached address for each call. The original address remains valid after the call.

```
int      FmWriteValue(  
Widget   widget,  
char     *name,  
XtArgVal value,  
String   type)
```

This function fetches the address attached to the specified active value and associated with the specified widget (possibly allocating a memory location of size `sizeof(XtArgVal)` if the active value is automatic), converts *value* to the type of the active value if *value* is of type `String`, stores *value* as the contents of the active value address, and calls the *set* script of the active value with the address passed as the \$ argument. If it succeeds, the function returns 1. If the active value does not exist, or if it is not attached and it is not automatic, the function returns 0. If it is immediate, or if *widget* is null, or if the value cannot be converted to the active value type, the function returns -1. Calling this function is similar to executing an instruction of the form:

```
<widget>:@<name> = value;
```

in a FACE script.

8.4.2 Old-style Active Value Functions

```
int    FmAttachAv(  
    char    *name,  
    caddr_t  address)
```

FmAttachAv associates the application variable whose address is specified to the active value called *name*. If *name* exists already, it returns 1, else 0.

```
Boolean    FmGetActiveValue(  
    Widget    w,  
    char    *name)
```

Called from the application, executes the *get* script of active value *name* for widget *w*. If *name* is null, the *get* scripts of all the active values of widget *w* are executed, whatever their name. If *w* is null, the *get* scripts of all the active values named *name* are executed, regardless of the widget they belong to. If the script contains the special variables @ or \$, the application variable is updated. If at least one script is executed, the function returns TRUE, else FALSE.

```
Boolean    GetActiveValue(  
    Widget    w,  
    char    *name)
```

Called from the interface, has the same action as **FmGetActiveValue**.

```
XtPointer    FmGetActiveValueAddr(  
    char    *name)
```

FmGetActiveValueAddr returns the address attached to the active value named *name*, or 0 if no address was attached to *name*. Called from the application, it returns the address of the active value defined by the string *name*.

```
XtPointer av_addr = FmGetActiveValueAddr ("SlideValue");
```

```
Boolean   GetActiveValueAddr(  
String    active_value_name)
```

Called from a FACE script, it returns the address of the active value defined by the string *active_value_name*.

```
av_addr = GetActiveValueAddr("SlideValue");
```

Which can also be written:

```
av_addr = $SlideValue;
```

FmGetActiveValueAddr and **GetActiveValueAddr** can access variables which have been declared **static** in another application module.

```
Boolean   FmSetActiveValue(  
Widget    w,  
char      *name)
```

Called from the application, executes the *set* script of active value *name* for widget *w*. If *name* is null, the *set* scripts of all the active values of widget *w* are executed, whatever their name. If *w* is null, the *set* scripts of all the active values named *name* are executed, regardless of the widget they belong to. If the script contains @ or \$ the interface resource to which the active value is assigned will be updated. If at least one script is executed, returns 1 else 0.

```
Boolean   SetActiveValue(  
Widget    w,  
char      *name)
```

Called from the interface, has the same action as **FmSetActiveValue**.

8.5 Using Active Values—Examples

This section illustrates how to use active values.

8.5.1 Defining an Active Value

In this example, we define an active value for an `XmScale` widget. The active value named `ScaleValue` is equivalent to the `value` resource of this widget. It would have the following *get* script:

```
@ = self:value;
```

which places the resource value into the active value, or *gets* the resource from the interface. The complementary *set* script:

```
self:value = @;
```

sets the resource in the interface, with the contents of the active value. The application declares, and may initialize, the variable that is to hold the value:

```
int scale_value = 2;
```

and the application and the interface are linked with the call:

```
FmAttachAv("ScaleValue", &scale_value);
```

which connects the name with the data address. To initialize the widget from the application, use:

```
FmSetActiveValue(0, "ScaleValue");
```

which would set the `scale_value` resource to 2. To get the new value entered by the user moving the scale requires:

```
FmGetActiveValue(0, "ScaleValue");
```

after which `scale_value` might be 5. The basic scripts above can be used for all variables of basic type which can be set by simple assignment: Integer, Pointer, Widget, etc.

8.5.2 Manipulating strings

As in C itself, it is not possible to use simple assignments to manipulate strings. Instead, use pointers and functions such as `strcpy`, `strcat`, and `strcmp`. Therefore, use the variable `$` as a pointer to the active value when dealing with strings. For example, for an `XmText` widget with an active value `Name` equivalent to the `value` resource of this widget, the application must declare a character array to hold the text:

```
char Name[1024];
```

```
FmAttachAv("Name", Name);
```

where `Name` is equivalent to `&Name[0]`. It would have the following *get* script:

```
name = self:value;
strcpy($, name);
free(name);
```

which copies the resource value into the active value. Use a temporary variable `name` to hold a pointer to the copy which can be freed after use. For a further discussion of `XmText` widgets and `XmString` resources see the chapter on Memory Management in the *XFaceMaker Reference Manual*. The *set* script is:

```
self:value = String($);
```

where the resource is set by the widget copying the text from the active value using the pointer. The typecaster `String` tells FACE that `$` points to a string as appropriate for the `XmText` `value` resource. If you had declared the active value to be a pointer:

```
char *name_ptr;
name_ptr = "Name of Customer";
FmAttachAv("Name", &name_ptr);
```

then the following *get* script would be correct:

```
tmp = self:value;
@ = tmp;
```

This assigns to the active value the pointer to the copy of the resource. Because `XmText` returns a copy of the buffer, the application should free the buffer when it no longer needs it. The associated *set* script is:

```
self:value = String(@);
```

where the resource is set by the widget copying the text using the pointer held in the active value. When using `$` to retrieve a string, e.g. the return value of the function `TextGetString`, you cannot say:

```
$ = TextGetString(w);           wrong
```

because `$` is a *read only variable*. But you can write:

```
strcpy($, TextGetString(w));
```

Although better still would be:

```
text = TextGetString(w);
strcpy($, text);
free(text);
```

because the function returns a pointer to a temporary buffer containing the text.

8.5.3 Getting a Widget ID

The simplest use of active values is to get a widget ID from the interface. This method has the advantage of being independent of the name or position of the widget:

```
Widget PopupTLShell;
FmAttachAv("PopupTLShell", &PopupTLShell);
```

with the *get* script:

```
@ = self;
```

A set script is not defined. Thus

```
FmGetActiveValue(0, "PopupTLShell");
```

loads the application variable `PopupTLShell` with the widget ID.

If we wanted the widget ID within a FACE script from the application we could use an active value that has a name, `PopupTLShell`, but *no* get or set scripts defined.

```
Widget PopupTLShell;
PopupTLShell = FmLoadCreate("PopupTLShell", "PopupTLShell",
                           parent, NULL, 0);
FmAttachAv("PopupTLShell", PopupTLShell);
```

The following `activateCallback` script would be valid:

```
Show(@PopupTLShell);
```

8.5.4 Defining Multiple Active Values

Several active values may be associated with a single widget. Moreover, with one restriction, a single active value can be associated with many widgets. This is useful when starting an application in which every widget that requires some form of initialization can be associated with the active value `Start` and have an appropriate *set* script. Then, on commencing, the application can initialize all the widgets with one call:

```
FmAttachAv("Start", dummy);
FmSetActiveValue(0, "Start");
```


The restriction mentioned above is that the use of `GetActiveValue` or `FmGetActiveValue` on `Start` will cause a multitude of *get* scripts to be called, if they exist, thereby leaving `@` and `$` in an undetermined state.

8.5.5 Using Per-Widget and Per-Call Active Values

This example is part of the `Edit` application which is included in the `XFaceMaker` distribution. The application implements a very simple text editor. To load the interface, open `Edit.fmp`. The example shows how *per-widget* and *per-call* active values are used to define pseudo-resources and methods, and build easy-to-use interface modules.

The example is a file selection module. It consists of a Motif file selection box to which active values have been added to make it easier to use in an interface. It is contained in the file `FileSel.fm` which is listed at the end of the section..

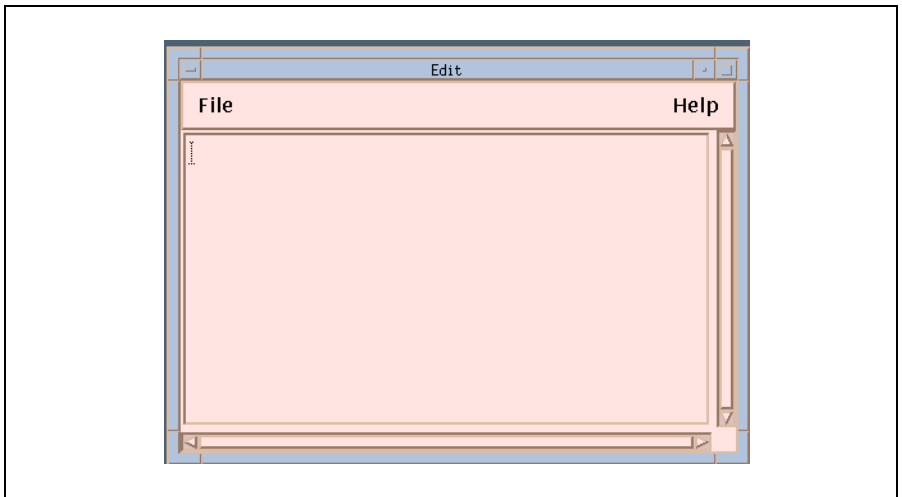


Figure 8.6: The “Edit”Main Window

Three active values are defined:

- fsFile
- fsFinished
- GetFileName

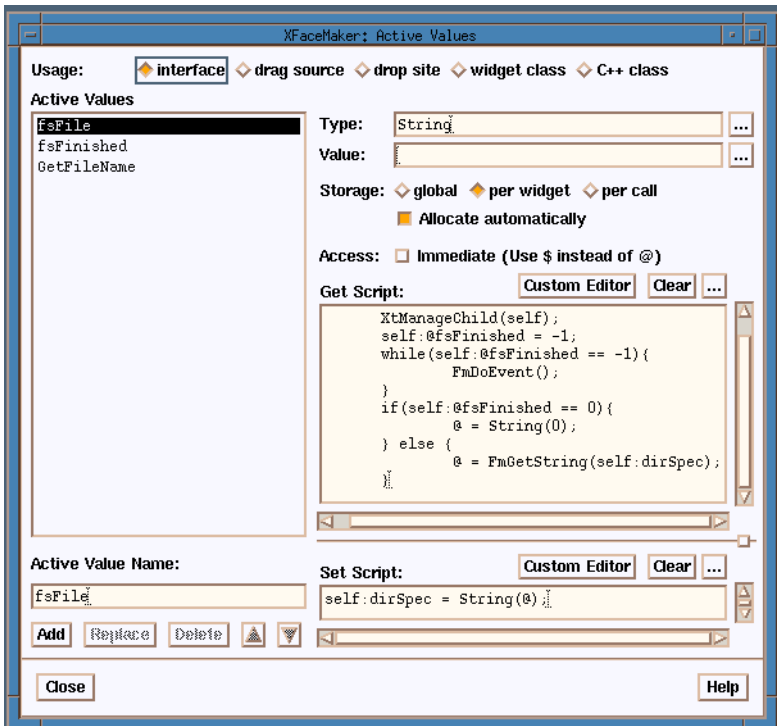


Figure 8.7: The Active Values for “Edit”

The `fsFile` active value is used to pass a default file name to the file selector, and to retrieve the file name selected by the user. It is defined as follows:

- The type of the active value is String.
- It is a *per-widget* active value because there may be more than one instance of the file selector module in the application, so each file selector must have its own storage for the file name.
- The “Allocate automatically” toggle is set: the storage for this active value will not be allocated by the application, but is provided by XFaceMaker. This means that the file selector is a completely independent and self-contained module which does not need anything from the application side and can be re-used as is in another interface. You could define it as a template and instantiate it from the Templates Toolkit.
- The “Immediate” toggle is not set: the active value will be accessed in FACE scripts using the `@` special variable which, in this case, will designate a String pointer allocated automatically by XFaceMaker.
- The `get` script will be called when the active value is read. It will handle all the interactions needed to let the user choose a file name. It will:
 - m manage the file selection box (if the file selector is inside a Motif DialogShell, this will also pop the dialog shell),
 - m wait until the user is done selecting a file name: the script enters a local event loop by repeatedly calling the Fm library function `FmDoEvent` (which dispatches one X event to the toolkit) until the auxiliary active value `fsFinished` is set, indicating that the user pressed the **OK** or **Cancel** button,
 - m store the selected file name (or 0 if the user pressed **Cancel**) in the active value’s storage `@`.
- The `set` script is used to initialize the default file name: it displays the file name by copying it to the Motif file selection box’s `dirSpec` resource.

The `fsFinished` active value is an auxiliary value used to record the state of the file selector during interaction. It is of type `Int`, it also has *per-widget* storage, it is allocated automatically, and it defines no side-effects in its `get` or `set` scripts. It is set by the `okCallback` and `cancelCallback` of the Motif file selection box as follows:

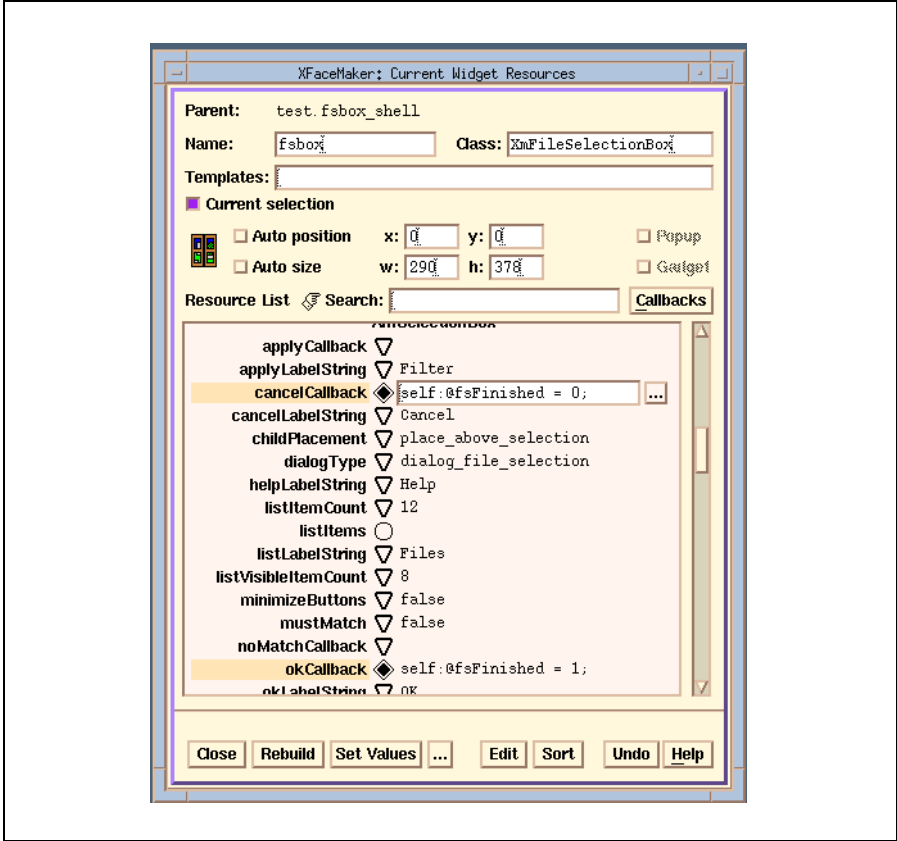


Figure 8.8: The Resources List

With these two active values, the file selection module is ready to use. For example, one could use the following in a FACE script to pop the file selector and obtain the selected file name:

```
root*fsbox:@fsFile = "unnamed";
String filename = root*fsbox:@fsFile;
if(filename)
    LoadFile(filename);
...
```

The `GetFileName` active value makes it even easier to use the file selector. It is a *per-call* active value, which means that it is called like a function. The difference between a *per-call* active value and a function is that an active value is associated to a widget, thus several *per-call* active values with the same name can be defined for different objects with different side-effects.

The type of the active value is `GetFileNameArgs`. It is the name of a structure, defined in the *set* script, which holds the arguments of the active value and its return value. Here, the return value *ret* (the name can be arbitrary, the return value is always the first field of the structure) is of type `String`, and the arguments are *current* (of type `String`) which will be the default file name and *pattern* (of type `String`) which will be the filtering pattern.

When the active value is called, the arguments are stored in a temporary `GetFileNameArgs` structure, and the address of the structure is passed to the *set* script as its *@* argument. The *set* script then fetches the arguments by initializing a variable of type `GetFileNameArgs` with the *@*¹ value, and accessing the structure fields.

Here, the *pattern* argument is copied into the Motif file selection box's *pattern* resource, and the *current* argument is copied into the `fsFile` active value. Finally, the `fsFile` value is read to pop the file selector and get the selected file name, which is returned by storing it into the *ret* field of the structure.

The `GetFileName` active value can be called as follows:

```
String filename = root*fsbox:GetFileName("unnamed", "*");  
...
```

1. The argument structure is contained in *@* if the “Immediate” toggle is not set. If the “Immediate” toggle is set, the argument structure is in *\$*. There is no other difference between immediate and non-immediate *per-call* active values.

8.5.6 FileSel.fm

```
widget XmDialogShell {
    flag {
        popup
    }
    resources {
        x = 261
        y = 352
        width = 290
        height = 378
    }
    children {
        widget XmFileSelectionBox {
            flag {
            }
            activevalue {
                name = fsFile
                get =
                XtManageChild(self);
                self:@fsFinished = -1;
                while(self:@fsFinished == -1){
                    FmDoEvent();
                }
                if(self:@fsFinished == 0){
                    @ = String(0);
                } else {
                    @ = FmGetString(self:dirSpec);
                }

                set = self:dirSpec = String(@);
                type = String
                storage = object
                automatic = true
            }
            activevalue {
                name = fsFinished
                type = Int
                storage = object
                scope = protected
                automatic = true
            }
            activevalue {
                name = GetFileName
                set = struct GetFileNameArgs {
```

```
        String ret;
        String current;
        String pattern;\
};\
GetFileNameArgs a = @;\
self:pattern = a->pattern;\
self:@fsFile = a->current;\
a->ret = self:@fsFile;
        type = GetFileNameArgs
        storage = none
        automatic = true
    }
    resources {
        x = 0
        y = 0
        width = 290
        height = 378
        defaultPosition = false
        dialogTitle = File Selector
        autoUnmanage = true
        okCallback = self:@fsFinished = 1;
        cancelCallback = self:@fsFinished = 0;
        dialogStyle = dialog_full_application_modal
    }
} fsbox
}
} fsbox_shell
```

CHAPTER 9: Modules: Groups and Templates

You can define three different types of reusable components or modules in XFace-maker: groups, templates, and classes. Although similar, these modules are used for different purposes. Groups and templates are described in this chapter. Classes are described in Chapter 13.

Groups A group is a collection of widgets that can be used as though it were a single widget. Groups have an important characteristic —instances of a group are *copies* of the original object. This means that when a group is modified, the instances of that group are not modified. Once a group is placed in an interface it is no longer recognizable to XFaceMaker as such, but acts like a collection of separate widgets.

Groups are most useful when an application needs to instantiate several copies of the same interface fragment during execution.

When a group is loaded in XFaceMaker, it is added to the Groups widgetstore.

Templates A template is a kind of *model* or pattern that can be instantiated, like groups, but can also be applied to other objects. Templates are similar to groups, but they add two features:

- m Changing the definition of a template will change all instances of that template throughout the interface that is currently loaded in XFaceMaker, as well as any widgets to which that template has been applied.
- m Templates can be used to define and enforce *style rules* during the design of a user interface.

Templates can be used like groups to instantiate several copies of an interface fragment. In compiled mode they behave exactly like groups. In interpreted mode, however, and within XFacemaker, they have the specific template behavior.

When a template is loaded in XFaceMaker, it is added to the Template widgetstore.

9.1 Using Groups

A *group* is a collection of widgets that can be used as though it were a single widget. Therefore, it is necessarily a widget or a widget and its descendants. Groups supplement the standard widget set and can be used in other interfaces. Groups can also be used to create modules of which several instances are loaded in the interface. Groups can be loaded dynamically by the application, as they are needed. This can dramatically reduce the load time of applications.

Groups allow you to make multiple copies of a module, either interactively, or programmatically, but they have an important drawback—instances of a group are merely *copies* of the original object. This means that when a group is modified, the instances of that group are not modified. Once a group is placed in an interface it is no longer recognizable to XFaceMaker as such, but acts like a collection of separate widgets. In general, groups should be created for the following purposes:

- Enabling an application to create multiple instances of the same module by multiple calls to `FmCreateObject` (or its equivalent). This is the most frequent use of groups.
- Defining reusable modules on a *per-session* basis. A group can be considered as a very short-lived reusable module, meant to be used during a small number of XFaceMaker sessions, and then forgotten. In this view, groups are only a quicker version of cut-and-paste.
- Providing library members of common interface elements which can be edited further before use.
- Dividing a large interface into smaller interface elements. But this is better done by building a *Project*. See Chapter 12, Building Projects.
- Creating variable numbers of interface elements at run-time.

Using groups will not reduce memory requirements, but their use gives a small executable and allows you to produce an interface faster. Several pre-defined groups are available for loading into XFaceMaker. They are in the `$NSLHOME/lib/xfm/Fm` directory. Some of the pre-defined groups are:

`ScrollList.fm` which is equivalent to `XmScrolledList`

`ScrollText.fm` which is equivalent to `XmScrolledText`

If you wish to use these or other predefined groups, load them into XFaceMaker using the **Modules** --> **Group** menu. They will be accessible for instantiation through the Groups widget store.

In order to have these groups loaded automatically every time XFaceMaker is started, use the **Save Pref's** command, after having loaded the groups. This creates a new `.xfm.startup` file in which the commands to load the groups are included. If you want remove groups from automatic startup, delete the groups you want to remove through the **Modules --> Group --> Delete** command, then use the **Save Pref's** command.

Section 9.1.2. lists the full collection of pre-defined groups. Once a group is placed in the Groups widgetstore, it can be used just like any other widget.

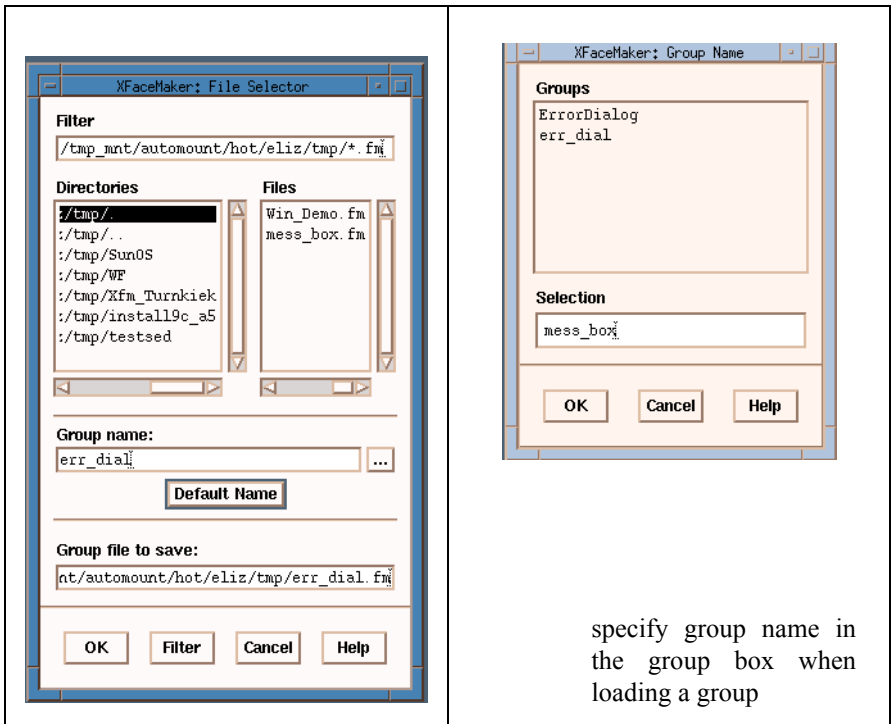


Figure 9.1: The File Selector specify group name and file in the file selector when saving a group

9.1.1 Defining a Group

A group can be a single widget, or a widget and its children. A group is defined by selecting the topmost widget of a subtree in the widget hierarchy, and then applying the commands in the Modules menu. Use the commands in the Groups sub-menu of the Modules menu to do the following.

To save the currently selected sub-tree (the current widget and any of its children) as a group:

1. Select the topmost widget of the group subtree. You can select the widget in the interface, the Widget List, or the Widget Tree. To define the whole interface as a group, select the highest *shell* widget.
2. Click on **Save As** in the Groups sub-menu to pop up the File Selector. If the group is to be used from a different directory, you must save it with a relative path name (by removing the directory from the file name in the File Selector when you save the group) before you do **Save Prefs** and use the FMFILEPATH environment variable to tell XFaceMaker in which directories it should look for the file.
3. In the "Group file to save" field, specify the name of the file (giving it the .fm extension) in which to store the group descriptor file.
4. In the "Group name" field, specify the name to identify the group in the widgetstore, and click **OK**. This name will appear in the widgetstore when you move the cursor to that group's icon. This is usually the same as the file name (use the default name) but can be different.

Once defined, the group is added to the Groups widgetstore and can be used just like any standard widget. A default icon for the group is added to the Groups widgetstore automatically. The group file is saved in the same format as a standard interface file, but the mapped flag is not set.

To load a previously defined group file into the Groups widgetstore.

1. Click on **Load** in the Groups sub-menu to pop up the File Selector.
2. Specify the pathname of the group descriptor file (a .fm file) which is to be loaded, and click **OK**. Then, if the file is found, the Group Name box pops up.
3. Specify the name of the group as it is known to XFaceMaker. (This is the name entered in the "Group name" field when saving the group as explained above.) XFaceMaker loads the group and a default icon---or the icon you specified for the group---appears in the widgetstore.

Previously defined groups can also be loaded in the widgetstore globally, using the **Save Prefs** command in the File menu. You must be careful not to subsequently change the pathnames of group files, as XFaceMaker will have no way of looking up the new paths.

If the group is to be used from a different directory, you must save it with a relative path name (by removing the directory from the file name in the file selector when you save the group) before you do a **Save Prefs**, and use the FMFILEPATH environment variable to inform XFacemaker how to locate the files.

To specify an icon to represent a group in the widgetstore.

1. Click on **Choose Icon...** in the Groups sub-menu. The Group Name box pops up.
2. Specify the name of the group, and click **OK**. The Icon Selection Box pops up. See Section 5.6 on how to specify pixmaps.
3. Create a new pixmap or choose an existing pixmap to represent the group, and click **OK**. The newly defined icon is immediately displayed in the Groups widgetstore.

To delete a group from the Groups widgetstore.

1. Select **Delete...** from the Groups sub-menu. The Group Name box pops up.
2. Specify the name of the group to delete. This is the name under which the group is known to the widgetstore, not the name of the group file.

This removes the group from the widgetstore. The group file, however, remains unchanged, and the group can be re-loaded later. If instances of the group have been added to the interface before the group is deleted, they remain unchanged.

9.1.2 Predefined Dialog Groups

XFaceMaker comes with a collection of predefined groups. The following is a list of the files contained in the XFaceMaker library directory `Fm`, which is installed as `$NSLHOME/lib/xfm/Fm`. Each `.fm` file contains a group which creates a simple dialog box. The file name is followed by the equivalent OSF/Motif library function it mimics. Group files have the same resources set as when created by the library functions where these are not the normal defaults of the widgets, except for a few important ones which are defined for clarity.

These predefined groups can be loaded into the Groups widgetstore, using the **Load** command in the Groups sub-menu, or they can be loaded as standard interfaces using the **Open** or **Read** commands in the File menu.

Group File Name	Equivalent Motif Function
ErrorDialog.fm	XmCreateErrorDialog
FileSelDial.fm	XmCreateFileSelectionDialog
InfoDialog.fm	XmCreateInformationDialog
MessageDial.fm	XmCreateMessageDialog
PromptDial.fm	XmCreatePromptDialog
QuestioDial.fm	XmCreateQuestionDialog
ScrollList.fm	XmCreateScrolledList
ScrollText.fm	XmCreateScrolledText
SelectDial.fm	XmCreateSelectionDialog
WarningDial.fm	XmCreateWarningDialog
WorkingDial.fm	XmCreateWorkingDialog

Table 9.1: Predefined Group Widgets

9.1.3 Predefined Interface Elements

The following files are contained in the XFaceMaker library directory `Fm`, which is nstalled as `$NSLHOME/lib/xfm/Fm`. Each `.fm` file contains a group which creates certain interface elements. Some have an equivalent Motif library function

which they mimic. Others have been created to provide basic pre-fabricated interface elements. Their use is slightly more difficult as they are not based on a Shell widget, and therefore the third column in the list states the type of the parent widget they require. The group files which mimic a Motif function have the same resources set as those created by the library functions.

Group File Name	Equivalent Motif Function	Parent Widget
CheckBox.fm	XmCreateCheckBox	XmBulletinBoard
MenuBar.fm		XmForm
OptionMenu.fm	XmCreateOptionMenu	XmBulletinBoard
PopupMenu.fm	XmCreatePpopupMenu	XmBulletinBoard
RadioBox.fm	XmCreateRadioBox	XmBulletinBoard
GroupIcon.fm		

Table 9.2: Predefined Group Interface Elements

Comments:

- The `CheckBox` has two buttons.
- The `MenuBar` follows the OSF/Motif Style Guide recommendations for a Menu Bar.
- The `OptionMenu` has two options.
- The `PopupMenu` has two items.
- The `RadioBox` has two buttons.
- `GroupIcon` is used by `XFaceMaker` internally and should not be changed.

For each group described above there is a corresponding bitmap in the `XFaceMaker` library directory `bitmaps`.

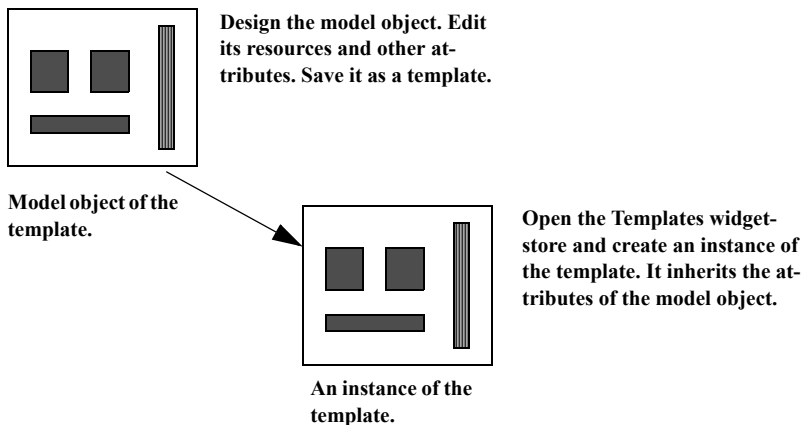
9.2 Templates

A template is a kind of *model* or pattern that can be applied to other objects, called the *instances* of the template. When you create an instance of a template, the instance inherits the attributes of its template. For example, you could design a pushbutton with specified resources and save it as a template. Then all the instances of that pushbutton template will inherit the template resources.

Templates are similar to groups, but they add two important features:

- Changing the definition of a template will change all instances of that template throughout the interface.
- You can define and enforce *style rules* by applying a template to other objects when you are designing the interface.

In addition, if the template is a composite object, the sub-objects of the template are also copied as children of the template instance. For example, you can define a dialog box consisting of a composite widget containing child widgets such as pushbuttons and scrollbars. You can then save this dialog box as a template and create instances of it. Every child inherited from a template is itself an instance of an internal template, of which the model object is the corresponding child of the master template.



An instance inherits the following attributes from its template:

- resource values (including callback scripts). See Section 9.2.7

- active values,
- the creation script,
- the values of the following flags, if they are set in the template: `mapped`, `popup`, `gadget`, `menu_pulldown`, `menu_option`, `menu_popup`, `menu_bar`, `dialog_shell`.

XFaceMaker distinguishes which attributes of a template instance are inherited from its template, and which are specific to the instance itself.

9.2.1 Defining a Template

A template is defined by an object which is the model for the template. To define a template, first select the model object, then use the commands in the Templates sub-menu of the Modules menu. Once the template is defined, instances of it can be created by clicking on its icon in the Templates widgetstore. To define a template:

1. Select the top widget of the interface object that will serve as the model for the template. To define the whole interface as a template, select the highest shell widget. If you select a composite object, all sub-objects will be included in the template.
2. Select **Save** or **Save As** from the Templates sub-menu in the Modules menu. This pops up the File Selector.
3. In the "Template file to save" text field, enter a file name. If the template is to be used from a different directory, you must save it with a relative path name (by removing the directory from the file name in the file selector when you save the template) before you do a **Save Prefs**, and use the `FMFILEPATH` environment variable to enable XFacemaker to locate the files.
4. In the "Template name" field, enter a name for the template. Or click on the "Default name" pushbutton, which specifies the file name as the default template name.
5. Select a "Template mode". See Section 9.2.7.
6. Click on **OK**. XFaceMaker saves the object description into a `.fm` file, and places a default icon in the Templates widgetstore. Select **Save Prefs** in the File menu to store the template in your `.xfm.startup` file for automatic re-loading in subsequent XFaceMaker sessions.

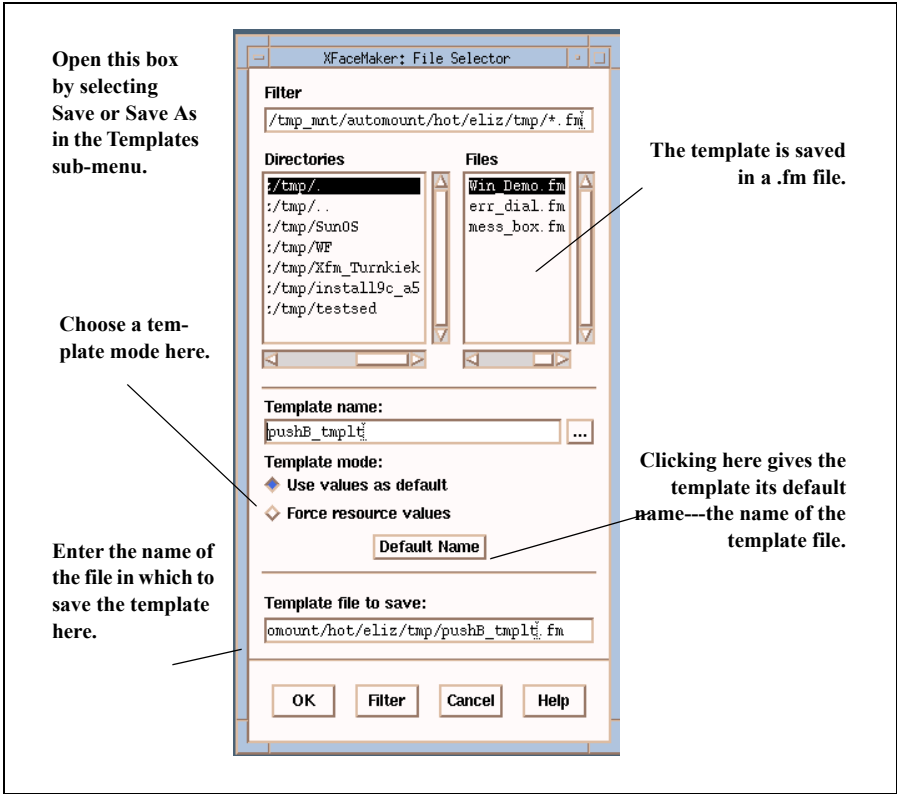


Figure 9.2: Saving a Template

The difference between *Save* and *Save As* is the following:

- *Save As* allows you to specify a name for the template, and the file in which to save the template, when there is no "current" template file.
- The *Save* command saves the template under the "current" template name and template file specified in the last *Save As* or Edit menu command. If there is no current template name and file, or if the selected object is different from the one selected during the last *Save As* or Edit menu command, an alert box will warn you.

9.2.2 Choosing a Template Icon

You can specify an icon to represent a template in the widgetstore.

1. Click on **Choose Icon...** in the Templates sub-menu. The Template Name dialog box pops up, listing the currently loaded templates.
2. Select the template, and click **OK**. The IconSelectionBox pops up. See Section 5.6 on using this box.
3. Create a new pixmap or choose an existing pixmap to represent the template, and click **OK**. The newly defined icon is immediately displayed in the Templates widgetstore.

9.2.3 Loading an Existing Template

Template files may be loaded automatically each time XFaceMaker is launched, or they can be loaded per-session.

- Templates can be automatically re-loaded each time XFaceMaker is launched. To do this, select the **Save Prefs** command in the File menu before quitting the XFaceMaker session. All templates from your current session will be stored in your `.xfm.startup` file.
- Templates can be loaded by the user at any time using the **Load** command in the Templates sub-menu.

To load a template using the commands in the Templates sub-menu:

1. Click on **Load** in the Templates sub-menu to pop up the File Selector.
2. Specify the pathname of the template description file (a `.fm` file) which is to be loaded, and the template name, and click **OK**. XFaceMaker loads the template, and a default icon (or the icon you specified for the template, if one exists) appears in the widgetstore.

9.2.4 Deleting a Template

Deleting a template removes it from the Templates widgetstore and un-applies it to all instances in the interface. However, the template file remains unchanged, and the template can be re-loaded for use later.

To delete a template:

1. Click on **Delete** in the Templates sub-menu. The Template Name box pops up listing the templates in the current interface. The currently loaded templates are listed in the box.
2. Select the template you want to delete. This is the name under which the template is known to the widgetstore, not the name of the template file which has the .fm suffix.
3. Click on **OK**. The template is immediately deleted from the widgetstore. However, its description file remains unchanged and can be re-loaded.

9.2.5 Modifying A Template

You can modify an existing template to change its mode or other attributes. There are two methods:

- Load the template file using the **Read** command in the File menu. Select the object and make the modifications. Use **Save As** in the Templates sub-menu to save the modified template.
 - Use the **Edit** command in the Templates sub-menu to load and select the template. The **Edit** command is in fact a short-cut. It is equivalent to loading the template file using the **Read** command and selecting the object. You can save the modified template with the same name using the **Save** command.
1. Click on **Edit**. The Template Name box pops up.
 2. Enter the name of the template you wish to modify. The selected template file is loaded, and the model object is mapped on the screen and selected, so that it can be modified.
 3. Modify the model object and save it.

Do not forget to delete the template model created by **Edit** or **Read** before you save the rest of your interface.

9.2.6 Editing a Template Instance

An instance of a default mode template can be edited just like any other widget in XFaceMaker. You can, for example, specify new resources or active values for that particular instance. You can also add new children to an instance of a composite template, and edit the inherited children. However, you cannot remove children inherited from a template.

However, you *cannot* change the resources inherited from an override mode template. If a composite template is in override mode, its sub-objects are also in override mode. Therefore, the resources of the children cannot be changed. See the next section for more on template modes.

To modify an instance, create it by clicking on its icon in the Templates widgetstore, in the same way you would create a widget or an instance of a group. If the toplevel object of the template is not a shell, a `TopLevelShell` will be created with the template model inside it.

9.2.7 Template Resources

There are two template modes, *default* and *override*. They define how the resources of the template will be applied to its instances:

- In **default** mode ("Use values as default"), the resources defined in the template are used as default values, and can be changed in the instance.
- In **override** mode ("Force resource values"), the resources of the template cannot be changed in the instance. If you select a resource inherited from an override template, an alert box will inform you that the resource cannot be changed.

The choice of a mode only concerns a template's resources, active values, and creation script. The flags are always copied from the template.

Some template resources available in the Widget Resources box must not be set. For example, the `x`, `y`, `w`, and `h`, coordinates must never be set because then all instances of the template would have exactly the same coordinates—clearly an undesirable option! If "autopos" or "autosize" is set in the template, the `x,y,width,height` will not be inherited by the instance. But "autopos/autosize" will not be inherited (except for children inherited from composite templates).

In the Resource List, inherited resources are marked with a special icon. In the XFaceMaker interface, attributes that are inherited from a template and that cannot be changed are preceded by two stars (**). Inherited attributes that can be changed, such as the resources inherited from a default template, are preceded by one star (*). This feedback is also present in the following windows:

- the Resource Editor box,
- the Active Values box,
- the Widget List (the names of the sub-widgets inherited from the template are marked with **),
- the Widget Tree (the names of the sub-widgets inherited from the template are marked with **),
- the name field in the Widget Resources window (marked with ** if the widget is a sub-widget inherited from a template).

Resources inherited from a *default* template are marked with an asterisk or icon in the Resource Editor only if they have their default value. If the value is changed, the resource is displayed normally in the Resource Editor, although it is still marked with a star in the resource list.

The option "Show Template Resource" in the Sort window of the Resource window controls whether template resources are listed or not.

The name of the template for the edited object is displayed in the **Template** text field in the Widget Resources box. This field is editable, allowing you to change the template of an object. Clearing the field removes the template from the object. Typing a template name for an object that has no template applies the template to the object, as if the object had been created using the template icon. Applying a template is described in section 9.2.8.

9.2.8 Applying a Template

You can apply a template to an existing object. In this way, templates can be used as *style sheets*. For example, a template with a given font could be applied to all the text widgets of an interface, in order to enforce a homogeneous appearance.

- *Applying* a template means that the template is attached to the object, as if the object had been created as an instance of the template, and the object is re-built.
- *Un-applying* a template means that the template is removed from the object. All inherited attributes are removed except for the object flags (popup, etc.) which remain as set by the template, and the children inherited from a composite template, which remain as *regular* children.

Since a template can be applied to an object of a different class, XFaceMaker applies only the resources of the template that are appropriate for the target object. You can apply several templates to the same object. In that case, the attributes of each template are applied in turn to the object. If there are conflicts, e.g. if the same resource is specified in several templates, then the last template overrides the previous ones. If an object has several templates, the template names are displayed in the *Template* field of the *Resources* window separated by commas.

The *Edit* menu has two items used to apply/unapply a template to an object: *Apply Template* and *Un-Apply Template*. When you click on one of these items a sub-menu appears, containing the options listed below.

- **To Selected Object** applies the template to every selected object.
- **To Whole Subtree** applies the template to every currently selected object, and to all their descendants.
- **To Widgets Of Same Class** applies the template to every currently selected object, and to all descendants that have the same widget class as the template model object. Gadgets are considered to be the same class as widgets. For menus, the type of the menu is taken into account: a pulldown menu is considered as having a different class from a popup or a *regular* RowColumn.

When you select an option, a prompt box pops up to specify which template you wish to apply/un-apply.

The same commands can be activated directly by pressing the *Custom* mouse button in a template icon. This pops up a menu with two sub-menus, **Apply** and **Un-Apply**, identical to the ones described above. Here, the template name is not requested: the template associated with the icon is applied or un-applied directly.

The model object of a template can itself be an instance of another template: templates can be *chained*. In that case, the templates are applied in *super-template to sub-template* order; i.e., the template of the template is applied before the template itself. Of course, template chains cannot loop: if you apply a template to an object and then try to save the object as the template you applied to it, a dialog box notifies you of the error. Different modes can be mixed when templates are chained: for example, a template in override mode can be an instance of a template in default mode. This allows you to specify both *override* and *default* resources in the same template.

9.2.9 Interfaces Containing Templates

When a file with a template reference is loaded, XFaceMaker loads the template automatically. This is also true for template descriptions. If a template is loaded and the template is itself an instance of another template, the other template is also loaded automatically.

The usual conventions are used to locate the template file if it does not begin with a slash / character: the current directory is searched first, then \$HOME, then the XFaceMaker library directory, then the default directories:

```
/usr/lib/X11/xfm  
/usr/local/lib/X11/xfm  
/usr/local/lib/xfm
```

In addition, if the file does not contain any / character, the template file will be searched for using the path specified by the FMFILEPATH environment variable. See Chapter 22.

In files other than .fm files, i.e., C-code, UIL or .rdb files, template instances are saved as *regular* widgets. All resources, active values, children, etc., inherited from a template or not, are saved for each template instance. Thus, whenever a template is modified, the C, UIL or .rdb files containing objects built using this template must be re-generated.

CHAPTER 10: The Application

XFaceMaker enables you to generate, compile, and test a working application interactively and easily. You can modify the application files directly using a customizable text editor. Applications can be built using `make`.

XFaceMaker generates the application files by applying the information that is specific to your interface to a set of file *patterns*. These patterns contain special comments which describe what must be output for each kind of object, such as application functions, active values, etc. The patterns can be modified to suit specific needs on a per-user, project-wide or system-wide basis.

The XFaceMaker application pattern format allows the programmer to modify some parts of the generated files, but still be able to re-generate the file when the interface is changed; for example, if a new function is added. This is achieved by saving the generation information in the file as comments. This process is described in the following sections.

- To generate application files automatically, see Sections 10.1 and 10.2.
- To write application code for XFaceMaker “by hand” (for existing applications), see Sections 10.3 to 10.6.

10.1 Generating Application Files Automatically

To generate the application files for an interface, use the *Save Appli Files* item in the File menu.

To generate code which does not contain the code preservation comments, and thus is easier to read, make sure that the *Preserve User Code* option in the Options menu is not set. This is the default case.

You can also generate the application files from the command-line, with the `-genappli` option. Thus, the command:

```
xfm -gen Example.fm
```

will generate the application files for the interface contained in `Example.fm`.

To generate the application files, XFaceMaker invokes the **genappli** program with the appropriate options to generate the four files described below:

- a C file called `<Name>Main.c`, containing the `main` function of the application;
- a C file called `<Name>App.c`, containing the skeletons of the application functions as well as the global active value variables.
- a C include file called `<Name>App.h`, containing the declarations of the application functions and active value variables;
- a Makefile called `<Name>Make`, contains rules to build the application.

10.1.1 The Main File

The `main` file contains the calls to:

- initialize the X Toolkit and the XFaceMaker library,
- attach the application functions (in interpreted mode),
- attach the global active values,
- load and map the interface modules (in interpreted mode), or call the creation functions (in compiled mode),
- enter the `main` loop, or call the `FmCallEditor` function to build an extended XFaceMaker.

10.1.2 The Functions File

The functions file contains:

- the declarations of the active value variables corresponding to global active val-

ues of your interface. New-style per widget active values are not saved in the application file. If an active value is typed, the corresponding C type is used when generating the application file; otherwise XtArgVal is used.

- the skeletons of the application functions declared in the FACE scripts of the interface using the **function** keyword. If the argument names were specified in the function declaration, they are used when generating the skeletons.¹

10.1.3 The Include File

The include file contains the external declarations of the variables and functions in the functions file.

10.1.4 The MakeFile

The Makefile contains the rules to build three programs:

- the interpreted version of the application, called **i<name>**, is linked with the **libFm.a** library.
- the compiled version of the application, called **c<name>**, is linked with the compiled interface modules and the **libFm_c.a** library.
- the “editable” version of the application, called **e<name>**, is linked with the **libFm_e.a**, calls the **FmCallEditor** function, and can be used as a Design process when launched with the **-client** option.

10.1.5 The Pattern Files

The pattern files used to generate the four application files are: **PattMain.c**, **PattApp.c**, **PattApp.h**, **PattMake**. These pattern files are located in the **GenAppli** sub-directory. To find them, XFaceMaker uses the data file search rules, i.e. XFaceMaker looks for the **GenAppli** directory in: the current directory, the **\$HOME** directory, the **\$FMLIBDIR** or the installation directory, under **\$NSL-HOME/lib/xfm**.

1. Functions declared with the **function** keyword are global, in XFaceMaker. This means that all the functions you define during a session of XFaceMaker are remembered, and they will be included in the functions file. To reset XFaceMaker’s internal functions table, you can use the **New** button in the **File** menu. This will also clear your design process so you should first save the interface you are working on, activate **New**, then **Open** to load your interface or project, then generate your Appli files. These will contain only the application functions that pertain to your interface or project.

If you want to modify the pattern files, make a copy of the full GenAppli directory in your current directory, or in \$HOME, and modify the copy. You should avoid modifying the distribution pattern files, so as to be able to use the standard patterns if you run into problems with the ones you modified. Or, make sure you have an unmodified copy to fall back on! Section 10.2.2 describes modifying application patterns.

10.2 Building the Application

The application files can be compiled immediately to build a running application, although of course you will later have to add your application code.

To build the application, you can run `make` in an xterm window:

```
make -f <Name>Make
```

The default target in the Makefile builds two versions of your application: interpreted and compiled.

You can also build the editable version of your application directly from XFaceMaker, using the **Build Appli** command in the 'File' menu. The output of the `make` command will be displayed in an XFaceMaker window which pops up automatically.

You can customize the `make` command line by creating an executable file called `xfmmkappli` in the current directory, your home directory or in the XFaceMaker data files directory (by default `/usr/lib/X11/xfm`). If this file is present, it will be executed instead of `make` with the same arguments.

You may have to modify the generated Makefile to compile and/or link your application if your system requires special compilation flags or libraries. However, the generated Makefile includes a platform-specific file which redefines macros such as include and library paths, special compilation flags and compiler options, etc. Application files generated on one of the reference platforms (i.e. platforms for which the NSL CD-ROM contains a distribution) can be compiled immediately on that platform without modifying the Makefile.

To compile the application on another platform, you must either modify the Makefile manually, or redefine the macros `XFM_TARGET_OS` or `XFM_TARGET_MK` as appropriate: `XFM_TARGET_OS` is the name of the target operating system prepended with a "-"; `XFM_TARGET_MK` is the full path name of the include file for the target operating system. Include files exist in the standard distribution for the following operating systems: `aixv3`, `hpux`, `irix-5.2`, `osf1`, `sco`, `solaris`, `sunos`. The corresponding include files have names beginning with `Mk-` and are installed in the same directory as the pattern files.

For example, the following commands would generate and compile an application on a Sun system, then compile it again on an HP-UX system, assuming the platform-specific files are installed in the same location:

```
sunos % xfm -gen Test.fm
sunos % make -f TestMake
hpux % make -f TestMake XFM_TARGET_OS=-hpux
```

If the configuration files were installed in different directories, the following should be used:

```
hpux % make -f TestMake \  
XFM_TARGET_MK=/nsl/hpux/lib/xfm/GenAppli/Mk-hpux
```

The application files take into account the various options with which the interface was saved. For example, if the **Save RDB Also** option was set, the Makefile will load the `.fm_rdb` files instead of the `.fm` files, and an application defaults file will be generated from the `.rdb` files. The **Save UIL Also** option is handled as well.

If a project file was saved for the application, it is used in the Makefile to re-generate all files: C code files, UIL files, RDB files. Note that, since project files include C code and RDB options, the options in the project file override the options in the Makefile. So, when you change the options, remember to save your project file again, or re-generate the application files if you don't use a project file.

The Makefile contains targets to build an application in interpreted, compiled, or editor mode as follows:

```
'make -f XyzMake interpreted'
```

makes the interpreted version of the application (i.e. iXyz plus all the files it needs like application defaults files).

```
'make -f XyzMake compiled'
```

makes the compiled version of the application (i.e. cXyz plus all the files it needs like application defaults files and UID files).

```
'make -f XyzMake eXyz'
```

makes the editor version of the application. The target `'all'` does not make the editor version.

```
'make -f XyzMake cleanall'
```

removes all the files generated by the Makefile, including C code files made from `.fm` files.

If the interface was created using `nslfm` and uses additional NSL or XRT widgets (for example `XnslDraw`, `XRT/Graph`, etc), the Makefile will contain the appropriate libraries and include paths to compile the application properly, and the necessary initialisation function will be called in the main file. As a result, applications can usually be compiled without modifying the application files.

You can modify the application file patterns if you want to change permanently the way `XFaceMaker` generates the application files on your system. See Section 10.2.2.

10.2.1 Modifying Application Files

To build a useful application, you will have to add code to the application files, specify additional object files in the Makefile, etc. There are two main options at this stage: enable or disable the *Preserve User Code* option.

1. You can decide that you will make XFaceMaker generate the application files only once, and then modify them and update them manually or by any means external to XFaceMaker. This solution has the drawback of requiring that you update your application files every time you change your interface. For example, suppose that you have generated your interface files, then you decide that you need a new application function in a callback script. You will have to add the calls necessary to declare and register the new function in the application files. In this case, you disable the *Preserve User Code* option. The application files are generated from the Pattern files, XFaceMaker includes no pattern information in the application files generated.
2. You can decide that you will make XFaceMaker generate the application files as many times as needed but that you will manually copy any application code that you have added to these files from the older version (the files are saved with a ~ suffix) to the more recent version, and will not rely on XFaceMaker to preserve your user code when re-generating the application files. The advantage of this solution as compared to the one described above is that XFaceMaker will automatically add to the application files the calls necessary to declare and register new application functions called from FACE scripts, and declare global active values. The risk is that you forget to copy your application code and lose it. Plus, copying the application code from version to version can be very tedious. As in the previous case, you disable the *Preserve User Code* option in this mode. The application files are generated from the pattern files, XFaceMaker includes no pattern information in the application files generated.
3. The recommended option is to set the *Preserve User Code* option. In this case, you can generate the application files as needed, e.g. every time the modifications made in the interface files imply changes in the application files: adding global active values, adding application function calls. Any application code that you have added to the application files will be preserved by XFaceMaker as it generates new application files, provided you follow a few simple rules when modifying your application files. In this case, you should set the *Preserve User Code* option. The application files are generated from the pattern files the first time (no application files are found in the current directory) and XFaceMaker includes the pattern information in the application files generated. For subsequent generations of the application files, XFaceMaker uses the application files as model, instead

of the pattern files. The advantage of this method is that you do not have to worry about copying your application code from version to version. The disadvantage is that the code is less readable, since the patterns are included in the file. However, once you become familiar with the format of the patterns, you get used to skipping the patterns and soon can read the code easily. If you choose this option, you should make sure that the ***Preserve User Code*** toggle in the Options menu is always set when you generate your application files as XFaceMaker relies on the state of this toggle to decide which model it should use to generate the application files.

Once you have decided whether to use the preserve user code option, you should set the ***Preserve User Code*** toggle as desired and save this information using the ***Save Prefs...*** command so that the toggle will be in the correct state next time you launch XFaceMaker.

In summary, if you choose the code preservation option, you must set the '***Preserve User Code***' toggle before saving your files from the first. Every time you will save your application files, XFaceMaker will use your existing application files as patterns instead of the original patterns, and preserve the application code you will have added in the meantime. If XFaceMaker does not find any pattern information in the existing application files, it will use the original pattern files. In all cases, a backup copy of the existing application files will be saved with a ~ suffix. You should check the correctness of the application files when you re-generate them.

The rules for adding application code to your application files are the following.

1. You may add/modify application code anywhere in a C file outside sections beginning with a line of the form:

```
/*## begin ... ##*/
```

and ending with a line of the form:

```
/*## end ... ##*/
```

2. You may add/modify application code anywhere in a Makefile outside sections beginning with a line of the form:

```
#### begin ...
```

and ending with a line of the form:

```
#### end ...
```

3. You may add/modify application code in a C file inside the sections described in rule 1, but only inside sub-sections beginning with a line of the form:

```
/*## begin user ##*/
```

and ending with a line of the form:


```
/*## end user ##*/
```

Example:

This is an example of a C application file with code preservation comments and added application code. The modified parts are marked with comments in upper-case letters.

```
/*
 * C file generated by XFaceMaker2.
 * Do not edit directly, and do not remove this comment.
 * Timestamp:          760353431
 */

/*
 * XFaceMaker 3.0 Application Functions Definitions.
 */
/*****
 *      SOME APPLICATION INCLUDES HERE
 *****/

#include <Fm.h>

/**## begin template 'active_values' ##*/
/**## $$AVTYPES AVNAMES;$\n$ ##*/
/**## end template 'active_values' ##*/

/**## begin 'active_values' Edit ##*/
String CurrentFile;
/**## end 'active_values' Edit ##*/

/*****
 *      SOME APPLICATION CODE HERE
 *****/

/**## begin template 'function' ##*/
/**## begin user ##*/
/**## end user ##*/
/**## #if NeedFunctionPrototypes ##*/
/**## CTYPE CNAME($$CARGTYPES CARGNAMES$, $) ##*/
/**## #else ##*/
/**## CTYPE CNAME($$CARGNAMES$, $) ##*/
/**## $$CARGTYPES CARGNAMES;$\n$ ##*/
/**## #endif ##*/
/**## { ##*/
```

```
/*## begin user ##*/
/*## end user ##*/
/*## } ##*/
/*## end template 'function' ##*/

/*****
 *      SOME MORE APPLICATION CODE HERE
 *****/

/*## begin 'function' LoadFile ##*/
/*## begin user ##*/
/*****
 *      SOME APPLICATION COMMENTS HERE
 *****/
/*## end user ##*/
#if NeedFunctionPrototypes
String LoadFile(String filename)
#else
String LoadFile(filename)
String filename;
#endif
{
/*## begin user ##*/
/*****
 *      APPLICATION FUNCTION CODE HERE
 *****/
FILE *f = fopen(filename, "r");
...
/*****
 *      END OF APPLICATION CODE HERE
 *****/
/*## end user ##*/
}
?/*## end 'function' LoadFile ##*/
```

10.2.2 Modifying Application Patterns

It may be useful or necessary to modify the application pattern files to suit the particular needs of a project or an application. This can be done by changing the installed pattern files (make sure you keep a copy of the original to fall back on), or by placing a copy in a different directory accessible by the XFaceMaker search rules, i.e. in a directory called **GenAppLi**, a sub-directory of the current directory, a user's home directory, or a project-wide directory specified using the **FMLIBDIR** environment variable. For example:

```
./GenAppLi/  
$HOME/Genappli/  
$FMLIBDIR/Genappli/
```

The rules explained in the previous section also apply to pattern files. This is the easiest way to modify pattern files, and allows you to customize, for example, the libraries used to link an application in a Makefile.

It is also possible to modify the pattern information to control how the application objects will be output in the application file.

See the *XFaceMaker Reference Manual* for more information on the **genappli** program and on patterns.

10.3 Write and Build Your Application

If you cannot or don't want to generate application files automatically, follow the procedures outlined in the following sections. In order to use an interface built with XFaceMaker, you must write a main program - called the application or the application program in this document. In addition to handling any application tasks, this main program will load the interface, either

- in its `.fm` form if you wish to execute the interface in interpreted mode
- in its `.c` form if you wish to execute the interface in compiled mode.

Interpreted mode is convenient in the prototyping phase of your interface development: you can make changes in the `.fm` file and re-launch your executable file, the changes will be visible immediately — there is no need to recompile and link the interface. This shortens the testing cycle considerably and makes the development more comfortable. In most instances, the final interface is generated in C-code, which has the following advantages:

- faster execution: the object code can be optimized and, in general, compiled programs execute faster than interpreted programs.
- portability: since the sources of the FACE functions are included in the XFaceMaker distribution, a C-code interface is entirely portable.

Whether you wish to generate an interpreted or a compiled interface, your main program should:

- open the display and initialize XFaceMaker structures with the application function `FmInitialize`
- attach any active values with the application function `FmAttachAv`
- create the interface, either by loading it (interpreted mode), or by calling the creation function(s) generated by XFaceMaker (compiled mode).
- enter the Xevent loop with the application function `FmLoop` or `FmEditLoop`.

10.4 Interpreted Interface

Link with -lFm

To execute an interface in interpreted mode, you load the `.fm` file which constitutes the interface. If the interface calls application functions, you must attach them, using `FmAttachFunction`. If it uses active values, these must be attached as well.

10.4.1 Minimal Main Program

The minimal main program in interpreted mode with no application function calls, no active values, and only one interface file has the following contents:

```
#include <Fm.h>
main (argc, argv)
int argc;
char **argv;
{
Widget def_as;/* def_as: default application shell*/
Widget as;/* as: application shell*/
def_as=FmInitialize("exmpl", "Exmpl", NULL, 0, &argc, argv);
as = FmLoad("exmpl.fm");
FmLoop();
}
```

To compile and link the program, do:

```
cc -c exmpl.c
cc -o exmpl exmpl.o -lFm -lXm -lXt -lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries—please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command.

10.4.2 Using Active Values and Application Functions

When using active values to share data between the interface and the application, you have to attach each active value; similarly, when the interface calls application functions, you have to attach each function. The main program for an interface with active values and application function calls contains:

```
#include <Fm.h>
int av1;
char av2[64];
void app_fn(w)
Widget w;
{
/* do application function stuff */
}
main (argc, argv)
int argc;
char **argv;
{
Widget def_as; /* def_as: default application shell*/
Widget as; /* as: application shell*/
def_as=FmInitialize("exmpl","Exmpl",NULL,0,&argc,argv);
FmAttachFunction("app_fn", app_fn, "None", 1, "Widget");
FmAttachAv("av1", &av1);
FmAttachAv("av2", av2);
/* FmLoad or FmLoadCreateManaged to pass arguments */
as = FmLoad("exmpl.fm");
FmLoop();
}
```

To compile and link the program, do:

```
cc -c exmpl.c
cc -o exmpl exmpl.o -lFm -lXm -lXt -lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries—please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command.

10.4.3 Loading Several Interface Files

Instead of creating an interface with one .fm file, you can subdivide your interface into several files. The main reasons for doing this are:

- Your .fm files being smaller, XFaceMaker response times as you develop and modify your interface files will be faster.
- If you generate C-code, smaller .c files will guarantee that you do not reach the limits of your compiler.

If an interface file contains several .fm files, you first load the file which contains the application shell, then load the other files as children of the interface fragments already loaded. Usually, the toplevel widget of an interface fragment is a shell, child of the application shell. However, this is not mandatory and you can specify any parent for the interface fragment you are loading. The only constraint is that there be only one widget at the top of the hierarchy being loaded.

The same holds true when loading groups. When loading several interface files, you must use `FmLoadCreate` or `FmLoadCreateManaged` for the interface files which do not contain an application shell, in order to specify the parent of the toplevel widget. The .fm file which contains the application shell may be loaded with `FmLoad` if desired.

```
#include <Fm.h>
int av1;
char av2[64];
void app_fn(w)
Widget w;
{
/* do application function stuff */
}
main (argc, argv)
int argc;
char **argv;
{
Widget def_as;/* def_as: default application shell*/
Widget as;/* as: application shell*/
Widget secnd; /*scnd: toplevel of second .fm file*/
def_as=FmInitialize("exmpl", "Exmpl", NULL, 0, &argc, argv);
FmAttachFunction("app_fn", app_fn, "None", 1, "Widget");
FmAttachAv("av1", &av1);
FmAttachAv("av2", av2);
```



```
/* can use FmLoad or FmLoadCreate plus FmShowWidget*/  
as=FmLoadCreateManaged("first", "first.fm", def_as, NULL, 0);  
secnd = FmLoadCreateManaged("secnd", "secnd.fm", as, NULL, 0);  
FmLoop();  
}
```

To compile and link the program, do:

```
cc -c exmpl.c  
cc -o exmpl exmpl.o -lFm -lXm -lXt -lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries—please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command.

10.4.4 Loading an Interface with Groups

Groups are interface fragments which may be instantiated more than once within a given interface. When loading groups, as in the case of multiple .fm files, you must specify the parent widget of the instance of the group being instantiated. You also specify the name of the instance of the group.

A main program which loads groups (and uses active values and calls application functions) has the following structure. You can also load groups in an application function called from the interface if you want to instantiate groups dynamically in response to user action.

Note: This case is functionally equivalent to the previous case, since the functions `FmLoadCreate` and `FmLoadCreateGroup` are equivalent. The latter is more efficient when many instances of an interface module are to be created.

```
#include <Fm.h>
int av1;
char av2[64];
void app_fn(w)
Widget w;
{
/* do application function stuff */
}
main (argc, argv)
int argc;
char **argv;
{
Widget def_as; /* def_as: default application shell*/
Widget as; /* as: application shell*/
Widget f_ts; /* toplevel of first instance of group */
Widget s_ts; /* toplevel of second instance of group */
def_as=FmInitialize("exmpl", "Exmpl", NULL, 0, &argc, argv);
FmAttachFunction("app_fn", app_fn, "None", 1, "Widget");
FmAttachAv("av1", &av1);
FmAttachAv("av2", av2);
/* can use FmLoad or FmLoadCreate plus FmShowWidget*/
as=FmLoadCreateManaged("first", "first.fm", def_as, NULL, 0);
f_ts = FmLoadCreatGroup("inst1", "grp.fm", def_as, NULL, 0);
s_ts = FmLoadCreatGroup("inst2", "grp.fm", def_as, NULL, 0);
/* call FmShowWidget to manage the groups */
FmShowWidget(f_ts);
FmShowWidget(s_ts);
```

```
FmLoop();  
}
```

To compile and link the program, do:

```
cc -c exmpl.c  
cc -o exmpl exmpl.o -lFm -lXm -lXt -lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries—please refer to your system documentation.

If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command.

10.5 Compiled Interface

Link with `-lFm_c`

and `-lXnsICtrl` if your interface uses any of the Control widgets.

When XFaceMaker generates C code for an interface, it creates a function called `FmCreatexx` where `xx` is the name of the `.fm` file (or the name you specify with `-cfile` when generating the C code). For a file called `mycif.fm`, the function is `FmCreatemycif`.

10.5.1 Minimal Main Program

```
#include <Fm.h>
main (argc, argv)
int argc;
char **argv;
{
    extern Widget FmCreatemycif();
    Widget def_as; /* def_as: default application shell */
    Widget as; /* as: application shell */
    def_as=FmInitialize("exmpl", "Exmpl", NULL, 0, &argc, argv);
    as = FmCreatemycif("exmpl", def_as, NULL, 0);
    FmLoop();
}
```

To compile and link the program, generate the C-code file of the interface—see the description of C-code generation options, in Section 11.3. To generate the C-code file you may either save the interface with the **Save C-Code** toggle set in the C Options within XFaceMaker or enter the following command line:

```
xfm -compile mycif.fm -cfile mycif.c
```

then compile:

```
cc -c mycif.c
cc -c exmpl.c
cc -o exmpl exmpl.o mycif.o -lFm_c -lXnsICtrl -lXm -lXt \
-lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries—please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command. `-lXnsICtrl` is required only if your interface includes Control widgets.

10.5.2 Using Active Values and Application Functions

The application functions called from the interface will be automatically linked to the functions in the main program by the linker, provided the functions have the same name in the interface and the application. It is not necessary to attach the functions explicitly as with an interpreted interface: the application functions are called directly by the generated C-code. Note: Application functions **must** be external. The main program for an interface with active values and application function calls has the following content:

```
#include <Fm.h>
int av1;
char av2[64];
void app_fn(w)
Widget w;
{
/* do application function stuff */
}
main (argc, argv)
int argc;
char **argv;
{
extern Widget FmCreatemycif();
Widget def_as; /* def_as: default application shell*/
Widget as; /* as: application shell*/
def_as=FmInitialize("exmpl", "Exmpl", NULL, 0, &argc, argv);
FmAttachAv("av1", &av1);
FmAttachAv("av2", av2);
as = FmCreatemycif("exmpl", def_as, NULL, 0);
FmLoop();
}
```

To compile and link the program: Generate the C-code file of the interface, either by saving the interface with the **SAVEC-Code** option set in XFaceMaker or with the following command line:

```
xfm -compile mycif.fm -cfile mycif.c
```

then compile:

```
cc -c mycif.c
cc -c exmpl.c
cc -o exmpl exmpl.o mycif.o -lFm_c -lXnsLCtrl -lXm -lXt \
-lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries— please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command. `-lXnsCtrl` is required only if your interface includes Control widgets.

10.5.3 Loading Two Interface Files

Instead of creating an interface with one `.fm` file, you can subdivide your interface in several files. In this case, you first load the interface file which contains the application shell, then load the other files as children of the interface fragments already loaded. Usually, the toplevel widget of an interface fragment is a shell, child of the application shell. However, this is not mandatory and you can specify any parent for the interface fragment you are loading. The only constraint is that there be only one widget at the top of the hierarchy being loaded. The same holds true when loading groups.

```
#include <Fm.h>
int av1;
char av2[64];
void app_fn(w)
Widget w;
{
/* do application function stuff */
}
main (argc, argv)
int argc;
char **argv;
{
extern Widget FmCreatecif1(), FmCreatecif2();
Widget def_as; /* def_as: default application shell*/
Widget as; /* as: application shell*/
Widget secnd_ts;
def_as = FmInitialize("exmpl", "Exmpl", NULL, 0, &argc, argv);
FmAttachAv("av1", &av1);
FmAttachAv("av2", av2);
as = FmCreatecif1("exmpl", def_as, NULL, 0);
secnd_ts = FmCreatecif2("secnd", as, NULL, 0);
FmLoop();
}
```

To compile and link the program, generate the C-code file of the interface, either by saving the interface with the `SaveC-Code` toggle set in `XFaceMaker` or with the following command lines:

```
xfm -compile cif1.fm -cfile cif1.c
xfm -compile cif2.fm -cfile cif2.c
```

then compile:

```
cc -c cif1.c
cc -c cif2.c
cc -c exmpl.c
cc -o exmpl exmpl.o cif1.o cif2.o -lFm_c -lXnslCtrl \
-lXm -lXt -lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries - please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command. `-lXnslCtrl` is required only if your interface includes Control widgets.

10.5.4 Loading an Interface With Groups

Groups are interface fragments which may be instantiated more than once within a given interface. When loading groups, as in the case of multiple .fm files, you must specify the parent widget of the instance of the group being created. You also specify the name of the instance of the group. A main program which loads groups (and uses active values and calls application functions) has the following structure. Note that you can also load groups from an application function called from the interface if you want to instantiate groups dynamically in response to user action.

```
#include <Fm.h>
int av1;
char av2[64];
void app_fn(w)
Widget w;
{
/* do application function stuff */
}
main (argc, argv)
int argc;
char **argv;
{
extern Widget FmCreatemycif(), FmCreatemygrp();
Widget def_as;/* def_as: default application shell*/
Widget as;/* as: application shell*/
Widget secnd_ts;
def_as = FmInitialize("exmpl","Exmpl",NULL,0,&argc,argv);
FmAttachAv("av1", &av1);
FmAttachAv("av2", av2);
as = FmCreatemycif("exmpl", def_as, NULL, 0);
f_ts = FmCreatemygrp("first", as, NULL, 0);
s_ts = FmCreatemygrp("secnd", as, NULL, 0);
FmLoop();
}
```

To compile and link the program: Generate the C-code file of the interface and of the group, either by saving the interface with the **SaveC-Code** toggle set in XFace-Maker (make sure you save the group through the Modules menu after having set the toggle) or with the following command lines:

```
xfm -compile mycif.fm -cfile mycif.c
xfm -compilegroup mygrp.fm -cfile mygrp.c
```

then compile:

```
cc -c mycif.c
```



```
cc -c mygrp.c
cc -c exmpl.c
cc -o exmpl exmpl.o mycif.o mygrp.o -lFm_c -lXnslCtrl \
-lXm -lXt -lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries - please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command. `-lXnslCtrl` is required only if your interface includes Control widgets.

10.5.5 Stand-alone C-Code

If you are generating stand-alone C-code for your interface, you will not want to link with any of the XFaceMaker libraries. Reminder: the source files for the FACE functions which can be used in compiled mode are provided with the product, thus ensuring full independence from NSL for compiled applications. Using stand-alone C-code thus does not have any significant advantage and deprives you of the use of FACE convenience functions and of active values. See Section 11.6 on how to write scripts for stand-alone C-code. If, however, you choose to work in stand-alone C-code, you will not want to use `FmInitialize` or `FmLoop` in the main program. The minimal main program for stand-alone C-code is:

```
main (argc, argv)
int argc;
char **argv;
{
extern Widget FmCreatemycif();
Widget def_as; /* def_as: default application shell*/
Widget as; /* as: application shell*/
def_as = XtInitialize("exmpl", "Exmpl", NULL, 0, &argc, argv);
as = FmCreatemycif("exmpl", def_as, NULL, 0);
XtMainLoop();
}
```

To compile and link the program, generate the C-code file of the interface either by saving the interface with the `SaveC-Code` toggle set in XFaceMaker (make sure the save options are properly set for stand-alone C) or with the following command lines:

```
xfm -compile mycif.fm -cfile mycif.c -standc
```

then compile:

```
cc -c mycif.c
cc -c exmpl.c
cc -o exmpl exmpl.o mycif.o -lXm -lXt -lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries— please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command.

10.5.6 C-Code With UIL

Link with Mrm

If you are working with UIL, the .c file does not contain the interface description; this is in the .uil file. You will need to generate a .uid file and link with the Mrm library. The main program has the following structure:

```
#include <Fm.h>
void main(argc, argv)
int argc;
char **argv;
{
extern Widget FmCreateexuil();
Widget top;
MrmInitialize();
top = FmInitialize("exuil", "Exuil", NULL, 0, &argc, argv);
FmCreateexuil("exuil", top, NULL, 0);
FmLoop();
}
```

To compile and link the program, generate the UIL and C-code files of the interface by saving the interface with the **Save C-Code** and **Save UIL** toggles set in XFaceMaker, or with the following command line:

```
xfm -compile cif.fm -cfile cif.c -uil
```

then create the uid file:

```
uil -o cif.uid cif.uil
```

then compile:

```
cc -c cif.c
cc -c exmpl.c
cc -o exmpl exmpl.o cif.o -lMrm -lFm_c -lXnslCtrl -lXm \
-lXt -lX11 -lm
```

The order in which the libraries are specified is important. On some machines you may need to specify some additional system libraries - please refer to your system documentation. If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command.

10.6 Building a New XFM

Link with Fm_e

You can create a new XFaceMaker easily, using the function FmCallEditor. You can thus add your application functions, new widget classes, new resource types and resource converters to XFaceMaker and obtain an executable which is better adapted to your specific needs. The simple example shown below adds a widget class and an application function.

```
#include <Fm.h>
#include <Newcl.h>
void new_fn()
{
/* do function stuff */
}
main (argc, argv)
int argc;
char **argv;
{
FmInitialize("myxfm", "MyXfm", NULL, 0, &argc, argv);
FmAddWidgetClass(&newclWidgetClass,
0,
FM_PRIMITIVE,
FM_NORMAL,
0,
#include <Newcl.h>",
"newclWidgetClass",
0,
0,
0,
"NewclIcon");
FmAttachFunction("new_fn", new_fn, "None", 0,);
FmCallEditor();
}
```

To compile and link the program, do:

```
cc -c myxfm.c
cc -o myxfm myxfm.o Newcl.o -lFm_e -lXm -lXt -lX11 -lm
```

If the class has been generated with XFaceMaker, use FmAdd<ClassName> instead of FmAddWidgetClass.

`FmCallEditor` is the main function of the XFaceMaker editor. Calling it starts XFaceMaker and loads the XFaceMaker interface. You then have a new version of XFM with the additional functions and classes built into it.

This is used to extend XFaceMaker with, for example, database functions. You can also use it to test your whole application while still being able to edit the interface. This can also be done without exiting the XFM process. (See “Controlling the Design Process”.)

The function `FmCallEditor` can be used only in interpreted mode.

If your compiler is not ANSI compliant, you must add `-D_NO_PROTO` to the compile command.

10.7 Compatibility with libFm_c and libFm_e

Since the *Fm_c* and *Fm_e* libraries define two different versions of several functions (such as active value functions, etc...), you must be careful to link the correct version of the functions. This is done by specifying the order of the two libraries in the link editor command line.

For example, let us consider a widget class *MyClass* designed using XFaceMaker and saved as non-stand-alone C code into the file *MyClass.c*. Let us assume that *MyClass.c* contains calls to the function `FmShowWidget`. To ensure that the `Fm_c` version of `FmShowWidget` will be used, we will place the `libFm_c` library before the `libFm_e` library on the command-line when linking a new XFaceMaker with the new class:

```
cc -o myxfm main.o MyClass.o -lFm_c -lFm_e -lXm -lXt -lX11
```

Notes:

1. This assumes that the link editor of your system uses the first definition of a function that it encounters. This is the case on most platforms, but it is not guaranteed for all systems, so please refer to the appropriate manual pages if necessary.
2. This also assumes that the link editor will not load an object file containing already defined symbols. If it does not work on your system, you can remove the appropriate object file from your Fm library. The name of the object file containing all the application-callable functions is `appf.o` for `libFm` and `libFm_e`, and `c_appf.o` for `libFm_c`.

For `FmInitialize`, you must specify explicitly which version you want (`Fm_c` or `Fm_e`). In the example above, the main program looks like:

```
main(argc, argv)
```

```
int argc;
char **argv;
{
    FmInitialize("myxfm", "MyXfm", 0, 0, &argc, argv);
    FmAddMyClassWidgetClass();
    FmCallEditor();
}
```

Since the `Fm_c` library is placed before `Fm_e`, the link editor will use the `Fm_c` version of `FmInitialize`. This is undesirable since `FmCallEditor` expects that the `Fm_e` version of `FmInitialize` was called previously. To solve this, two special versions of `FmInitialize` are defined: `FmInitializeI` is the version contained in the `Fm_e` library, while `FmInitializeC` is the version defined in `Fm_c`. To initialize both the `Fm_c` and `Fm_e` libraries, you must call both `FmInitializeI` and `FmInitializeC`. You must also use `FmSetToplevelC` to prevent `XtInitialize` from being called twice. So, for the example above, the correct code is:

```
main(argc, argv)
int argc;
char **argv;
{
    Widget toplevel;

    toplevel=FmInitializeI("myxfm", "MyXfm", 0, 0, &argc, argv);
    FmSetToplevelC(toplevel);
    FmInitializeC("myxfm", "MyXfm", 0, 0, &argc, argv);
    FmAddMyClassWidgetClass();
    FmCallEditor();
}
```

CHAPTER 11: Generating C-Code

This chapter discusses how to generate interfaces in the C language from a previously created Fm interface. We will call the C file generated "C-code interface". It is compiled and linked with the application in contrast with the .fm file which is loaded and interpreted.

C-code interfaces have the following advantages:

- The interface file can be compiled on any Unix target machine.
- The Fm_c library source code is contained in the distribution so that it too can be compiled on any Unix target machine.
- Execution times are slightly faster, since no interpretation is required.
- You do not have to attach application functions.
- No separate .fm interface file is required.

C-code interfaces have the following characteristics:

- Any changes to the interface require recompiling and relinking.
- Modifications of the C-code generated by XFaceMaker should not be attempted.
- NSL does not guarantee C-code compatibility for different releases of the product. C-code must be re-generated from the .fm file.
- C and Fm interfaces may display some minor differences as described in section 11.7, C and Fm Mode Differences.
- An interpreted interface fragment cannot be parented from a compiled fragment.

It is difficult to mix Fm and C-code interface files in the same program because the application must be linked with both the interpreted library (Fm) and the C library (Fm_c) which have common function names.

C-code files generated by XFaceMaker cannot be reloaded in XFaceMaker. If you edit the .c file these changes will be lost. To modify an interface, you should always use XFaceMaker to change the .fm file and regenerate the .c file. A description of how to port the Fm_c library is included in the *XFaceMaker Reference Manual*.

11.1 C-Code Interfaces

The C-code file generated by XFaceMaker contains all the declarations and definitions that make up the interface:

- By default, every widget of the interface is stored in an array which is allocated when the interface creation function is invoked. Other widget storage options are available, as explained below.
- The code is compatible with both X11R4/Motif 1.1 and X11R5/Motif 1.2. This means that an interface generated in an X11R5/Motif 1.2 environment can be compiled in an X11R4/Motif 1.1 environment (except if you use special Motif 1.2 functions, e.g. Drag-and Drop functions).
- Every FACE script of the interface is compiled into a separate private C function; this includes all callback and active value scripts and the user-defined translations specified by using the `Eval` action.
- It does not contain a `main` function, which is assumed to be in the application.
- The last function of the file, called `FmCreate<name>`, contains all the Intrinsic and Motif function calls necessary to build and display the widgets of the interface, and to link the functions to the callbacks, active values and actions.

From the application's point of view, the C-code file defines one single *creation function* which creates the interface. This function is called `FmCreate<name>`, where *name* is the base name of the `.fm` file, and has the following arguments:

```
Widget FmCreate<name>(name, parent, args, num_args)
    String      name;
    Widget      parent;
    Arg         *args;
    Cardinal    num_args;
```

The creation function `FmCreate<name>()` can be thought of as an equivalent to `XtCreateWidget`. It returns the topmost widget of the interface--if there were several topmost widgets, the first one is returned. This topmost widget is created by passing the *name*, *parent*, *args* and *num_args* arguments to `XtCreateWidget`, or an equivalent function.

The resource values specified by *args* are added after the values specified with XFaceMaker. The topmost widget of the interface is mapped automatically if it was visible when the interface was saved. If your `.fm` file contains an `ApplicationShell`, then the *name* argument is ignored: the name of the application (defined as the last

component of `argv[0]`) is used instead of the name of the `ApplicationShell`. If your `.fm` file does not contain an `ApplicationShell`, then the parent argument should be a descendant of a previously created `ApplicationShell`.

11.2 Generating the C-Code File

You can generate a C code version of:

- the entire interface
- a part or of the interface
- a widget class.

See Section 11.5 for details on generating C code for a widget class.

There are two ways to generate a C code version of the *entire* interface:

- *In the C Code Options window.* Set the **Save C Code** checkbox. When this is *on* (highlighted), a C language version of the interface will be generated whenever you save the interface. If **Save** is used, then the C file will be saved with the same base name as the interface file. If **Save As** is used, then the File Selector pops up, allowing you to save the C file under a different name. The code generated is affected by the options set in the C Code Options window.

- *In the command line.* Use the `-compile <name>` option. For example:

```
xm -compile filename.fm
```

generates the C-code file `filename.c`, with the default options of the C-Code Options box.

There are several ways to generate a C-code file corresponding to a *part* of the interface of which there will be more than one instance; i.e. a group:

- *In the C Code Options window.* Select the top widget of the group. Set the **Save C Code** checkbox, then select **Modules -> Group -> Save As...** (This is equivalent to **Save C Code** with the option "Dynamic Widget References" as described below.) The group may or may not contain a shell widget.

- *In the command line.* Use the `-compilegroup <name>` option.
`xfm -compilegroup groupname.fm -cfile groupname.c`
generates a C-code file named `groupname.c` (and is equivalent to selecting **Modules -> Group -> Load**, followed by **Save C Code file** in the C Code Options window, and then **Modules -> Group -> Save As...**).
- *Store widget references in arrays.* Use the default option in the C code options window or the following command line:
`xfm -compile -cflags va`

To generate the C code for a new *widget class*, you can use the command line options or set the options in the C Code Options window and use the Modules menu to save the class: **Modules -> Classes -> Save** or **Save As...** For more details on this subject, see Section 11.5, C Code for a New Widget Class.

11.3 C Code Options

There are a number of C code options available, which are described in Section 11.3.1 below. You can select and apply options in the C Code Options window, or use the equivalent command line options. The command line options are listed in Table 11.1.

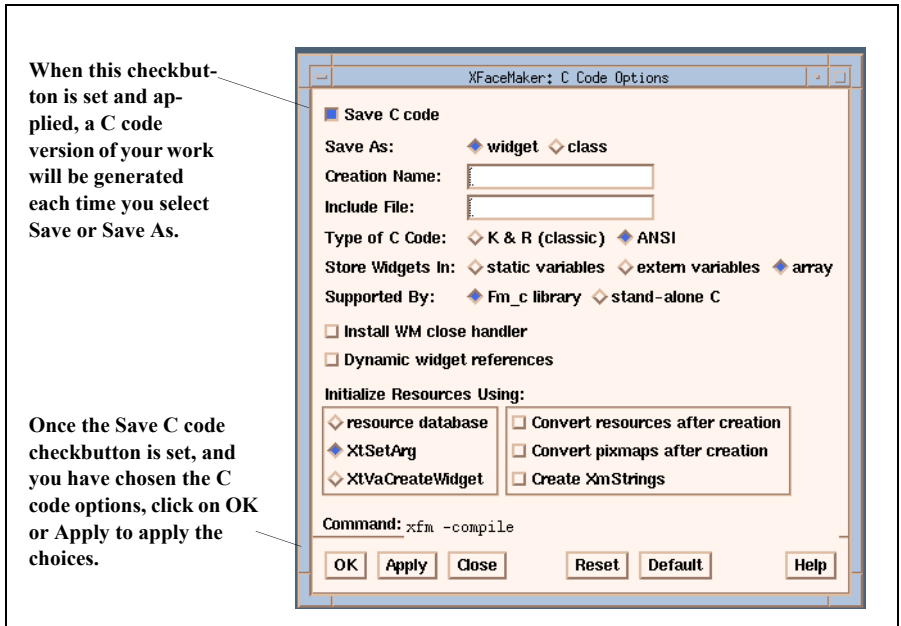


Figure 11.1: The C Code Options Window

To set the C code options in the window:

1. Open the window by clicking on **C Code...** in the Options menu.
2. Select the **Save C code** checkbox. It is highlighted when set.
3. Select the options. They are described below.
4. Click on **OK** to apply the options and close the window, or click on **Apply** to apply the options and keep the window open.

Once the options are set and applied, a C code version of your interface will be saved whenever you save your interface. The choices remain valid for every save of your interface, until you change the options.

11.3.1 Description

The following section describes the C code options that are available. Table 11.1 shows the equivalent command line option for each. The options are described in the order in which they appear in the C Code Options window.

You can use the `-cflags` flag character string for the different options available, which can be concatenated to give the total specification.

When the `-compile` (or `-compilegroup`) options are used, XFaceMaker does not normally display anything on the screen, but the X Toolkit must be initialized so that the interface can be checked for errors. This means that the `xfm` command must always be run in an X server environment (e.g. from an `xterm`). Because the interface widgets are created as the `.fm` file is parsed, any creation scripts defined will be executed.

The following list describes the C code options:

Save As: widget

Compiles the current interface into a single creation function that can be called to create the interface, that is, `FmCreate<Name>`. The default value for *Name* is the basename of the current filename, i.e., the `.fm filename`.

Save As: class

Compiles the interface into the code necessary to create a new widget subclassed from the class of the top widget in the interface. Code is also generated to integrate the new widget class into XFaceMaker. The option `Store Widgets In: array (-cflags a)` will be forced to True.

Creation Name

Used to enter the name used in the creation function so that it becomes `FmCreate<Name>`. The default value for *Name* is the basename of the current filename; i.e. the `.fm filename`.

Class Prefix

Used to enter the prefix string to be used for the widget class when generating the class. The prefix is used in conjunction with the creation name to give `FmCreate<PrefixName>` which will create an instance of the new widget class `<PrefixName>`. The default value for *Prefix* is `Xfm`.

Include File

Used to specify an include file name to be inserted in the C-code file. The string given is inserted directly into the file to give the line `#include Text`. Any special characters must be protected when used from the shell. Only one file can be included.

Type of C Code: K&R(classic)

Generates C-code conforming to the standard defined by Appendix A, of Kernigan and Ritchie's *The C Programming language*.

Type of C Code: ANSI

Generates C-code with prototyped functions conforming to the *ANSI C* standard. The code can also be compiled using a C++ compiler. Types defined in the application must be declared. (See Section 11.4, Specifying C/C++ Type Names.)

Store Widgets In: static variable

Specifies that when the C-code file is generated, every resolvable widget of the interface will be stored in a private C variable. For any unresolvable widget names, a dynamic widget reference is made so that the name will be resolved at runtime. This option *must not* be used for interface fragments which will be instantiated more than once.

Store Widgets In: extern variables

Specifies that widget ID's be stored in global (extern) C variables in the generated C file. A widget whose name is *Name* will be stored in a C variable called `FmNameWidget`. If two widgets in the interface have the same name, one of the variables is re-named as `FmNameWidget_N`, where *N* starts at 1 and is incremented for every synonym. This option *must not* be used for interface fragments which will be instantiated more than once.

Store Widgets In: array

Specifies that widgets be stored in an array that is allocated every time the inter-

face creation function is called. When this option is set, XFaceMaker ignores the `-compilegroup` option (`-cflags d`), and outputs dynamic widget references only for widgets defined outside of the module. This option is general: it can be used for interface fragments that are to have multiple instances.

Supported by: Fm_c library

Generates code which makes use of the Fm_c library functions. The functions simplify the application and reduce code size. The source code of the Fm_c library is included in the standard distribution.

The following two options are valid with “Use Fm_c library”:

Install WM close handler

Installs a default handler that will be called when the Motif Window Manager *Close* menu item is selected, as in Fm mode. The default handler exits the application if the shell is an ApplicationShell, or else unmaps the shell. The handler can be changed using FmSetCloseHandler.

Dynamic widget references

Specifies that XFaceMaker produce dynamic widget references for all widgets.

Standalone C

Specifies that the code generated contain calls to the X and Motif libraries only. The application will have to use XtInitialize and XtMainLoop instead of FmInitialize and FmLoop to be fully independent of the Fm_c library.

Initialize Resources Using: resource database

Specifies that an internal resource database be generated as an array which is passed to the X resource manager. The widget creation functions consult the resource manager which handles all resource conversions.

Initialize Resources Using: XtSetArg

Tells XFaceMaker not to generate a resource database, but to call `XtSetArg` for all resources. The resources are either passed directly in one call, or converted from their `String` representation to their own representation and then passed as parameters to `XtSetArg`. The following resources are passed directly:

m Strings

m Booleans

m KeySyms (call to `XStringToKeysym`)

m integers (Int, Position, Dimension, etc.)

m Widgets

m enumerated constants (LabelType, Orientation, etc.)

m XmStrings (optionally, see *Create XmStrings*)

All the other resources are converted by calling `XtConvertAndStore` *before* creating the widget, passing the parent widget.

If the `XtSetArg` option is set, you have the following options:

Convert resources after creation

Specifies that for resources that need to be converted, `XtConvert` be called *after* the widget has been created (passing the new widget to the converter), and set using `XtSetValues`.

Convert Pixmaps after creation

Specifies that for Pixmap resources that need to be converted, `XtConvert` be called *after* the widget has been created (passing the new widget to the converter), and set using `XtSetValues`. For other resources that need to be converted, `XtConvert` is called with the parent widget before creating the widget.

Create XmStrings

Specifies that resources of type `XmString` do not call the resource converter, but directly call the function `XmStringCreateLtoR`, to produce shorter code.

Initialize Resources Using: XtVaCreateWidget

Tells XFaceMaker to use the Xt “varargs” functions to create the widgets (XtVaCreateWidget, etc.). The resources of the widget are passed as a varargs list of pairs (name,value), or as a tuple (XtVaTypedArg, name, type, value, size) for resources that need to be converted.

The following three options are valid with XtVaCreateWidget:

5 Creation Arguments allowed

When set, the maximum number of arguments that can be passed to the creation function is five.

20 Creation Arguments allowed

When set, the maximum number of arguments that can be passed to the creation function is twenty.

100 Creation Arguments allowed

When set, the maximum number of arguments that can be passed to the creation function is one hundred.

11.3.2 The Command Line Options Table

The following table lists the C code options and their equivalent command line. -cop-

Table 11.1: C Code Command Line Options

Option	Command Line Option	-coptions (obsolete)	-cflags
Save As: widget	-compile		
Save As: class	-compile	4096	g
Creation Name	-cname<Name>		
Class Prefix	-cprefix<Prefix>		
Include File	-cinclue<Text>		
Type of C code: K&R(classic)	set no flags		k
Type of C code: ANSI	-ansic	2048	C
Store Widgets In: static variables	set no flags		
Store Widgets In: extern variables		8192	G
Store Widgets In: array		32	a
Supported by: Fm_c library	set no flags		
Supported by: stand-alone C	-standc	512	S
Install Wm close handler	-instclose	1024	i
Dynamic widget references	-compilegroup	256	d
Initialize Resources Using: resource database	set no flags		r
Initialize Resources Using: XtSetArg		1	c
Convert resources after creation		4	s
Convert pixmaps after creation		8	p
Create XmStrings		16	x
Initialize Resources Using: XtVaCreateWidget		2	v
5 creation arguments allowed	default		
20 creation arguments allowed		64	m
100 creation arguments allowed		128	M

tions values are listed for backward compatibility only.

11.4 Specifying C/C++ Type Names

XFaceMaker has an internal table which defines the C or C++ types into which the arguments of functions called from FACE scripts must be cast in the resulting C/C++ code. This built-in table contains the C/C++ form of all the argument types of all the pre-defined FACE functions. When it outputs a function call and the `-ansic` option (or equivalent) is set, the FACE compiler looks up its internal table for the FACE argument type. If a corresponding C/C++ type is found, it outputs a cast to this type. Otherwise, no cast is output.

Note that XFaceMaker does not output the declarations of the C/C++ functions called in FACE scripts. The declarations must be included in the generated C/C++ file using the `-cinclude` option. However, for functions of the XFaceMaker libraries, the `Fm.h` file is now included automatically when the `-ansic` option is set.

If you call functions of your application which have arguments that XFaceMaker does not know about, you will have to specify the C/C++ form of the unknown arguments. This can be done in two ways:

- by specifying the C/C++ form of a FACE argument type, or
- by specifying the C/C++ prototype of a given function.

Both methods can be used either in FACE scripts or from the application. If a function argument has no associated type caster, XFaceMaker issues a warning during C code generation, and no cast will be generated.

11.4.1 Declare Types and Prototypes in FACE Scripts

To specify the C/C++ form of a FACE type in a FACE script, use the syntax:

```
type MyType "struct my_type *";
```

The `type` keyword accepts a second type name which specifies the C/C++ type into which function arguments will be cast in the generated C code. The new C/C++ type is recorded in XFaceMaker's internal table and will be used for all function calls with the given argument type.

Examples of this construct are:

```
type IntPtr "int*";
type XnslTreeEdgeId "XnslTreeEdgeId";
type XnslTreeNodeIdPtr "XnslTreeNodeId*";
```

To specify the C/C++ prototype of a given function, use the keyword `cprototype`:

```
cprototype MyFunction "struct s1 *" MyCFunction
```

```
("struct s2 *", ...);
```

This declaration defines the C name and the prototype of function `MyFunction`. It will be used only for calls to the specified function.

11.4.2 Declaring Types and Prototypes from the Application

The following functions have an effect similar to the FACE keywords described above. They are useful when you want to build a new instance of `XFaceMaker` with additional built-in functions. The `FmNewTypeDef` application function is equivalent to the new `type` construct:

```
FmNewTypeDef  
("MyType", "struct my_type *", sizeof(struct my_type *));
```

The third argument specifies the size of the C type in bytes. This is not used for C/C++ prototyping, but for widget class generation: if the given type is used as the type of a new resource, `XFaceMaker` will take the specified size as the size of the resource. The `FmSetFunctionPrototype` is equivalent to the `cprototype` construct:

```
FmSetFunctionPrototype  
("MyFunction", "MyCFunction", "struct s1 *",  
                                "struct s2 *", ...);
```

The `MyFunction` function takes a variable number of arguments. If the number of C types for the arguments of the FACE function does not match the number of arguments specified in the corresponding call to `FmAttachFunction`, the behavior is undefined.

If the new type is a resource type (i.e. the type of a resource of a widget added to `XFaceMaker`), use `FmAddRepresentation` instead.

11.5 C Code for a New Widget Class

To generate the C code for a widget class, you must first set the **Save C code** flag in the C Code Options window, or use the equivalent command line options. XFace-Maker will then generate a .c file (and the associated .h and P.h files) when the interface is saved using the commands in the File menu, in the Modules menu, or in the Active Values Editor.

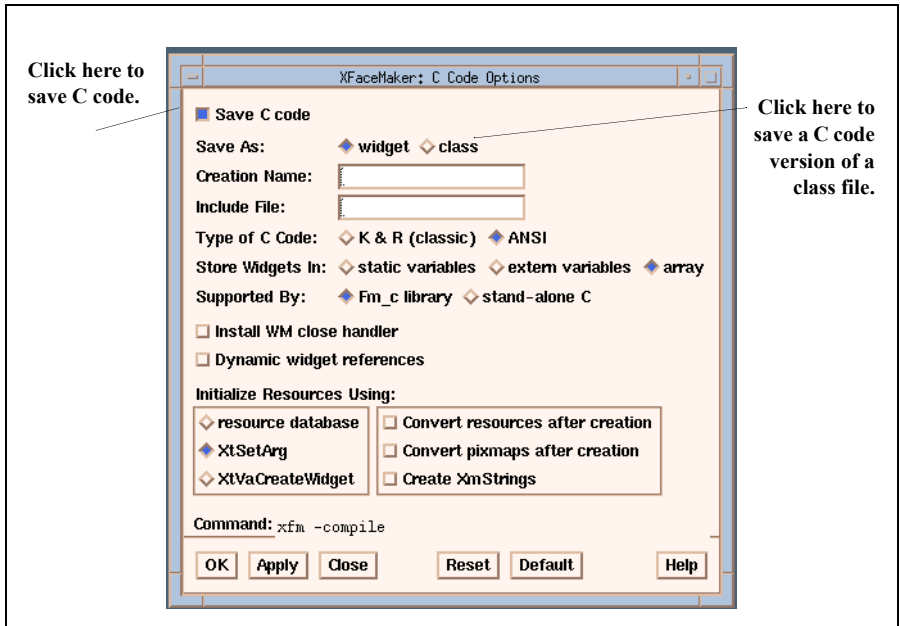


Figure 11.2: Saving a C Code Version of a Class

To save a C code version of a new widget class:

1. Set **Save C code** in the C Code Options window, then use the Modules menu to save the class: **Modules -> Classes -> Save** or **Save As...** The C code will be generated with the *save as class* option without your having to set the flag in the C code Options window.
2. Set **Save C code** and **Save As: class** in the C Code Options window, then click on **OK** or **Apply** to apply it. Don't forget to unset the **Save As: class** toggle once your class has been saved!

You can also set the Save C code option, then save the class in the Active Values box as described in Chapter 8.

Note: if you have defined a prefix for your class, you should arrange to have your `.h` file for the class in a directory with the same name as your prefix. For example, for a class `MyClass` with prefix `MyPrefix`, you should have `MyPrefix/MyClass.h`. To achieve this, you may need to create the directory `MyPrefix` and move the file `MyClass.h` into it.

Alternatively, you can use the command line options to save a C code version of a class:

```
xfm -compile Myclass.fm -cflags g
```

to produce your C-code widget class file. You can combine this option with other options, but note that the option *Store Widgets in Array* (`-cflags a`) will be forced to True.

Also, the option *Use XtSetArg* (or `-cflags c`) is forced for the creation of sub-widgets in the class, unless the option *Use XtVaCreateWidget* (or `-cflags v`) was set.

11.6 Stand-alone C Code

If you set the *standalone C* option, XFaceMaker outputs a C file which does not contain any implicit calls to the `libFm_c.a` library functions.

The advantages of this option are:

- you do not need to link your application with the `libFm_c.a` library,
- you do not need to port the library when you port your application to a new platform.
- you generate pure Xt and Motif code that you can use to trace problems: if a problem still occurs with standalone C, it is not due to XFaceMaker. It is due to the implementation of X and/or Motif on your platform.

The drawback is that you lose the functionality implemented in the convenience functions of `Fm_c` and in the active values. In addition, you must follow the rules summarized below when writing your FACE scripts:

- No calls to functions in the `Fm_c` library; i.e. no FACE functions.
- No Active Value references.
- Resource references in FACE functions and in translation table scripts must be explicitly typed.
- Resource references for widgets contained in variables must be explicitly typed.

- Resource assignments from a variable must use a statically typed variable.
- Widget references in a `ScrolledWindow` or in a menu must use the `*` character to skip clip windows and menu shells.
- No X properties.

The following modifications must be made in the application code.

- Instead of calling `FmInitialize`, you must call `XtInitialize` (or an equivalent initialization sequence). If you do not use `XtInitialize` you should create an unrealized `ApplicationShell` and pass it as the parent.
- Instead of calling `FmLoop`, you must call `XtMainLoop` (or an equivalent event handling sequence).
- If the interface you want to create contains an `ApplicationShell` as its root widget and you have not specified any of the C-code flags `c`, or `v` (i.e. you want to use the resource database to set widget resources), then the `name` argument that you pass to the `FmCreate<filename>` *must* be the same as the name of the application, which is the last component of `argv[0]`.
- You must specify a non-null `parent` if the top level widget is not an `ApplicationShell`.

Finally, if you have used `Pixmaps` in your interface, they might look slightly different from the same interface linked with the `libFm_c.a` library, because the special `Pixmap` converters contained in `libFm_c.a` are not registered, so only the standard `Motif` converters are available. This means that:

- you cannot use color `Pixmaps` in `XWD` format,
- there is no converter for the type `XtRPixmap`; so for instance, the resources `XmNIconPixmap` of a shell or `XmNLabelInsensitivePixmap` for a label will not work.

Example: The following script, when translated into stand-alone C, will produce warnings.

```
1  v = 10;
2  self:value = v;
3  function None f()
4  {
5      self:value = 10;
6      printf("%d\n", self:value);
7  }
8  f();
```

The first assignment of the `value` resource (line 2) has a dynamically typed variable as the right-hand side: you should use a statically typed variable instead. The second assignment of the `value` resource (line 5) is inside a function definition: you should specify the type of the resource explicitly. The reference to the `value` resource (line 6) is also inside a function: you should again specify the type of the resource explicitly.

The modified script below will produce no warning at compilation:

```
1  Int v = 10;
2  self:value = v;
3  function None f()
4  {
5      self:value:Int = 10;
6      printf("%d\n", self:value:Int);
7  }
8  f();
```


11.7 C and Fm Mode Differences

The most frequent differences that can appear between an application using an XFaceMaker interface in Fm format, and the same application using the same interface in C-code format, are described here.

Undefined FACE Functions The use of the interpreter to create the interface means that private data structures are maintained. FACE provides a number of convenience functions in interpreted mode that make use of this additional information, which do not exist in the libFm_c library. Other functions are not portable between different machines and/or operating systems. Since the source of all library functions used in C mode has to be provided to ensure a full portability of XFaceMaker applications in C mode, these non-portable functions cannot be included in the C mode library.

FACE Type Checking and Error Handling In interpreted mode, the FACE scripts attached to callbacks of the interface are interpreted by the XFaceMaker library. The FACE interpreter checks if the arguments passed to FACE functions have the correct type. If an error occurs during the evaluation of a script, a warning or an error is printed. When FACE scripts are translated to equivalent C functions, there is no type checking or error handling. For this reason, an application that just prints an error message in interpreted mode can encounter a fatal condition and exit (possibly dumping a core file) when compiled. The only solution to this kind of problem is, of course, to correct all errors in FACE scripts.

Close Handlers The default Window Manager close handler is installed automatically in interpreted mode, but not in the C-code file. To install the close handler you must select the option “Install WM ‘Close’ Handler” or its command line equivalent.

Pixmaps When a file name is used to specify a Pixmap, the file is searched using the various paths defined by XmGetPixmap and contained in the XBMLANGPATH and XMPIXMAPSPATH environment variables, if defined. If the XBMLANGPATH variable is not set, XFaceMaker will set it to a default value so that it can find its own bitmaps. An application using a C-code file does not contain this code. The solution to this problem is to set XBMLANGPATH or XMPIXMAPSPATH to the required path before launching the application if you store files in non--standard locations. The manual page for XmGetPixmap gives detailed information on the standard search paths used. You can use the `-verbose` command line option when loading `xfm` to display the bitmap paths used by XFaceMaker.

Alternatively, the C-Code application can ensure XBMLANGPATH is set correctly by manipulating the value itself, with code like the following:

```
/* Fallback default directories to find files.*/
static char def1[] = "/usr/local/lib/X11/appli";
static void SetXBMLangEnv(LibDir)
char *LibDir; /* LibDir set by cmd line option */
{
    /* default is /usr/lib/X11/appli */
    char buf[1024]; char *p;

    p = getenv("XBMLANGPATH");
    if(!p)
        p = "."; /* never start with : */
    sprintf(buf,
        "XBMLANGPATH=%s/%B:./%B:./bitmaps/%B:\n"
        "%s/bitmaps/%B:%s/bitmaps/%B:%s/bitmaps/%B:\n"
        "/usr/include/X11/bitmaps/%B",
        p, LibDir, def1, def3);
    putenv((char *)XtNewString(buf));
}
```

Program Name When generating a C-code file with the “Use resource database” option, the resource database facilities provided by the X Toolkit are used to specify resource values for widgets. For each resource specified in your interface, XFaceMaker will add a line of the form:

```
<prog_name>.<widget1>...<widgetn>.<resource>: \ <value>
```

to the resource database. For this reason, if the name of the executable file containing your application has a dot (‘.’) character in it, then the resource database will get confused and none of the resources of your interface will be correct. The solution is, of course, not to give a name containing a dot to your program.

CHAPTER 12: Building Projects

Interfaces can become quite large, with multiple secondary windows. In such situations, it is desirable to split an interface into several files that can be worked on separately, although they still conceptually belong to the same global interface.

In XFaceMaker, subdividing an interface into separate components consists in building a *Project*: the complete interface is described in a project file (`.fmp` extension), which is the list of individual `.fm` files that belong to the interface, and, for each such file, where it is connected in the interface, i.e. its parent path, and whether it is mapped.

Similarly, XFaceMaker keeps track of the file names of model objects so that, if you work on a model object, e.g. a template, while an interface is loaded, and you do a Save or Save As, the model object (the template) is saved to its specific file (the template file), not to the interface file.

You can build a project from a complex interface that you want to subdivided into several `.fm` files, or you can build a project from separate `.fm` files that you want to combine to form a complex interface. Both methods are described here.

12.1 Sub-dividing a Complex Interface

Let's assume we have an interface that contains an `ApplicationShell` which is the main window, two `TopLevelShell`'s that correspond to secondary windows, and a `DialogShell`.

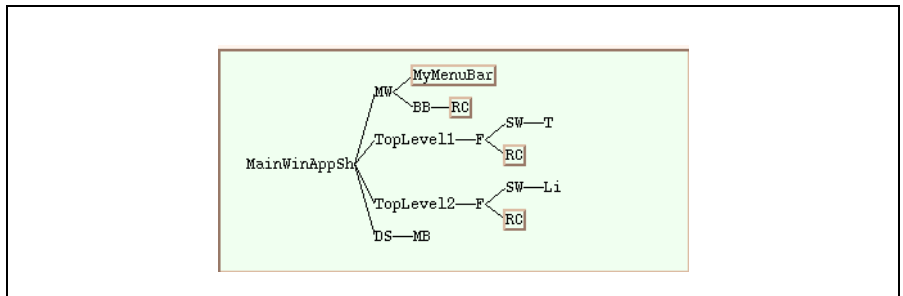


Figure 12.1: The Complete Interface

A logical thing to do, though not the only possible one, is to subdivide the interface at the shell level, i.e. to create four .fm files, one for each shell with its associated sub-tree. Once the interface is broken up, each fragment can be worked on separately. XFaceMaker will still know that the interface fragment belongs to the global interface.

Assume we want to save the sub-tree starting from DS (the dialogShell) in a file called `if_dialog.fm`, the sub-tree under and including `TopLevel2` in file `if_top2.fm`, the sub-tree associated to `TopLevel1` in file `if_top1.fm`, and the remainder of the interface, i.e. the `applicationShell`, the `mainWindow` and its sub-tree in file `if_main.fm`.

Copy the interface to the file named `if_main.fm` and load it into XFaceMaker with the **Open** option in the **File** menu. Select the DS widget in the tree or list and, in the File menu, choose the **Save Selected As ...** option. The File Selector pops, specify the file name: `if_dialog.fm`. XFaceMaker pops a Question dialog asking if you want to save the remainder of the interface. This is because, since you are saving the `dialogShell` separately, that interface fragment will have to be removed from the full interface file in order to not be loaded twice when you load your project.

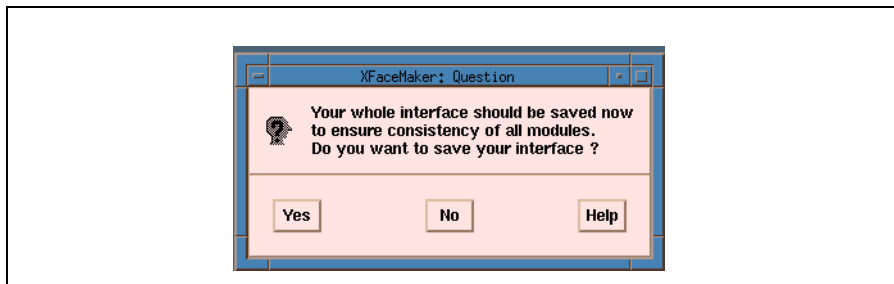


Figure 12.2: Save Whole Interface Question Dialog

Since we want to sub-divide the interface further, we don't need to save the full interface now but we could if we wanted to.

Now, select `TopLevel2`, choose the **Save Selected As ...** option, and enter `if_top2.fm` as the file name. The Question dialog will pop again, click No for now. Select `TopLevel1`, choose the **Save Selected As ...** option, and enter `if_top1.fm` as the file name. The Question dialog will pop again, and, this time, make sure you click Yes, to save the remainder of the interface, in the `if_main.fm` file which we started out from.

At this point, we have the four separate interface files. The original file, `if_main.fm` has been stripped of the contents of the three other files. It now contains only the `ApplicationShell`, the `MainWindow`, and the associated sub-tree. Now, we can create the Project file. To do so, choose the **Save Project As...** option in the File menu. The File Selector is popped, with a default project name which is the name of the main interface file, with the extension `.fmp` (here `if_main.fmp`).

The contents of the file created is:

```
_FmSetSaveOptions(' ', ' ', 2081, 0, 0, 1, 0, ' ', ' ', ' ');
_FmPOpenFile("/hot/eliz/proj/if_main.fm");
_FmPReadSubFile("/hot/eliz/proj/if_top1.fm", "", "", 1);
_FmPReadSubFile("/hot/eliz/proj/if_top2.fm", "", "", 1);
_FmPReadSubFile("/hot/eliz/proj/if_dialog.fm", "", "", 0);
```

The first line represents the set of interface generation options that will be used when building the C code for the interface. The current setting of the options is stored in the project file when the project is saved. These options include the parameters for C code generation, RDB files, Message files, UIL files. The option settings that are applied when the project is built are those that were saved in the project file, not the values that are current in `XFaceMaker`. Thus, you should set these options as desired before saving the project. Or, save the project again, after you have set the options as desired.

The rest of the `.fmp` file contains the set of commands required to load the full interface. It is executed if you choose the Open option in the File menu, and ask to load the file `if_main.fmp`. It is also executed when you generate the C code using the command line option `-compile`, e.g.

```
xfm -compile if_main.fmp
```

The first file that is loaded is the file that contains the `applicationShell`. The other files are then concatenated with the first one. The meaning of the arguments of `FmPReadSubFile` are:

- `m file_name` the name of the file to read.
- `m parent_path` the name, in `FACE`, of the widget to which the interface fragment should be connected, i.e. the name of the interface fragment's parent. Here, the field is empty, because the parent is directly the `applicationShell`.
- `m next_path` specifies the name of the toplevel widget to load next for sibling interface fragments. This field is usually empty: it is unusual to save siblings in different interface files.
- `m mapped` a flag that specifies if the interface fragment was saved as mapped (1),

or unmapped (0).

12.2 Combining Interface Files

A project can also be built from pre-existing interface files. To keep the example simple, let us assume that you have two interface files that you want to combine into a project:

- a file, `ess.fm` that contains the applicationShell, a manager and its sub-tree
- a file, `ds.fm` that contains a dialogShell and its sub-tree.

To build the project, first load file `ess.fm` into XFaceMaker, with the **Open** option in the **File** menu. Then load file `ds.fm` with the **Read** option in the **File** menu. Then, save the project file, with the **Save Project As ...** option in the **File** menu. In the File selector, specify a file name with the `.fmp` extension for the project file. This will look like this:

```
_FmSetSaveOptions(' ', ' ', 2081, 0, 0, 1, 0, ' ', ' ', ' ');  
_FmPOpenFile("/hot/eliz/proj2/ess.fm");  
_FmPReadSubFile("/hot/eliz/proj2/ds.fm", "", "", 0);
```

The contents of the file is similar to that described in the previous section.

12.3 Working with a Project

Once you have loaded a project into XFaceMaker, any **Save** or **Save As...** command will save the objects currently edited in the appropriate file, i.e. in the `.fm` file that the hierarchy containing the objects belongs to. This includes any group, template or class files. When a **Save** or **Save As...** command is called after a Save Template command, for instance, the template model will be saved to the template file, not to the main interface file. A warning is issued, however, to alert you to the fact.

Similarly, if you modify the widgets in the hierarchy under `TopLevel2`, for instance, in the example above, the changes will be stored in the file `if_top2.fm`, and `if_main.fm` will remain unchanged.

While working on the interface, there are times when you don't need to load the entire project, you can work locally on one of the interface fragments. You just have to load it with the **Open** command, and work normally on it. You won't need to load the full project, as long as you don't want to generate the application files, nor to reference widgets that belong to one of the other interface fragments.

Do not use the project mechanism to create multiple instances of an interface fragment: the existence of two identical modules in your interface could produce undesired effects. For this reason, the ***Read*** command verifies that an interface fragment is not already loaded and issues an error message if applicable.

To create multiple instances of the same interface module, load the module as a template or a group, then create multiple instances of the template or group.

12.4 Projects and Application Files

XFaceMaker generates only one set of application files for a project. To create the application files for a project, load it into XFaceMaker, set the desired save options, then do a ***Save Appli Files***.

The <name>Main.c file contains the calls to attach the active values and the functions for all the modules in the interface. It has the calls to load and map (interpreted mode), or call the creation functions (compiled mode), of all the modules that comprise the interface. If you want to instantiate some modules dynamically from the application, instead of having them all be instantiated at start up, all you need to do is move the interface module creation call to where you want it in your source code. You will also need to make sure that the arguments, such as *parent*, are known in your application code.

The <name>App.c file contains the global active value variable declarations for all the interface modules, and the skeletons of the application functions declared in all of the interface modules that belong to the project.

The <name>App.h file contains the external declarations of all the variables and functions that are in the <name>App.c file.

The <name>Make file has all the rules to build all the interface files, according to the C code and other options that were set the last time the project was saved. This includes the rules to build any new classes included in the project files.

Do not attempt to save the application files from individual modules of your project, as you would have files that pertain only to that particular module; they would be meaningless.

12.5 Undoing a Project

Occasionally, you might build a project, then change your mind on the way you want to break up the interface into individual files. To undo a project, you have to make XFaceMaker forget that the files belong to a project, that the sub-trees that constitute the interface belong to different files. The easiest way to undo a project is to:

- m load the project into XFaceMaker
- m select the applicationShell (the interface's topmost widget)
- m choose the *Save Selected As ...* option in the **File** menu
- m provide a new file name with the `.fm` extension in the File Selector.

The new file contains the complete interface, with no project sub-divisions. Working from that file, you can build a new project from scratch, with new sub-divisions and file names. Eventually, you should delete the previous project files and any associated application files, in order to avoid confusion between the new and the old project

CHAPTER 13: Creating New Widget Classes

You can define new Xt Intrinsics *classes* of widgets. The new classes are automatically integrated into XFaceMaker and used just like any standard Xt Intrinsics class. This enables you to extend the OSF/Motif toolkit to create widgets that are customized to your needs. User-defined Xt classes loaded in XFaceMaker are added to the *User Defined* widgetstore.

To create a new class you should be familiar with the following concepts:

- The **superclass** is the existing widget class that you use as a basis for describing the new widget class.
- The **subclass** is the new widget class. Thus, superclass and subclass describe the relationship between the existing widget class and the new widget class.
- A **resource** is a property of the widget. Normally, the superclass has a number of resources that are inherited by the subclass. In defining a new widget class, you may add new resources to those inherited.
- A **method** is a function associated with a widget class. XFaceMaker does not allow you to define new methods for your subclass, but you may re-define a method inherited from the Xt Intrinsics.

A new widget class is created by writing a subclass of an existing class (its superclass), using the object-oriented architecture of the Xt Intrinsics. You use XFaceMaker both to define a new widget class, and to instantiate widgets of the new class. From this starting point, you can give it additional resources and features.

A new widget class has the same resources as its superclass, (although the resources may have different default values). Normally, you will also want to define new resources for your new widget class. In XFaceMaker, you do this using active values. The active values are transformed into resources and class methods by XFaceMaker. By defining the behavior of your new widget class in active value scripts, you can build and test your new widget class interactively.

The new class may have pre-defined children. Widgets created from the new class will automatically have children created according to your description. The children will have the same resource values and hierarchical structure as those of the object you use to create the new class.

13.1 Defining and Saving a Widget Class

This section explains how to define and save a new widget class. The widget class definition is saved in a `.fm` file, in the same manner as a group, template, or any other interface description. You may define up to twenty new classes in one session. To define a new class you first create an interface object that defines the new class's superclass. Once the model object is created, *either*:

- save it as a class using the commands in the Modules menu, or
- open the Active Values window to define *new* resources and behavior for the class, and then save it.

13.1.1 Saving a Class in the Modules Menu

Once you have created the model object of the class, you can save it using the commands in the **Classes** sub-menu of the **Modules** menu. To save a new widget class:

1. Select the top level object which defines the new class.
2. Choose **Save As** from the Classes sub-menu of the Modules menu. A File Selector pops up.
3. Enter the name of the file in which to store the new class. **The first character must be an uppercase letter.**
4. Enter a name for the new class in the Class Name field, or click on the Default Name pushbutton, which automatically enters the class file name as the class name.

Following the Xt convention, the class name begins with an upper-case letter. If you intend to compile the interface, the name must be a valid C identifier. If the class is to be used from a different directory, you must save it with a relative pathname (by removing the directory from the file name in the file selector when you save the class) before you do a Save Prefs, and use the FMFILEPATH environment variable to enable XFaceMaker to locate the file.

5. Click on **OK**.

XFaceMaker saves the new widget class description in a `.fm` file, and places a default icon in the User Defined widgetstore. The icon can be changed using the **Choose Icon** command in the Classes sub-menu. Select the **Save Prefs** command in the File menu to store the new widget class in your `.xwm.startup` file for automatic re-loading in subsequent XFaceMaker sessions.

As soon as the new widget class is saved, instances of it can be created in the usual way by clicking on its icon in the Toolkit. You should delete the model object once it has been saved as a class.

The difference between **Save** and **Save As** is the following:

- **Save As** allows you to specify a name and filename for the new widget class, even

if there is a current file.

- The **Save** command saves the class definition using the name and file as last specified. If there is no current class name and file, or if the selected model object is different from the one selected during the last **Edit** or **Save As**, an alert box will warn you.

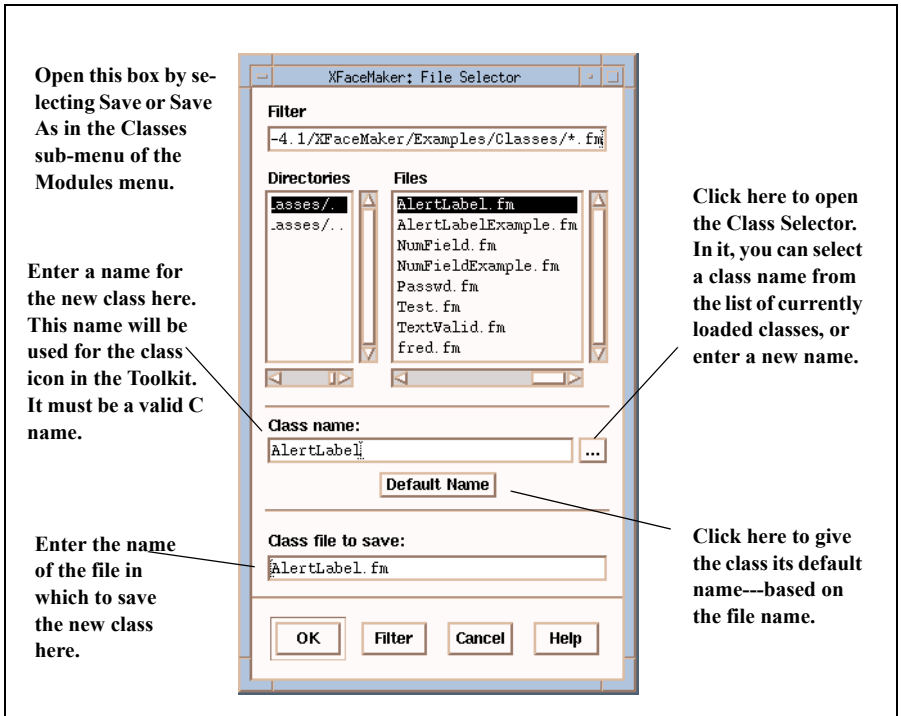


Figure 13.1: The Save Class File Selector

Note: Once you create and save a new widget class, you should destroy the object that defines the class. If you attempt to save the interface containing the object, an alert box will appear. It warns you that once a class or template is defined and saved, the model object should be destroyed. You can, of course, place an instance of the widget on the interface in the usual way, by selecting it from the Toolkit.

13.1.2 Defining a Class in the Active Values Editor

Once you have created the model object of the class, you can define new resources and behavior for it, and then save it as a class using the Active Values window.

1. Select the object and open the Active Values window.
2. Select “widget class” in the Usage field.
3. Enter a name for the new widget class in the Class Name field.
4. Choose an icon to represent the class by specifying a name in the Class Icon field. The class icon will appear in the Toolkit under User Defined. It also appears to the right of the ... (*ellipsis*) button.
5. Define resources and describe behavior for the Xt Intrinsic class methods. This is described in the sections below.
6. Click the *Save* pushbutton.

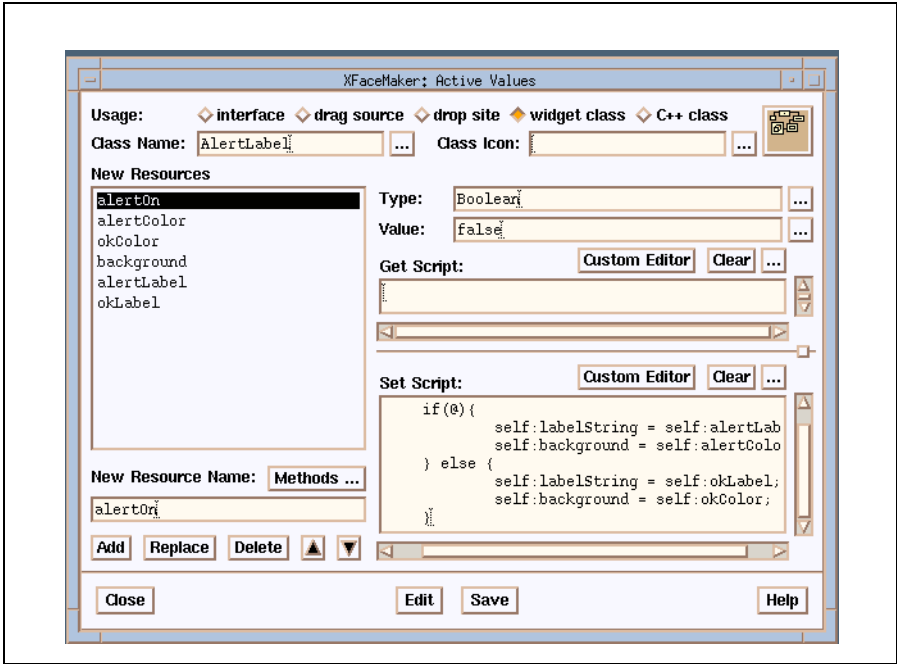


Figure 13.2: Defining a Class

13.1.3 Defining a New Resource

To define a new resource in the Active Values window:

- Enter its name in the “New Resource Name” field.
- Enter its type in the “Type” field. Or click on the ellipsis button to select a type from a list of types known to XFaceMaker. XFaceMaker knows about all Xt Intrinsics and OSF/Motif types. You may also enter the name of a new type that you define. For compatibility with the previous version, you can define the type by calling the type caster at the beginning of the *Set* script; e.g. `MyType(@)` ;
- Enter a default value in the Value field.

If you define a new type, you must also provide a type converter which will be called by the X Toolkit to convert strings to the new type. You will not be able to access resources of this type until you provide its converter. Refer to the X Toolkit documentation for a complete description of type converters and the way they must be registered. Also, you might want to design a special editing box for the new type of resource.

For enumerated types, the FACE language allows you to specify the new type, in which case XFaceMaker will automatically register a converter for the new type. You specify the names of the different values using the `type` keyword in the *Set* script of the resource. For example:

```
type Fruit = apple, banana, passion_fruit;
```

This defines a new enumerated type `Fruit`, and registers automatically a resource converter from `String` to `Fruit`. It also defines the FACE constants *PrefixAPPLE*, *PrefixBANANA*, and *PrefixPASSION_FRUIT*, where *Prefix* is the class prefix (if any) that has been specified in the C Code Options box, or `Xfm` otherwise.

Finally, you may also enter the resource’s default value in the “Value” field. If not, the default value depends on the type. To open a dialog box specific to the named type, press the ellipsis button next to the “Value” field.

XFaceMaker automatically allocates space for the new resource, returns a value to `XtGetValues`, and copies the user value given to `XtSetValues`. To specify additional behavior, such as memory allocation or to modify a resource value, use the *Get* and *Set* scripts for the resource. In *Get* scripts, the special FACE variable `@` refers to the resource. In *Set* scripts, `@` refers to the new value, `@1` refers to the old value, and `@2` contains a pointer to the widget. If the resource’s value is being set for the first time during the widget’s initialization, `@2` will contain a NULL pointer.

13.1.4 Xt Intrinsic Class Methods

A widget class *method* is a function that you can call to operate on an object. You can define behavior for an Xt Intrinsic class method using the Active Values window.

- Click on the **Methods** button in the Active Values editor to pop up a list of methods.
- Select a name from the list. The name you select will appear in the “New Resource Name” field, although it really signifies the name of the Xt Intrinsic class method. You should leave the “Type” field, “Value” field, and *Get* script blank.
- Use the *Set* script to describe the behavior for the named method.

For simple widget classes, most of the methods need not be defined—the inherited methods can usually be used. In most cases, you will only need to define the *Get* and/or *Set* scripts for your new resources. You may also need to initialize variables, declare functions or register converters in the Initialize method. Other methods such as *Realize*, *SetValuesAlmost*, etc... are used less often.

Only the methods of the Core and Composite classes can be defined using active values. Widget designers who wish to define other methods defined by a sub-class of Core or Composite must do so by calling a C function that sets the appropriate pointers in the class record. This function must be called in the *ClassInitialize* method.

The special FACE variable \$ contains a pointer to a structure containing the arguments to the method. See the reference manual and your Xt documentation for details on Xt Intrinsic class methods.

13.1.5 Loading a Widget Class

User created widget class files can be loaded automatically each time you launch XFaceMaker, or they can be loaded during a session.

- To load an existing widget class automatically at the next launch time, use the **Save Prefs** command in the File menu to save your current configuration, which includes any user-defined classes.
- To load a widget class file during a session:
 1. Select **Load** from the Classes sub-menu of the Modules menu. The File Selector pops up.
 2. Select the class definition (.fm) file to load and click **OK**. The icon is displayed in the User Defined widgetstore, and you can now instantiate a widget from the new class. Only the class name will appear in the Widget Tree and/or List if open.

The number of classes that can be dynamically loaded into XFaceMaker is limited to 20.

13.1.6 Editing a Widget Class

You can modify an existing widget class as follows:

1. Select **Edit** from the Classes sub-menu. The Class Name box pops up, listing the user-defined classes that are currently loaded in XFaceMaker.
2. Select the name of the widget class to be edited and click on **OK**. The entire hierarchy of widgets which comprise the class will be displayed in the Widget Tree and/or List if open.
3. Edit the widget class and save it using the commands in the Classes sub-menu of the Modules menu, or the Active Values editor.

13.1.7 Deleting a Widget Class

Deleting a widget class removes it from the User Defined widgetstore. However, the class definition file remains and can be re-loaded later. To delete a widget class:

1. Select **Delete** from the Classes sub-menu. The Class Name box pops up listing all user-defined widget classes currently loaded.
2. Select the name of the widget class to be deleted, and click **OK**. This removes the class icon from the User-Defined widgetstore but does not delete the corresponding widget class because the X Toolkit provides no support for that. The instances of the class that you may have created are not destroyed.

13.2 Generating C Code for a New Widget Class

To generate the C code for a widget class, you must set the necessary options in the C Code Options window before you save the class.

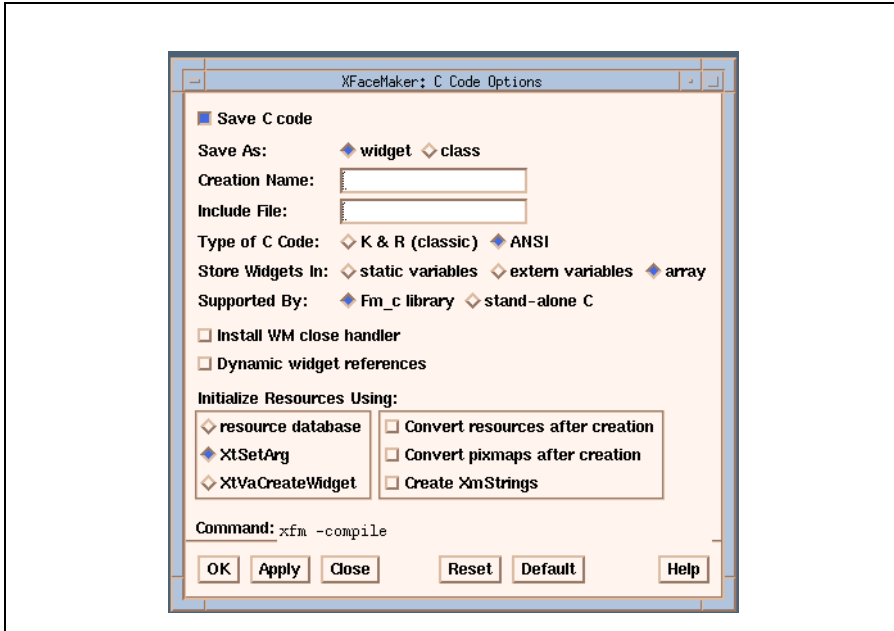


Figure 13.3: Saving a C code version of a Class

To save a C code version of a new widget class:

1. Set the **Save C code** checkbox in the C Code Options window.
2. Select the appropriate options. See Section 11.3 about C code options.
3. Click on **OK** or **Apply** to apply the options.
4. Save the class as explained in Section 13.1 using the commands in the Classes sub-menu of the Modules menu. Or use the Active Values Editor with the Usage field set to **widget class** (see Section 8.3.4).

Note: if you have defined a prefix for your class, you should arrange to have your .h file for the class in a directory with the same name as your prefix. For example, for a class `MyClass` with prefix `MyPrefix`, you should have `MyPrefix/MyClass.h`. To achieve this, you may need to create the directory `MyPrefix` and move the file `MyClass.h` into it.

Using the Command Line Options

Alternatively, you can use the command line options:

```
-compile -cflags g
```

to produce your C-code widget class file. You can combine this option with other options, but you should note that the option *Store Widgets in Array* (or `-cflags a`) will be forced to True.

Also, the option *XtSetArg* (or `-cflags c`) is forced for the creation of sub-widgets in the class, unless the option *XtVaCreateWidget* (or `-cflags v`) was set. See section 11.3 for a description of the C code options.

13.3 Contents of the Class File

In the simplest case, for a widget class called *Name*, XFaceMaker will generate three files, along with the .fm file:

- *Name.c* which contains the C-code for the definition of the widget class, including the function `FmCreate<Name>`, which creates an instance of the new widget,
- *Name.h* which contains the public declarations used by programs that create instances of the class, i.e., widget class pointer declaration and symbolic resource names,
- *NameP.h* which contains the private declarations used to derive the widget class, i.e., widget and class record definitions.

The file names can be overridden by using the `-cfile` option; where, for instance, the full widget class name would breach the fourteen character pathname limit.

Naming is also influenced by the *Class prefix* option, or `-cprefix <Prefix>`. The prefix defined is used in much the same manner as that of `Xm` for the OSF/Motif widgets. The class name becomes `PrefixName`, the include files are expected to be in the `Prefix` directory, (as in `#include<Prefix/NameP.h>`), and resource values begin with the prefix.

The names follow the conventions set by the Xt Intrinsics. For a class called *MyBox*, the name of the class pointer is *myBoxWidgetClass*. If the Creation Name option is used with the text *OtherName*, the name of the class will be *OtherName*, and the class pointer will be *otherNameWidgetClass*. With a prefix string of `Prefix` this becomes `PrefixOtherName` and `prefixOtherNameWidgetClass` respectively.

The generated C-code file contains a convenience function `FmCreateName`, which creates an instance of the new widget class, or you can call `XtCreateWidget` directly. If you set the *stand-alone C* option, the files contain only Intrinsic and Motif library calls, unless you have made calls to XFaceMaker functions in any FACE scripts.

The generated C-code file also contains a function `FmAddNameWidgetClass` which can be called to register the new widget class and its associated resource types in a new instance of XFaceMaker. The definition of this function is enabled by the compilation flag `XFM`. `FmAddNameWidgetClass` calls `FmAddWidgetClass()` and `FmSetSuperclassInfos()` with the appropriate values for the new widget class, so that the new widget class is added to the User Defined widget-store and can be used to create widgets and also to create new sub-classes of the added class. See Chapter 16 for details.

13.4 Hints

This section contains a few hints that are useful for creating new widget classes with XFaceMaker.

13.4.1 Widget Size

Usually, you will not want to specify a special size for your new widget class, but rather use the default size of its superclass. To do this, set the *autosize* flags for the toplevel model object of your class. Otherwise the values of the *width* and *height* resources will be saved in the new class, and all instances of the class will have these default values. XFaceMaker never saves *x,y* resources for classes, but only *width* and *height*.

13.4.2 Compound Widgets

New widget classes often consist of several sub-widgets inside a composite widget. For example, a dialog box might consist of a `XmBulletinBoard` inside which buttons and labels are created. When you define such “compound” widget classes with XFaceMaker, you should pay attention to the following:

- Only the active values defined in the *toplevel* widget of the model object are saved as new resources for the new class. This means that if you want to be able to change a resource of a sub-widget, you must add a new resource to the toplevel widget that will propagate the resource value to its child. An example of this technique is the `fontList` resource of the `NumField` example described in Section 13.6.2.
- You must handle *colors* properly. In the OSF/Motif toolkit, colors are not inherited automatically by the children of a composite widget. For example, if you change the background color of a composite, it will not affect the background colors of its descendants. This means that you should define *Set* scripts for the color resources of the toplevel widget of your class. The *Set* scripts will simply propagate the color change to the descendants. See the `NumField` example in Section 13.6.2.

13.4.3 File Organization

In most cases, a new widget class will require additional C code to be written by the designer, because it would not be possible (or reasonable) to program all the functionalities of your new class in FACE scripts. This is the same as for a normal application: the look of the widget and the interactions between its various components and resources are programmed in FACE, while the basic functions are programmed in C.

The additional C code of your widget class will be called from the FACE scripts of your class methods as functions. The C functions used by your new widget class should be put in a separate file. A useful convention is to use the name of the new widget class with a **F** (for “Functions”) suffix.

For example, if your widget class is called **MyClass**, you could put your C functions in a file called **MyClassF.c**. The declarations of the functions should be put in **MyClassF.h**, which can be included in the C file generated by XFaceMaker for the new class with the `-cinclude` option.

Thus, for the class **MyClass**, you will have the following files:

MyClassF.c: contains the C functions implementing the functionalities of the new class; this file is written by the widget designer;

MyClassF.h: contains the external declarations of the functions in **MyClassF.c**; this file is written by the widget designer;

MyClass.fm: contains the description of the widget class in `.fm` format; this file is created by XFaceMaker when the widget class is saved;

MyClass.c: this file is created by XFaceMaker if the widget class is also saved as C code, and contains the definition of the class in C;

MyClass.h: this file is created by XFaceMaker if the widget class is also saved as C code, it contains the public declarations for the widget class;

MyClassP.h: this file is created by XFaceMaker if the widget class is also saved as C code. It contains the private declaration of the class and the instance records for the new class.

The C files **MyClass.c**, **MyClass.h** and **MyClassP.h** are typically created using the command line:

```
xfm -compile MyClass.fm -cfile MyClass.c -cname \
    MyClass -cflags gvSC
```

The flags mean:

g generate widget class definition,

v use **XtVaCreateWidget**,

S generate stand-alone C code (so that you don't need the `libFm_c.a` library with your widget)

C generate ANSI C.

Note that if you put your widget class in a library, it is good practice to prefix the functions of `MyCLASSF.c` to avoid name clashes with the applications using your widget. For example, we at NSL use the prefix `_Xnsl` for all our private widget functions.

13.4.4 Stand-alone C for a New Class

Since widget classes are often packaged as libraries, you may want the code for the widget class to be self-contained; i.e. to contain no calls to `libFm_c.a` functions in the C code. To achieve this, you must specify the `-cflags S` (or equivalent `-standc`) option when you translate your widget class into C code. You must also follow a certain number of rules in the FACE scripts of your widget class to be sure that XFaceMaker generates correct C code. The rules for generating stand-alone C code are explained in Section 11.6.

However, widget classes may need to use FACE functions. In that case, they cannot be compiled in stand-alone C and must be linked with the `Fm_c` library.

To link the widget into XFaceMaker, the `Fm_e` library is also needed. See section 10.7, Compatibility with `libFm_c` and `libFm_e`.

13.4.5 Callbacks

New callbacks for a class are defined exactly like normal resources: simply create an active value with the name of your callback, and with the type `Callback`:

```
activevalue {
    name = myCallback
    type = Callback
}
```

Callback resources usually have no side-effects: they just contain a callback list. Note that you *must* use the type `Callback` for your callbacks so that the toolkit can initialize its internal data structures properly for callback lists.

The callbacks of your new widget must be activated from another action of the widget by calling `XtCallCallbacks`. Note that you have to specify a `call_data` argument: this can be either a simple value, or a structure created by a C function defined in the auxiliary file of the class `MyClassF.c`. See the `NumField` example in Section 13.6.2.

13.4.6 Building A New XFaceMaker

You might want to design and test your widget class interactively with your functions. To do this, you must do the same as for normal applications: you have to build a new `XFaceMaker` linked with the functions of your widget class contained in `MyClassF.c`. For example, if you have two functions `MyFunction1` and `MyFunction2` in `MyClassF.c`, you can use the file generated by `XFaceMaker`, for example:

```
#include <Fm.h>
#include <MyClassF.h>

main(argc, argv)
int argc;
char **argv;
{
    Widget toplevel =
        FmInitialize("my_xfm", "Xfm",
                    NULL, 0, &argc, argv);

    FmAttachFunction("MyFunction1", MyFunction1, ...);
    FmAttachFunction("MyFunction2", MyFunction2, ...);

    FmCallEditor();
}
```

Then, you must link this main program with the `libFm_e.a` library to produce a new `XFaceMaker` with your functions linked in:

```
cc -o my_xfm main.o MyClassF.o -lFm_e -lXm -lXt -lX11 -lm
```

With this new `XFaceMaker`, you will be able to load your widget class and test it interactively. If you save your new widget class as C code, you might want to test the C version as well as the interpreted version. To do this, you must generate the C files of your widget class, for example using the command line options given in this chapter. Then, compile the file `MyClass.c` with the `-DXFM` flag, to enable the function `FmAdd<ClassName>WidgetClass`:

```
cc -c -DXFM MyClass.c
```

```
mv MyClass.o MyClass_XFM.o
```

The `main.c` should then be modified to cope with both the interpreted and compiled cases:

```
#include <Fm.h>
#include <MyClassF.h>

main(argc, argv)
int argc;
char **argv;
{
    Widget toplevel =
        FmInitialize("my_xfm", "Xfm", NULL, 0, &argc, argv);

#ifdef CCODE
    FmAddMyClassWidgetClass();
#else
    FmAttachFunction("MyFunction1", MyFunction1, ...);
    FmAttachFunction("MyFunction2", MyFunction2, ...);
#endif

    FmCallEditor();
}
```

The file `main.c` must be compiled with the flag `-DCCODE` to produce an object file suitable for linking with the compiled widget class:

```
cc -c -DCCODE main.c
mv main.o main_CCODE.o
cc -o my_xfm_CCODE main_CCODE.o MyClass_XFM.o \
    MyClassF.o -lFm_e -lXm -lXt -lX11
```

Note that since `MyClass.c` was generated with the `S` (standalone) flag, you do not need to link the `libFm_c.a` library. Finally, once your new widget class is usable, you can package it in a library. You must compile the C file of your class without the `-DXFM` flag. Otherwise, your object file would contain undefined references to `libFm_e.a` functions called by `FmAddMyClassWidgetClass`:

```
cc -c MyClass.c
ar cv libMyWidgets.a MyClass.o MyClassF.o ...
```

13.4.7 Sub-Classing At Arbitrary Levels

New widget classes designed using `XFaceMaker` can in turn be sub-classed to an arbitrary level: an instance of a class *MyClass* made with `XFaceMaker` can be used to build a new sub-class *MySubClass* and so on.

To generate the C code file for *MySubClass* using an `xfm` command line in a `Makefile`, you will need to load the class *MyClass*, because `XFaceMaker` needs to know a widget class to produce a sub-class. To do this, you can use the option `-startup`. You must create a FACE startup file, *MyClass.face*, containing a call to the `LoadWidgetClass` function:

```
LoadWidgetClass("MyClass", "MyClass.fm");
```

and pass this file to `XFaceMaker` when compiling the class *MySubClass*:

```
xfm -startup MyClass.face -compile MySubClass.fm ...
```

This option avoids the need to create a new `XFaceMaker` with **MyClass** loaded to compile the sub-class **MySubClass**.

Similarly, to compile an application that instantiates a widget of class *MySubClass*, you will have to use the `-classes` option to specify both the class and the subclass:

```
-classes MyClass:MyClass.fm, MysubClass:MySubClass.fm
```


13.5 How XFM Generates Sub-Classes

13.5.1 The Class Record

The class record for the new class is an aggregate structure composed of the class parts of all superclasses (including the class part of the model object class), plus the class part of the new widget class, which is empty.

All fields of the class record are initialized to the appropriate inheritance value, except the following fields of the `CORE` class structure, which have values computed from the model object: **superclass**, **class_name**, **widget_size**, **class_initialize**, **initialize**, **actions**, **num_actions** **resources**, **num_resources**, **destroy**, **resize**, **expose**, **set_values** **get_values_hook**, **tm_table**.

13.5.2 The Widget Instance Record

The widget record for the new class is an aggregate of the widget parts of all the superclasses, plus the widget part of the new class. The new widget part contains an array containing the widget IDs of the widget's children, and locations where the values of the new resources, corresponding to the active values of the model object, will be stored.

13.6 Examples

This section contains a few commented examples of new widget classes defined with XFaceMaker. The files containing these examples are included in the XFaceMaker distribution. Each example includes a listing of the .fm file containing the description of the class.

13.6.1 Alert Label

```

1:  widget XmLabel {
2:      flag {
3:          autopos
4:      }
5:      activevalue {
6:          name = alertOn
7:          set = if (@) {
8:  self:labelString = self:alertLabel;
9:  self:background = self:alertColor;
10: } else{
11:  self:labelString = self:okLabel;
12:  self:background = self:okColor;
13: }
14:         type = Boolean
15:         storage = object
16:         automatic = true
17:         usage = class
18:         default_type = String
19:         default_value = False
20:     }
21:     activevalue {
22:         name = okLabel
23:         set = if (!self:alertOn)
24:  self:labelString = @;
25:         type = XmString
26:         storage = object
27:         automatic = true
28:         usage = class
29:         default_type = String
30:         default_value = EVERYTHING'S OK
31:     }
32:     activevalue {

```

```
33:         name = alertLabel
34:         set = if (self:alertOn)
35:     self:labelString = @;
36:         type = XmString
37:         storage = object
38:         automatic = true
39:         usage = class
40:         default_type = String
41:         default_value = ALERT ...
42:     }
43:     activevalue {
44:         name = alertColor
45:         set = if (self:alertOn)
46:     self:background = @;
47:         type = Pixel
48:         storage = object
49:         automatic = true
50:         usage = class
51:         default_type = String
52:         default_value = red
53:     }
54:     activevalue {
55:         name = okColor
56:         set = if (!self:alertOn)
57:     self:background = @;
58:         type = Pixel
59:         storage = object
60:         automatic = true
61:         usage = class
62:         default_type = String
63:         default_value = green
64:     }
65:     resources {
66:         width = 124
67:         height = 65
68:         recomputeSize = false
69:         fontList = *adobe-helvetica-medium-r-*-12-*
70:     }
71: } La
```

- (1) Shows that the model object is an `XmLabel`, so the new class will be a subclass of `XmLabel`.
- (3) The `autopos` flag indicates that the new widget class does not define a default position. If this flag were not set, the `x,y` resources for the model object would appear in the class `.fm` file, but they would be ignored by `XFaceMaker`.
- (6) The active value defines the new resource `alertOn`. There is no get script, so the resource implements no side-effects when it is read by `XtGetValues`.
- (7-13) The set script defines the side-effects desired when this resource is set (also executed at widget initialization). If the new value of the resource, contained in `@`, is `True`, then the label string of the widget is changed to the value of the new resource `alertLabel`, and the background color is changed to `alertColor`; otherwise, the label and color are set to `okLabel` and `okColor`. Note that, in this example, there is no verification that the resource value has actually changed: the script will be executed even if the application sets the `alertOn` resource to its current value.
- (14) The resource is of type `Boolean`.
- (15-17) These active value flags are set automatically by `XFaceMaker`.
- (18-19) The default value type is `String`, the default value is `False`.
- (21-31) Define another new resource, `okLabel`. It is of type `XmString`. Its default value type is `String`, and the default value is `EVERYTHING'S OK`. The side effect for this resource is to test the state of the `alertOn` resource. If it is `False`, the new value of `okLabel` (contained in `@`) is copied into the widget's `labelString` resource.
- (32-42) The resource `alertLabel` is similar to `okLabel`.
- (43-53) The resource `alertColor` defines the widget's background color when `alertOn` is `True`. It is of type `Pixel`, its default type is `String`, the default value is `red`.
- (54-64) The resource `okColor` is similar to `alertColor`.
- (66-67) Specify the default size for instances of the `AlertLabel` class.
- (68) Prevent widget resizing when the `labelString` changes.
- (69) Specify a new default value for the `fontList` resource.

13.6.2 The *NumField* Class

The *NumField* widget is a compound widget, composed of an *XmForm* containing an *XmTextField* widget and an *XmScale* widget. The widget can be used to select a numeric value, either by typing the value in the text field, or by dragging the scale to the desired value. Dragging the scale prints the value in the text field, and typing Return in the text field updates the scale position. Here is the description of the *NumField* in .fm format.

```

1:  widget XmForm {
2:      flag {
3:          autosize
4:          autopos
5:      }
6:      activevalue {
7:          name = value
8:          set = if (@ > self:maximum)
9:              {@ = self:maximum; self:value = @;}
10:         else if (@ < self:minimum)
11:             {@ = self:minimum; self:value = @;}
12:         self.scale:value = @;
13:         String buf = XtMalloc(40);
14:         sprintf(buf, "%d", @);
15:         self.text:value = buf;
16:         XtFree(buf);
17:         if (@ != @1){
18:             XtCallCallbacks(self,"valueChangedCallback",$); }
19:         type = Int
20:         storage = object
21:         automatic = true
22:         usage = class
23:         default_type = String
24:         default_value = 0
25:     }
26:     activevalue {
27:         name = minimum
28:         set = self.scale:minimum = @;
29:         type = Int
30:         storage = object
31:         automatic = true
32:         usage = class

```

```

33:         default_type = String
34:         default_value = 0
35:     }
36:     activevalue {
37:         name = valueChangedCallback
38:         type = Callback
39:         storage = object
40:         automatic = true
41:         usage = class
42:     }
43:     activevalue {
44:         name = foreground
45:         set = self.scale:foreground = @;
46:         self.text:foreground = @;
47:         type = Pixel
48:         storage = object
49:         automatic = true
50:         usage = class
51:     }
52:     activevalue {
53:         name = background
54:         set = self.scale:background = @;
55:         self.text:background = @;
56:         type = Pixel
57:         storage = object
58:         automatic = true
59:         usage = class
60:     }
61:     activevalue {
62:         name = fontList
63:         set = self.text:fontList = @;
64:         type = FontList
65:         storage = object
66:         automatic = true
67:         usage = class
68:     }
69:     activevalue {
70:         name = maximum
71:         set = self.scale:maximum = @;
72:         type = Int

```

```
73:         storage = object
74:         automatic = true
75:         usage = class
76:         default_type = String
77:         default_value = 100
78:     }
79:     resources {
80:     }
81:     children {
82:         widget XmScale {
83:             flag {
84:                 mapped
85:                 autosize
86:                 autopos
87:             }
88:             resources {
89:                 orientation = horizontal
90:                 bottomAttachment = attach_form
91:                 leftAttachment = attach_form
92:                 rightAttachment = attach_form
93:                 valueChangedCallback = parent:value = self:value;
94:                 dragCallback = parent:value = self:value;
95:             }
96:         } scale
97:         widget XmTextField {
98:             flag {
99:                 mapped
100:                 autosize
101:                 autopos
102:             }
103:             resources {
104:                 rightAttachment = attach_form
105:                 topAttachment = attach_form
106:                 leftAttachment = attach_form
107:                 bottomAttachment = attach_widget
108:                 bottomOffset = 10
109:                 bottomWidget = scale
110:                 columns = 10
111:                 activateCallback =
112:                 parent:value = atoi(self:value);
```

```

113:         }
114:     } text
115: }
116: } Form

```

- (1) Shows that the toplevel object of the class model is an `XmForm`, so the `NumField` class will be a sub-class of `XmForm`.
- (6-25) The active value `value` defines a new resource, `value`, which is of type `Int`. When this resource is initialized or set, it has the following side-effects:
 - m Check the value passed is within bounds. If it is larger than `maximum`, set it to `maximum`. If it is smaller than `minimum`, set it to `minimum`.
 - m Set the value of the `XmScale` to the same value (line 12).
 - m Print the value in decimal format into the text field using `printf` (lines 13-16).
 - m If the value has actually changed (test line 17), call the new callback `valueChangedCallback`; the `call_data` is the argument of the set script, i.e. a structure containing the new and current values.
- (26-35) The `minimum` resource is an example of how a resource of a sub-widget can be made accessible from the toplevel object. Its set script simply assigns the value to the corresponding resource of the `XmScale` widget. The `maximum` and `fontList` resources are similar to `minimum`.
- (36-42) Define the new callback `valueChangedCallback`. This resource has no side-effects. The callback is activated by the `value` resource.
- (43-51, 52-60) Shows how you can propagate changes to the colors of the toplevel widget to the sub-widgets.
- (81-111) Define the sub-widgets of the model object. These sub-widgets will be created by the `Initialize` method of the new class every time an instance is created.
- (82-96) Define the `XmScale` sub-widget. Its resources will be set when it is created by the `Initialize` method. Note that the callbacks of the scale reference the new resource `value` of the toplevel widget. Of course, if you switch to Try mode while you design the widget and you move the scale, `XFaceMaker` will complain that the resource does not exist yet: it will be created only when the `XmForm` widget is saved as a new class.
- (97-114) Define the `XmTextField` sub-widget.

CHAPTER 14: Developing in C++ with XFM

Interfaces built with XFaceMaker can be integrated into C++ applications. There are two ways to do this:

- You can generate C++ classes corresponding to interface components as described in Chapter 15.
- You can generate a C code version of the interface (as described in Chapter 11), and compile it with a C++ compiler.

In both cases, XFaceMaker enables you to call C++ application code from FACE scripts. FACE accepts new constructs in scripts to define the C names of FACE functions and types, and to access C++ objects.

14.1 Generating C++ Code

The existing option `-ansiC` (or its equivalent `-cflags C`) generates fully prototyped code which can be compiled either by an ANSI C compiler or a C++ compiler. This means that all functions (static or external) generated by XFaceMaker are declared with a prototype. When the syntax differs between ANSI C and C++ (essentially for functions without arguments), both forms are output with conditional compilation flags. This also means that the arguments of all function calls in FACE scripts are cast to their C or C++ types. See Chapter 11.

14.2 Calling Member Functions from FACE Scripts

It is possible to call a C++ member function in a FACE script for a pointer to an instance of that class using the same syntax as in C++. For example, if `o` is a pointer to an instance of class `Foo`, and `bar` is a member function of the class, `bar` can be called for object `o` by writing:

```
Foo o; i = o->bar("hello");
```

You must declare the member function like an ordinary C function, but with its full name of the form `Foo::bar`. The FACE parser recognizes functions whose name contains the two characters `::` as C++ member functions.

For instance, the method declaration in the application will look like:

```
class Foo {
    ...
public:
    ...
    int bar(char*);
    ...
};

...
FmAttachFunction
("Foo::bar", (int(*)())&Foo::bar, "Int", 1, "String");
...
```

The address passed to `FmAttachFunction` can be either the address of the C++ member function if your C++ compiler allows it, or the address of a static member function which calls it. See Section 14.3.2 for a discussion of the two options. The member function can also be declared in the FACE script with the `function` keyword:

```
function Int "Foo::bar" (String);
```

Pointers to instances of C++ classes are ordinary variables, but they must be declared statically with the same type as the class name, for example:

```
Foo o;
```

Pointers to instances of C++ classes are always obtained from the C++ application (usually as the result of a function call or through active values). They cannot be generated by FACE. The FACE parser transforms calls of C++ methods into ordinary function calls by adding the implicit `this` parameter of the member function as the first argument. The address of the C++ method is obtained by concatenating the type of the instance pointer with the name of the member function, and then looking up the FACE functions table.

When the FACE script is compiled into C++ code, the FACE compiler generates a plain C++ call unless another C++ type name has been specified using the `type` keyword. For example, in the following script:

```
Foo o;
type Foo "class foo *";
o->bar(...);
```

the member function call will be compiled as:

```
((class foo *)o.value)->bar(...);
```

14.2.1 A Complete Example

Let us assume we have C++ class with a member function **welcome**, which takes a string argument and returns void. The .h file (let us call it `myClass.h`) in which this class is defined is like this:

```
class myClass {
public:
    void welcome(char* str);
};
extern class myClass * create_myClass();
```

We will build a simple interface to illustrate how the member function can be called from a FACE script.

Our interface consists of an ApplicationShell, a Form, a PushButton. We will call the interface description file `excpp.fm`.

In the `xfmCreateCallback` of the PushButton, we create an instance of the class **myClass** with the following script:

```
1: type myClass "class myClass *";
2: function myClass create_myClass();
3: global myClass mc;
4: mc = create_myClass();
```

Line 1 declares a new FACE type, the C++ class **myClass**

Line 2 declares a C function, **create_myClass()**, whose definition can be found in the `main.c` file below. When called, this function creates an instance of the class **myClass**.

Line 3 declares a global variable, **mc**, with type **myClass**

Line 4 creates an instance of the class; **mc** is a handle to the instance.

In the PushButton's `activateCallback`, we call the **welcome** member function with the following script:

```
1: global myClass mc;
2: function None "myClass::welcome" (String str);
3: mc->welcome("welcome member function has been called");
```

Line 1 declares the global variable, **mc**, so that the value returned to mc in the `createCallback` script is accessible in the `activateCallback` script.

Line 2 declares the member function **welcome**; it is defined for class **myClass**, returns nothing, takes one argument of type String.

Line 3 calls the member function with the text that is to be printed.

We want to be able to run this interface in compiled mode and in interpreted mode. For the compiled mode, we need to generate the C code for the interface file. In order to have it compile without errors, we need to make the class **myClass** known in the .c file by including the class definition file. To do so, you can open the C code options box and enter “myClass.h.” in the text field next to the ***Include File:*** label, set the ***Save C code*** toggle and save your interface; or you can use the `-cinclude “myClass.h”` command option when generating the C code using batch mode. In both cases, make sure you are generating C code with the ANSI C option, as you will need to compile the file with a C++ compiler.

Now, let us look at the main program. Because it is so simple, we can create it directly. It has to:

1. Include the class description file **myClass.h**.
2. Define the C function **create_myClass()** and the class member function **welcome**.
3. Load and create the interface. In compiled mode, we just instantiate the interface (CCODE defined at compile time). In interpreted mode, we attach the functions, load and instantiate the interface, show the widgets. When attaching the member function, we have assumed that the GNU C++ compiler is used.

Here is code for the main program:

```
#include <stdio.h>
#include <Fm.h>
#include "myClass.h"
extern Widget FmCreateexcpp(String,Widget,ArgList,Cardinal);
class myClass * create_myClass(){
class myClass * mc;
mc = new myClass();
return(mc);
}
void myClass::welcome(String str){
printf("-----\n");
printf("%s\n", str);
}
main (int argc, char **argv) {
Widget toplevel, parent;
toplevel = FmInitialize("excpp", "Excpp", 0, 0, &argc, argv);
#ifdef CCODE
    toplevel = FmCreateexcpp("excpp", toplevel, 0, 0);
#else
```

```

    FmAttachFunction("create_myClass",
                    (FaceFunctionPtr)create_myClass, "myClass", 0);
    FmAttachFunction("myClass::welcome", (FaceFunctionPtr)
                    &myClass::welcome, "None", 1, "String");
    toplevel=FmLoadCreate("excpp", "excpp.fm", toplevel, 0, 0);
    FmShowWidget(toplevel);
#endif
    FmLoop();
}

```

The compiled version can be built with:

```

g++ -c excpp.c
g++ -DCCODE -c excppMain.c
g++ -o cexcpp excppMain.o excpp.o -lFm_c -lXm -lXt -lX11 -lm

```

The interpreted version can be built with:

```

g++ -c excppMain.c
g++ -o iexcpp excppMain.o -lFm -lXm -lXt -lX11 -lm

```

When executing the program, (here cexcpp but iexcpp yields the same) you should get:

```

chryso: cexcpp &
chryso: -----
welcome member function has been called

```

When you generate the application files with XFaceMaker, rather than writing the main program directly as we did in this simple example, the member function is attached in `xxMain.c`. It is not declared in `xxApp.h` or defined in `xxApp.c` since it is assumed that member functions are defined with the C++ class.

14.3 Limitations

Due to the static nature of the C++ class mechanism as opposed to the dynamic nature of the FACE interpreter, there are a number of limitations concerning the way FACE can call C++ functions. These limitations concern only the *interpreted* mode: the FACE compiler should always generate correct C++ code.

14.3.1 Argument Passing Rules

In the FACE language, all values have the same size (except floating-point variables). This is also true for function arguments: when the FACE interpreter calls a C or C++ function, all the arguments are passed as single-word (32 bits) values (or double-precision floating-point values).

Because of the different argument-passing rules in C++ and in C, problems can arise when a C++ function is called in a FACE script in interpreted mode. To avoid this, all C++ functions of an application which are to be called from FACE scripts should accept only single-word or floating-point values.

For example, a C++ function which takes a `short` or a `float` argument will not be called correctly by the FACE interpreter, because the argument will be converted to an `int` or a `double`. Such a function should accept an `int` or a `double` instead.

14.3.2 C++ Member Functions

The mechanism described above to attach a C++ member function so that it can be called from a FACE script in interpreted mode, relies on nonstandard and undocumented features of C++ compilers. The first problem is to obtain the address of a member function so that it can be passed to `FmAttachFunction`. If you use the GNU C++ compiler (`g++`), this can be done using the syntax:

```
&Foo::bar
```

whereas if you use the AT&T C++ compiler (`CC`), the syntax is:

```
Foo::bar
```

The second problem is to call a C++ class through an ordinary function pointer. The FACE interpreter does this by adding the implicit ‘this’ argument as the first argument, which assumes that the C++ compiler is implemented using this technique. Although it is the case for the most commonly used C++ compilers `g++` and `CC`, it is not guaranteed to be true for all future implementations.

If you do not wish to use pointers to functions, here is a cleaner (although more tedious) solution:

1. For every member function that you want to attach to the FACE interpreter, you must write a *static* member function which takes an object pointer as the first argument. The *static* member function calls the member function for the object.
2. You can then pass the address of the *static* member function to `FmAttachFunction` instead of the address of the member function.

Using this technique, the example above would look like:

```
class Foo {
    ...
public:
    ...
    int bar(char*);
    static int Foo_bar(Foo*, char*);
    ...
};

...
int Foo::Foo_bar(Foo* foo, char *s)
{
    return(foo->bar(s));
}

...
FmAttachFunction("Foo::bar",
    (int(*)())Foo::Foo_bar, "Int", 1, "String");
...
```

In this way, the FACE interpreter transforms the call `o->bar(...)` into a call to the static member function, but in the generated C code the actual member function will be called. This means that static member functions need be defined only if your application runs in interpreted mode.

CHAPTER 15: Creating C++ Classes

XFaceMaker enables you to generate C++ classes corresponding to interface modules. A C++ class created using XFaceMaker can be viewed as a user-defined wrapper for a widget tree. It provides a set of constructors and a destructor to control the creation and destruction of instances of the widget tree, plus a set of public member functions (methods) which permit its manipulation while hiding the details of its internal structure.

A C++ class is defined from a widget tree, characterized by its top widget. The members of the class are defined by means of active values associated with this widget. You can define data members as well as member functions: data members correspond to active values with a *per object* storage, whereas member functions correspond to active values with a *per call* storage. Members can be declared public, protected or private. For data members, you can require that access functions (set and get) be automatically generated. Member functions are always declared virtual; their body is defined in FACE, like any active value script.

Two predefined “styles” of code generation are supported: the NSL style, and the Rogue Wave© style. Choosing a style determines which class to use as base class of the class generated, as well as the signature of the member functions.

- The NSL style: the class generated is derived from a predefined base class, called NslUIC, which provides constructors and useful member functions (show, hide, getWidget...).
- The Rogue Wave style: the class generated is derived from a class of the View.h++¹ library, which is chosen automatically among the Widget wrappers (also called Controllers) provided by this library. Constructors and access functions for data members are defined following the conventions adopted in View.h++. Using this style generates classes which are easy to combine with classes of the View.h++ library.

1. View.h++ is a trademark of Rogue Wave Software, Inc.

15.1 Defining a C++ Class

To define a C++ class with XFaceMaker, you first have to define the widget tree which you want to encapsulate by the class; each future instance of the class will hold its own copy of this widget tree. Then, select the top widget of the tree and open the Active Values box.

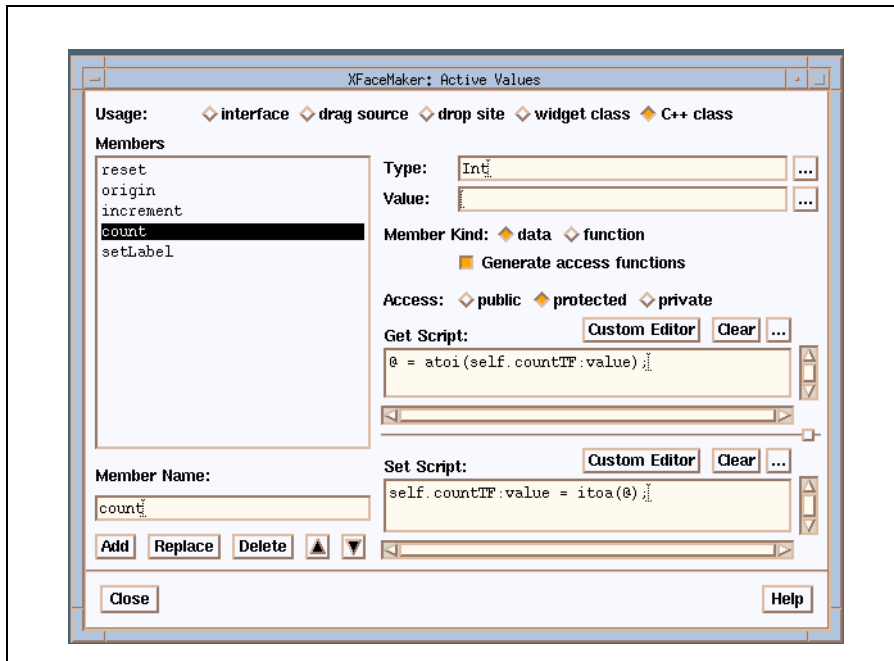


Figure 15.1: Defining a C++ Class

Specify “C++ class” in the “Usage” field. The window gets configured in a way which is appropriate for editing class members. Existing members appear in the list on the left.

The following sections describe how to edit the remaining fields in this window.

15.1.1 Data Members

To edit a data member, choose “data” in the “Member Kind” field, and enter the necessary data as follows:

1. Enter its name in the “Member Name” field.
2. Enter its type in the “Type” field, typing the type name as you would do in a FACE script (see Chapter 6 on writing scripts).

You can specify an initial value for this member in the “Value” field: it will be assigned to the member at initialization time for every instance of the class.

Select the access level for the member, by choosing “public”, “protected” or “private” in the “Access” field.

You can require that access functions be generated automatically. If so, XFaceMaker will define two public member functions for read and write access. The names of these functions will be built by concatenating “get” and “set” with the name (beginning with a capital letter) of the member. (Note: this naming scheme implies that you must not create two members whose names differ only by the case of their first letter, because the resulting access functions would have the same names).

The prototypes of these functions are the following (for a member named `data` of type `MyCppType`):

```
MyCppType getData()  
void setData(MyCppType)
```

where `MyCppType` is the C++ type corresponding to the FACE type `MyCppType`.

For example, for a member named “origin” declared as being of FACE type `Int`, “getOrigin” and “setOrigin” will have the following prototypes:

```
int getOrigin()  
void setOrigin(int)
```

You can control the behavior of these functions by means of the *get* and *set* scripts: the *get* script is inserted in the body of the “get” function and evaluated before returning the member’s value; the *set* script is inserted in the body of the *set* function and evaluated before performing the assignment to the member.

These scripts are like any other active value scripts. You can refer to other members of the object (other active values) using the usual syntax. See Section 8.1 for details on writing active value scripts.

15.1.2 Member Functions

To define a member function select “function” in the “Member Kind” field. Give it a name in the “Member Name” field.

To define the return type and the argument list of the member function, you must define a structure whose fields will be interpreted as follows:

- The first field corresponds to the value returned by the function. Its type (a FACE type) is the return type of the function. Its name will be used to specify the return value (unless the return type is None, which corresponds to the C++ void type).
- The remaining fields correspond to the arguments of the function. The order in which they appear in the structure will be their order in the argument list of the C++ member function. Each type is a FACE type, whose C++ counterpart will be used in the actual definition of the function.

The right place to define this structure is at the beginning of the function’s body (i.e. its *set script*). You must also type its name in the “type” field of the active values window. Note that you can choose any name for this type. The only constraint is that it must not conflict with another existing type known by FACE.

Thus, it is good discipline to give this type a name which contains in some way the name of the class and the name of the function. However, there is no risk of conflict with a type name in the generated C++ code.

To be able to return a value and/or pass arguments, you must declare a variable whose type is the argument structure, and initialize it to @ (the “value” of the current active value). This variable will actually hold a pointer to an instance of the argument structure, through which you will be able to access the return value and the arguments of the function.

For example, assume that you have defined a class `MyClass` with a data member called `a`, of type `Int`, and that you want to write a member function named `multiplyA`, which multiplies the current value of `a` by its argument, stores the result in `a` and returns the new value of `a`.

The set script of this function will be the following:

```
/* define the argument structure */
struct MyClass_multiplyA_args {
    Int ret;
    Int coef;
};

/* define a variable of this type and initialize it to @ */

MyClass_multiplyA_args args = @;

/* now the "real" function body starts */

self:@a = self:@a * args->coef;
args->ret = self:@a;
```

Once you have defined a member function, you can call it from any other FACE script of your interface, using the following syntax :

```
<widget> : <avname>(arg, ...)
```

15.1.3 The this Pointer

When writing the body of a member function, you may want to refer to the implicit `this` pointer. You can do this by typing the following expression:

```
self:$this
```

However, note that the type of this expression will always be `void*`. In order to pass it to a function expecting an argument of a more specific type, you will have to cast it explicitly.

15.2 Saving a C++ Class

Saving a C++ class creates three files:

- A `.fm` file, which contains its description like a normal interface. This file will be used to reload the class into XFaceMaker, to use it in new interfaces, or to modify it.
- A header file (with suffix `.h` by default), which contains the C++ definition of the class. This file has to be included by programs using the class.
- An implementation file (with suffix `.C` by default), which contains the implementation of the class. The object file produced by compiling it will have to be linked into executable programs using the class.

To save a class, select its top widget, then the **C++ class** item in the Modules menu. This opens a file selector which allows you to choose the name of the implementation file (by default, its suffix is `.C`), the suffix of the header file (its base name is always the same as for the implementation file), and the name of the class (by default, the basename of the implementation file).

This file selector also allows you to choose the style that must be used to generate the C++ files. Two styles are currently available:

- the NSL style (default)
- the Rogue Wave style, used to generate classes compatible with the View.h++ library.

15.3 Predefined Styles

There are two predefined styles: NSL style and Rogue Wave Style.

15.3.1 The NSL Style

When the NSL style is used, the class created is derived from a predefined class called `NslUIC`. `NslUIC` is an abstract class which provides a minimal behavior for interface components defined as C++ classes. It defines the following public member functions, inherited by all classes created according to the NSL style:

```
Widget  getWidget()
```

Returns the toplevel widget of the class instance.

```
void    show()
```

Makes the object visible (corresponds to the FACE function Show).

```
void    hide()
```

Hides the object. (Corresponds to the FACE function Hide).

In addition to the members you have explicitly defined, `XFaceMaker` automatically defines two constructors and a (virtual) destructor. For a class named `MyClass`, the constructors are defined as follows

```
MyClass(String name, Widget parent, Arg *args, Cardinal
                               num_args, Boolean managed)
```

Initializes a class instance naming its top widget *name*, gives it *parent* as parent and initializes its resources using the values passed in the arglist *args*. The *managed* argument controls whether this interface component should be managed or not when created. (Note: this works only if the toplevel widget has been saved mapped. Otherwise, the only possibility to show the component will be an explicit call to the `Show` member function).

```
MyClass(String name, NslUIC& parent, Arg *args, Cardinal
                               num_args, Boolean managed)
```

This function is similar to the previous one, but it takes as the *parent* argument an instance of a class created by `XFaceMaker` (derived from `NslUIC`).

15.3.2 The Rogue Wave Style

When the Rogue Wave Style is used, the base class of the class created is chosen depending on the class (in the Xt sense) of its top widget. It is always a controller class, chosen to realize the best match between the Xt type and a subclass of `RWController`. The constructors defined for the class depend on the RW class selected.

The correspondence table between the class of the top widget (in the XFaceMaker sense) and the Rogue Wave class is the following:

top widget	Rogue Wave Class
ApplicationShell	RWApplicationShell
OverrideShell	RWOverrideShell
TopLevelShell	RWTopLevelShell
TransientShell	RWTransientShell
XmDialogShell	RWDialogShell
XmMenuShell	RWMenuShell
XmArrowButton	RWArrowButton
XmBulletinBoard	RWBulletinBoard
XmCascadeButton	RWCascadeButton
XmCommand	RWCommand
XmDrawingArea	RWDrawingArea
XmDrawnButton	RWDrawnButton
XmFileSelectionBox	RWFileSelectionBox
XmForm	RWForm
XmFrame	RWFrame
XmLabel	RWLabel
XmList	RWList
XmMainWindow	RWMainWindow
MenuBar	RWMenuBar
XmMessageBox	RWMessageBox
OptionMenu	RWOptionMenu
XmPanedWindow	RWPanedWindow
PopupMenu	RWPopupMenu
PulldownMenu	RWPulldownMenu

top widget	Rogue Wave Class
XmPushButton	RWPushButton
XmRowColumn	RWRowColumn
XmScale	RWScale
XmScrollBar	RWScrollBar
XmScrolledWindow	RWScrolledWindow
XmSelectionBox	RWSelectionBox
XmSeparator	RWSeparator
XmText	RWText
XmTextField	RWTextField
XmToggleButton	RWToggleButton

For more information about the Rogue Wave classes and the methods they provide, see the `View.h++` manual.

15.4 Example

In this section, you will be guided through the construction of a C++ class representing a simple interface component. The example we are going to consider is a counter, which could be used, for instance, to display and update a player's score in a game, or a parameter in a control panel.

The function of this component is to hold and display a numeric value, called **count**, which is identified by a label, and which can be incremented or reset to a default value, called **origin**. It must support two sets of actions:

- from the user:
 - m Increment count by pressing the button holding the name.
 - m Reset count to a default value, by pressing the reset button.
- from a program:
 - m Set the label.
 - m Read, set or reset count.
 - m Set the default value of count.

Its visual layout is the following:

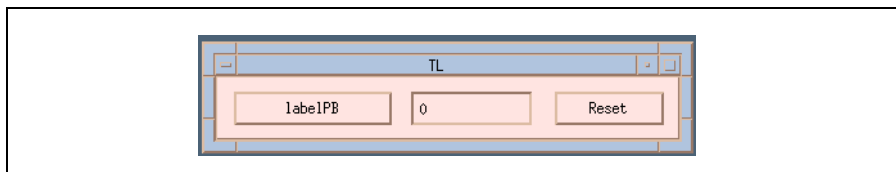


Figure 15.2: The Counter

Moreover, we want to be able to group several instances of this component in composite widgets (such as a rowcolumn) to display and manage several counts.

The source files for this example are in the **Examples** directory of the XFaceMaker distribution, in the **CxxClass** subdirectory.

15.4.1 Building the Widget Tree

Using XFaceMaker, build the following widget tree, where `form` is an `XmForm` widget, `labelPB` and `resetPB` are two `XmPushButton` widgets, and `countTF` is an `Xm-TextField` widget (a shell widget is added automatically by XFaceMaker when you create the form widget).

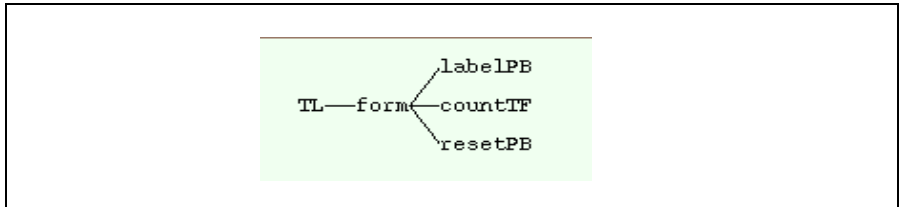


Figure 15.3: The Hierarchy

Set the following resource values:

- for `resetPB`:
`labelString = reset`
- for `countTF`:
`editable = false`
`cursorPositionVisible = false`

15.4.2 Defining the Class Members

Since we want our component to be embeddable in composite widgets, its top widget will not be the `Shell`, but the `Form` below it, named `form`. To define the class members, select the top widget of the class, i.e. `form`, and open the Active Values window (using the Windows menu). Then select the C++ class in the usage field. The Active Values box is now configured to help you create and edit the members of your class.

The Counter class will have the following members:

- public member functions:
`void increment()` *increment count by one.*
`void reset()` *sets count to its default value.*
`void setLabel(String)` *changes the label identifying the*
counter.
`void setCount(int)` *sets count to the value passed as argument.*

`int getCount()` *returns the current value of count.*

`void setOrigin(int)` *sets the default value of the counter.*

`int getOrigin()` *returns the current value of origin.*

- two private data members:

`int origin` *stores the default value of the counter.*

`int count` *stores the current value of the counter.*

In addition, we require that any newly created instance of `Counter` be initialized by setting `origin` and `count` to 0.

15.4.2.1 Data Members

To define the `origin` data member, first enter its name in the “Member Name” field of the Active Values window. Then select “data” in the “Member Kind” field. Since this member has been specified `private`, select “private” in the Access field.

Finally, specify the type of this member by typing `Int` in the “Type” field, and its initial value by typing 0 in the “Value” field.

Apply these changes by clicking the **Add** button. The name `origin` now appears in the list of defined members. The `count` member is defined in a similar way.

15.4.2.2 Member Functions

We shall now define the member functions of the class, according to their specification given above.

The `increment` function is very simple. It takes no argument and increments the value of `count` by one. To define it, first type its name in the “Member Name” field, then select “function” in the “Member Kind” field. Specify that this function is a public member function by selecting “public” in the “Access” field. The last thing to do is to define the body of this function in the Set script window. This will be done according to the principles presented in section 15.1.2.

The first thing to do is to define the argument structure of the function; it will be used to hold its return value and its arguments. In this case, the function returns no value, and takes no arguments. Therefore, the argument structure is defined like this:

```
struct increment_args { None ret; };
```

By convention, the argument structure must always have at least one field, and its first field represents the return value. Here, only the type of this field is used: `None` specifies that the member function has return type `void`.

Then, the following statement implements the required behavior:

```
self:@count = self:@count + 1;
```

To complete the definition, copy the name of the argument structure, i.e. `increment_args`, into the “Type” field, and click on **Add** to apply the definition.

The reset function is defined in a very similar way. Its argument structure is:

```
struct reset_args { None ret; };
```

The rest of its body consists in a single statement:

```
self:@count = self:@origin;
```

The `setLabel` function takes one argument of type `String`, and returns nothing. Therefore, its argument structure is defined as follows:

```
struct setLabel_args { None ret; String s;};
```

The `ret` field will not be used here since the function has return type `void`. However, the second field will be used to hold the argument. Therefore, you must declare a variable whose type is the argument structure and initialize it to `@`, representing the value of the current active value.

```
setLabel_args args = @;
```

Then the actual body of the function can be written, using the `args` variable to access the argument string:

```
self.labelPB:labelString = args->s;
```

The remaining member functions, `setCount`, `getCount`, `setOrigin` and `getOrigin`, are read and write access functions corresponding to the private data members `count` and `origin`. Such functions are useful if you want to allow users of the class to read and write values from and into data members without losing the benefits of encapsulation. By defining code for these functions, you can provide controlled access to the data member, which would not be the case if these members had been defined public.

`XFaceMaker` provides you with an easy way of dealing with this. For each data member you define, you can enable automatic generation of access functions by setting the “Generate Access Functions” option. You will define their body in the `get` and `set` scripts.

To define `getCount` and `setCount`, select `count` from the Members list, then click on “Generate Access Functions”. There is no need to define a `get` script: the `getCount` function will simply return the current value of `count`. However, any change to `count` performed by the `setCount` function must be propagated to the graphic level, that is, the string displayed by the `countTF` text field must be updated.

This is achieved by the following statement, which calls the predefined FACE function `itoa` to translate the numeric value into a character string:

```
self.countTF:value = itoa(@);
```

Click on ***Replace*** to apply these changes.

Similarly, define the functions `getOrigin` and `setOrigin` by selecting the `origin` active value, and clicking on “Generate Access Functions”. You don’t need to write a script for `set` or `get`, as the default behavior suffices.

15.4.3 Callbacks

To complete the definition of the Counter class, we have to define callbacks for the two Pushbuttons used, in order to allow the user to perform the actions that have been specified. Both of them simply invoke functionalities of the component.

When pressed, the left button (which holds the label of the counter), causes the counter's value to be incremented by one. This can be achieved by sending the message “increment” to the counter, that is, invoke the function active value “increment” associated with the top widget, i.e. the form. To do this, select the `labelPB` widget and use the resource window to define its activate callback as follows:

```
parent:increment();
```

When pressed, the right button (`reset`) simply sets the counter's value to its default. This is simply written:

```
parent:reset();
```

15.4.4 Saving the Class

To save the class, select its top widget (here the form) and choose the ***C++ class*** item from the Modules menu. This opens an enhanced file selector which allows you to specify the name of the class and the files in which its definition are to be saved.

Enter a new file name “`Counter.fm`”. To specify the name of the class and the files you can click on the ***Default Name*** button (the names will be derived from the `.fm` file name). Then click on ***OK*** to save the class. This actually saves three files:

- `Counter.fm` which contains the description of the interface component in the XFaceMaker format.
- `Counter.h`, containing the interface (in the C++ sense) of the class.
- `Counter.C` which contains its implementation.

You can also specify names as you like by entering them in the appropriate text fields.

15.4.5 Using the Counter Class

We will now build a small program, called `counters`, using the class just defined. It will group two instances of `Counter` into a `RowColumn` widget, itself contained in an `applicationShell`.

To do this in a natural way, we will create a second class allowing us to manipulate the `RowColumn` as a C++ object. In other words, we will create our own, very simple, C++ wrapper for the `XmRowColumn` widget class. Then we will use it to write the `main` function of the `counters` program in a simple and homogeneous way.

15.4.5.1 Building a Simple C++ Wrapper for `XmRowColumn`

Let us create a C++ class called `RC` corresponding to the `XmRowColumn` widget. For the sake of simplicity in this example, the class will not define any specific members. In particular, it will not provide member functions for resource handling. Building a real wrapper would require more work because it would have to provide a complete set of member functions implementing resource handling and convenience functions for this widget class.

Select **New** in the File menu of `XFaceMaker`'s main window, and lay out an `XmRowColumn` from the Toolkit. Select it and save it as a C++ class using the Modules menu, under the name `RC`. This creates the files `RC.fm`, `RC.h` and `RC.C`

15.4.5.2 The Application Program

Here is the code of the application program `counters.C`:

```
#include "Counter.h"
#include "RC.h"
#include <Fm.h>

main(int argc, char** argv) {
    Widget toplevel =
        FmAppInitialize(0, "Counters", 0, 0, &argc, argv, 0, 0, 0);

    RC rc("rc", toplevel);
    Counter c1("c1", rc);
    Counter c2("c2", rc);

    c1.setLabel("Green");
    c1.setOrigin(10); // Green starts with a bonus of ten points
    c1.reset();

    c2.setLabel("Blue");

    c1.show();
```

```
c2.show();
rc.show();
FmShowWidget(toplevel);

FmLoop();
}
```

Let's look in more detail at its main parts.

```
#include "Counter.h"
#include "RC.h"
```

Header files containing the definitions of the two classes we have defined must be included in order that these classes can be used.

```
#include <Fm.h>
main(int argc, char** argv) {
    Widget toplevel =
        FmAppInitialize(0, "Counters", 0, 0, &argc, argv, 0, 0, 0);
```

Like every program built with XFaceMaker, `counters` must initialize its interface. The call to `FmAppInitialize` returns an `ApplicationShell`.

```
RC rc("rc", toplevel);
Counter c1("c1", rc);
Counter c2("c2", rc);
```

Objects corresponding to the interface are created: one instance of `RC` containing two instances of `Counter`. Each object is initialized by passing it a *name* and a *parent*, which can either be a widget or an instance of a class deriving from the generic interface component class `NslUIC`.

```
c1.setLabel("Green");
c1.setOrigin(10); // Green starts with a bonus of ten points
c1.reset();

c2.setLabel("Blue");
```

The counters are configured to represent the initial state of the interface.

```
c1.show();
c2.show();
rc.show();
FmShowWidget(toplevel);
```

The widgets are now managed.

```
FmLoop();
```


Finally, the event loop is started.

To build the program, compile the classes and the program with a Cplusplus compiler, and link the objects obtained with the libraries `Fm_c`, `Xm`, `Xt`, `X11`, `m` as for any compiled application built with XFaceMaker.

CHAPTER 16: Extending XFaceMaker

You may need to extend XFaceMaker to include non standard Motif widgets (third-party or user-defined widget classes). You may need, also, to add functions to the FACE language so that they can be executed when you are in Try mode. The XFaceMaker extensions are designed to help you achieve this as easily as possible.

An extension is defined by a description file written in XFaceMaker's scripting language, FACE. The extension file contains all the declarations of the new objects that are to be added to XFaceMaker: widget classes, functions, constants, resource types, etc.

To load an extension into XFaceMaker, the extension file must be processed by the **xfmextend** command, which produces a new version of XFaceMaker linked with the extension objects. The **xfmextend** command can be called from within XFaceMaker through the *Load Extensions* item in the *File* menu.

The XfaceMaker distribution includes some frequently used extensions. Users who simply need to load an existing extension should read the section "Loading an Extension" on page 312.

Users who want to define new extensions (or who want to understand how extensions work) should read the entire chapter.

16.1 Loading an Extension

To load an extension into XFaceMaker, use the **Load Extensions** item in the **File** menu.

XFaceMaker will display a list of extension names, from which you will select the extensions you wish to load. There are two kinds of extensions: *default* extensions and *user-defined* extensions.

Default extensions are shipped and installed with XFaceMaker: they appear as single names in the list (for example: **XnslDraw**). The current list of default extensions is:

XnslDraw	integration of the XnslDraw widget and the Draw library gadgets
XnslHtml	integration of the VistaPoint widget
XnslTree	integration of the XnslTree widget
XnslStand	integration of common functions needed by the other NSL integrations; it is loaded automatically by all other extensions, so you do not need to select it.
Xrt3d	integration of the Xrt3d widget from KL Group
XrtField	integration of the XrtField widgets from KL Group: XrtStringField, XrtIntField, etc.
XrtGear	integration of the XrtGear widgets from KL Group: XrtTabManager, XrtTabButton, XrtProgress, XrtGearCombo, etc.
XrtGraph	integration of the XrtGraph widget from KL Group.
XrtTable	integration of the XrtTable and XrtLabel widgets from KL Group
XrtCommon	common functions needed by all KL Group extensions: this extensions is loaded automatically when you select at least one of the XrtGraph, Xrt3d, XrtField or XrtTable extensions, so you do not need to select it.

Initially, the extension selection box lists only the default extensions. To load a default extension, select its name in the list. Note that you will be able to use the KL Group widget extensions only if you have purchased the appropriate libraries and licenses.

A user-defined extension is not part of the standard distribution of XFaceMaker: it can be obtained, for example, from the author of a widget, or it can have been written by you (the next section explains how to write an extension). To load a user-defined extension, click the **Browse** button and select the name of the FACE file describing the extension. Extension files have a **.face** suffix. The extension file will be added to the list and selected.

Once you have selected the extensions you wish to load, click **OK**. You will be prompted for the name of the new XFaceMaker executable that is to be produced. You can accept the default name or type a new one, then click **OK**.

The **xfmextend** command will be launched to build your extended XFaceMaker, which can be started as your new Design Process if the build completes successfully.

After that, the objects defined in the extensions will be available for use in your interfaces. Note, though, that if an extension defines specialized resource editors for new resource types, you will have to exit XFaceMaker completely and restart it before you can use the new resource editing boxes.

Note: When you generate applications with an extended version of XFaceMaker (i.e. either **xfmwf** or a version built by **xfmextend** or with the **Load Extensions** command), any applications generated by the resulting XFaceMaker will be linked with the same set of extensions as the extended XFaceMaker by which they were generated. This means that, for example, if you have created an interface using only the XnslDraw widgets with a fully extended executable, the application will be linked with all the default extensions (all NSL widgets + KL Group widgets). As a consequence, the include files and libraries must be found for all the default widgets that the executable is built with.

Thus, you should not hesitate to re-generate a version of XFaceMaker with the extensions you need at a given time (for example XnslDraw and XrtGraph), and use this XFaceMaker executable to generate your applications.

16.2 Extension Libraries

Extensions (for example third-party widgets) are linked to XFaceMaker through *extension libraries*.

Suppose you want to integrate with XFaceMaker a widget you have written. With previous versions of XFaceMaker, you had to write a C file where you would call XFaceMaker library functions such as `FmAddWidgetClass`, `FmAttachFunction`, `FmAddResourceType`, etc. to declare the new widget class, its functions, its resource types. Then, you would compile this C file, and link it with the XFaceMaker library and your widget library to produce a new XFaceMaker (See sections 16.6 and 16.7).

With the XFaceMaker extension mechanism, this C file is produced automatically and packaged in a library called an extension library. The extension library will be used, together with your widget library, to link a new XFaceMaker, or the interpreted version of applications using your widget. For example, for NSL's XnslTree widget, the extension library is XnslTreeFm. An extended XFaceMaker that includes the XnslTree widget will generate a Makefile that links interpreted applications and a new XFaceMaker with the XnslTreeFm and XnslTree libraries. Compiled applications are linked with the XnslTree library.

In addition to the extension library, you can define a library containing utility functions which may be needed to access data structures or functions of your widget in FACE scripts. For example, if your widget uses structures with fields that cannot be accessed in FACE, you will have to write utility functions to access these fields. These utility functions will be packaged in a second library that will also be linked with XFaceMaker and interpreted mode applications using your widget. For example, for the XRT/Graph widget, the extension libraries are Xrt2dFm, XnslGraph, and XnslXrtPS. Xrt2dFm is the widget description library, built from XrtGraph.face. XnslGraph defines the utility functions. XnslXrtPS is built from XrtCommon.face which describes features that are common to several XRT widgets.

Both types of extension libraries can be generated automatically by **xfmextend**. All you need to supply is the name of the libraries and the source files containing the utility functions, if any. This information is contained in the extension file, as explained in "Build Information Declarations" on page 317.

16.3 The “xfmextend” Command

The **xfmextend** command is normally called automatically by XFaceMaker through the *Load Extension* menu item. It may be convenient, though, to call it directly from a Makefile, so here is a full description of the command.

The **xfmextend** command is a shell script which accepts the following syntax:

```
xfmextend [-h] [-l] [-f] [<xfmname>] [<extension>...]
```

where:

- h** provides a short help,
- l** lists available extensions,
- f** specifies that xfmextend must rebuild the extension libraries even if they are already installed,

xfmname is the name of the new XFaceMaker that will be produced,

extension specifies either an extension name or an extension file.

If no **xfmname** is specified, "myxfm" is used. If no extensions are specified, all the extensions installed in "\$NSLHOME/lib/xfm/Extensions/Fm" are loaded.

The "-f" option can be used to rebuild a local version of an existing extension library, for testing purposes for example.

If an extension is specified by its name only (for example Xnsldraw), **xfmextend** tries to locate the corresponding FACE file (Xnsldraw.face) in the directory

\$NSLHOME/lib/xfm/Extensions/Fm

or using the directory path \$FMFILEPATH (with the substitution character %F). An extension can also be specified directly as an absolute or relative file name.

xfmextend operates in two steps:

1. For each extension, it tries to locate the extension libraries specified in the extension file. Extension libraries are searched in the current directory and in \$NSLHOME/lib. If an extension library is not found, **xfmextend** generates it.
2. Then, **xfmextend** links the new XFaceMaker with the extension libraries of all the extensions specified.

xfmextend uses new XFaceMaker options to generate the extension libraries. These options are:

-extensions "<ext1> <ext2> ...": Specifies FACE extension description files to load

at startup.

- extlib** *<library-name>*: This option is used with the **-gen** option to generate the source files of an extension library. The pattern files for extension libraries are `PattLibMake` and `PattLibExt.c`.
- newxfm** *<xfm-name>*: This option is used in conjunction with the **-gen** option to generate the source files of a new XFaceMaker. The pattern files used are `PattXfmMake` and `PattXfmMain.c`.
- extpath** *<dir1/%F:dir2/%F...>*: Specifies the search path for extension files.

16.4 Defining a New Extension

An XFaceMaker extension consists mainly of a FACE description file, which contains two kinds of declarations:

- The *build information declarations* instruct **xfmextend** on how to link a new XFaceMaker with the extension libraries. This information is also recorded in the new XFaceMaker and is used to generate the application source files. Build information declarations have the form of FACE constants whose names begin with **_XFM_EXT_**.
- The *extension object declarations* define the new objects defined by the extension. These are usual FACE declarations for the objects (functions, constants, enumerations, etc.) contained in the extension.

The .face description files are used to generate the files (Makefile, .h file, .c file) that will be used to build a new XFaceMaker. Their only purpose is to provide the information needed to build the files that will be compiled, and linked with the XFaceMaker *editor* library (Fm_e), to make a new extended XFaceMaker executable.

You can have a look at existing extensions shipped with XFaceMaker while reading this section. They are located in the directory \$NSLHOME/lib/xfm/Extensions. For each extension, the FACE extension file is provided, as well as the source files for auxiliary functions if any.

16.4.1 Build Information Declarations

Build information declarations have the form:

```
define _XFM_EXT_<NAME>_<EXTENSION-NAME> ' <VALUE> ';
```

where *NAME* identifies the information and *EXTENSION_NAME* is the name of the extension (usually the same as the extension file). For example, the line:

```
define _XFM_EXT_BUILD_LIBRARIES_XrtGraph ' -lxrtm ';
```

defines the *LIBRARY* information of the **XrtGraph** extension as **-lxrtm**. All build information constants are of type String.

Here is the description of all the build information constants that can be used in an extension file.

_XFM_EXT_BUILD_COMMAND_: Specifies an additional command to execute on an executable built with the extension. This is used, for example, to call *xrt_auth* to enable applications linked with the XRT widget libraries.

_XFM_EXT_BUILD_C_INIT_FUNCALL_: Specifies the function call which initializes the extension library in an application using this extension. The value is a full C function call instruction (including the semi-colon character). For example, in the Xnsldraw extension, this line is:

```
define _XFM_EXT_BUILD_C_INIT_FUNCALL_Xnsldraw
    'XnsldrawInit(toplevel);';
```

_XFM_EXT_BUILD_C_LIBRARIES_: Specifies the libraries to use when linking with an application in compiled mode. Use this declaration only if the extension requires special additional libraries in compiled mode. Usually, only **_XFM_EXT_BUILD_LIBRARIES_** and maybe **_XFM_EXT_BUILD_E_LIBRARIES_** will be specified. The value will be added to the link command line and is usually of the form '-l<library>'. For example, in the Xnsldraw extension, this line is:

```
define _XFM_EXT_BUILD_C_LIBRARIES_Xnsldraw
    '-lXnsldraw -lXnsltiff -lXnsldf';
```

_XFM_EXT_BUILD_E_LIBRARIES_: Specifies the libraries to use when linking with an application in editor or interpreted mode. The value will be added to the link command line and is usually of the form '-l<library>'. This is usually the library which contains the calls to register the extension objects (functions, constants, etc.) with the FACE interpreter, and which was specified with the **_XFM_EXT_E_LIBRARY_** declaration. For example, in the Xrt/Graph extension, this line is:

```
define _XFM_EXT_BUILD_E_LIBRARIES_XrtGraph
    '-lXrt2dFm -lXnsldraw';
```

_XFM_EXT_BUILD_INCLUDES_: Specifies the include files to add to the source files of applications using this extension. The value is a full C include directive. For example, in the Xnsldraw extension, this line is:

```
define _XFM_EXT_BUILD_INCLUDES_Xnsldraw
    '#include <Xnsldraw.h>';
```

_XFM_EXT_BUILD_INCLUDE_PATH_: Specifies the include path to use when compiling application source files using this extension. The value is of the form '-I<directory>'. For example, in the Xnsldraw extension, this line is:

```
define _XFM_EXT_BUILD_INCLUDE_PATH_Xnsldraw
    '-I.. -I../Xnsldraw -I$(NSLHOME)/include';
```

_XFM_EXT_BUILD_INIT_FUNCALL_: Specifies the function call which initializes the extension library in an application using this extension, when the application runs in interpreted or editor mode. This is usually a call to the function

specified as the value of `_XFM_EXT_BUILD_INIT_FUNCTION_`, which is automatically generated. The value is a full C function call instruction (including the semi-colon character). For example, in the `XnslDraw` extension, this line is:

```
define _XFM_EXT_BUILD_INIT_FUNCALL_XnslDraw
    'XnslInitDrawLibrary(toplevel);';
```

`_XFM_EXT_BUILD_INIT_FUNCTION_`: Specifies the name of the initialization function of the extension library. This function will be automatically generated by **`xfmextend`** and will register all the objects (functions, constants, etc.) declared in the extension with the FACE interpreter. For example, in the `Xrt/Graph` extension, this line is:

```
define _XFM_EXT_BUILD_INIT_FUNCTION_XrtGraph
    'XnslInitGraphLibrary';
```

`_XFM_EXT_BUILD_LIBRARIES_`: Specifies the libraries to use when linking with an application using this extension. These libraries will be used both for compiled and for interpreted applications. The value will be added to the link command line and is usually of the form `'-l<library>'`. For example, in the `Xrt/Graph` extension, this line is:

```
define _XFM_EXT_BUILD_LIBRARIES_XrtGraph
    '-lXrtm';
```

`_XFM_EXT_BUILD_LIBRARY_PATH_`: Specifies the library path to use when linking applications using this extension. The value is of the form `'-L<directory>'`. For example, in the `XnslStand` extension, this line is:

```
define _XFM_EXT_BUILD_LIBRARY_PATH_XnslStand
    '-L../Lib -L$(NSLHOME)/lib';
```

`_XFM_EXT_BUILD_OBJECTS_`: Specifies additional object files to include in the extension libraries built by **`xfmextend`**. These files contain auxiliary functions which may be needed to use the extension with `XFaceMaker`, and/or functions used by specialized resource editors. Several blank-separated object files can be specified. For example, in the `Xrt/Graph` extension, this line is:

```
define _XFM_EXT_BUILD_OBJECTS_XrtGraph
    'XrtGraphFm.o';
```

`_XFM_EXT_BUILD_SOURCES_`: Specifies the additional source files necessary to build the extension libraries. **`xfmextend`** will link these source files in the current directory when building the extension libraries. Several blank-separated source files can be specified. For example, in the `Xrt/Graph` extension this line is:

```
define _XFM_EXT_BUILD_SOURCES_XrtGraph
    'XrtGraphFm.c';
```

_XFM_EXT_CODE_: Specifies additional C code which will be included in the extension libraries built by **xfmextend**. This code will typically contain calls to the function **FmAddWidgetClass** to register new widget classes, and maybe **FmAddResourceType** to declare new resource type editors. For example, for the Xrt/Graph extension, this line is:

```
define _XFM_EXT_CODE_XrtGraph'
    (*fns>add_widget_class)(&xtXrtGraphWidgetClass,
        0,
        FM_PRIMITIVE,
        FM_NORMAL,
        0,
        "#include <Xm/XrtGraph.h>",
        "xtXrtGraphWidgetClass",
        0,
        0,
        0,
        "XrtGraph.bm");';
```

_XFM_EXT_C_LIBRARY_: Specifies the name of the library that will be added to the Makefile generated by XFaceMaker in the rules for compiled applications. It is the name of the extension library containing the auxiliary functions defined for the extension. For example, for the Xrt/Graph extension, this line is:

```
define _XFM_EXT_C_LIBRARY_XrtGraph
    'Xns1Graph';
```

_XFM_EXT_E_LIBRARY_: Specifies the name of the library that will be added in the Makefile generated by XFaceMaker in the rules to build interpreted or editor applications. It is the name of the extension library containing the calls that register the extension objects with XFaceMaker. These calls are automatically generated from the declarations contained in the extension file. It can also include auxiliary functions used by specialized resource editors. For example, for the Xrt/Graph extension, this line is:

```
define _XFM_EXT_E_LIBRARY_XrtGraph
    'Xrt2dFm';
```

16.4.2 Declaring Extension Objects

In addition to *build information declarations*, the extension file contains declarations for the objects defined in the extension. These declarations use the usual FACE syntax. Here are the objects you will typically declare in an extension file:

- functions (function keyword)

- constants (**define** keyword)
- structures (**struct** keyword)
- types (**type** keyword): enumerations, representations, C type casters.

To implement the declaration of all extension objects, the syntax of some FACE keywords has been extended. See the *XFaceMaker Reference Manual* for a basic description of these declarations.

16.4.2.1 Functions

The **function** keyword now accepts the ANSI C triple-dot syntax to declare functions with a variable number of arguments:

```
function None FooBar(...);
```

16.4.2.2 Constants

The **define** keyword has been modified to specify the type of a constant, for example an enumerated type that is not already defined as a resource. You specify the name of the constant in the Face language, the name of the constant in the generated C code, and the associated Face type:

```
define Xyz_FACE_CONSTANT_NAME Xyz_C_CONSTANT_NAME  
    XyxFacetype;
```

e.g. in the XRT/Field extension file:

```
define XRTFLD_DIRECTION_NEXT XRTFLD_DIRECTION_NEXT  
    XrtFldDirection;  
define XRTFLD_DIRECTION_PREVIOUS XRTFLD_DIRECTION_PREVIOUS  
    XrtFldDirection;
```

16.4.2.3 Structures

The **struct** keyword now accepts quoted strings as field names. This allows the declaration of nested structures, unions, or array fields. For example, suppose you want to declare the following C structure in FACE:

```
struct XyzStruct {  
    union {  
        int i;  
        char *s;
```

```

    } u;
    struct {
        int n;
    } sub;
    float values[2];
};

```

Here is how you would declare it in FACE:

```

struct XYZStruct "XYZStruct*" {
    Int      "u.i";
    String    "u.s";
    Int      "sub.n";
    Float     "values[0]";
    Float     "values[1]";
};

```

You must also use quotes to access the special structure fields in FACE scripts:

```

XYZStruct s = new_struct(XYZStruct);
s->"u.i" = 10;
printf("%g\n", s->"values[0]");

```

Note, though, that you still cannot use array fields as FACE arrays (i.e., you cannot get the address of the "values" field and use it to create a FACE array). You could declare the field address, e.g.

```
"Pointer values"
```

and use it to create a FACE array, e.g.

```
"Float values[Int] = new_c_array(s->values, 2);"
```

but this would work only for applications in compiled mode, not in interpreted mode.

16.4.2.4 Types

The **type** keyword syntax has been extended to specify the names of C constants corresponding to enumeration resources:

```

type EnumType = value1 = XYZENUM_VALUE_1,
               value2 = XYZENUM_VALUE_2,
               ...;

```

Use this syntax only for resources. If you want to specify enumerated types that are not used in resources but are used, e.g., in function calls, use the **define** keyword as described in the section “Constants” on page 321.

In the example above, *EnumType* is the character string of the resource's Representation type, *value1..n* are the character strings for the converter, and *XyzENUM_VALUE_1..N* are the enum values in the .h. For example, in the Xrt/Field extension file:

```
type xrtFldCase =
    case_as_is =    XRTFLD_CASE_AS_IS,
    case_lower =    XRTFLD_CASE_LOWER,
    case_upper =    XRTFLD_CASE_UPPER;
```

Remember that the `type` keyword can have two forms: the "enumeration" form (as explained above) and the "representation" form:

```
type MyType "struct my_type *" XtrMyType;
```

This form declares the FACE type "MyType" as corresponding to the C type "struct my_type *" and also says that the C preprocessor symbol holding the name of the representation for MyType is XtrMyType. The representation name symbol can be omitted if the type is never used for a resource, but only for function arguments or return types.

Similarly, to declare a resource that accepts arrays of values of another type, you would write:

```
type MyTabType "my_tab_type*" XtrMyTabType;
```

The "****" is interpreted as an indication that this is a "table resource". For an example, refer to the XrtField extension.

16.4.3 Additional Source Files

For a simple extension, you will need to write only a FACE extension file. In more complex cases, though, you will have to write additional C code because of the limitations of the FACE language. For example, you cannot use functions with more than 12 arguments. To use such a function in FACE, you would have, say, to define a structure containing all the arguments, and to write an auxiliary C function which accepts this structure as argument and calls the "real" function.

Utility functions are defined in additional source files that you specify with the constants `_XFM_EXT_BUILD_OBJECTS_` and `_XFM_EXT_BUILD_SOURCES_`.

`_XFM_EXT_BUILD_OBJECTS_` is a blank-separated list of object files that are to be added to the extension libraries.

_XFM_EXT_BUILD_SOURCES_ is a blank-separated list of source files (.c and .h) used to build the additional objects.

The additional source files will be compiled in two version:

- one version will be compiled with the -DCALLEDITOR flag and will be included in the extension library used for building a new XFaceMaker and applications in interpreted mode;
- another version will be compiled with the -DCCODE flag and will be included in the "auxiliary" extension library used for building all applications.

16.4.4 New Resource Editors

If you want to write an extension to integrate a new widget in XFaceMaker, you may need to define additional resource editors for complex resources of the new widget.

If you need to define a new resource editor, or if you want to provide a specialized user interface to edit a resource, here is what you should do.

1. You should build a .fm file which describes your resource editor. This file can be defined using XFaceMaker. The resource editor must obey the following rules:
 - The toplevel object of the .fm file must be a composite (for example an Xm-Form), not a Shell, because the editor will be loaded inside the Resource Editor shell in XFaceMaker.
 - The toplevel object must have a set of active values which are used to exchange data with the XFaceMaker resource editor. These active values are:

<Type> This active value has the name of the resource type for which the editor will be registered. Its "set" script will be called when the editor is popped, with the \$ or @ value (depending on the "avtype" parameter passed to FmAddResourceType: see this function in the *XFaceMaker Reference Manual*) set to the current value of the resource. The "set" script of this active value must initialize the editor according to the current value.

_FmResourceReturnValue This active value is called when the user clicks OK after using the resource editor. Its "get" script must copy the new resource value (always as a String) into the \$ value.

_FmResourceEditManage This active value can optionally be defined. It will be called just before the resource editor is mapped to the screen.

_FmResourceEditUnmanage This active value can optionally be defined. It will be called when the resource editor is unmapped.

2. You must register the new resource editor with the `FmAddResourceType` function (see this function in the *XFaceMaker Reference Manual*). The call to `FmAddResourceType` can be issued:
 - directly from the `_XFM_EXT_CODE_` declaration (see “Build Information Declarations” on page 317), or
 - from a function contained in an additional source file and called from the `_XFM_EXT_CODE_` declaration (see “Build Information Declarations” on page 317 and “Additional Source Files” on page 323).

In both cases, the call to `FmAddResourceType` must be surrounded by `"#ifdef CALLEDITOR ... #endif"` statements.

3. Your new resource editor may need to call C functions from its FACE scripts. The code of these C functions will have to be added to an additional source file linked in the extension library, and registered with `FmAttachFunction`.

You should have a look at the standard XFaceMaker resource editors in `"$NSLHOME/lib/Fm"`:

- `"String.fm"` (the default editor for String or unknown types) is a simple example;
- `"Colors.fm"` (the color palette and RGB editor) is a more complex one.

You should also look at the source of the `XrtGraph` extension (in `"$NSLHOME/lib/xfm/Extensions/XrtGraph"`) for an example of how to register new resource editors (look at the `XnslInitXrt2dResources` function).

16.4.5 New Table Resource Editors

You can also declare a resource as being a "table resource", i.e. a resource which accepts arrays of values of another type. For example, resources of type `"FloatTable"` accept arrays of floats. XFaceMaker can handle these resources automatically by providing a generic `"Table"` editor which can call an appropriate sub-editor for each item in the table. Table resources are declared using the `type` keyword, as follows:

```
type XrtDataStyles "XrtDataStyle*" XtrXrtDataStyles;
```

16.5 Example - Adding an Extension

This section shows how you can extend XFaceMaker with a new widget class built with XFaceMaker.

Let's assume that you have defined a widget class that is subclassed from XmPrimitive, whose name is StateLabel. This class defines new resources, but no new resource types. Let's assume that its description is in a file called StateLabel.fm. For the sake of simplicity, we will use the directory where this file resides as the current directory for the entire example.

The contents of the file used for this example is printed below for the sake of completeness; you need not read it to understand what follows.:

```

1:  widget XmLabel {
2:      flag {
3:          autosize
4:          autopos
5:      }
6:      activevalue {
7:          name = state
8:          set = if (@ == 0)
9:  self:background = self:colfalse;\
10:     else
11:  self:background = self:coltrue;
12:         type = Int
13:         storage = object
14:         automatic = true
15:         usage = class
16:         default_type = String
17:         default_value = 0
18:     }
19:     activevalue {
20:         name = colfalse
21:         set = if (self:state == 0)
22:  self:background = @;
23:         type = Pixel
24:         storage = object
25:         automatic = true
26:         usage = class
27:         default_type = String
28:         default_value = green
29:     }
30:     activevalue {

```

```
31:         name = coltrue
32:         set = if (self:state != 0)
33:     self:background = @;
34:         type = Pixel
35:         storage = object
36:         automatic = true
37:         usage = class
38:         default_type = String
39:         default_value = red
40:     }
41:     resources {
42:         borderColor = pink
43:         borderWidth = 1
44:     }
45: } La
```

Let's assume also that you have created a pixmap file, `StateLabel.bm` that represents the class icon as you want it in the XFaceMaker toolkit and that it is in the current directory.

Here are the steps you should follow to create your extension for this class, and build an extended XFaceMaker executable that contains the class.

1. Create the `StateLabel.c`, `StateLabel.h`, and `StateLabelP.h` files.

To do so, you need to load the class in XFaceMaker, and specify the following *C-code options*: set the **Save C code** toggle, set the **class** toggle, specify the **class prefix** (here we will use `Xorg` as the class prefix), set the **stand-aloneC** option, click Apply, close the C options box.

Open the modules menu, **save** the class. The `StateLabel.c`, `StateLabel.h`, and `StateLabelP.h` have been created in the current directory. We have specified a class prefix `Xorg` and, in order to respect conventions we will instruct XFaceMaker to build Makefiles that look for the include files under `Xorg`; we need to create a directory `Xorg` in the current directory, and move the two `.h` files into `Xorg`.

2. Build the widget library.

First **compile** the `StateLabel.c` file, then make a widget library called `libStateL.a` from the `.o` file. Your commands will look like the following, depending on your system:

```
cc -c StateLabel.c -I.
ar cr libStateL.a *.o
ranlib libStateL.a
```

These commands have created a library called libStateL.a¹. Because of the class prefix Xorg and widget conventions, we will instruct XFaceMaker to create Makefiles that look for this library in the Xorg directory. So, we need to move libStateL.a to the Xorg directory.

3. Build the .face extension file.

Let's call this file StateL.face. The name of the .face file determines the name of the extension. The name that is appended to the build constants must be the same as the name of the .face file.

Here is the contents of the file:

```

/*****
* Class information.
*****/

define _XFM_EXT_CODE_StateL '
(*fns->add_widget_class)(&xorgStateLabelWidgetClass,
0,
FM_PRIMITIVE,
FM_NORMAL,
0,
#include <Xorg/StateLabel.h>,
"xorgStateLabelWidgetClass",
0,
0,
0,
"StateLabel.bm");
';

define _XFM_EXT_C_LIBRARY_StateL '';
define _XFM_EXT_E_LIBRARY_StateL 'StateLfm';
define _XFM_EXT_BUILD_C_INIT_FUNCALL_StateL
'XtInitializeWidgetClass(xorgStateLabelWidgetClass);';
define _XFM_EXT_BUILD_INIT_FUNCTION_StateL
'XStateLInit';
define _XFM_EXT_BUILD_INIT_FUNCALL_StateL
'XStateLInit();';

```

1. If our class had C functions associated to it, these would be defined in a separate .c file, which would have to be compiled and included in the library. Note that the function declarations would have to be added to the class .h files.

```
define _XFM_EXT_BUILD_INCLUDES_StateL
    '#include <Xorg/StateLabel.h>'
define _XFM_EXT_BUILD_LIBRARIES_StateL '-lStateL';
define _XFM_EXT_BUILD_C_LIBRARIES_StateL '';
define _XFM_EXT_BUILD_E_LIBRARIES_StateL '-lStateLFm';

define _XFM_EXT_BUILD_INCLUDE_PATH_StateL
    '-I$(NSLHOME)/include';
define _XFM_EXT_BUILD_LIBRARY_PATH_StateL
    '-L$(NSLHOME)/lib -L./Xorg';
```

_XFM_EXT_CODE_StateL specifies how to add the widget class. This will be in the **StateLFm** library, as specified by **_XFM_EXT_BUILD_E_LIBRARIES_StateL**.

_XFM_EXT_C_LIBRARY_StateL does not need to specify a library, as the extension does not have any auxiliary functions.

_XFM_EXT_E_LIBRARY_StateL specifies the additional **StateLFm** library, which is needed to build interpreted or editor application.

_XFM_EXT_BUILD_C_INIT_FUNCALL_StateL specifies the function call that initializes the widget library in applications using the widget.

_XFM_EXT_BUILD_INIT_FUNCTION_StateL specifies that the name of the function that will be generated by xfmextend to register the objects declared in the extension with the FACE interpreter. In this example, no structures or functions have been defined so that this function will just add the widget class and register the variables defined in this .face file.

_XFM_EXT_BUILD_INIT_FUNCALL_StateL specifies the explicit function call. Here, no parameters are needed.

_XFM_EXT_BUILD_INCLUDES_StateL specifies the include directive that is to be added in application files generated by XFaceMaker with this extension.

_XFM_EXT_BUILD_LIBRARIES_StateL specifies that **StateL** is to be linked with all applications built with this extension.

_XFM_EXT_BUILD_C_LIBRARIES_StateL does not specify any library, as no additional library need be added in the Makefile rules generated by XFaceMaker for compiled applications.

_XFM_EXT_BUILD_E_LIBRARIES_StateL specifies that **StateLFm** is to be added to the Makefile rules generated by XFaceMaker for interpreted or editor applications.

_XFM_EXT_BUILD_INCLUDE_PATH_StateL specifies the paths where include files are to be looked for in addition to the default ones. Note that the Makefile generated by XFaceMaker always has a **-I.** directive to include files in the current directory. The XFaceMaker include files required are in **\$(NSL-HOME)/include.**

_XFM_EXT_BUILD_LIBRARY_PATH_StateL specifies paths where libraries are to be fetched, in addition to the standard ones. In our case, **-L./Xorg** has been specified as this is where the **StateL** library is located. **\$(NSL-HOME)/lib** is where the XFaceMaker libraries are located.

4. Build a new extended XFaceMaker

Launch an XFaceMaker executable from the directory where the .face file has been defined. In the **File** menu, select **Load Extensions...** In the extensions box, select **Browse..** . A file selector is popped, select the StateL.face file. It is added to the list of extensions available, and it is selected. Select any other extension you want to include in your new XFaceMaker executable, click OK. Specify the name of the executable you want to build in the box that is popped. The default name is myxfm. A the commands being executed to build your new executable are displayed in a command output box. The last line in the box should say *command succeeded*. You can choose to launch your new executable as a design process or, better, exit XFaceMaker and launch your new executable as the server process.

If problems occur during the build, find their cause, correct them, and redo the above steps. Remember that the extension libraries are built only if they do not already exist. You may want to remove the extension libraries you have just built (in our case libStateLFm.a and libStateLFm.so) as well as any source files built from the .face file before you re-launch the build extension mechanism.

16.6 XFaceMaker Extension Functions

The XFaceMaker extension mechanism discussed earlier in this chapter is based on a number of functions that extend the tool so as to include third-party widgets and/or utility functions. The extension functions are included in the `libFm_e.a` library, with which the extension code is linked every time a new XFaceMaker is produced.

In the remaining pages of this chapter, we give an overview of the extension functions. If you use the automatic extension mechanism, you should only exceptionally need the information in these pages.

16.6.1 New Widget Class

The `FmAddWidgetClass` function is used to add a new widget class to the set of classes that XFaceMaker already knows. The class name of the new class is the name contained in the `WidgetClass` record. During the next call to the `FmCallEditor` function, a new icon will be created for the new widget class in the *User defined* panel of the *Widget Toolkit*.

`FmAddWidgetClass` can be used several times in the same program to add several new widget classes. XFaceMaker limits the number of classes you add to 50.

Once a new widget class is added, it can be used in XFaceMaker exactly in the same way as standard OSF/Motif classes. The only difference is that unless the function `FmAddResourceType`, or `FmAddEnumeratedType` is used to define resource types for the new widget, resources that have a type which is not known by XFaceMaker will be edited with the *String* resource edit box.

For a detailed description of the function and its arguments refer to the *XFaceMaker Reference Manual*.

16.6.2 New Resources

If you have added a new widget class to XFaceMaker using the function `FmAddWidgetClass`, you may have to describe a new resource type to XFaceMaker. XFaceMaker will need two types of information:

1. What to do when editing the resource.
2. How to generate the C code for the new resource, and how to manipulate the resource values internally.

The function `FmAddResourceType` (or `FmAddEnumeratedType` for enumeration resources) should be used to inform XFaceMaker on what to do when editing a new resource. The function must be called after `FmAddWidgetClass` and before `Fm-`

CallEditor. You cannot add a resource type that is already declared in XFaceMaker. Also, you must make sure the new resource type has the appropriate resource converter.

FmAddResourceType returns 1 on success, 0 on failure.

Its parameters are:

```
Boolean FmAddResourceType (  
    char      *type,  
    char      get_value,  
    char      avtype,  
    char      *avname,  
    char      *boxname,  
    char      *default_value,  
    char      *enumeration,  
    int       offset)
```

For example:

```
FmAddResourceType("XrtXLabel", RT_OBJECT, RT_STRING,  
    "_XrtXLabel", "XrtLabel", NULL, (char *) 0, 0);
```

This call declares the new resource type "XrtXLabel". When a resource of this type is selected in the XFaceMaker resource list, an active value called "_XrtXLabel" will be set, and the shell called "XrtLabel" will pop up.

For a full description of the FmAddResourceType function, and a list of the resource types declared in XFaceMaker, refer to the *XFaceMaker Reference Manual*.

The FmAddEnumeratedType function can be used to add a new enumerated resource type instead of FmAddResourceType. This lets you define a prefix for the enumeration values in place of the default Xm.

FmAddEnumeratedType returns 1 on success, 0 on failure.

Its parameters are:

```
Boolean FmAddEnumeratedType (  
    char      *type,  
    char      *values,  
    int       offset,  
    char      *prefix)
```

For example:

```
FmAddEnumeratedType
```



```
(XnsLrNewType2, "choice1\nchoice2\nchoice3\n", 10 "Xnsl"))
```

declares the new resource type `XnsLrNewType2`. This resource is an enumerated type, which can have 3 values `choice1`, `choice2`, `choice3`, corresponding to integer values 10 (`XnslCHOICE1`), 11 (`XnslCHOICE2`) and 12 (`XnslCHOICE3`) respectively.

For classes built within XFaceMaker, an alternative and easier option if you define an enumerated type, is to give the string values when you make the type declaration. In this case XFaceMaker will automatically create the resource converter and call `XtAddConverter` in the `Initialize` method. A call to `FmAddEnumeratedType()` is added within the `FmAddNameWidgetClass()` function. The definitions required are also added to the `Name.h` file.

When adding a new resource type, you also have to specify how XFaceMaker should generate the corresponding C code to create correct and portable code. For this, you use the function `FmAddRepresentation`:

```
int      FmAddRepresentation(  
        const      char *type;  
        const      char *repname;  
        const      char *ctype;  
        int         size;  
        int         aligned;  
        int         indirect;)
```

This function informs XFaceMaker on how to manipulate resources of the new resource type, and how to generate C code when the resource is set for a widget or used in a FACE script.

For example, a resource representing a bit mask could be declared as:

```
FmAddRepresentation("Mask", "XxRMask", "unsigned int",  
    sizeof(unsigned int), 0, 0);
```

`XxRMask` is assumed to be the name of the C symbol holding the string value "Mask", traditionally defined in a widget's public include file as:

```
#define XxRMask "Mask"
```

The `aligned` parameter is set to 0 because an `XtArgVal` is usually defined as an unsigned long, and an `int` can be smaller than a `long` (e.g. on alpha processors).

When adding new variable types for function calls (not for new widget class resources), you can use the function `FmNewTypeDef`.

Calling

```
FmNewTypeDef(type, ctype, size)
```

is equivalent to calling

```
FmAddRepresentation(type, 0, ctype, size, 0, 0)
```

For application value types, i.e. types defined by the application and which are used only in function call parameters or return value, but which are not resource types, `FmNewTypeDef` is sufficient

All new widget resource types should be declared with `FmAddRepresentation`.

The `FmAddEnumeratedType` function calls `FmAddRepresentation` automatically, and assumes that the enumerated value is of type `unsigned char`.

16.6.3 Using A New Resource Type in XFaceMaker

When a resource of representation type `type` is selected in the resource list of XFaceMaker, the following happens.

- The active value `avname` is attached to an internal memory location depending on `avtype`: one word if `avtype` is `RT_INTEGER`, a dynamically allocated buffer if `avtype` is `RT_STRING`.
- The current value of the resource is fetched according to the value of `get_value`. If `get_value` is `RT_OBJECT`, then the value is the string contained in the resources of the XFaceMaker objects, as saved in the `.fm` file: this value is copied into the active value buffer. If `get_value` is `RT_WIDGET`, `XtGetValues` is called for the resource: if the resource is of type string, the string pointed to by the value is copied into the active value buffer, otherwise only the value is copied into the active value. If `get_value` is `RT_OBJECT` and the resource is not set, `default_value` is used if it is not `NULL`.
- If `enumeration` is non-null, then the *Enumeration* box list is filled with the specified items.
- The active value *Set* script is called using `FmSetActiveValue(0, avname)`. The editing box must take whatever action is necessary to display the current value.
- The *Get* script of the active value `_FmResourceReturnValue` is invoked when the user presses the **OK** or the *Apply* button. This get script must do whatever is necessary to copy the new resource value into `$` as a String, as if the value had been entered by the user directly in the resource line text field.

16.6.4 Adding a New Resource Editing Group

You may choose to define a new resource editing box to edit the new resource type by specifying the parameter `boxname`. You will then have to define a non-modal group which will be mapped inside the XFaceMaker resource window.

In XFaceMaker 3.x, resource editors are described as groups contained in the `Fm` subdirectory of the XFaceMaker library files directory, usually `$NSLHOME/xfm/Fm`. To add a new resource editor, simply add a `.fm` file to this directory, and pass the basename of the file to `FmAddResourceType`. For example, you can add a file named `MyTypeEditor.fm` and call:

```
FmAddResourceType("MyType", RT_OBJECT, RT_STRING,  
                  "MyTypeAv", "MyTypeEditor", 0, 0, 0);
```

You must choose a name which does not already exist in the `Fm` directory. You must also choose a name different from all toplevel windows already present in the `Fm.fm` file, because XFaceMaker first looks in the toplevel windows already loaded into its interface to see if there is an old-style box present, and if so, uses it as the resource editor. To be sure not to use an existing name, you can prefix your box name: `XyzMyTypeEditor` for example.

You need not modify the `Fm.fm` file nor use `FmAddBootFile` with this method. The group file will be loaded and instantiated automatically when you first edit a resource of the new type.

The resource editing group must be designed as follows:

- The toplevel widget of the group must not be a shell, but a composite widget (usually a Form).
- An active value named like the `avname` parameter passed to `FmAddResourceType` must be defined for the toplevel widget of the group. The *Set* script of this active value will be called when the box is popped to initialize the box fields with the initial resource value. The resource value is contained in `@` if the `avtype` parameter passed to `FmAddResourceType` was `RT_INTEGER`, or in `$` if it was `RT_STRING`. The *Set* script of the `avname` active value can initialize whatever sub-widgets of the editing group are appropriate.
- An active value named `_FmResourceReturnValue` must be defined for the toplevel widget of the group. The *Get* script of this active value will be called when the user presses **OK** or **Apply** in the resource window to get the resource value to apply. The *Get* script must copy the selected resource value into `$`, ****as a string****, as if the value had been typed in the resource line text field. For example, if the integer value 100 had been selected using a scrollbar, the *Get* script

of the `_FmResourceReturnValue` would copy the string “100” into \$ by `strcpy($, “100”);`

- an active value named `_FmResourceEditManage` can optionally be defined for the topmost widget of the group. It will be called after the `avname` active value, and immediately after the group is managed in the XFaceMaker resource window. This allows arranging the geometry of widgets once all size calculations are done.
- an active value named `_FmResourceEditUnmanage` can also optionally be defined for the topmost widget of the group. It will be called after the group is unmanaged from the XFaceMaker resource window.

The resource editing group can be composed of an arbitrary widget tree. It must *not* contain the **OK** / **Apply** / **Set** / **Reset** buttons: these are provided by the resource window. As an example, you can look at the Boolean editor, in `Fm/Boolean.fm`.

16.6.4.1 Adding an “Old-style” Resource Editing Box

In XFaceMaker version 2.1 and before, resource editing boxes were implemented as modal popup-windows. This type of box still works for backward compatibility reasons but this method is obsolete and existing resource editors should be converted to use the non-modal group method described above. The old-style method is given here only so as to help you translate old-style windows to the new-style editing group. An old-style window is popped when the . . . (ellipsis) button of a resource list line is pressed, and it blocks all input to the rest of XFaceMaker until the user presses **OK** or **Cancel**.

Modal resource editing windows followed the rules below:

- The topmost widget of the box had to be a `TransientShell`, and had to have its `popup` flag set. The shell name had to be the same as `boxname` or `type`.
- The shell had to be a direct child of the main XFaceMaker `ApplicationShell`. The description of the shell had to be contained in a special `Fm.fm` file, which could be located, for instance, in the current directory, in your home directory, or in a directory defined by `-fmlibdir`. You could also use the `FmAddBootFile` function (see below).
- An active value called `avname` (or `type` if `avname` was `NULL`) had to be defined for one or more widgets of the box. The `Set` scripts of this active value had to update the widgets of the box according to the current value of the resource stored in \$ for a string or @ for an immediate value.
- The box had to include two push buttons: an **OK** button and a **Cancel** button. Clicking in the **OK** button would call `Return (self, value)`, where `value` was a

String containing the new value of the resource. The Cancel button, when selected, called `Return(self, String(0))`.

The easiest method was to copy and edit a resource edit box from `Fm.fm`. The editing box returned the new value as a string. The widget class writer had to define the appropriate resource converter for the new resource type.

The function `FmAddBootFile` allowed you to specify additional `.fm` files to be loaded after the `Fm.fm` file as part of the XFaceMaker user interface. In that way, you didn't need to copy the whole `Fm.fm` file and add your resource editing box to it: you just needed to put your box in a separate `.fm` file, say `MyBox.fm`, and call:

```
FmAddBootFile( "MyBox.fm" );
```

`FmAddBootFile` should be called after `FmInitialize` and before `FmCallEditor`, for example just after `FmAddResourceType`.

16.7 Building a New XFaceMaker

Building a new instance of XFaceMaker containing a new widget class is similar to building an XFaceMaker application. The process is described in detail below so that you understand it. However, this is done automatically by the **xfmextend** command so that you can skip this section entirely.

A small C program has to be written, and linked with the `libFm_e.a`, and the OSF/Motif and X libraries. a canvas of the C program can be generated by XFaceMaker through the **File** menu, **Save Appli Files, New XFM**. The canvas program will include the calls to `FmInitialize` and `FmCallEditor`. Calls to the appropriate extension functions can be added as needed. The C program should have the following structure:

- `FmInitialize`,
- `FmAddWidgetClass` for each new widget class (optional),
- `FmAddResourceType` or `FmAddEnumeratedType` for each new resource type (optional),
- `XtAddConverter` to register any resource converters required by the new resources added (optional).
- `FmAttachAv` for each new active value (optional),
- `FmAttachFunction` for each new function. This may be a widget function similar `XmTextReplace`, or a new application or library function (optional),
- `FmCallEditor` to load the interface of XFaceMaker .

If you only wish to extend FACE then only the calls to `FmAttachFunction` will be required.

The following program can be used to build a new XFaceMaker with the new widget class `New`.

```
#include <Fm.h>
#include <New.h>

main(argc, argv)
int argc;
char **argv;
{
    FmInitialize("myxfm", "Myxfm", 0, 0, &argc, argv);
    FmAddWidgetClass(&newWidgetClass,
                    0,
                    FM_PRIMITIVE,
```

```
        FM_NORMAL,  
        0,  
        "#include <New.h>",  
        "newWidgetClass",  
        0,  
        0,  
        0,  
        "NewIcon");  
    FmAttachFunction("new_func", new_func);  
    FmCallEditor();  
}
```

Once saved into the file `main.c`, this program can be compiled and linked with the object module of the `New` widget class to build a new `XFaceMaker` :

```
cc -o myxfm main.o New.o -lFm_e -lXm -lXt -lX11
```

An example `main.c` is given in the `Examples` directory `NewXfm`.

To add new functions so that they are available in *Face* in *Try* mode, the function `FmAttachFunction` may be used before the call to `FmCallEditor` (with or without `FmAddWidgetClass`). The functions attached could be calls to functions defined for the new widget, or non-widget functions in a library or object file which is linked when creating `myxfm`.

16.7.1 Linking with Alternate Libraries

The same mechanism can be used to link `XFaceMaker` with alternate Motif or X libraries. This might be required so that the newest release can be used, for shared libraries or for using a debugging library. To do this construct, or copy, the main program above without the call to `FmAddWidgetClass` and link as follows:

```
cc -o myxfm main.o -lFm_e -Lndir -lXm -lXt -lX11
```

where `ndir` is the path of the directory containing the new libraries.

16.7.2 Generating and Linking C-Code Files

The mechanism used to generate C-code files from the `.fm` files is described in Section 11.2. When the C-code interface is compiled, the header files for the new widget classes must be accessible, since they are included in the C file. When linking you must include the object code of your new widget classes (or the library which contains it) with the other libraries.

16.7.3 Subclassing User-Defined Widgets

In order to create sub-classes of existing classes, XFaceMaker needs some information about the widget classes. For the predefined Motif widgets, this information is built into XFaceMaker, but for user-defined widgets added with the function `FmAddWidgetClass`, this information must be specified. Therefore if you want to create sub-classes of your own widgets, you must call the `FmSetSuperclassInfos` function after the call to `FmAddWidgetClass`, and before the call to `FmCallEditor`.

```
int      FmSetSuperclassInfos(
WidgetClass *widget_class;
char      *includes;
char      *class_parts;
char      *widget_parts;
char      *class_init;
char      *class_part_init;
WidgetClass *subclass_prototype;
int      subclass_prototype_size;)
```

`FmSetSuperclassInfos` returns 1 on success, 0 on failure. See its detailed description in the *XFaceMaker Reference Manual*.

When using subclassed widgets created with XFaceMaker, the generated C-code file contains the function `FmAddNameWidgetClass` which calls `FmAddWidgetClass()` with the appropriate values for the new widget class. The compilation flag **XFM** enables this code to link the new widget with XFaceMaker. However, it is up to you to register any resource converters that may be required by your new widget, even if this is automatically done on widget instantiation, because XFaceMaker may have to parse the new resource types before this point. The *XFaceMaker Examples* directory contains the sub-classed widget example `TextValid`.

16.8 Calling Extension Functions Indirectly

You can register new widget classes without directly referencing Fm functions by using `FmGetFunctionsVector`. This is useful in a library of widgets, which may be used with or without the `libFm_e.a` library. This function returns a structure containing pointers to XFaceMaker functions used to register new widget classes.

This is used in the extension `.c` file generated for the extension library in the example of section “Example - Adding an Extension” on page 326. Instead of calling the `FmAddWidgetClass` function directly, the `.c` file generated by **xfmextend** contains:

```
FmFunctionsVector *fns = FmGetFunctionsVector();
    (*fns->add_widget_class)(&xorgStateLabelWidgetClass,
                            0,
                            FM_PRIMITIVE,
                            FM_NORMAL,
                            0,
                            "#include <Xorg/StateLabel.h>",
                            "xorgStateLabelWidgetClass",
                            0,
                            0,
                            0,
                            "StateLabel.bm");
```

With this method, the Fm functions are not referenced directly by the object file of the class, but only through function pointers, so applications using the widget need not be linked with the `Fm_e` library.

The structure returned by `FmGetFunctionVector` is defined in `FmCommon.h`. The fields are pointers to the XFaceMaker extension functions.

CHAPTER 17: Controlling the Design Process

This chapter describes XFaceMaker's architecture, the *dual process mode*.

17.1 The Process Functions

The **XFaceMaker process** (also referred to as 'XFaceMaker') manages the entire XFaceMaker interface; i.e. the XFaceMaker Command window, editing windows, dialog boxes, etc. and the behavior associated with them. The XFaceMaker process also memorizes the editing state of the design process, in order to restore it when an error occurs or when the design process is restarted.

The **Design process** creates the interface being designed, and executes the FACE scripts written by the designer. The design process also handles some of the editing commands and feedback, such as saving and loading the interface files, drawing the selection handles, applying templates, etc.

Thanks to the XFaceMaker extension mechanism, the design process can be linked with the whole application code, allowing you to test the application functions while designing the interface. Since the design process can be launched while XFaceMaker is running, you can modify, compile and restart the application code as many times as required without exiting XFaceMaker.

In addition, although the design process handles an important part of the editing functions, it is generally much smaller than the XFaceMaker process in terms of memory usage and startup time, so restarting a design process is fast compared to restarting XFaceMaker itself.

XFaceMaker and the design process communicate through a UNIX socket and use a communication protocol implemented by the FACE interpreter. The FACE protocol allows two processes running a FACE interpreter to communicate through FACE function calls and through global active value calls. The protocol is not publicly accessible in this version of XFaceMaker, although it might be made public in future versions for application designers who wish to split their user interface and their application code into two separate processes.

Both processes are implemented by the same program, `xfm`. In most cases, the dual-process mechanism is transparent: you just type `xfm` (or `xfmwf` to also launch the XFaceMaker executable with the Draw Library) . The connection between the two processes is explained below.

17.2 Connection

By default, XFaceMaker runs in server mode, i.e. implements the XFaceMaker editor process. In server mode, XFaceMaker loads its interface, sets up a socket to accept connections from design processes, launches an initial design process and waits for it to connect.

A design process is launched by running `xfm` with the `-client` option. When this option is set, `xfm` (or `nslfm`) runs in client mode: it does not load the XFaceMaker user interface, but instead tries to connect to the XFaceMaker process. Once both the XFaceMaker process and the design process are ready, the XFaceMaker windows become active.

The initial design process is launched automatically by the XFaceMaker process when it starts, using the same executable name. All the command-line options specified for the XFaceMaker process are passed to the initial design process, with an additional `-client` option.

There may be at most one design process connected to a given XFaceMaker process. If a second design process tries to connect, XFaceMaker displays an error and the connection is refused.

Dual-process mode can be disabled using the `-monoprocess` option. With this option, XFaceMaker works as it did in previous versions, with only one process acting as both the XFaceMaker process and the design process.

17.3 Socket Addresses

The XFaceMaker process sets up two sockets for design processes to connect to: a UNIX domain socket and an INTERNET domain socket. The path name of the UNIX socket and the port number of the INTERNET socket are allocated by the XFaceMaker process, and stored in a property of the X root window together with the name of the host machine where XFaceMaker is running, and with an identifying name for the XFaceMaker server, by default "xfm". The design process fetches XFaceMaker's socket addresses from the same root window's property and connects to it using the UNIX socket if it is running on the same host, or the INTERNET socket otherwise.

By default, there can be only one XFaceMaker process for a given X server. If an XFaceMaker process is launched while a previous one is running, a dialog box asks the user if the first process should be discarded. If yes, the address of the first XFaceMaker process will be discarded from the X root window's property, and it will not be possible to connect a design process to it any more. The dialog box will also appear if an XFaceMaker process has previously terminated abnormally, for example if it has been killed using a KILL signal: in that case, you should answer 'Yes'.

To run several XFaceMaker processes on the same display, one should use the `-servername` option, which specifies a different identifying name under which the XFaceMaker socket addresses will be stored in the X root window's property. The same server name has to be specified for the XFaceMaker process and the design process (which is done automatically when the design process is launched by XFaceMaker).

The two processes can run on different host machines, but they must be connected to the same X display. Of course, the versions of the two processes must match: for example a Motif 1.1 XFaceMaker will not run properly with a Motif 1.2 design process.

The socket addresses can also be specified directly using command-line options, although this is seldom necessary. The `-serverhost` option specifies the host on which the XFaceMaker process is running. The option `-socketfile` specifies the UNIX socket path name to which to connect. The `-socketport` option specifies the INTERNET socket port number to which to connect.

17.4 Error Protection

Error protection is the first major functionality offered by the dual-process mode. In most UIMS's, as in previous versions of XFaceMaker, the interface and its behavior are implemented in the same process as the editor itself. This means that if the interface crashes (which can happen, for example, if the user calls a Motif function in a callback with wrong arguments), then the whole program crashes. XFaceMaker solves this problem by separating the interface being designed by the user, from the XFaceMaker editor itself. In the XFaceMaker architecture, the design process can crash, but the XFaceMaker process stays alive. Error recovery is explained in Chapter 6.

17.4.1 Design Process Termination

When a fatal error occurs in the user's interface, the Design process terminates. A fatal error can result from:

- a call to XtError, usually caused by a widget used in an abnormal way, or an X Toolkit or Motif function called with wrong arguments;
- a fatal UNIX signal, typically 'Bus error', 'Segmentation violation', or 'Illegal instruction', which has usually the same origin as an XtError but has not been checked by the toolkit programmer;
- an error occurring during the execution of a FACE script, if the option ***Restart on FACE Errors*** was set in the Options menu.

The first two kinds of errors cannot be disabled: they always terminate the design process, because they indicate that some data may be corrupted. Termination of the design process when a FACE error occurs can be disabled by clearing the ***Restart on FACE Errors*** item in the Options menu of XFaceMaker. If this toggle is cleared and a FACE error occurs, an error message will be displayed but the FACE script will only be aborted, as in the previous version. It must be noted that such errors may also corrupt data in some cases, so it is wiser to set the ***Restart on FACE Errors*** option.

When the design process terminates, the nature of the error is displayed in the XFaceMaker message box, and an alert box pops up to inform the user. It may happen that the design process gets so corrupted that it cannot even display the error before exiting. In this case, only the alert box will pop up.

Once the alert box is acknowledged, a dialog box prompts the user for a command line which will be executed to restart the Design process. In most cases, one just needs to click 'OK' to restart the Design process. The following two sections explain in more detail the options available at that time.

17.4.2 Restarting the Design Process

The default command line displayed by the restart dialog box launches the same design process as the one which just terminated, usually `xfm` (or a new user-built instance of XFaceMaker), with all the command-line options specified initially, plus the `-client` option and the `-restore` option explained below.

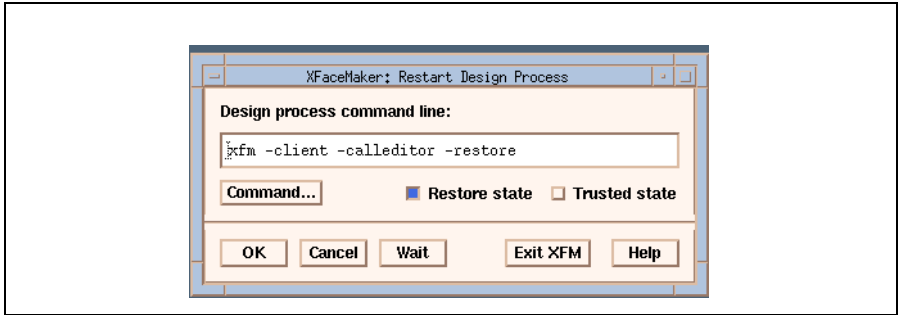


Figure 17.1: The Restart Process Box

- The **Command** pushbutton pops a File Selector for choosing a different XFaceMaker executable.
- The **Restore state** and **Trusted state** buttons toggle the `-restore` and `-trusted` options (explained below) respectively in the command line.
- **OK** launches the specified command line. XFaceMaker will then wait until the new design process connects to it.
- **Cancel** will not launch any design process. The user will then be warned that XFaceMaker is running in mono-process mode. Clicking ‘No’ at this time will go back to the command line dialog box.

- The ***wait*** button will not launch any design process, but XFaceMaker will still wait for a connection (the XFaceMaker interface is disabled until the connection occurs). This is useful if you want to launch the design process manually from a shell, or by any other external means. In this case, don't forget to set the ***-cli-ent*** option when launching the design process.

17.4.3 Restoring the Design Process State

The XFaceMaker process keeps a copy of the state of the Design process. The ***-restore*** option informs the starting Design process that it must fetch this state from the XFaceMaker process, and modify its own state accordingly. In short, the new Design process will restart where the old one had stopped. The Design process state includes:

- the whole hierarchy of widgets,
- the sets of loaded groups, templates and classes,
- the current editing mode (Try or Build),
- various internal informations: current directory, current file name, current edited object, template or class model objects in edition.

If the Design process was restarted by the user, or if it terminated due to an execution error in Try mode, the Design process state is saved just before the Design process exits.

If the Design process crashed during a widget modification (i.e. when the user clicked 'Apply' in an editing box after modifying a resource or another attribute), then XFaceMaker restores the state of the Design process "just before" the widget was rebuilt, and things will be as if the user had just made the modifications, but not yet clicked 'Apply'. In this case, the modifications should be corrected and another 'Apply' should be attempted, or the modifications can be discarded by selecting another object (the 'Resource modified/ignore changes?' warning will be issued).

It may happen that the Design process state cannot be saved before it exits, for example if its data is too corrupted. For this reason, XFaceMaker keeps a second security state, called the 'trusted state', which is saved periodically and every time the user switches to Try mode. You can specify how often you want the trusted state to be saved in the Options menu, ***Auto Save State***. The trusted state can be restored by using the ***-restore*** and ***-trusted*** options simultaneously. So, the typical scenario is to try first ***-restore*** alone (the default), and if this fails try ***-restore -trusted***.

In addition to having the Design process state kept by the XFaceMaker process, it is possible to save the state to disk. This can be useful, for example, to protect against a power failure without saving a `.fm` file. The '**Save State**' command in the 'File' menu saves the current Design process state into a set of files starting with `.x_fm_` in the current working directory of XFaceMaker. Once saved, this state can be restored by passing the `-restore` (and perhaps `-trusted`) option(s) to the XFaceMaker process. The state will first be loaded from disk files to the XFaceMaker process, and then restored into the initial Design process.

17.4.4 Launching a New Design Process

The user can control the Design process using the **Restart Design Process** command in the 'File' menu. This command terminates the Design process (with a confirmation if the current interface was modified). The Design Process Restart dialog box appears. See the previous sections on how to choose the Design process and its options.

The **Restart Design Process** command is used to start a different Design process, and gives access to the 'Dynamic Extension' functionality in XFaceMaker. For example, suppose you want to define a new C function and add it to XFaceMaker so that it can be called in a callback. As in previous versions, you have to write your function, compile it, and link it with a main program containing calls to `FmInitialize`, `FmAttachFunction` and `FmCallEditor`. In fact, XFaceMaker will do everything – except writing the function – automatically, as explained in chapter 10. The difference is that you don't have to exit your current XFaceMaker and start the new one from the beginning. Instead, you just have to call the **Restart Design Process** command and specify the new program as the Design process. Your new function is now available.

When XFaceMaker is running in mono-process mode (without any Design process connected), the **Restart Design Process** switches back to dual-process mode by starting a new Design process.

CHAPTER 18: Working with UIL

This chapter describes the facilities provided by XFaceMaker to work with the OSF/Motif User Interface Language (UIL). It also explains how the interface description is translated from one format to the other, the problems you may encounter while translating your interface, and how to change your application to use the new format. Before you read this chapter, you should be familiar with UIL in order to understand the technical details involved in the UIL/Fm translation.

XFaceMaker enables you to:

- generate a UIL description of an XFaceMaker interface. You can do this by setting the **Save UIL file** option, or by using command-line options. This saves your interface in UIL format along with the normal `.fm` file
- translate an existing UIL interface into an `.fm` file to continue developing an application using XFaceMaker. You can translate the existing file, then edit it with XFaceMaker using a separate translator program `ui2fm`.

Translating a UIL file to XFaceMaker format, editing it using XFaceMaker, and then saving it back to UIL format is a complex process. The two languages have different functionalities, so you might not obtain a file with the same structure as the original UIL file. Therefore, although this should work with very simple interfaces, it is not recommended.

To check whether you have the UIL feature, look in the Options menu. If UIL is available on your system, the **UIL...** toggle button will be sensitive.

18.1 Generating UIL Files

There are two ways to ensure that XFaceMaker will generate a UIL file of the interface at save time:

- Set the **Save UIL file** checkbox in the UIL Options window.
- Use the `-uil` command line option.

18.1.1 The Save UIL file Option

If you set the **Save UIL file** option, XFaceMaker will save the description of your interface in UIL format every time you save your interface using **Save**, **Save As**, or **Save Selected As**. Open the UIL Options window by selecting **UIL...** in the Options menu.

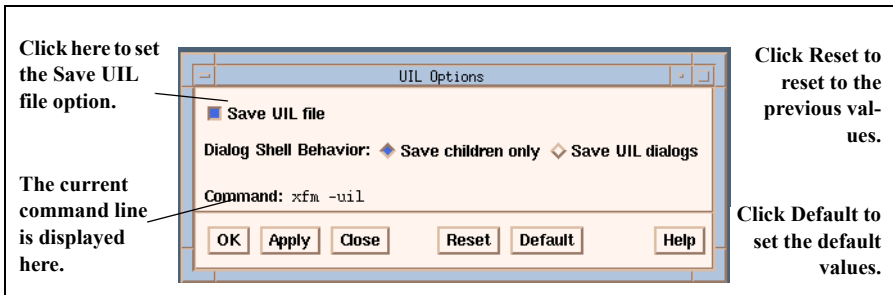


Figure 18.1: The UIL Options Window

To set UIL options:

1. Set the **Save UIL file** checkbox.
2. Set the desired options. You have the following choices:

Save children only When set, XFaceMaker will generate all dialogs as `XmDialogShell`'s with a `XmMessageBox` widget child.

Save UIL dialogs When this option is set, XFaceMaker will generate predefined combinations of objects called *Dialogs* in the UIL file. These use the motif convenience functions e.g., `XmCreateWarningDialog`. See Section 18.3.7 for more information.

3. Click **OK** to apply the options and close the box, or **Apply** to apply the options and keep the box open.

- You can save the state of the **UIL Options** window in the `.xfm.startup` file for the next time you use XFaceMaker, by selecting **Save Prefs** in the File menu.
- All options have a corresponding command line option and these are shown in the **Command line** field for copying to the shell prompt or a Makefile. Table 18.1 shows the equivalent command line options. These options can be used with all C Code and RDB options.
- When you save UIL, the C code generated, if any, does not contain the widget hierarchy and the resource settings since they are in the associated UIL file.

18.1.2 The -uil Command Line Option

You can also generate an UIL file from a `.fm` file by launching XFaceMaker with the `-uil` command-line option, and specifying the name of the `.fm` file you want to translate:

```
xfm -uil <file>.fm
```

This is equivalent to launching XFaceMaker normally, loading the file with the **Open...** command, setting the **Save UIL file** option, and saving the file with **Save**. Note that XFaceMaker still needs to open a connection with the X server, so the `DISPLAY` environment variable must be set correctly, or the `-display` option must be used. The `-uil` option can also be used in conjunction with the `-compile` and `-compilegroup` options. When the `-uil` option is set, `-compile` is equivalent to `-compilegroup`: all widget references in FACE scripts will be resolved at run-time.

```
xfm -uil -compile <file>.fm [-cfile <cfile>.c]
```

If you specify a name for the C-code file with `-cfile`, then the UIL file will have the same base name as the C-code file. If you use `-compile` or `-compilegroup` in conjunction with `-uil`, XFaceMaker will generate the same UIL and C-code files as if you had used the **Save UIL file** and **Save C Code file** options.

Table 18.1: UIL Command Line Options

Option	Command Line Option	Use with
Save DialogShell children only	default	-uil
Save UIL Dialogs	-uildialogs	-uil

18.2 Saving UIL and C Code Files

The ***Save UIL file*** option can be used in conjunction with the ***Save C Code file*** option. In that case, XFaceMaker generates a UIL file and a C file. The UIL file is the same as if you had used only ***Save UIL file***. However, the C-code file is similar to the file normally output by XFaceMaker, except that the Motif function calls that create the widgets of the interface are replaced by calls to the **Mrm** library that will fetch the widgets from the UID file.

The C-code file contains:

- C functions corresponding to the FACE scripts of the Fm interface,
- a creation function, which contains Mrm calls to load the UID file and fetch the widgets of the interface, and also Motif calls to create the Shells of your interface that cannot be saved in the UIL file (see Section 18.3.3).

While generating UIL files, the C-code file is always generated as a *Group*, which means that the widget references in FACE scripts will be resolved at run-time.

The UIL option can be used for any application using C-code files generated by XFaceMaker. All you have to do is generate the UIL file along with the normal C-code file of your interface, and compile the UIL file into a UID file using the Motif UIL compiler `uil`.

Externally, the C-code file generated in this way is equivalent to the normal C-code file, so the rest of the application need not be changed. This provides an interesting intermediate level of interpretation: all your callback scripts are compiled into C-code, but the UIL description can be changed without re-compiling the application.

Also, this technique reduces the size of your application, since part of your interface is stored as a separate UID file instead of C-code. Your application can still be ported to different platforms on which UIL is supported.

18.3 Contents of the Generated UIL File

This section describes how an XFaceMaker interface is translated to a UIL description, and emphasizes a few aspects of the files.

18.3.1 Structure of the UIL File

Every UIL file generated by XFaceMaker consists of one UIL module, and contains:

- *the module declaration*. The module name is the base name of the UIL file.
- *procedure declarations*. Every FACE script of the interface is transformed into one or more procedures (see Section 18.3.4). The procedures are declared without arguments.
- *object declarations*. Every widget of the interface is saved as a UIL object (except for Shell widgets. See Section 18.3.3).

No special clauses are included in the UIL file, which means that the module is case-sensitive. All the UIL keywords are output in lowercase.

For every object in the UIL file, the following is output:

- its name,
- its widget class,
- whether it is a gadget or not,
- its callbacks,
- its arguments (resources),
- its control list, i.e., its children widgets.

18.3.2 Widget Names

In a UIL module, every widget (and every other named object) must have a unique name. Since this is not the case in `.fm` files, the program checks for duplicate names while generating the UIL files.

When two widgets have the same name, the second one is saved with a different name, obtained by post-fixing a number. When this happens, a warning is issued, because this might affect the behavior of the interface (for instance, if the widget was referenced in a FACE script).

In that case, you should change the name of the widget to a unique name, and also all the references to it in FACE scripts, before generating the UIL file.

Also, all UIL names have a maximum length of 31 characters. XFaceMaker truncates widget names to this length, and then checks for duplicate names. Again, if a name is changed, a warning is given.

18.3.3 Shell Widgets

The standard UIL compiler does not recognize Shell objects, which means that Shell widgets cannot be saved in the UIL file. If your interface contains a shell (which is most often the case), only the child of the shell and all its descendants will be saved in the UIL file. It is then the responsibility of your application to create the Shell widget using the Motif library, and then to fetch its child from the UID file.

XFaceMaker does this for you if you generate a UIL file along with a C-code file (either by setting both the ***Save UIL file*** and ***Save C Code file*** options, or by using the `-uil` and `-compile` command line options). In that case, the creation function contained in the C-code file will create the Shell widgets of your interface by direct calls to the Intrinsics and the Motif library. The creation function also contains calls to the Mrm library to fetch the widgets from the UID file.

Note: The C-code file contains the name of the UID file, which is obtained by replacing the `.uil` suffix by `.uid`, so you must compile the UIL file into a UID file with the same base name.

18.3.4 FACE Scripts

It is not possible to specify a complex program as the value of a callback in UIL, but only the name of one or more procedures to be added to the callback list. For this reason, in general, FACE scripts cannot be saved in the UIL file. Instead, a procedure name is generated and saved as the value of the callback resource in the UIL file. The name of the procedure has the form:

`<widget-name>_<callback-name>`

and is truncated to 31 characters. It is the responsibility of the application to define and to register the procedure, but again, if you generate C-code code along with UIL code, XFaceMaker does it for you in the C-code file. The FACE script is compiled into a C function, which is registered in the creation function of your interface.

An exception to what is explained above occurs when the UIL file is generated alone without a C-code file, and the script contains ONLY calls to functions of the form:

`<function-name>([self] [, <string-argument>] [, $]);`

Since these match the calling sequence of a UIL callback procedure, XFaceMaker generates the direct procedure call in the UIL file. For example, the following .fm callback specification:

```
activateCallback =
    function_0();
    function_1(self);
    function_2(self, "Hello");
    function_3(self, "Good bye", $);
```

would be translated in UIL as:

```
activateCallback =
    procedure function_0();
    procedure function_1();
    procedure function_2('Hello');
    procedure function_3('Good bye');
```

If the FACE function has at least two arguments, then the second argument must be a string, which will be passed as the callback tag to the UIL procedure. If the script does not fulfill these conditions, then a single procedure will be generated instead as explained before.

18.3.5 Active Values

UIL does not provide any mechanism equivalent to XFaceMaker active values, so active values are not saved in the UIL file. UIL *identifiers* are similar to active values in that they are values provided by the application at run-time, but they are not used in the same way as active values.

If you are generating a UIL file alone, you will get a warning when XFaceMaker encounters an active value. If you are saving your interface in C-code along with the UIL file, XFaceMaker will generate code to create and handle active values in the C-code file. In this case, your interface will be equivalent to the interface generated in C-code only.

18.3.6 Create Callbacks

Both XFaceMaker and UIL provide *create callbacks*; that is, callbacks that are activated when a widget is created. When a `.fm` file is translated into UIL format, the FACE scripts attached to the create callbacks of your widgets are output as UIL create callbacks.

The UIL create callbacks are equivalent to XFaceMaker create callbacks, with one exception. In XFaceMaker, the create callback of a widget is called after the widget has been created *and* all its children have been created and managed, whereas in UIL, the create callback is called *immediately after* the widget has been created and before its children are created. This does not seem to be specified in the UIL documentation, but can be seen in the source code of the Mrm library.

Therefore, you should *not* reference the descendants of a widget in its create callback if you want to translate it to UIL. Otherwise, you will get an error because the children will not yet exist when the create callback is executed. Unfortunately, create callbacks are often used in XFaceMaker to set a resource of a widget to one of its sub-widgets.

For instance, this is how to set the `menuHelpWidgetresource` of a `MenuBar` in XFaceMaker with the following `xfmCreateCallback` script of the same widget:

```
self:menuHelpWidget = self.help_cascade;
```

Since this is the only way to specify a sub-widget as a resource value in XFaceMaker, this case is handled especially during translation to UIL format, and the resource assignment in the create callback is transformed into an argument statement in the UIL file:

```
object menu_bar : XmMenuBar widget {
    controls {
        managed XmCascadeButton help_cascade;
        ...
    }
    arguments {
        XmNmenuHelpWidget = help_cascade;
    }
}
object help_cascade : XmCascadeButton widget {
    controls {
        XmPulldownMenu help_pulldown;
    }
}
object help_pulldown : XmPulldownMenu widget {
    ...
}
```

18.3.7 Dialogs

The UIL compiler recognizes some combinations of widgets as single special objects. For instance, an XmMessageBox widget whose parent is an XmDialogShell can be described in UIL as a single object called an XmMessageDialog. XFaceMaker can optionally generate such UIL special objects. The `-uildialogs` command-line option will cause XFaceMaker to recognize such objects and to generate the correct UIL description. By default, only the child object, the XmMessageBox in the previous example, will be output in the UIL file.

18.3.8 Menus

Menus are another case where the UIL file will not have the exact same structure as the XFaceMaker interface. In XFaceMaker, a PulldownMenu is a sibling of its controlling CascadeButton, whereas in UIL, it is contained in its control list. This case is handled during `.fm` to UIL translation.

18.3.9 User-defined widgets

XFaceMaker supports generation of interfaces containing *user defined* widget classes. User defined classes include:

- the widget classes that you can add using the `FmAddWidgetClass` function.
- any widget classes included in XFaceMaker by NSL, e.g. in the `ns1fm` executable.

For these user-defined classes, XFaceMaker outputs in the UIL files all the declarations that are needed by the UIL compiler.

- argument (resources) declarations,
- reasons (callbacks) declarations,
- creation function declaration.

The resources of a user-defined widget class are declared with the `any` UIL type, so there will be no type checking at UIL compilation time on these resources.

The resources and callbacks are declared with a suffix of the form `NameN`, where `Name` is the name of the user-defined class.

The creation function for the class is declared and registered automatically in the associated C-code file if you generate C-code and UIL files. The creation function is called `NameCreate`.

18.4 Changing the Application

As we have already explained, when you generate your XFaceMaker interface in UIL format, you can either produce just the UIL file, or the UIL file along with a C-code file.

In the first case, you will have to:

- replace the calls to functions of the `libFm.a` (or `libFm_c.a`) library in your application by calls to functions of the `libMrm.a` library provided by Motif,
- add Motif function calls to create the shells that are not described in the UIL file.

If you choose this solution, please refer to the UIL and Mrm documentation for a description of the Mrm functions used to load and instantiate an interface from a UID file.

The second solution is much easier. If you produce your UIL file along with a C-code file, you will have no extra work to do, except to:

- add `-lMrm` when linking your application,
- make sure that you always call the `FmCreateName` function generated by XFaceMaker in the C-code file with a non-null `parent` argument. This is because the `parent` argument might be used to convert resources for the Shell widgets of your interface. For an applicationshell, you should pass the default application shell returned by `FmInitialize`.

18.5 Translating UIL Files to Fm files

This section explains how to translate an interface in UIL format into one or more XFaceMaker files, using the `uil2fm` program. This is provided on the XFaceMaker distribution tape with the `xfm` editor itself.

`uil2fm` translates the UIL file in two major steps:

1. The UIL file is parsed and compiled into an internal form by use of the `Uil` function of the UIL library.
2. The UIL parse tree is then traversed, and every *root* widget (i.e. a widget that is not a child of another widget) is saved with all its descendants.

Note: The `uil2fm` program uses the UIL library to parse UIL files. For this reason, the program is only available on platforms supporting UIL.

18.5.1 `uil2fm` Syntax and Options

`uil2fm` has the following command syntax:

```
uil2fm [-d] [-u] [-m] [-a] [-g] [-p <bitmaps-path>]
        [-x [-f <color>] [-b <color>] [-r <rgb-path>] ]
        [-i <uil-file>] [-o <fm-file>]
        [<uil-file>] [<fm-file>]
```

`<uil-file>` is the UIL source file. The default name is `default.uil`.

`<fm-file>` is the new `.fm` file created unless the `-g` option is used.

If `<uil-file>` was specified and `<fm-file>` not specified, then the `.fm` file will have the same base name as `uil-file` with the `.fm` suffix: otherwise `fm-file` is `default.fm`.

The options are:

- d** dump the UIL symbol table of the UIL file using the function `_UilDumpSymbolTable`.
- u** produce a UID file as well as the `.fm` file. The UID file has the same base name as the UIL file with a `.uid` suffix.
- m** output toplevel objects of the interface as *mapped* in the `.fm` file, so that they will be immediately visible when the file is loaded. By default, the toplevel objects are saved unmapped.
- a** output an application shell as the toplevel object of the `.fm` file, all other objects of the UIL interface will be children of this application shell.

- g** output every toplevel object of the UIL interface in a separate `.fm` file for use as an XFaceMaker group. Every toplevel object is saved as a file named `<fm-base-name>_<name>.fm`, where `fm-base-name` is the base name of `fm-file` and `name` is the name of the toplevel object.
- p bitmaps-path** save UIL icons in the directory `bitmaps-path`. The default is `"."` (dot) the current directory.
- x** output UIL icons as XWD format files instead of X bitmap files.
- f** specify the foreground for icons when `-x` is used.
- b** specify the background for icons when `-x` is used.
- r rgb-file** specify an alternate color database text file when `-x` is used. The default is `/usr/lib/X11/rgb.txt`.
- i uil-file** alternate way to specify the `uil-file`.
- o fm-file** alternate way to specify the `fm-file`.

18.5.2 UIL to Fm Translation Rules

This section explains how the UIL objects will be translated into XFaceMaker objects.

18.5.2.1 Structure of the Files

If the `-g` option is set, every toplevel object is saved in a separate `.fm` file. Otherwise, all the toplevel objects will be saved in the same file, in the order in which they appear in the UIL symbol table, which might not be the order in which they appear in the source UIL file.

If the UIL file does not contain any widget definitions, but only value definitions, an ApplicationShell widget with no children is output in the `.fm` file.

A UIL description is sometimes used as a template, and fetched (instantiated) several times in an application. For instance, a UIL file can contain the description of one XmPushButton object, but the application will fetch this button several times to create several identical buttons. In this case, `uil2fm` will output only one object in the `.fm` file.

If the UIL file also contains other objects (for instance the application main window), it will not be possible with XFaceMaker to fetch only the button. To do that, use the `-g` option so that the template is saved into a separate file, and every toplevel object should be loaded individually.

18.5.2.2 Values

In UIL, *values* are compile-time variables that can be used to hold the contents of widget resources and callback arguments. Values can hold constants of several types, or expressions which are evaluated during the compilation of the UIL module. UIL values are output at the time they are encountered during output of the widget entries, for instance as the right-hand side of a resource assignment, or as a procedure argument.

UIL values can be *private*, *exported* or *imported*.

- Private values are used only in the UIL module where they are defined. `uil2fm` outputs the contents of the value, which has already been evaluated by the UIL parser.
- Exported values are output the same way as private values, but since they can be referenced from another UIL module, `uil2fm` has to provide a mechanism to do this, which is by means of an XFaceMaker active value. For every exported value in the UIL file, an active value is associated with the root widget currently being saved. The active value has the name of the exported value, and its *Get* script returns the contents of the value. For example, if the UIL file contains:

```
value twenty : exported 20;
```

then the following active value will be defined on the toplevel object of the resulting `.fm` file:

```
activevalue {  
    name = twenty  
    get = @ = 20;  
}
```

- For imported values---those defined in another UIL file---active values are also used. If the imported value is used as the value of a resource, then the resource assignment will be done dynamically in the XFaceMaker create callback of the widget: the active value corresponding to the imported value will be fetched, and the result will be assigned to the correct resource. If the imported value is used as an argument to a callback procedure, then the active value is fetched before the function is called, and the result is passed to the function.

For example, the following UIL description:

```
value i : imported integer;  
procedure print(integer);  
object button : XmPushButton widget {  
    arguments {  
        XmNwidth = i;  
    };  
};
```



```
    callbacks {  
        XmNactivateCallback = procedure print(i);  
    };  
};
```

is translated into:

```
widget XmPushButton {  
    flag {  
    }  
    createproc {  
        AttachAv("i", PointerTo(0));  
        GetActiveValue(Widget(0), "i");  
        AttachAv("i", @i);  
        self:width = @i;  
    }  
    resources {  
        activateCallback =  
            function Any print(Widget, None, None);  
            AttachAv("i", PointerTo(0));  
            GetActiveValue(Widget(0), "i");  
            print(self, @i, $);  
    }  
    activevalue {  
        name = i  
    }  
} button
```

The contents of a value are generated in the `.fm` file in such a way that they can be interpreted by the existing resource converters of the OSF/Motif toolkit.

The following special cases should be noted:

- **icons** UIL supports *icon* values, which are pixmaps defined inside the UIL file with a special format. Since XFaceMaker does not support icons, these are saved as separate files by `uil2fm`. The icon file is located in the directory specified by the `-p` option, and has the name of the icon value, possibly extended with a number if the file already exists. The value is then output as the name of the bitmap file.

If the `-x` option is not set, then an X bitmap file is produced. In this case, only the “black-and-white” data of the icon is saved, so multi-colored icons will become bitmaps.

If the `-x` option is set, then a file in XWD format is produced. The RGB values for colors are read from the file `/usr/lib/X11/rgb.txt` (or another file specified with `-r`). Foreground and background colors can be specified with `-f` and `-b`. The type converter used in XFaceMaker for Pixmaps recognizes XWD format files.

- **tables** All values of type *table*, XmString tables, font tables, etc., are saved as comma-separated strings. This can cause a problem for types which do not have a resource converter defined, such as ASCII string tables. In this case, the application will have to provide the appropriate resource converter.

18.5.2.3 User-Defined Objects

In UIL, user-defined objects are not declared using their class name, but using the name of the creation function as in: *MyClassCreate*. Since `uil2fm` needs to know the name of the class, it tries to guess it from the name of the creation function. If the creation function name is of the form *NameCreate*, then `uil2fm` assumes that the class name is *Name*. Otherwise, the whole name of the creation function is taken as the class name. In all cases, a warning is issued for every user-defined class encountered.

18.5.2.4 Identifiers

UIL identifiers are similar to imported values, except that the value of an identifier is not defined by another UIL file, but by the application itself. In fact, an identifier is an XFaceMaker active value without conversion scripts.

`uil2fm` translates every UIL identifier into an active value, without Get or Set scripts. When an identifier is used as a resource value, this resource is assigned in the create callback of the widget as for imported values, but here the active value must have been attached to a valid address in the application.

For example:

```
identifier i;
object button : XmPushButton widget {
    arguments {
        XmNwidth = i;
    };
};
```

is translated into:

```
widget XmPushButton {
    flag {
    }
    createproc {
        self:width = @i;
    }
    resources {
    }
    activevalue {
        name = i
    }
} button
```

Note: In the current implementation `uil2fm` does not always handle identifiers passed as callback procedure tags correctly, as described in the next section.

18.5.2.5 Procedures

UIL procedure references in callbacks are translated into FACE function calls, with the three standard arguments of a callback procedure:

```
XmNactivateCallback = procedure my_function('hello');
```

translates into:

```
activateCallback = \  
    function Any my_function(widget, None, None); \  
    my_function(self, "hello", $);
```

If a callback tag is passed to the procedure, `uil2fm` will convert it in the appropriate way, because UIL procedure functions always expect the tag to be passed by reference.

For example, if an integer tag is passed, the function in the generated FACE script will be passed a pointer to the integer. For this, a special function `PointerTo` has been added to FACE.

Note: In the current implementation, if an identifier is passed to a callback procedure, it will *not* always be converted to a reference. Instead, the contents of the active value is simply passed without conversion. This implies that you must attach the right value in your application. For example:

```
identifier i;  
...  
XmNactivateCallback = procedure my_function(i);
```

translates into:

```
activateCallback = \  
    function Any my_function(widget, None, None); \  
    my_function(self, @i, $);
```

If `i` is attached to the address of an integer, `my_function` will receive the integer instead of its address. Therefore, you should attach `i` to the address of a pointer to the integer.

18.5.2.6 Dialogs and Convenience Objects

The UIL compiler recognizes some combinations of Motif widgets as single objects, which are not known by XFaceMaker. The table below summarizes the transformations made by `uil2fm` during its translation.

Table 18.2: UIL: Dialogs and Convenience Objects

UIL Object	Fm Parent	Fm Child	Fm Additional Resources
Radio Box	-----	RowColumn	C:packing = pack_column C:radioBehavior = true C:isHomogeneous = true
ErrorDialog	DialogShell	MessageBox	C:dialogType = dialog_error
InformationDialog	DialogShell	MessageBox	C:dialogType = dialog_information
QuestionDialog	DialogShell	MessageBox	C:dialogType = dialog_question
Warningdialog	DialogShell	MessageBox	C:dialogType = dialog_warning
WorkingDialog	DialogShell	MessageBox	C:dialogType = dialog_working
SelectionDialog	DialogShell	SelectionBox	C:dialogType = dialog_selection
PromptDialog	DialogShell	SelectionBox	C:dialogType = dialog_prompt
ScrolledList	ScrolledWindow	List	P:scrollingPolicy=application_defined P:visualPolicy = variable P:scrollBarDisplayPolicy = static P:shadowThickness = 0
ScrolledText	ScrolledWindow	Text	P:scrollingPolicy=application_defined P:visualPolicy = variable P:scrollBarDisplayPolicy = static P:shadowThickness = 0

C: Resource -- applied to the Fm child widget

P: Resource -- applied to the Fm parent widget.

18.5.3 UIL and the Application

To convert an application from UIL to XFaceMaker, you will have to change the part of your application that manages the interface to use XFaceMaker functions instead of Mrm functions.

This section provides specific descriptions of how to do this. We will not provide a general method, since applications can be very different, but we will show examples of how the most frequent cases can be handled.

In the following examples, the code used for the XFaceMaker application is surrounded with `#ifdef FM ... #endif`, and the code for the UIL application is surrounded with `#ifdef MRM ... #endif`. It is good practice to keep the Mrm code in your application so that you can test the two cases.

18.5.3.1 Initializing a UIL Application

The initialization sequence of an UIL application usually consists of initializing the Toolkit and the Mrm library. This can be replaced by a single call to `FmInitialize`:

```
Display      *display;
XtAppContext app_context;
#ifdef MRM
    MrmInitialize ();
    XtToolkitInitialize();
    app_context = XtCreateApplicationContext();
    display = XtOpenDisplay(app_context, NULL,
                           "example", "Example",
                           NULL, 0, &argc, argv);
#endif
#ifdef FM
    Widget toplevel = FmInitialize("example", "Example",
                                   NULL, 0, &argc, argv);
    app_context = XtWidgetToApplicationContext(toplevel);
    display = XtDisplay(toplevel);
#endif
```

18.5.3.2 Creating an Application Shell

In a UIL application, the application shell must be created “by hand”, using `XtAppCreateShell` for instance. With `XFaceMaker`, you have two solutions:

- You can specify the `-a` option when translating your main UIL module so that `uil2fm` includes an `ApplicationShell` in your `.fm` file automatically, in which case you don’t have to do anything in your application.
- Or, if you have not specified the `-a` option, you will have to load a `.fm` file containing only an `ApplicationShell`:

```
#ifdef MRM
    Arg arglist[10];
    int n;
    ...
    n = 0;
    XtSetArg(arglist[n], XmNallowShellResize, True); n++;
    toplevel = XtAppCreateShell("example", NULL,
                                applicationShellWidgetClass,
                                display, arglist, n);

#endif
#ifdef FM
    toplevel = FmLoad("AppShell.fm");
#else /* FM */
```

The file `AppShell.fm` contains:

```
widget ApplicationShell {
    resources {
        allowShellResize = true
    }
} example
```

18.5.3.3 Registering Callback Procedures

In a UIL application, callback procedures are registered using `_MrmRegisterNames`. This has to be replaced by calls to `_FmAttachFunction`:

```
static MrmRegisterArg regvec[] = {
    {"my_function_1", (caddr_t)my_function_1},
    {"my_function_2", (caddr_t)my_function_2}
};
static MrmCount regnum = XtNumber(regvec);
...
#ifdef MRM
    MrmRegisterNames(regvec, regnum)
#endif
#ifdef FM
    for(n = 0; n < regnum; n++)
        FmAttachFunction(regvec[n].name,
                        regvec[n].value,
                        "Any", -1);
#endif
```

In this example, the functions are declared without the argument number or types, so there will be no type checking in FACE, but the functions could also be declared with their prototypes.

18.5.3.4 Loading the Interface

To load an interface from a UID file, the UID file is first opened and then the toplevel widget fetched. This can be replaced by a call to `FmLoad` or `FmLoadCreateGroup`, depending on whether your file contains an `ApplicationShell` or not:

```
#ifdef MRM
MrmHierarchy mrm_hier;
MrmCode      class;
Widget       main_win;
char uid_file[] = "example.uid";
...
MrmOpenHierarchy (1, uid_files, NULL, &mrm_hierarchy);
MrmFetchWidget (mrm_hierarchy,
                "example_main",
                toplevel,
                &main_win,
                &class);
#endif
#ifdef FM
#ifdef APPSHELL
/* .fm contains an ApplicationShell */
main_win = FmLoad("example.fm");
#else
main_win = FmLoadCreateGroup("example_main",
                             "example",
                             toplevel,
                             NULL, 0);
#endif
#endif
#endif
```

If your interface is contained in several files, you can do the same for every component. If you use `FmLoadCreateGroup` several times with the same group name, the group will be loaded the first time only.

18.5.3.5 Fetching Values from UIL

UIL exported values are translated into active values, calls to `_MrmFetchLiteral` must be replaced by calls to `FmGetActiveValue`. For instance, to fetch an exported string value named `name` from your interface:

```
#ifdef MRM
    MrmCode return_type;
    char *string;
    ...
    MrmFetchLiteral(mrm_hierarchy,
                    "name",    dpy,
                    &value,
                    &return_type);
#endif
#ifdef FM
    FmAttachAv("name", &string);
    FmGetActiveValue(0, "name");
#endif
```

18.5.3.6 Setting Identifiers for the Interface

To set the value of a UIL identifier for use by your interface, use active value functions instead of `MrmRegisterNames`:

```
static Position x, y;
...
#ifdef MRM
    MrmRegisterArg arg[2];
    ...
    arg[0].name = "x";
    arg[0].value = (caddr_t)x;
    arg[1].name = "y";
    arg[1].value = (caddr_t)y;
    MrmRegisterNames (arg, 2);
#endif
#ifdef FM
    FmAttachAv("x", &x);
    FmAttachAv("y", &y);
#endif
```

There is no need to call `FmSetActiveValue`, since the active values generated by `uil2fm` do not have any *Set* scripts. All you have to do is attach the active values so that the `@name` expressions return the right values.

18.5.3.7 Linking your Application

Having completed the changes, to make your application, link it with the `libFm.a` library instead of `libMrm.a`.

18.5.3.8 Generating a C-Code Interface

Once your application has been correctly changed to use Fm instead of UIL, you can generate C-code from your `.fm` files. The application will again have to be slightly changed, as is explained Section 11.1. By combining these two translations, XFace-Maker provides a way to compile an UIL interface into C.

CHAPTER 19: RDB Files

In a “classical” X application many resources are defined in a resource database (RDB) file. This file may be installed as an *application defaults* file in the `/usr/lib/app-defaults` directory, or the `XAPPLRESDIR` directory or added to, or overridden by, the user’s `$HOME/.Xdefaults` file. This flexibility enables you to customize the application, and perhaps more importantly, to speed up development when building the application.

When using XFaceMaker this becomes less important and one could argue that *total* flexibility in the hands of the final end-user is also slightly dangerous and prone to problems of correct installation, or incorrect editing, of the resource database. However, color and font information should be separated as these do change between different displays and systems.

Normally, XFaceMaker forces widget names to be unique by adding a suffix if the names are otherwise identical. This allows different children of the same widget parent to have the same name. However, you might want to use common widget names, e.g. `OKButton`, `MenuItemButton`, etc., for interface-wide resources. You can do so by setting the editor option `-nonuniquenames`. This allows your resource specification files to use common widget names.

19.1 Setting RDB Options

To set RDB options, open the edit window by selecting **RDB...** from the Options menu. All options have a corresponding Command Line option, shown below. These options can be used with all C Code and UIL options, but when you save different file versions, there are important consequences, as explained in Section 19.3.

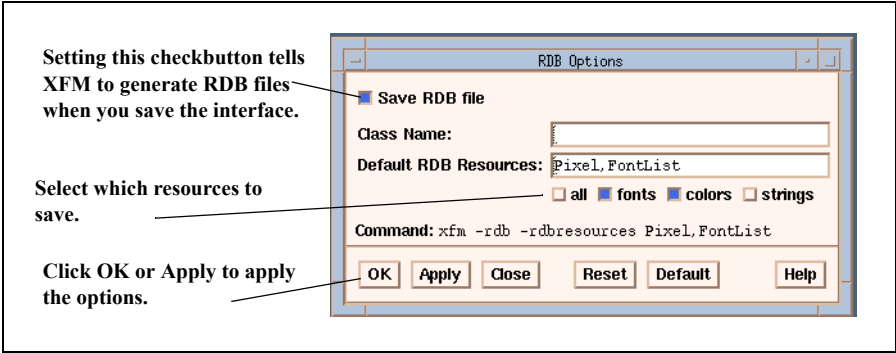


Figure 19.1: The RDB Options Window

Whenever you save an interface with the **Save RDB file** option set, two files are saved in addition to the standard .fm file:

- An .rdb file, the Resource Data Base File. This contains the resource specifications for the resources selected in the RDB Options Box.
- An .fm_rdb file. This contains the interface description file in .fm format, containing the full interface description except for the resource specifications which are in the .rdb file. C code and UIL files are generated from this file if those save options have been set.

Table 19.1: RDB Command Line Options

Option	Cmd Line Option	Use with
Class name	-rdbclass<Name>	-rdb
Save in RDB file: All resources	default	-rdb
Save in RDB file: Fonts	-rdbresources FontList	-rdb
Save in RDB file: Colors	-rdbresources Pixel	-rdb
Save in RDB file: Others	-rdbresources [Class],[Type],[Resource]	-rdb

To set the RDB file options do the following:

1. Select the **Save RDB file** checkbox.
2. Enter a name in the **Class name** field. This specifies the first component of the resource specification. Thus the name `MyAppli` will produce a resource file containing lines of the form:

```
MyAppli.name1.name2...namen: value
```

This allows multiple `.rdb` files to be generated when an interface has been split into several `.fm` files. Each RDB file can be given the same class name so that they can all be concatenated together to produce one application RDB file. If the class name is not defined then the name of the `.rdb` will be used.

3. Choose options by selecting the corresponding checkbox:

all This generates an RDB file with the extension `.rdb` containing all the interface resources, as well as a version of the `.fm` file with no resources, whose extension is `.fm_rdb`.

fonts This generates an RDB file with the extension `.rdb` containing all `FontList` resources, as well as a version of the `.fm` file with no font resources, whose extension is `.fm_rdb`.

colors This generates an RDB file with the extension `.rdb` containing all `Pixel`, i.e. color resources, in the `.rdb` file as well as a version of the `.fm` file with no color resources, whose extension is `.fm_rdb`.

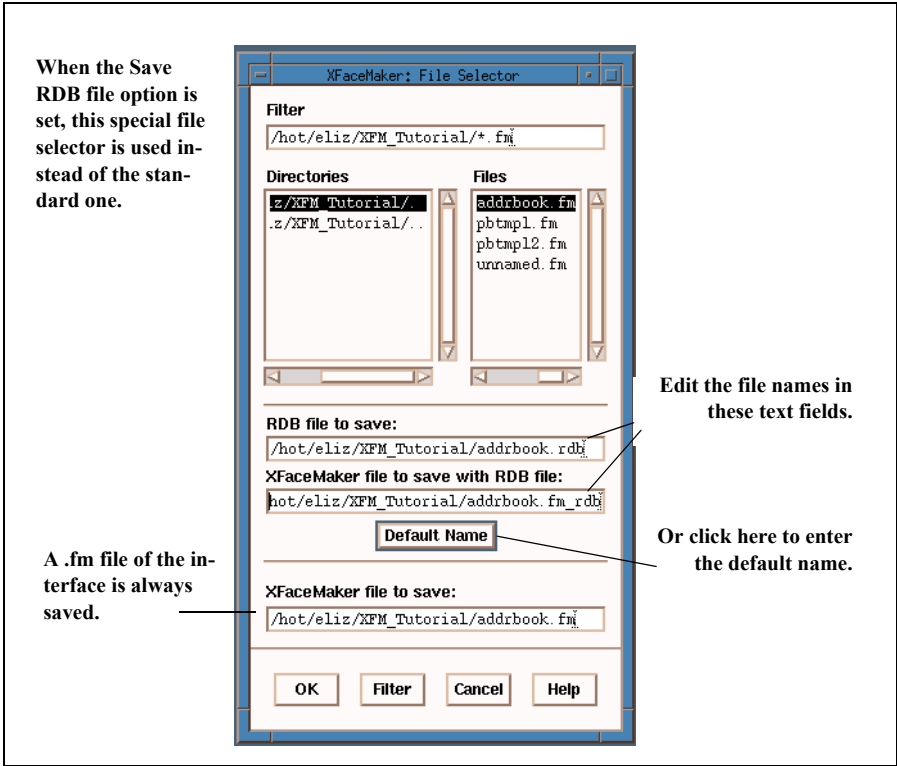
strings This generates an RDB file with the extension `.rdb` containing all `XmString` resources, as well as a version of the `.fm` file with no compound string resources, whose extension is `.fm_rdb`.

4. If desired, add other resources in the **Default RDB resources** field. This field shows which resources will be saved in the `.rdb` file. In addition to the selections above, you can add any other resources by defining the resource by its Class (with `XmC` removed), Type, or its name directly, e.g., `Foreground`, `mnemonic`, `Pixel`, `labelString`. The list is comma separated without intervening spaces. The widget resource Type as given in the standard OSF/Motif documentation is not always correct.
5. Click **OK** to apply the RDB options when an interface is saved. Or click **Apply** to apply the options but keep the window open.

19.2 Saving an RDB file

Once the options are set and applied using the RDB Options window, a special File Selector is used instead of the standard one.

- When you select **Save** in the File menu, then an `.rdb` file and the `.fm_rdb` file will be saved with the same root name as the `.fm` file.
- When you select **Save As, or Save Selected As**, then the File Selector will appear, allowing the `.rdb` and the `.fm_rdb` files to be saved under a different name from the `.fm` file. If a C-code file is saved at the same time, it will be a C-code version of the `.fm_rdb` file.



19.3 Generating Multiple Files

C code and UIL files

When you generate a C code or UIL file along with an RDB file, the resources that you have chosen to store in the RDB file will *not* be included in the C code or UIL file.

When you save an RDB file, you actually save three `.fm` files:

- `.fm` contains everything, as though you had not saved an RDB file.
- `.fm_rdb` contains everything *except* the resources you specified should be saved in the RDB file.
- `.rdb` contains *only* those resources you specified should be saved in the RDB file.

If you set the Save C code or Save UIL code option as well as the RDB option, the C code or UIL file is generated from the `.fm_rdb` file, and therefore does not include the resources you specified to be saved in the RDB file.

For example, if you specify that fonts be saved in an RDB file, and you save a C code version of the interface as well, the C code version will not include any font specifications.

Internationalized files

If you intend to internationalize the interface, do not place any strings in the `.rdb` file which are also in the `.msg` file.

CHAPTER 20: Using Drag and Drop

XFaceMaker makes it easy to set up a widget as a drag source or as a drop site using a single function call, and to exchange data between drag sources and drop sites using active values. The “native” Motif 1.2 functions are also available directly for more advanced uses of drag-and-drop.

The first part of this chapter describes how to use drag-and-drop in simple cases. This part can be read by interface designers who are unfamiliar with how drag-and-drop works in Motif 1.2 (although it assumes that the reader is familiar with XFM concepts and with X and Motif programming in general).

The second part of this chapter contains a detailed description of the XFaceMaker drag-and-drop functions and how they interact with the Motif toolkit. This part requires that the reader be familiar with the Motif drag-and-drop functions and objects.

Drag and Drop is unavailable in versions of Motif earlier than Motif 1.2.

Examples of simple and advanced drag-and-drops can be found in XFaceMaker’s examples directory, in the DragAndDrop sub-directory.

20.1 Simple Drag-and-Drop

A *drag and drop* is an interactive operation with visual feedback which is meant to transfer data between two objects of a user interface. For example, it is possible to drag a piece of text from a list to a text editor.

The *drag source* is the object which starts the drag operation and which provides the data to be transferred. The drag source generally initiates a drag operation when the user presses a mouse button (by convention, the second or middle button) in the drag source. An icon is then substituted for the mouse cursor to show that a drag is in progress. The drag icon, which can be dragged anywhere on the screen, can have different shapes that represent the type of data being dragged.

The user *drops* the data by releasing the mouse button on another interface object (which can belong to a different application). The drop is successful only if the object has been prepared to receive data: such an object is called a *drop site*.

Because drag-and-drop transactions can occur between different applications, the transfer of data uses the X selection mechanism, which provides a way to transmit data through the X server connections and to specify how the data must be converted. The type of data that a drag source can provide or that a drop site can receive is identified by a character string called a *target*: a drag source can *export* one or several targets whereas a drop site can *import* one or several targets. A drag-and-drop transaction is possible only between two objects that share at least one target. Targets can be arbitrary strings, but since all applications are potentially expected to send or receive data, target names should obey the conventions defined in the “Inter-Client Communication Conventions Manual” by the X Consortium. For example, if a widget can provide a Latin-1 character string to be dragged, it should export a target named **STRING**.

The visual feedback that is displayed during a drag-and-drop operation can be controlled through special Motif widgets and functions. The drag icon consists of a basic Pixmap, with optional *state* and *operation* Pixmaps which indicate, respectively, if the cursor is currently on a valid drop site and the type of drag operation in progress.

20.1.1 Simple XFM Drag and Drop Functions

XFaceMaker provides two functions which allow the interface designer to set up drag-and-drop operations very easily: the `FmStartSimpleDrag` function is called to start a drag operation, and the `FmRegisterSimpleDropSite` function is called to declare a widget as a drop site which can receive data from a drag source. These func-

tions can be called from FACE scripts in XFaceMaker. The types listed in the function descriptions are the actual FACE types, and may not correspond exactly to C types.

Widget	FmStartSimpleDrag(
	Widget <i>widget</i> ,
	XEvent <i>event</i> ,
	String <i>targets</i> ,
	Pixmap <i>pixmap</i>)

This function starts a drag operation.

widget specifies the drag source. It is generally the widget in which the user has pressed mouse button two.

event is a pointer to the X event generated by the user's action (this must be a button event).

targets is a character string which contains the name(s) of the target(s) that the widget can provide. If several targets are specified, they must be separated by commas.

pixmap specifies the Pixmap which will be displayed during the drag operation by the drag icon. If *pixmap* is null, a default pixmap provided by Motif is used.

This function is generally called from a widget's translation table, in a FACE action script for a button press. In this case, the *widget* argument is `self` and the *event* argument is `$`.

Example: The following translation table could be defined for any widget:

```
translations = #override\  
<Btn2Down> eval("FmStartSimpleDrag(self, $, 'STRING', Pixmap(0));
```

```
void      FmRegisterSimpleDropSite(  
          Widget    widget,  
          String    targets)
```

This function registers *widget* as a drop site, which accepts the targets specified in the *targets* argument. If several targets are specified, they must be separated by commas. This function is generally called from a widget's *xfmCreateCallback* at widget creation time.

Example: The following creation script could be defined for any widget:

```
xfmCreateCallback =  
    FmRegisterSimpleDropSite(self, 'STRING');
```

20.1.2 Transferring Data Using Active Values

FmStartSimpleDrag and *FmRegisterSimpleDropSite* call Motif functions and set up Motif callbacks and resources so that the dragged and dropped data will be transferred using XFaceMaker active values.

In the source widget (i.e. the widget which called *FmStartSimpleDrag*), there should be an active value defined for every target specified in the *targets* argument passed to *FmStartSimpleDrag*. When the icon is dropped on a drop site, the toolkit will fetch the value of the data from the source widget by calling the active value's *get* script. This script must store the data to be transferred into the *@* argument. In some cases, the source widget must also specify more information about the data such as its type, size and format, but in most simple cases you just need to return the value: XFaceMaker will supply everything else automatically (see below).

The drop site widget (i.e. the widget which called *FmRegisterSimpleDropSite* on which the icon was dropped) must also define an active value for every target it has passed in the *targets* arguments to *FmRegisterSimpleDropSite*. Once the data to transfer has been fetched from the source widget as explained above, the toolkit will transmit the value to the drop site widget by calling its active value's *set* script. This script can read the transmitted data from the *@* argument (plus other information on the value, if needed, as explained later), and use it as required.

The allocation of memory space to hold and transfer data is handled automatically, so the active value script of the source widget need not allocate any memory to hold the data. If the value is a pointer value, it should be freed by the drop site once it has been used. If it is an immediate value, no memory must be freed.

For targets **STRING**, **TEXT** and **COMPOUND_TEXT**, the length of the data is computed automatically by XFaceMaker and is the length of the string (which must be null-terminated). The length is stored in @2. For other target names, the length and format of the data must be specified. The format of the data must be stored in @3. It is 8, 16 or 32, and is used by the X library to swap bytes correctly for transfers between different machines. The length of the data must be stored in @2. It is the size of the data in units defined by the format (i.e. in bytes if format is 8, in short words if format is 16, etc...). See Section 20.3.6 for a complete description of the data structures passed to the drag-and-drop active value scripts.

Example:

The following interface description contains an example of drag and drop between a scale widget and a label widget. The scale widget acts as a drag source and provides a string value (the current scale value printed in decimal). The label widget is a drop site that accepts a string value and stores it as its label string.

```
1: widget ApplicationShell {
2:     ...
3:     children {
4:         widget XmForm {
5:             ...
6:             children {
7:                 widget XmScale {
8:                     ...
9:                     activevalue {
10:                        name = STRING
11:                        get = @ = itoa(self:value);
12:                    }
13:                    resources {
14:                        ...
15:                        translations = #override\
16:<Btn2Down>: eval("FmStartSimpleDrag(self, $, 'STRING',
17:                                     Pixmap(0));")
18:                    }
19:                } scale
20:                widget XmLabel {
21:                    flag {
22:                        mapped
23:                    }
24:                    activevalue {
25:                        name = STRING
26:                        set = self:labelString = String(@);
27:                        XtFree(@);
```

```
26:                                }
27:                                createproc {
28:                                    FmRegisterSimpleDropSite(self,
                                                                    "STRING");
29:                                }
30:                                resources {
31:                                    ...
32:                                }
33:                                } label
34:                                }
35:                                } form
36:                                }
37: } shell
```

The scale widget's active value **STRING** (lines 9-12) defines how the scale exports a string value. Its *get* script is called when a drop occurs to fetch the data to transmit: this script converts the integer value into a decimal string. The string value stored in @ will be automatically copied into a newly allocated string, which will be freed once the data is transmitted. The scale's translations (lines 15-16) start a drag when mouse button 2 is pressed in the scale's background. The target exported for this drag is **STRING**.

The label widget's active value, also named **STRING** (lines 23-26), defines how the label widget imports a string value. Its *set* script is called when a drop occurs on the label: this script stores the string value as the label's string. The string value is allocated by the toolkit, and must be freed when no longer needed.

The label's creation script (lines 27-29) registers the label as a drop site which accepts drops with a target type of **STRING**.

20.2 Defining Drag and Drop Operations

XFaceMaker Active Value window helps you to define simple drag-and-drop operations between widgets of your interface. The window allows you to set up a widget as a drag source or as a drop site, to define the list of targets exported or imported by the widget, and to choose a drag icon pixmap. The resources of the widget are updated automatically to call the simple drag-and-drop functions described above. Active values are also defined automatically to transfer the data between the widgets.

20.2.1 Drag Source

An active value whose usage is set as *drag source* specifies a target that the widget can drag. To edit an active value of this kind for a selected widget, select *drag source* in the **Usage** field.

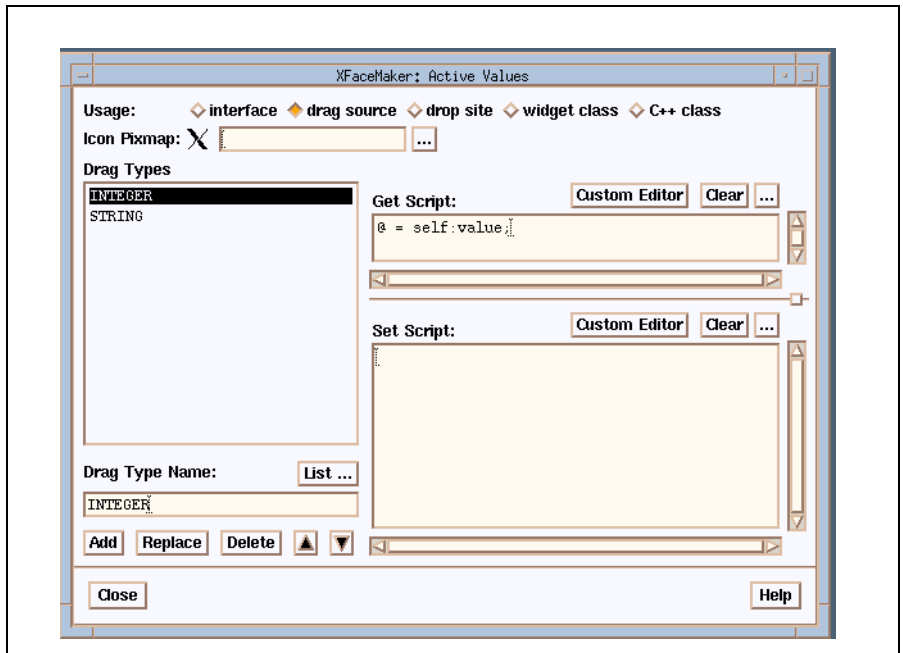


Figure 20.1: Drag source

1. Enter a drag type (the targets exported by the drag source) in the “**Drag Type Name**” field. Or click on the **List . . .** button to pop up the Drag Target Selector which contains a list of commonly accepted selection targets defined in the ICCCM. Select a drag target from the list, then click on the **OK** button. A widget can have more than one target.
2. Enter the file name of the pixmap that the pointer displays during a drag operation in the Icon Pixmap field. The ellipsis pops up the Pixmap editor for choosing or defining a pixmap.
3. Define the *get* script for the active value in the corresponding field. The *get* script should store the data to be transferred into the @ argument. Clicking on the ellipsis opens an edit box for multi-line scripts. Clicking on **Custom Editor** opens the editor that you specified in your FMEDITOR environment variable.
4. Click on **Add** to register the changes and apply them to the current widget. The name of the newly defined active value will be listed in the **Active Values** display area.

You can also remove a target name from the list by selecting it in the “Drag Types” list (or entering its name directly), then clicking on the **Delete** button.

20.2.2 Drop Site

An active value whose usage is set as *drop site* specifies a target that the widget can receive. To edit an active value of this kind for a selected widget, select “drop site” in the **Usage** field

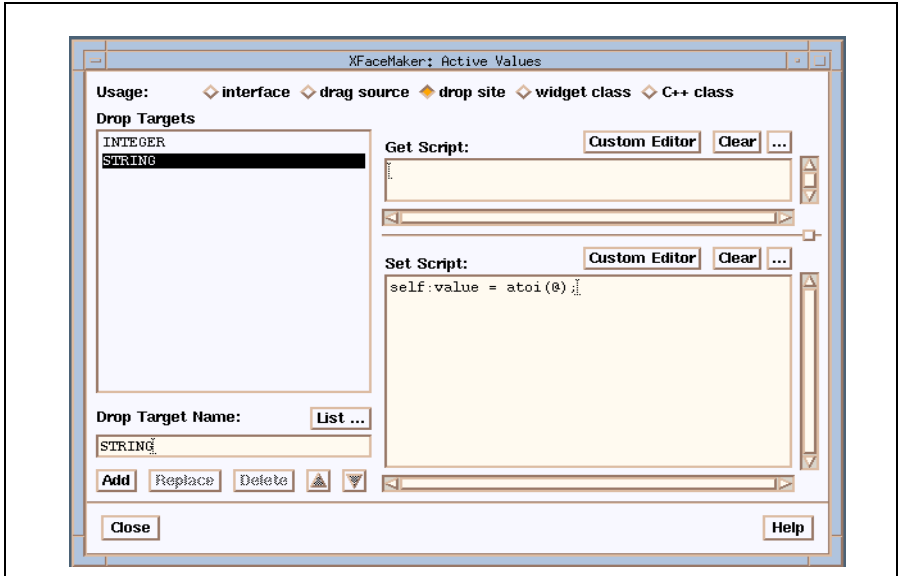


Figure 20.2: Drop site

1. **Drop Target Name:** Enter a drop target (the targets imported by the drop site) in the "Drop Target Name" field. Or click on the **List...** button to pop up the Drop Target Selector which contains a list of commonly accepted targets defined in the ICCCM. Select a drop target from the list, then click on the **OK** button. A widget can have more than one drop target.
2. Define the *set* scripts for the active value in the corresponding fields. The set script should read the data received from the *@* argument and use it as needed. Clicking on the ellipsis opens an edit box for multi-line scripts. Clicking on **Custom Editor** opens the editor that you specified in your FMEDITOR environment variable.
3. Click on **Add** to register the changes and apply them to the widget. The name of the newly defined active value will be listed in the **Active Values** display area.
4. You can remove a target by selecting it in the "Drop Targets" list (or entering its name directly), then clicking on the **Delete** button.

20.3 Drag and Drop Functions

This section describes the FACE functions which provide direct access to the Motif drag-and-drop functionalities. It also describes how the simple drag-and-drop functions are implemented on top of the Motif functions. The XFaceMaker drag-and-drop functions are similar to the Motif functions, but the arguments are specified in a different way:

- instead of passing argument array / argument count pairs, you must pass a single pointer to an argument list built using the FACE function described below;
- instead of callback or function addresses, you can supply XFaceMaker active values with the same names as the resources: XFaceMaker automatically sets the addresses of internal functions which call the appropriate active values.

20.3.1 Argument List Functions

Since several Motif drag-and-drop functions take Xt argument lists as parameters, XFaceMaker provides functions to manipulate argument lists in FACE. FACE argument lists are dynamically allocated arrays of Args and can have an arbitrary length.

FmArgList **FmNewArgList()**

FmNewArgList allocates an empty argument list.

```
void            FmAddArg(  
                 FmArgList list,  
                 String    name,  
                 None      value)
```

FmAddArg adds an argument to an argument list.

```
void            FmClearArgList(  
                 FmArgList list)
```

FmClearArgList removes all arguments from a list (but does not free the list itself).

```
void            FmFreeArgList(  
                 FmArgList list)
```

FmFreeArgList removes all arguments from a list and frees the list itself, which must not be used again.

20.3.2 Drag Context Functions

Widget	FmDragStart
	Widget <i>widget</i> ,
	XEvent <i>event</i> ,
	String <i>targets</i> ,
	FmArgList <i>arglist</i>)

FmDragStart calls *XmDragStart* with the arguments specified by *arglist*. The *XmNexportTargets* and *XmNnumExportTargets* arguments are specified as a comma-separated string in *targets*. Since function pointers and callback lists cannot be specified in FACE, *FmDragStart* passes default callbacks that call the active value set script of the source widget with the same name as the callback resource. For example, *FmDragStart* sets up a callback list for *XmNdragMotionCallback* with the address of an internal function which calls the set script of the source widget's *XmNdragMotionCallback* active value. The \$ argument passed to the active value script is the callback's *call_data* argument. The self argument passed to the active value script is the source widget with which *FmDragStart* was called, not the resulting drag context widget. The drag context widget can be fetched using the *XmDragContext* function, passing it the time stamp contained in the callback structure, like this:

```
Widget drag_context = XmGetDragContext(self, @2);
```

Similarly, the *XmNconvertProc* function is set to the address of an internal conversion procedure supplied by XFaceMaker which calls the get script of the source widget's active value with the selection target's name. The \$ argument passed to this get script is the address of an *FmConvertProcStruct*, described later. The XFaceMaker conversion selection procedure automatically handles the **TARGETS** target name and returns the list of all exported targets specified in *FmDragStart* (plus the targets **MULTIPLE** and **TIMESTAMP** which are handled by the Xt Toolkit selection mechanism). The conversion procedure also automatically handles the special **DELETE** target (if it is not already handled by an active value) by returning a type of NULL and a length of 0. In short, only the targets actually exported need to be handled by active values.

```
Widget    FmStartSimpleDrag(  
    Widget    widget,  
    XEvent    event,  
    String    targets,  
    Pixmap    pixmap)
```

FmStartSimpleDrag is similar to *FmDragStart*, but the argument list is replaced by a single *pixmap* argument. This pixmap is used to create a drag icon using *XmCreateDragIcon*. The icon is then passed as the *XmNsourcePixmapIcon* argument to *XmDragStart*.

```
Widget    FmCreateDragIcon(  
    Widget    widget,  
    String    name,  
    FmArgList arglist)
```

FmCreateDragIcon calls *XmCreateDragIcon* with the arguments specified by *arglist*.

20.3.3 Drop Site Functions

```
Widget    FmDropSiteRegister(  
    Widget    widget,  
    String    targets,  
    FmArgList arglist)
```

FmDropSiteRegister calls *XmDropSiteRegister* with the arguments specified by *arglist*. The *XmNimportTargets* and *XmNnumImportTargets* arguments are specified as a comma-separated string in *targets*. The *XmNdragProc* and *XmNdropProc* resources are set automatically to internal functions that call the set scripts of active values with the same names in the drop site widget. The \$ argument passed is the callback procedure's *call_data* argument.

Furthermore, if there is no active value defined for the *XmNdropProc* callback, *XFaceMaker* automatically calls the *XmDropTransferStart* function for all import targets of the drop site. If the drop site is valid (i.e. the *XmNdropSiteStatus* field of *XmDropProcCallback* structure is **XmVALID_DROP_SITE**), then *XFaceMaker* passes an internal drop transfer procedure as in *FmDropTransferStart* (see Section 20.3.4). The client data is the *XmNdropProc*'s *call_data* argument (of type *XmDropProcCallback*). If the drop site is not valid (*XmNdropSiteStatus* = **XmINVALID_DROP_SITE**), then *XmDropTransferStart* is called with an *XmN-transferStatus* of **XmTRANSFER_FAILURE**.

```
Widget    FmRegisterSimpleDropSite(
```

```
Widget    widget,
String    targets,
```

FmRegisterSimpleDropSite is similar to *FmDropSiteRegister* but does not have the *arglist* argument: all arguments other than *XmNimportTargets* and *XmNnumImportTargets* have their default values.

```
Widget    FmDropSiteUpdate(
Widget    widget,
String    name,
None      value)
```

FmDropSiteUpdate calls *XmDropSiteUpdate* with an *Arg* built using *name* and *value*.

```
Any    FmDropSiteRetrieve(
Widget    widget,
String    name)
```

FmDropSiteRetrieve calls *XmDropSiteRetrieve* with an *Arg* built using *name*, and returns the retrieved value.

```
XRectangle    FmCreateRectangle(
Int           x,
Int           y,
Int           width,
Int           height)
```

FmCreateRectangle allocates and fills an *XRectangle* structure with the specified geometry. This structure can be passed to *FmDropSiteRegister* as the value of the *XmNdropRectangles* resource. The pointer returned should be freed when no longer needed, using *XtFree*.

20.3.4 Drop Transfer Functions

```
Any    FmDropTransferStart(
Widget    widget,
Widget    drag_context,
String    targets,
None      client_data,
Boolean    incremental,
Int       status)
```

FmDropTransferStart calls *XmDropTransferStart* with the specified *drag_context*. The value of the *XmNdropTransfers* resource is built using the *targets* and *client_data* arguments. The incremental and status arguments are passed as the *XmNincremental* and *XmNtransferStatus* resources respectively.

The *XmNtransferProc* resource is set to the address of an internal procedure which calls an active value of the drop site widget. The transfer procedure first looks for an active value of name *transferProc*. If it exists, its set script is called. Otherwise, it calls the set script of an active value with the same name as the selection target for which a transfer is requested, unless the selection conversion has been refused or if it has failed due to a timeout.

The *widget* argument should be the drop site in which a drop has occurred, and will be used to find the active values called by the transfer procedure.

The *\$* argument passed to the active value scripts is an *FmTransferProcStruct*. The content of this structure is detailed in the next section. Note that if the *transferProc* active value is defined, it is always called for each transfer, even if the conversion was not successful. If the conversion was refused, for instance because there was no owner, the *type* field of the *FmTransferProcStruct* is the string **FM_CONVERT_REFUSED**. If the conversion failed because of a timeout, the *type* field is the string **XT_CONVERT_FAIL**. If the *transferProc* active value is not defined, the active value with the same name as the target is called only if the selection was successful.

```
Any FmDropTransferAdd(  
    Widget    widget,  
    Widget    drag_context,  
    String    targets,  
    None      client_data)
```

FmDropTransferAdd calls *XmDropTransferAdd* with a drop transfer entry list built using the *targets* and *client_data* arguments.

20.3.5 Display and Screen Functions

These functions return global Motif objects which can be used to set global data used in drag-and-drop operations.

```
Widget      FmGetXmDisplay(
Widget      widget)
```

FmGetXmDisplay calls *XmGetXmDisplay* with the widget's display, and returns the global XmDisplay object.

```
Widget      FmGetXmScreen(
Widget      widget)
```

FmGetXmScreen calls *XmGetXmScreen* with the widget's display, and returns the global XmScreen object.

20.3.6 Conversion and Transfer Structures

The drag source and drop site active value scripts for the exported and imported targets can (and sometimes must) access additional information about the data being transferred. This information is passed to the active values scripts as the address of a structure. The structure fields can be accessed in FACE using the `->` operator.

The `$` argument of a drag source's active value for an exported target points to an **FmConvertProcStruct**:

```
typedef struct _FmConvertProcStruct {
    XtPointer      value;
    String         type;
    int            length;
    int            format;
    Widget         drag_context;
    String         selection;
    String         target;
    XtPointer      client_data;
    unsigned long  max_length;
    XtRequestId    request_id;
} FmConvertProcStruct;
```

The fields of this structure are derived from the values passed to the *convertProc* of the drag context, which is provided by XFaceMaker. The Atoms are changed to strings, and some return values are initialized to default values. Refer to the X Toolkit Intrinsic manual, section 11.5.2.1, for a complete description of selection conversion procedure arguments.

value is where the active value's *get* script must store the value to transmit to the drop site. If the value is immediate (i.e. it is an integer or a one-word value), then it must be stored directly in the *value* field, and the *length* field must be left to its initial value of 0. For a string or a pointer then, the pointer must be stored in the *value* field. For targets **STRING**, **TEXT** and **COMPOUND_TEXT**, the *length* field is updated automatically to the length of the string (which must be null-terminated). For other targets for which a pointer is stored, the *length* field must be updated explicitly to the size of the data.

type is where the type of the data must be stored. This field is initialized to a default value which is taken from the table of "generally accepted" atoms listed in the ICCCM, section 2.6.2. For the commonly used targets **STRING** and **COMPOUND_TEXT**, the type is initialized to the target name. For the **TEXT** target, the type is initialized to **TEXT**, but it is changed to **STRING** if left alone. For targets not in this table, the *type* field is initialized to the target name.

length is where the length of the converted data must be stored if the data is a pointer (and not a string). The length is in units defined by *format*.

format is where the format of the converted data must be stored. This can be 8, 16, or 32. This field is initialized to 8 if type is **STRING**, **TEXT** and **COMPOUND_TEXT**, and to 32 otherwise.

drag_context is the **XmDragContext** widget returned by **XmDragStart**.

selection is the name of the selection used by the Motif drag and drop mechanism: **_MOTIF_DROP**.

target is the name of the selection target. It is the same as the active value name.

client_data is the value of the **XmNclientData** resource passed to **XmStartDrag**, if any.

max_length, *request_id* these are the arguments passed to the conversion procedure for incremental transfers (refer to the X Toolkit Intrinsics documentation).

The `$` argument of a drop site's active value for an imported target points to an **FmTransferProc**:

```
typedef struct _FmTransferProc {
    XtPointer    value;
    String       type;
    int          length;
    int          format;
    Widget       drop_transfer;
    String       selection;
    String       target;
    XtPointer    client_data;
} FmTransferProc;
```

The fields of this structure are derived from the values passed to the `transfer-Proc` of the drop transfer object, which is provided by `XFaceMaker`. The Atoms are changed to strings. Refer to the X Toolkit Intrinsic manual, section 11.5.2.2, for a complete description of selection callback procedure arguments.

value is where the active value's *set* script can read the value transmitted from the drag source. If the value is immediate (i.e. it is an integer or a one-word value), then it is stored directly in the `value` field, and the script must NOT free any memory. For a string or a pointer, then the pointer is stored in the `value` field, and the script can free the memory when no longer needed.

type contains the type of the data.

length contains the length of the converted data.

format contains the format of the converted data.

drop_transfer contains the `XmDropTransfer` widget returned by `XmDropTransferStart`.

selection contains the name of the selection used by the Motif drag and drop mechanism.

target is the name of the selection target.

client_data contains the value of the `XmNclientData` resource passed to `XmDropTransferStart`, if any. If `XmDropTransferStart` was not used directly but called automatically by the drop site's `XmNdropProc`, the `client_data` is the `XmNdropProc`'s call data (of type `XmDropProcCallback`).

CHAPTER 21: Internationalization

XFaceMaker provides the tools to build *internationalized* interfaces; i.e., interfaces that are customized according to the particular language and other local conventions of a country. These interfaces can be tested or ported on another machine which has GLS.

The handling of international messages in XFaceMaker is based on the X/Open Global Language Support (GLS) library. You should first read your local documents for a full understanding of the concepts and methods of GLS. Only a brief outline is given in this chapter.

This library is only available on some systems. To check whether you have this feature, open the File menu and look for the **Messages...** item. If it is not displayed, the library is not available on your system. If it is unavailable, you can still prepare your files for internationalization, but you will not be able to test, load, or port the internationalized files

21.1 General Concepts of GLS

The following rules should be followed if you want to internationalize an application using GLS.

1. All literal strings of the application that will be visible to the user must be listed. This includes buttons, messages, prompts, help texts, etc.
2. A *message number* must be assigned to each distinct string. For better modularity, messages can be grouped into *message sets*. Thus, every internationalized message is uniquely identified by a message number and a set number.
3. A *message catalog* must be created for each different language by compiling a message file. The message file contains a list of all the message numbers with the associated message text, in the national language. Using the system command **gencat**, this message file is compiled to produce the corresponding message catalog.
4. The application must call the GLS **catopen** function, which opens the catalog file suitable for the particular country. Typically, **catopen** is called at the beginning of the application.
5. Every use of a message in the application must be replaced by a call to the GLS **catgets** function. This takes four arguments:
 - m a catalog descriptor (returned by **catopen**)
 - m a message number
 - m a set number
 - m a default string.

The catalog is searched for the specified set/message number, and the corresponding string is returned. If the catalog is not found, the default string is returned. If the string is not found within the given catalogue a NULL or empty string may be returned depending on the system.

For example, the following C program:

```
main(argc, argv)
char **argv;
{
    Initialize();                /* My Initialize */
    printf("Hello, world!\n");
    Compute();                  /* Do work */
    printf("Good bye!\n");
}
```

must be transformed into:

```
#include <nl_types>
main(argc, argv)
char **argv;
{
    nl_catd ex_cat = catopen("example");

    Initialize();
    printf(catgets(ex_cat, 1, 1, "Hello, world!\n"));
    Compute();
    printf(catgets(ex_cat, 1, 2, "Good bye!\n"));
}
```

21.2 Internationalizing Strings

Every resource value in an XFaceMaker interface can be defined so that it will be internationalized. This includes resources such as `labelString`, `pattern`, `mnemonic`, and strings in FACE scripts that are assigned to a widget resource. The function `FmInternationalize` (or `Internationalize` in FACE scripts) can be used to explicitly process an internationalized string. For example, this function can be used for a function parameter in a FACE script:

```
function String MyFunction();  
s1 = MyFunction();  
s2 = Internationalize(s1);
```

An internationalized string in XFaceMaker has the following form:

```
%MSG%<set number>%[<message number>] <default string>
```

XFaceMaker provides a simplified way to internationalize strings in an interface. When you edit a resource value in the Resource Editor, you can internationalize it automatically by selecting the **GLS** button, and then applying the value changes to the widget.

When a string is defined using this format, XFaceMaker calls `catgets` with the specified set number, message number, and default string, and displays the result instead of the original string. For example, a `PushButton` with an internationalized label would have the following resources:

```
labelString = %MSG%1%1 Hello  
activateCallback = self:labelString = "%MSG%1%2 World";
```

This defines two messages in set number 1: message 1 corresponds to the default string "Hello", and message 2 corresponds to the default string "World".

The message number in an XFaceMaker internationalized string is optional: if it is omitted, then XFaceMaker does not call `catgets`, but always displays the default string. In this case, the message number will be allocated automatically the next time the interface is saved, if the *Save message catalog file* option is checked.

21.3 Saving Message Catalogs

When you save an interface, XFaceMaker will save all the internationalized messages in one or several message catalog source files. You can specify this and set other options in the Message Catalog Options editor. Open it by clicking on **Messages Catalog...** in the Options menu.

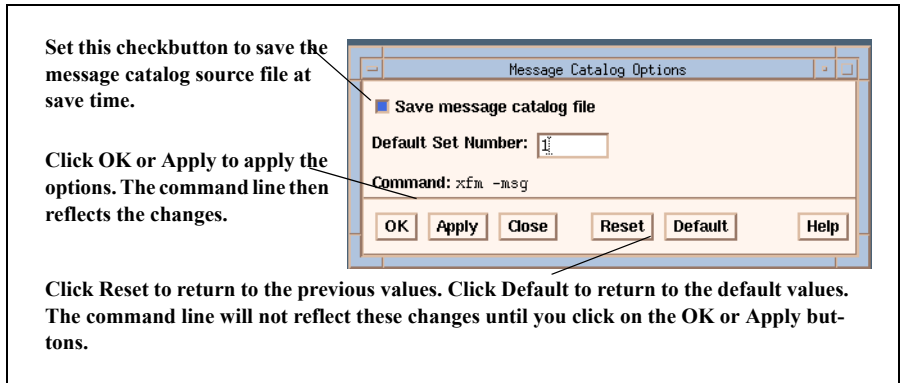


Figure 21.1: The Message Catalog Box

To set the message catalog options:

1. Select the **Save message catalog file** checkbutton. When this option is set and applied, XFaceMaker generates one or several catalog files (with the **.msg** suffix), containing all the internationalized messages of the current interface when you save an interface.
2. Enter a default set number. When you internationalize a string without specifying the catalog set number, XFaceMaker will enter this default set number for you, and the message will be stored in that catalog.
3. Click on **OK** to apply the options and close the window, or **Apply** to apply the options and keep the window open. The command line reflects the changes once you have clicked on **OK** or **Apply**.

Once you have set these options, the next time you save the interface, XFaceMaker will generate a message catalog source file (**.msg** file) along with the **.fm** file.

Since you may not have specified a message number for all the internationalized strings in the interface, XFaceMaker first checks which message numbers, for each set (or catalog), are used. XFaceMaker then assigns a number to any messages that are not yet numbered. This number will be different from any already used, and as small as possible—messages are numbered starting at one. If, by mistake, you have given the same catalog set number and message number to more than one message, only the first one is kept, and an error message is displayed in the command window giving the message numbers and associated text string. If this happens, re-number the messages and save the interface again.

If two messages have the same default string, no error message is displayed and the two strings will be output to the message catalog.

If you do not supply a message number but you do supply a default string, XFaceMaker will not search for an identical default string and set the same message number.

Set numbers greater than or equal to 200 are reserved. Any message whose set number is greater than 199 will not be saved. Therefore, when numbering messages, it is good practice to change the default set number for each main part of the interface. Doing so stores the messages in separate sets in your catalogs, and allows you to keep message numbers manageable. In addition, the translation of some messages may vary with the context in which they are used. Such messages should be numbered differently, or belong to a different set number in order to fetch a different message in the translated version. The message numbers should be as small as possible and as contiguous as possible. This is because the catalog will create entries for all possible numbers up to the largest defined number.

The message file is in a format that can be accepted as input by the `gencode` command. For instance, if the messages have been saved into `example.msg`, then the command:

```
gencat example.cat example.msg
```

will produce a catalog file `example.cat` which can then be loaded by the *Open Catalog* command.

21.4 Loading Message Catalogs

In order to obtain a translated string, one or more message catalogs must have been opened.

21.4.1 Within XFaceMaker

To use your message catalogs within XFaceMaker:

1. Select **Messages...** in the File menu. This opens a cascade menu.
2. Select the **Open Catalog...** command. This pops up the File selector for choosing the catalog file to open.
3. Specify a catalog file name and click on **OK**.

The next time that XFaceMaker encounters an internationalized string of the form described in the previous section, it will use this catalog to search for the specified message.

XFaceMaker uses `cotypes` to open the message catalogs. Thus, they must be in the suitable format, which is produced by the `gencat` command.

The name of the catalog follows the same rules as the argument to `catopen`:

- If the name contains a `/` (slash), then it specifies the pathname of a catalog file.
- Otherwise, the name is used in conjunction with the `NLSPATH`¹ environment variable to find the actual message catalog file (see your local GLS documentation).

Several message catalogs can be opened successively: in that case, XFaceMaker will search in all the open catalogs (last one first).

All currently open message catalogs can be closed by using the **Close catalogs...** command in the File menu.

21.4.2 In Your Application

To use your message catalogs in your application, whether in interpreted or in compiled mode, you must open the message catalogs that you need, i.e. do an equivalent of the **Open Catalog ...** command of XFaceMaker². The **FmCatOpen** function is pro-

1. The format for `NLSPATH` is: `<mydir>/%L/%N.cat` where `%L` is a substitution variable for the `LANG` environment variable and `%N` for a catalog file-name.

vided for that purpose. You should add calls to ***FmCatOpen*** in your `xxMain.c` file to open all the message catalogs that you will need. The catalogs must be in the format produced by `gencat`.

Just as with `XFaceMaker`, the application can open several catalogs successively. The open catalogs are examined, (last one first) until the message referenced is found, or the default message is used. Once a catalog is no longer needed, it can be closed with the ***FmCatClose*** function.

The catalog search rule is the same as for `XFaceMaker` - see above.

Opening and closing catalogs is the application developer's responsibility; it is not handled automatically by the application generation utility. When you modify your application files to handle the opening/closing of catalogs, make sure you include `nl_types.h` before the line `#include <Fm.h>` since, otherwise, `nl_catd` would be defined by `Fm.h`.

21.5 Internationalization Functions

The message catalog facility described above can also be used directly in the application, by means of the functions described below. To use these functions, you must include the `nl_types.h` file before `Fm.h`.

On machines without GLS the functions still exist but return a fixed default value as given below. If you do not include `nl_types.h` then `nl_catd` will be defined by `Fm.h`. Therefore, `nl_types.h` must be included before `Fm.h`.

-
2. An example of an internationalized interface and application is provided in the product distribution: see the `Adder` directory in the `XFaceMaker Examples` directory.

```
nl_catd  FmCatOpen (  
    char *    sl,  
    int       i)
```

FmCatOpen performs the equivalent of the *Open Catalog...* command. *sl* specifies the catalog name. *i* is currently unused and should be set to 0. The return value is the value returned by **catopen**. As the application opens catalogs with this function, the catalog descriptors are stored by XFaceMaker so that the interface message strings are fetched in the message catalogs that the application has opened. Non-GLS machines return -1.

```
void      FmCatClose (  
    nl_catd i)
```

FmCatClose closes the catalog identified by the catalog descriptor *i*. A value of -1 means “close all catalogs”.

```
nl_catd  FmGetCatD ()
```

FmGetCatD returns the descriptor of the last opened catalog. Non-GLS machines return -1.

```
nl_catd  FmCatFind(  
    char *    name)
```

FmCatFind returns the descriptor of the catalog named *name*, or -1 if it is not open. Non-GLS machines return -1.

```
char *    FmCatGets(  
    int     setn,  
    int     msgn,  
    char *   dflt)
```

FmCatGets is equivalent to **catgets**, except that it looks for the specified message in all message catalogs opened using **FmCatOpen**. The message catalogs are searched in the reverse order of the opening order. Non-GLS machines return *dflt*.

```
char * FmInternationalize (  
    char * str)
```

FmInternationalize scans the string *str* which must be of the form %MSG%n%m xxx, calls **FmCatGets**, and returns the resulting string, or NULL if *str* does not have the correct format.

CHAPTER 22: Command Line Options

This chapter lists the command line options available for use in XFaceMaker. Each option has an associated resource whose name is listed in bold.

22.1 The Command Line Options

The full command line is:

```
xfm [options] <file>.fm
```

or if you are loading XFM *and* the NSL Draw Library:

```
xfmwf [options] <file>.fm
```

22.1.1 General Options

- help : (**help**) Print the command line information given here.
- lazy : (**lazy**) Specify that XFaceMaker interface widgets are created when needed.
- nonlazy : (**lazy**) This is the opposite of -lazy, which is the default in this release. If you specify -nonlazy, all the windows of XFaceMaker will be created at startup time, otherwise, they will be created the first time they are used.
- postordermanage : (**postOrderManage**) In interpreted mode, specify that children are managed first (compatibility with v2.1).
- revision: (**revision**) Print revision number for this release of XFaceMaker.

22.1.2 Environment Options

- appclass <application shell name>: (**applicationClass-Name**) Specify XFaceMaker's default application shell class name. Defaults to XfmApplication if not specified.
- autoplace <true/false>: (**autoPlace**) Specify automatic/interactive placement of XFM windows.
- bufsize <number>: (**bufferSize**) Specify size of internal buffer used to display resources, scripts, etc.
- calleditor : (**callEditor**) Call the FmCallEditor function when entering FmLoop.
- classes <classes>: (**startupClasses**) Specify classes to load at startup as a comma-separated list of classes. The syntax is

Class1:Classfile1.fm,Class2:Classfile2.fm

If the name of the .fm file is the same as that of the class, you can specify Class only. For example, to load classes C11 and C12, you could specify C11:C11.fm,C12:C12.fm or just C11,C12.

- coloricons <true/false>: (**colorIcons**) On color screens, specify whether to use/not use color icons in the XFaceMaker windows.
- defaultnames <names>: (**defaultNames**) Specify default object names template. Specify a comma separated list of default object names; e.g. XmBulletinBoard=BB,XmPushButton=PB,...
- display <displayname:0.0>: (**display**) Standard X client option, specifies the display to be used. May be used instead of defining the DISPLAY environment variable.
- fastlist: (**fastResourceList**) This option specifies that the “fast” resource list be displayed in the Resources box instead of the list containing icons, which is the default. The “fast” list option refreshes resources faster on edit widget change.
- fmbootfile <alternate boot file>: (**fmbootfile**) The default boot file is Fm.fm. You should not need to change it. This specifies which .fm file is to be loaded and executed by XFaceMaker.
- fmc colors <name>: (**fmc colors**) Specify <name> as the .rdb file for XFaceMaker interface colors.
- fmfonts <name>: (**fmfonts**) Specify the name of the resource file to be used. By default, the file XfmDb in the directory \$NSLHOME/lib/xfm or the directory specified by -fmlibdir is used.
- fmlibdir <alternate_fm_directory>: (**fmlibdir**) Specify an alternate directory from which to fetch the XFaceMaker interface description file, Fm.fm. The default directory is /usr/lib/X11/xfm for installations in the default directory, \$NSLHOME/lib/xfm for installations in a user-specified directory (where **NSLHOME** is set to the user-specified directory), /usr/local/lib/xfm for installations in /usr/local. The environment variable **FMLIBDIR** may be used instead of the command line option.
- fmresources: (**fmfonts**) This option has been kept for backward compatibility only. Use -fmfonts<name> instead.
- ignorexerrors: (**ignorexerrors**) Ignore “Bad Drawable” X11 protocol errors generated by some versions of the OSF/Motif Toolkit.
- nocalleditor: (**calleditor**) Do not call the FmCallEditor function when

entering FmLoop.

- token <type> : (**token**) Specify which token **type** you wish to use. When different versions of XFaceMaker are available from the same token server a particular token can be requested. Current types are XFM, XFM Entry, and XFM IDT. When using NSLd license server, token names are: 007, 008, 006 respectively. When using the NSLLicense license server, token names are: xfm3, xfm3/el, xfm3/idt, xfm3/demo.

22.1.3 Editor Options

- build <char> : (**build**) Specify <char> as the accelerator to **switch to Build mode**.
- copy <char> : (**copy**) Specify <char> as the accelerator for the *copy widget* operation.
- cut <char> : (**cut**) Specify <char> as the accelerator for the *cut* operation.
- duplicate <char> : (**duplicate**) Specify <char> as the accelerator for the *duplicate widget* operation.
- hidetmplres : (**hidetmplres**) This option causes resources inherited from templates to be omitted from the Resources list.
- kill <char> : (**kill**) Specify <char> as the accelerator for the *delete widget* operation.
- nonuniquenames : (**nonuniquenames**) This disables the XFaceMaker feature that forces widget names to be unique by adding a suffix, allowing different children of the same parent widget to have the same name. This lets resource specification files use common widget names.
- paste <char> : (**paste**) Specify <char> as the accelerator for the *paste widget* operation.
- try <char> : (**try**) Specify <char> as the accelerator to **switch to Try mode**.
- undo <char> : (**undo**) Specify <char> as the accelerator for the *undo* operation.

22.1.4 Compilation Options

When using the compilation commands the interface of XFaceMaker is not loaded, and it exits immediately after the compilation without displaying windows on the screen. However, XFaceMaker still connects to the X Display, so the X server must be running and the `DISPLAY` environment variable must be set correctly. This allows certain X data structures and functions to be used to validate the input file.

- ansic** : (**ansic**) Generate C-code with prototyped functions conforming to the *ANSI C* standard. This is the default.
- c++** : (**cxx**) Generate a C++ class.
- c++lib** <name> : (**cxxClassLib**) Specify the base C++ class library.
- c++name** <name> : (**cxxClassNam**) Specify the C++ class name.
- c++file** <name> : (**cxxCFile**) Specify the C++ implementation file name.
- c++header** <name> : (**cxxHFile**) Specify the C++ header file name.
- cfile** <file.c> : (**cfile**) Explicitly specify the name for the C file generated by the `-compile` or `-compilegroup` options. The default is to use the `.fm` basename with the suffix `.c`.
- cflags** <character> : (**cflags**) Specify an option for C-code generated by the `-compile` or `-compilegroup` options with a character. Characters may be concatenated to give a string of options. See the Command Line Options table for character values.
- cinclude** <string> : (**cinclude**) Specify an include file name for the C file generated by the `-compile` or `-compilegroup` options. The string is inserted directly after the text `#include`. Quotes and other characters must be protected when defined from the shell. e.g. `xfm -compile my.fm -cinclude 'myinclude.h'`
- cname** <name> : (**cname**) Specify a different name for the main function `FmCreate<name>` in the C file generated by the `-compile` or `-compilegroup` options. The default is to use the name `FmCreate<filename>` where `filename` is the basename of the `.fm` file being compiled.
- compile** : (**compile**) Compile a `.fm` file to create a C file. If the option `-rdb` is used, then <file>.`rdb` is created together with the C-code version of <file>.`fm_rdb`. All references to unknown widgets are resolved dynamically.
- compilegroup** : (**compilegroup**) Compile a `.fm` group file to create a C-code file. All references to widgets are resolved dynamically.

- options** <number> : (**options**) Obsolete, use **cflags** instead.
- cprefix** <Prefix> : (**cprefix**) When generating a widget class this option defines the prefix string to be used for the widget class as in **Xm** or **XnsL**. The prefix defined is used in much the same manner as that of **Xm** for the **OSF/Motif** widgets. The class name becomes **PrefixName**, the include files are expected to be in the **Prefix** directory, (as in **#include<Prefix/Name.h>**), and resource values begin with the prefix.
- force** : (**force**) When this is set, **XFaceMaker** stays silent when, while executing the **xfm -compile** command, it finds files with mismatching timestamp. If this option is not set, it pops a confirmation dialog when it encounters such files.
- instclose** : (**instclose**) Used with the **-compile** or **-compilegroup** options, causes **XFaceMaker** to install, on shells, a default handler that will be called when the **Motif Window Manager mwm Close** menu item is selected, as in interpreted **Fm** mode. The default handler exits the application if the shell is an **ApplicationShell**, or unmaps the shell otherwise. To change the handler use **Fm-SetCloseHandler**.
- msg** : (**msg**) Assign all message numbers in <file>.fm, and generate a message catalog source file corresponding to all the internationalized strings in <file>.msg. <file>.fm is changed if some message numbers were not assigned previously, i.e., all messages are numbered.
- nowarnings** : (**nowarnings**) This option prevents the **FACE** compiler from issuing certain frequent warnings when compiling **FACE** scripts into standalone **C** code, i.e. when the **-standc** option is used. The deleted warnings are: **Resource <resource name> might need conversion** and **Unknown resource <resource name>**. Note that these warnings should be suppressed or ignored only if you are sure that the specified resource does not need special conversions. See also **-silentmode**.
- rdb** : (**rdb**) Create files <file>.fm_rdb and <file>.rdb. In conjunction with **-compile** or **-compilegroup** the **C**-code version of the file <file>.fm_rdb is also generated.
- rdbclass** <name> : (**rdbclass**) When used in conjunction with **-rdb**, this option specifies the first component of the resource specification to be *name*.
- rdbresources** <type, ...> : (**rdbresources**) Specify resource types or names to be saved in **RDB** resource files
- standc** : (**standc**) Used with the **-compile** option causes the generation of *standalone* **C**-code; i.e., **C**-code that need not be linked with the **Fm_c** library.
- startup** <file.fm> : (**startup**) This option allows an **FACE** script file to be loaded before compilation begins. Ordinarily, this is used to load the description of a widget sub-class which exists in the .fm file to be compiled but has not

been linked into XFaceMaker.

- silentmode : (**silentmode**) This option disables all warning messages from the FACE interpreter and C-code generator. This can be used to suppress warning messages issued by FACE when generating standalone C code, if you are sure that all resource conversions in your scripts are OK. See also nowarnings.
- uil : (**uil**) Compile a .fm file to create a UIL file. If a C-code file is generated at the same time, all widget references are resolved dynamically. The Cfile generated contains all the scripts and none of the interface description which is in the UIL file.
- uildialogs : (**uildialogs**) Causes XFaceMaker to generate predefined combinations of objects called “Dialogs” in the UIL file. Can only be used in conjunction with the -uil option..

	Option	Cmd Line Option	-cflags	Use with
1	Creation function	-compile		
2	Generate widget class		g	1
3	Class prefix	-cprefix <Prefix>		2
4	Creation name	-cname <Name>		1
5	Include file	-cinclude <Text>		1
6	K &R C		k	1
7	ANSI C	default	C	1
8	Use resource database		r	1
9	Use XtSetArg for all resources	default	c	1
10	Use XtVaCreateWidget		v	1
11	Convert resources after creation		s	9 Only
12	Convert Pixmaps after creation		p	9 Only
13	Create XmStrings		x	9 Only
14	5 creation arguments allowed	default		10 Only
15	20 creation arguments allowed		m	10 Only
16	100 creation arguments allowed		M	10 Only
17	Store Widgets in static variables		l	1
18	Store Widgets in allocated array	default	a	1
19	Store Widgets in extern variables		G	
20	Use Fm_c library	default		1
21	Standalone C	-standc	S	1
22	Install WM 'Close' Handler	-instclose	i	1
23	Dynamic Widget references	-compilegroup	d	Instead of 1

Table 22.1: C-Code Command Line Options

22.1.5 Dual Process Options

- autoclient : (**autoClient**) Launch client automatically in server mode.
- autosave <number> : (**autoSaveInterval**) Specify interval (in seconds) of automatic saving of design process state.
- client : (**client**) Run in client mode
- designprocess <name>: (**designProcess**) Specify the design process to be launched by XFaceMaker at startup.
- monoprocess : (**monoprocess**) Run XFaceMaker in mono-process mode.
- nointerface : (**nointerface**) Do not load XFaceMaker's interface.
- projmgrname <name> : (**projectManagerName**) Specify the name used by the project manager process to connect to XFaceMaker.
- restore : (**restore**) Restore editing state stored in the XFaceMaker server.
- server : (server) Run in server mode.
- serverhost <name> : (**serverHost**) Specify the name of the host where the editing server runs.
- servername <name>: (**serverName**) Specify the name with which the XFaceMaker server is registered.
- socketfile <name> : (**socketFile**) Specify an alternate UNIX domain socket path name.
- socketport <number> : (**socketPort**) Specify an alternate INTERNET domain socket port number.
- toolkit <name>: (**toolkitClasses**) Specify a comma separated list as the initial toolkit; e.g. -toolkit Primitive,Shell,Composite
- trusted : (**trusted**) Restore trusted editing state stored in XFaceMaker server.

22.1.6 Application Generation Options

- applifiles <name> : (**appliFiles**) List of application files to generate.
- cleanappli : (**cleanAppliFiles**) Generate “clean” application files (without code preservation information).
- cpattfiles <files>: (**cPattFiles**) This option is reserved for future use.
- genappli : (**generateAppliFiles**) Generate application files.
- onlygen : (**onlyGen**) Generate application file template only (use with -genappli).
- preserve : (**cleanAppliFiles**) Generate application files with code preservation information.
- target <Target>: (**targetEnvironment**) This option is used to specify the environment for which the interface and the application files are to be generated, e.g. Windows.

22.1.7 Debug Options

- debugface : (**debugFace**) Enable debugging of FACE scripts.
- nodebugface : (**debugFace**) disable debugging of FACE scripts.
- dumpcore : (**dumpcore**) Cause a core dump on receipt of signal 11. By default XFaceMaker will exit without producing the file **core** on receipt of signal 11.
- synchronous : (**synchronous**) Standard X client debug flag.
- verbose : (**verbose**) Print messages during execution.

22.1.8 XFaceMaker Extension Options

- extensions "<ext1> <ext2> ...": Specifies FACE extension description files to load at startup.
- extlib <library-name>: Used with the -gen option to generate the source files of an extension library. The pattern files for extension libraries are **PattLibMake** and **PattLibExt.c**.
- newxfm <xfm-name>: Used in conjunction with the -gen option to generate the source files of a new XFaceMaker. The pattern files used are **PattXfmMake** and **PattXfmMain.c**.
- extpath <dir1/%F:dir2/%F...>: Search path for extension files.

22.1.9 Standard X client

All standard X client command line options will be recognized although not all may be meaningful for XFaceMaker. For a full list consult the standard X documentation. The *X Window System User's Guide* from O'Reilly has a description in the chapter "Command Line Options".

Finally `<file>.fm`:

file The name of the interface file that you wish to load with XFaceMaker if you want to load it directly and not with the *Open* command in the File menu.

All of the above command line options can be set in your `$HOME/.Xdefaults` file, although only a few are really meaningful. This is done by using the option name without `-`, followed by a colon, a tab character, and the value required.

For example, to disable lazy mode every time you use XFaceMaker add the following to your `$HOME/.Xdefaults` file at the beginning of a new line.

```
Xfm.nonlazy:           true
```

Other examples are:

```
Xfm.fmresources: /home/my/xfm/MyXfmDb
Xfm.kill:         \^X
```

When using the `CallEditor` function any command line options for the application will be used by XFaceMaker when the function is called.

CHAPTER 23: Environment Variables

The following environment variables may be used with XFaceMaker or its applications. You can set them from within XFaceMaker, through the

Windows menu in XFaceMaker's main window.

DISPLAY Standard X display variable. The option `-display` may also be used.

FMEDITOR Defines the text editor that XFaceMaker will call when the Custom Editor button is selected in one of the XFaceMaker editing windows. Set this to `wx242` to use NSL's text editor. You may set it to `vi` or `emacs` if you prefer. Make sure they are included in your `PATH`.

FMFILEPATH An environmental variable used by XFaceMaker to define the directories searched to find all Fm files. `%F` is used as the substitution character for the `.fm` file name. For example, if your `.fm` files are in the directory `/dir/Appli/Fm`, set this variable as follows:

`/dir/Appli/Fm/%F`

FMLIBDIR An environment variable which may be used to specify the directory where the XFaceMaker interface description file `Fm.fm` should be fetched. See also the `-fmlibdir` command option.

FMPIXMAPSPATH An environment variable used by XFaceMaker to define the directories searched to find color pixmap files. `%P` is used as the substitution character for the `XWD` file name.

LANG Defines the language element of `NLSPATH`.

LD_LIBRARY_PATH (sun,dec,sgi,sco)

LIBPATH (ibm)

LPATH (hp) Standard environment variable to specify search paths for libraries when loading a dynamically linked executable. The name differs on different operating systems. When setting this variable, include the directory `$NSLHOME/lib`.

NLSPATH When using internationalized interfaces this defines the directories searched to find the message catalog files.

NSL_LICENSE_HOST Specifies the name of the host machine for the server if you are using NSLlicense.

NSL_LICENSE_PORT specifies the port number the product should use to communicate with the license server. The default is 5002.

NSLHOME Used to access the XFaceMaker auxiliary files such as include files, application patterns, libraries, XFaceMaker interface files, etc. The default value of FMLIBDIR is initialized to \$NSLHOME/lib/xfm.

PATH Standard environment variable to specify directories where executables should be fetched. When setting this variable, include the directory \$NSLHOME/bin .

XAPPLRESDIR Standard X variable used to define the directories searched to find application resource files.

XBMLANGPATH Standard X variable used to define the directories searched to find bitmap files. %B is used as the substitution character for the bitmap file name.

XFM_LICENSE_NAME Specifies the name of the license if you are using NSLlicense as the license server daemon. This should be set to xfm3 or xfm3/el, or xfm3/idt depending on the type of product you have purchased.

Obsolete - XFaceMaker version 3.2.2 and above is incompatible with NSLd:

NSLSERVER If you are using the NSLd token server, this environment variable is used to specify the name of the machine running the NSL token server, from which tokens will be fetched. Useful especially when several NSL token servers are running on a network.

CHAPTER 24: The Fm File Structure

This chapter describes the Fm file structure, to enable developers to read or edit a file. An Fm file consists of a number of different blocks:

widget Each widget in an Fm file starts with:

```
widget <Widget Class>
```

followed by an opening brace (`{`) denoting the start of the **widget** block. The block continues until the matching close brace (`}`).

flag The **flag** block follows the **widget** block. It can contain the following fields:

- **mapped** Present if the widget was mapped when the Fm file was saved. If a parent of the widget was not mapped, the widget may not be visible.
- **popup** Present if the **popup** togglebutton in the Resources window was set. The widget is created as a **PopupShell**.
- **autosize** Present if the **autosize** toggle button in the Resources window was set. This denotes that the **width** and **height** resources will be ignored when creating the widget.
- **autopos** Present if the **autopos** toggle button in the Resources window was set. This denotes that the **x** and **y** resources will be ignored when creating the widget.
- **gadget** Present if the gadget version of the widget is to be created.
- **menu_bar** Present for **XmRowCoLumn** widgets if they are acting as a menu bar.
- **menu_pulldown** Present for **XmRowCoLumn** widgets if they are acting as a pull-down menu.

active value The **flag** block may be followed by one or more **active value** blocks which define any active values associated with the widget, along with their **Get** and **Set** scripts.

createproc The **flag** block may be followed by a **createproc** block which defines the **xfmCreateCallback** script for this widget.

template If an interface contains one or more templates then a **template** block defines the template files to be included.

resources The **resources** block contains all resources defined for this widget, including callbacks and translations. The resources defined may be modified by the **flag** block entries.

children If the widget is a parent, then a **children** identifier will mark the

start of the child widget definitions block. Further **widget** definition blocks will occur within this block.

Finally, please note the following points:

- The name of the widget comes after the closing brace of the widget block.
- A line starting with the characters **#** or **!** is a comment and therefore ignored.
- The indentation of a widget block in the Fm file shows the relationships of the widget, i.e., a child is indented with respect to its parent. When reading a file this indentation is *not* taken into account to determine the relationship between widgets, this depends entirely on the **children** block marker. Indentation uses the space character; **tab** characters exist only as formatting characters in FACE scripts where **tab** represents the continuation of the previous line at the start of a line. Beware of editors which insert **tab** characters automatically after newlines.
- The commands **xfmprint**, **xfmtableprt** and **fmafvprt** can give you a compact view of the interface if they are available on your system. They are described in the appendix.

CHAPTER 25: Editing Graphics in XFaceMaker

If you are running `xfmwf` or an extended XFaceMaker executable that includes the Draw widget and gadgets, and there is an XnsIDraw widget, in your interface, you may want to edit your drawing from within XFaceMaker.

NSL's graphic editor XDrawMaker is integrated with XFaceMaker to add dynamic graphics in Motif interfaces. The XFaceMaker command ***Edit Graphics with XDrawMaker*** (in the ***Edit*** menu) can be called when any composite widget is selected, and in particular an XnsIDraw widget. It launches XDrawMaker and sets up a connection between XFaceMaker and XDrawMaker.

Once the connection is established, the existing graphics in the selected XFaceMaker widget are transferred to XDrawMaker and can be edited using XDrawMaker. If the graphics have been imported into XFaceMaker in XFaceMaker format, or if they have been defined directly with XFaceMaker, the gadgets are accessible for editing in XFaceMaker or in XDrawMaker. If the graphics have been imported into XFaceMaker as a draw file, the XnsIDraw widget is accessible for editing directly in XFaceMaker but the draw objects can be edited only within XDrawMaker.

Once you have launched XDrawMaker using the *Edit Graphics with XDrawMaker* command, XDrawMaker is updated automatically every time you select a composite widget or an XnsIDraw gadget in XFaceMaker. If a composite widget contains XnsIDraw gadgets, the gadgets are loaded in XDrawMaker. If an XnsIDraw widget is selected, all the gadgets contained in the parent widget are loaded. Otherwise, the XDrawMaker window is cleared.

You can exit the connected XDrawMaker program at any time, and launch it again using the command *Edit Graphics with XDrawMaker*.

25.1 Graphics in XFaceMaker Format

If the graphics are in XFaceMaker format, the draw objects can be selected individually in XFaceMaker, their resources can be edited in the XFaceMaker resources box. The objects can also be edited in XDrawMaker, from XFaceMaker, if XDrawMaker has been launched with the *Edit Graphics with XDrawMaker* command.

Object selection is not carried over from one tool to the other. You must select the object you want to work on explicitly in XDrawMaker and, when you save your XDrawMaker work, if you want to edit an object's resources in XFaceMaker, you have to explicitly select the object in XFaceMaker.

When you **Save** your work in XDrawMaker, it transfers back the graphics to XFaceMaker and the selected object is rebuilt with the new graphic gadgets in it.

Important - resources such as `xfmCreateCallback` and `drawActivateCallback`, active values, attachments, ... can be set in XFaceMaker but are not known in XDrawMaker. This means that, when you edit with XDrawMaker a gadget for which any of these resources have been set, you will lose the scripts. An alert box will warn you of this. If you confirm your decision to edit your drawing with XDrawMaker, all settings for these resources will be lost.

XnsIDraw graphic gadgets can also be edited directly in XFaceMaker like any other widgets, and you can save them as part of a `.fm` file to build your application. XFaceMaker handles Draw gadgets much like XDrawMaker: when you take a gadget from the toolkit to place it in the interface, you specify its points as you would in XDrawMaker. In the XFaceMaker resources box, the auto position and auto size toggles are checked: even within XFaceMaker, the gadgets are defined through the `drawPoints` resource, not the width and height resources as with Motif widgets and gadgets.

Some resources are easier to work with in XnsIDrawMaker than directly in XFaceMaker. Installing slots, or specifying activation scripts, for example, is easier with XDrawMaker than with XFaceMaker. The same is true of group or path definitions. XDrawMaker has a group/path editing dialog box that you can use to re-order members, to control how attributes defined for a group are passed to its members, etc. Moreover, XDrawMaker represents groups and paths in tree form, where the members appear as *leaves* connected to a *root*. When a Draw group or path is exported to XFaceMaker, this hierarchical *representation* is lost, the group (path) object appears in the XFaceMaker widget tree or list at the same level as its members. This does

not mean that the group (path) object has lost its information. The `drawChildren`, `drawNumChildren`, etc. resources have the correct value, the group (path) will continue to *behave* as a group (path).

Other resources are easier to work with in XFaceMaker than in XDrawMaker. All the resources that do not have any specific editing box in XDrawMaker, the resources that you can access only through the **Expert Box** are definitely easier to reach and modify in XFaceMaker than in XDrawMaker.

25.2 Graphics in Draw File Format

When the command *Edit Graphics with XDrawMaker* is called on an XnsIDraw widget which has its `drawFile` resource set, only the name of the `.draw` file is transferred between XFaceMaker and XDrawMaker. This means that, when the *Save* command is called in XDrawMaker, the XnsIDraw widget is simply rebuilt in XFaceMaker to read the modified `.draw` file. The XnsIDraw gadgets do not appear in the XFaceMaker widget tree and cannot be edited using XFaceMaker: they are recorded separately in the `.draw` file.

To have access to the XnsIDraw gadgets of a drawing inside XFaceMaker, you have to first export the drawing from XDrawMaker in XFaceMaker format, then load the drawing sub-tree in XFaceMaker. You can subsequently edit your drawing using XDrawMaker, through the *Edit Graphics with XDrawMaker* command.

25.3 Recommended Procedure

Use XDrawMaker to do your basic drawing and animate the objects. This is where you will define groups, paths, animation scripts.

Save your drawing as a `.draw` file, and export it as an XFaceMaker file.

Load in an extended XFaceMaker (xfmwf or any XFaceMaker executable in which the Draw extension is included) the fragments of your interface to which you will want to add the drawing.

Read the `.fm` file of your drawing into XFaceMaker. This will always include an XnsIDraw widget.

Select the drawing and place it in desired position in your widget tree using **Cut** and **Paste** widget selection.

If you want to include the XnsIDraw widget in your interface, you just need to select the draw widget that you have just read, **cut** it, **paste** it in the desired widget, and delete the shell widget that XFaceMaker created when reading the `.fm`

file of the drawing.

If you want to place your draw gadgets directly in an existing manager, select all the gadgets, *cut* the selection, *paste* it where desired, then delete the shell parent of the draw widget, thus deleting the draw widget along with it.

Specify any additional resource settings on the gadgets in the drawing.

Save your .fm file as an interface file, or as a class depending on the type of object you are defining. Test it within XFaceMaker in Try mode if desired.

Generate, test the application as for any other type of interface. Note that you can generate a Makefile with XFaceMaker

25.4 Graphics and C-code Generation

If the graphics are loaded in XFaceMaker as a draw file, when you generate the C code of your .fm file, the C code for the graphics will not be generated, the graphics will run in interpreted mode.

If the graphics are loaded into XFaceMaker in XFaceMaker format, when you generate the C code for your .fm file, the C code for the graphics will be generated, the graphics will run in compiled mode. Loading your graphics into XFaceMaker in XFaceMaker format is the only way to generate the C code for a drawing made with XDrawMaker.

Index

- newxfm 418

Symbols

\$ 158, 160, 177
 .bm 110
 .bm file. 110
 .draw files. 425
 .face. 313
 .fm file
 environment variables 421
 .fm file See also fm file
 .fm_rdb 55, 380, 415
 .fmp file 255
 .msg files 405
 .ps file 77
 .rdb 380
 .uid 356, 362
 .Xdefaults 104, 419
 .xfm.color file 101
 .xfm.startup file. 53
 .xwd file 110
 @ 158, 160
 _FmResourceEditManage 324
 _FmResourceEditUnmanage 324
 _FmResourceReturnValue 324

A

accelerator 111, 144, 413
 activateCallback 128
 active values
 _FmResourceEditManage. 324
 _FmResourceEditUnmanage 324
 _FmResourceReturnValue 324
 access 158
 C++ class 171, 294
 compiled mode 225
 defining methods 181
 defining new 163
 defining resources. 181
 drag source 166, 389

drop site 167, 390
 examples. 178
 functions. 172
 get scripts 135, 159, 295
 immediate access 158
 interface 164
 interpreted mode 219
 names 158
 new style 163
 old style 163
 per call 181
 per widget. 181
 properties 158
 scripts 135, 160
 set scripts 135, 159, 295
 storage 158
 types 158
 used for 157
 widget class. 169
 aligning widgets 74
 alignment box 74
 -ansic 285, 414
 ansic 414
 -appclass. 411
 application 206, 217
 adding functions 232
 and internationalization 407
 and projects 259
 building 209
 compiled mode. 224
 files 206
 functions file 206
 generating 206
 include file 206
 interpreted mode. 218
 loading groups 228
 main function. 206
 main program. 218
 Makefile 206
 pattern files 207
 saving files 52
 standalone C-code. 230
 with active values 219
 with UIL. 231
 applicationClassName 411
 ApplicationShell 58

- name. 62, 86
- appliFiles. 418
- applifiles. 418
- apply options 92
- applying resources 83, 92
 - by rebuilding the object. 83
 - by setting values 83
 - the window icon 82
- armCallback. 128, 129, 144
- attachments 116
 - and Auto position. 86
 - displaying 118
 - editing. 116
- auto position 86
- auto size 87
- autoclient 417
- autoPlace 411
- autoplace 411
- autosave 417
- Auto-Save State 55

B

- bitmaps 110
- Boolean box. 98
- breakpoints 137
- bufferSize 411
- bufsize 411
- build. 413
- build 413
- Build mode 45, 58
- BUILD_C_INIT_FUNCALL. 318
- BUILD_C_LIBRARIES_ 318
- BUILD_COMMAND 317
- BUILD_E_LIBRARIES_ 318
- BUILD_INCLUDE_PATH_ 318
- BUILD_INCLUDES_ 318
- BUILD_INIT_FUNCALL_ 318
- BUILD_INIT_FUNCTION_ 319
- BUILD_LIBRARIES_ 319
- BUILD_LIBRARY_PATH_ 319
- BUILD_OBJECTS_ 319

- BUILD_SOURCES_ 319
- building
 - a project 255
 - an application 209
 - an interface 58

C

C code

- and UIL 354
- ANSI C 241
- command line options 245
- compile commands 414
- files. 236
- for new classes 249
- K&R 241
- linking files. 339
- options. 239
- save option 55
- standalone 250, 415
- templates in 204
- vs. Fm 253

C++

- and XFaceMaker 285
- called from FACE scripts 285
- integrating XFM interfaces 285
- c++. 414
- C++ classes 293
 - active values 294
 - data members 295
 - defining 294
 - get and set scripts 295
 - member functions. 296
 - NSL style 293, 298
 - Rogue Wave style. 293, 298
 - saving 298
- c++file 414
- c++header 414
- c++lib. 414
- c++name. 414
- C_LIBRARY_ 320
- callbacks 128
 - in Eval scripts 114
 - vs. translations. 111
- callEditor 411, 412
- calleditor. 411

-
- CallEditor() 419
 - catgets() 402
 - catopen() 402
 - cfile 414
 - cfile 414
 - cflags 414
 - cflags 414, 416
 - cflags C 285
 - cinclde 414
 - cinclde 414
 - class
 - XfmApplication 121
 - class record 277
 - classes 411
 - classes
 - adding to XFM 331
 - C code for 268
 - C++ classes 294
 - defining 262, 264
 - deleting 267
 - limited number 267, 331
 - loading 267
 - methods 266
 - new resources for 264
 - saving 262
 - cleanappli 418
 - cleanAppliFiles 418
 - client 344, 417
 - client 417
 - client option 140
 - cname 414
 - cname 414
 - CODE_ 320
 - color resources 101
 - colorIcons 412
 - coloricons 412
 - colormap 104
 - command line options 411
 - overridden by project files 54
 - command window 41
 - commands
 - from keyboard 19, 43
 - from menu 43
 - compile 237, 414
 - compile 414
 - compilegroup 414
 - compilegroup 238, 353, 414
 - compiling
 - C code 235
 - coptions 415
 - coptions 415
 - copy 413
 - copy 413
 - copying widgets 68
 - cPattFiles 418
 - cpattfiles 418
 - cprefix 415
 - cprefix 415
 - cprototype 247
 - createCallback 131
 - creation function 224
 - creation script 131, 240
 - cross-handle 59
 - current file 42
 - current widget 42
 - Custom Editor 127
 - cut 413
 - cut 413
 - cutting widgets 68
 - cxx 414
 - cxxCFile 414
 - cxxClassLib 414
 - cxxClassNam 414
 - cxxHFile 414
 - D**
 - data members 295
 - debugFace 418
 - debugface 418
 - debugging scripts 136
 - default
 - icon in resource list 90
 - resources 121

- default set number 405
- defaultNames 412
- defaultnames 412
- deleting widgets 69
- demo mode 40
- demos 17
- Design process 343
 - client 140
 - new 349
 - restarting 51, 140, 142
 - restore 141
 - state of 348
 - terminating. 139, 346
 - trusted state 55
- designProcess 417
- designprocess 417
- destroyCallback 128
- DialogShell 73
- DISPLAY 353, 412, 414, 421
- display 412
- display 412
- display and edit icons 44
- drag and drop 384
 - dragging resource values 89
 - editor 389
 - functions 384, 392
 - transferring data 386
- drag source 384
 - defined by active values 166, 389
- draw files
 - editing in XFaceMaker 427
- drop site 384
 - defined by active values 167, 390
- drop target 391
- dual process 139
 - error recovery 139
- dumpcore 418
- dumpcore 418
- duplicate 413
- duplicate 413
- duplicating widgets 69

E

- E_LIBRARY_ 320
- editing .draw files 425
- editing text
 - choosing an editor 127
- enumeration 98, 323
- environment variables 421
- error messages 49
- error protection 139, 346
 - dual process mode 139
 - Restart on FACE errors 139, 346
- Eval scripts 114, 133
- event 144
- exiting XFM 56
- expand widget tree 65
- extending XFaceMaker 311
- extensions 418
- extensions 315
 - constants 321
 - default 312
 - extension libraries 314
 - file 313
 - functions 321
 - linking the application 313
 - structures 321
 - user-defined 312
- extlib 418
- extpath 418
- extpath 418

F

- FACE
 - creation scripts 131
 - debugging scripts 136
 - defining functions 132
 - scripts 124
- fast resource list
 - displaying 91
 - no drag and drop 89
- fastlist 412
- fastResourceList 412
- fatal error. See error protection

-
- files
 - .bm 110
 - .c 217
 - .face 313
 - .fm 217
 - .fm_rdb 380
 - .msg 405, 415
 - .ps file 77
 - .rdb 380
 - .xwd 110
 - C code 236
 - color description. 103
 - Fm 423
 - Fm.fm 412
 - generating. 206
 - inserting. 51
 - opening 50
 - renaming 52
 - saving 52
 - structure 423
 - UIL. 352
 - xfm.color 101
 - float 98
 - fm file
 - for C++ classes 298
 - for widget classes 262
 - icon in resource list. 90
 - loading. 40
 - local and global functions. 133
 - save options 55
 - saving 52
 - translating UIL to fm files. 351
 - using active values 163
 - Fm file structure 423
 - Fm.fm 412
 - Fm_e 232
 - FmAddArg(). 392
 - FmAddEnumeratedType() 332, 333
 - FmAddRepresentation(). 333
 - FmAddResourceType() 331, 332
 - FmAddWidgetClass() 270, 274, 331, 333, 340
 - FmAttachAv(). 161, 176
 - FmAttachFunction() 218
 - FmAttachValue(). 160, 172
 - fmbootfile 412
 - fmbootfile 412
 - FmCallEditor() 338, 340
 - FmCallValue(). 172
 - FmCatClose(). 409
 - FmCatFind(). 409
 - FmCatGetS(). 409
 - FmCatOpen(). 409
 - FmChangeValueAddress(). 173
 - FmClearArgList(). 392
 - fmcolors 412
 - fmcolors 412
 - FmConvertProcStruct. 397
 - FmCreate(). 236
 - FmCreateDragIcon(). 394
 - FmCreateRectangle(). 395
 - FmDragStart(). 393
 - FmDropSiteRegister(). 394
 - FmDropSiteRetrieve(). 395
 - FmDropSiteUpdate(). 395
 - FmDropTransferAdd(). 396
 - FmDropTransferStart(). 396
 - FMEDITOR 421
 - choosing a text editor 127
 - FmFetchValue(). 173
 - FmFetchValueAddress(). 174
 - fmfile 419
 - FMFILEPATH. 421
 - fmfonts 412
 - fmfonts 412
 - FmFreeArgList(). 392
 - FmGetActiveValue(). 176
 - FmGetActiveValueAddr(). 176
 - FmGetCatD(). 409
 - FmGetFunctionsVector(). 341
 - FmGetValue(). 174
 - FmGetXmDisplay(). 397
 - FmGetXmScreen(). 397
 - FmInitialize(). 338, 370
 - FmInternationalize(). 410

FMLIBDIR 421
FmLibDir. 104
-fmllibdir 412
fmllibdir 412
FmLoad(). 373
FmLoadCreateGroup(). 373
FmNewArgList(). 392
FMPIXMAPSPATH 110, 421
FmPReadSubFile 257
FmReadValue(). 174
FmRegisterSimpleDropSite(). . . 386, 394
-fmresources 412
FmSetActiveValue(). 177
FmSetCloseHandler(). 415
FmSetSuperclassInfos(). 270, 340
FmSetValue(). 175
FmStartSimpleDrag(). 385, 394
FmTransferProc 399
FmWriteValue(). 175
fonts 105
Fonts box. 105
-force. 415
force 415
Form widgets
 attachments 116
functions
 active values. 172
 adding to XFaceMaker 339
 global 133
 local 133
 new-style 172
 old-style 176
functions file 206

G

gadgets
 changing widgets to 87
 editing. 70
 handles 59
 raising and lowering 72
 selecting 59
GenAppli. 216

-genappli 418
genappli 206
genecat 402
generateAppliFiles 418
generating files
 application files 206
 C code and UIL 354
 multiple files 381
 RDB files 378
 UIL files 351
get scripts 135, 159
 for C++ class active values . . . 171, 295
 for drag source active values . . . 167, 390
 for interface active values 159, 165
 for widget class active values . . . 170
GetActiveValue(). 176
GetActiveValueAddr(). 177
global (keyword). 125
global functions 133
global objects 48, 100
global storage 158
GLS. 98, 402
grids
 to align widgets 76
groups 190
 defining 192
 deleting 193
 in the application 222
 loading 192
 predefined 194
 Save Prefs 193
 saving 192
 widgetstore. 192

H

handles. 59
-help 411
help 411
-hidetmplres 413
hidetmplres. 413
hiding widgets 73
HSV color values 102

I

icons
 display and editing 44
 message 49
 -ignorexerrors 412
 ignorexerrors 412
 immediate access 158
 include files 207
 needed in XFM 19
 -instclose 415
 instclose 415
 integer box 98
 interface
 C code 235
 clearing 51, 56
 compiled mode 217
 creating multiple files 255
 description file 378
 displaying 73
 generating multiple files 381
 hiding 73
 internationalized 401
 interpreted mode 217, 218
 loading 50
 main program 217
 new 56
 save options 55
 saving as project 257
 sub-divided 220, 226
 templates in 204
 internationalization 55, 401
 functions 408
 Internationalize() 404
 Intrinsics 261

K

keyboard commands 19, 43
 -kill 413
 kill 413

L

LANG 421
 -lazy 411

lazy 411
 LD_LIBRARY_PATH 421
 -lFm 218
 -lFm_c 224
 -lFm_e 232
 libFm_c 233, 250
 libFm_e 233
 libMrm 361
 LIBPATH 421
 libraries
 needed in XFM 18
 Load Extensions 312
 LoadWidgetClass() 276
 local (keyword) 125
 local functions 133
 lower 72
 LPATH 421

M

main file 206
 main function 236
 main program 218
 compiled mode 225
 interpreted mode 218
 with active values 219
 with function calls 219
 Makefile 207
 member functions 296
 menu commands 43
 Menu Editor box 145
 menus
 accelerators 144
 creating with Menu Editor 148
 creating without Menu Editor 152
 mnemonic 144
 types 143
 message catalogs 402
 files 405
 loading 407
 NLSPATH 421
 saving 405
 message icons 49

message number 402
message section 49
method 261
mnemonic 144
-monoprocess 344, 417
monoprocess 417
Motif 152
mouse button 20
moving widgets 67
Mrm 231, 354
MrmFetchLiteral() 374
-msg 415
msg 415
mwm 104

N

named scripts 132
naming files 52
naming widgets 62
 in the Resource window 86
 legal names 86
newxfm 418
NLSPATH 421
-nocalleditor 412
-nodebugface 418
-nointerface 417
nointerface 417
-nonlazy 411
-nonuniquenames 413
nonuniquenames 62, 86, 413
-nowarnings 415
nowarnings 415
NSL style 299
NSL_LICENSE_HOST 421
NSL_LICENSE_PORT 421
NSLHOME 40, 422

O

objects
 saving 52
onlyGen 418

-onlygen 418
opening files 50
OptionsMenu 156
options 351
 C code 240
 Restart on FACE errors 139
 Restore state 140
 Trusted state 140

P

parent 125
-paste 413
paste 413
pasting widgets 69
PATH 40, 422
pattern files
 for the application 207
 modifying 216
pbmplus converter 110
per call storage 158
per widget storage 158
pixmap 106
 creating 108
 loading 107
PointerTo() 368
PopupMenu 155
PopupShell 87
postOrderManage 411
-postordermanage 411
-preserve 418
Preserve User Code 55, 211
printing 77
 options 78
program
 xfm 411
project
 and application files 259
 building 255
 combining multiple files 258
 undo 259
 working with 258
project files
 opening 51

saving 54
 projectManagerName 417
 -projmgrname 417
 prototypes
 declaring 247
 PulldownMenu 152

Q

quitting XFM 56

R

raise widget 72
 -rdb 415
 rdb 415
 RDB files
 and C files 381
 and internationalizing 381
 and UIL files 381
 contents of 381
 icon in resource list 90
 options 378
 saving 380
 templates in 204
 -rdbclass 415
 rdbclass 415
 -rdbresources 415
 rdbresources 415
 Read command 50
 Rebuild option 83
 reference name 63
 renaming files 52
 reparent
 widgets 68
 resource data base file 377
 Resource List 88
 resources
 adding new editors 324, 335
 adding to XFM 331
 apply options 92
 attachments 116
 Boolean 98
 callbacks 128
 color 101
 creation script 131
 data base file 377
 dragging values in list 89
 editing 88
 editing procedure 82
 editing window 84
 enumeration 98, 323
 float 98
 fonts 105
 icons 90
 in the Resource Editor 96
 integer 98
 internationalizing 404
 list 88
 listing preferred 95
 multi-line resources 96
 opening edit windows 84
 pixmap 106
 RDB file 377
 removing 97
 representation 323
 restricted 95
 saving sort options 95
 searching in list 82, 89
 selecting 88
 sorting the list 94
 string 98
 table editor 325
 translations 111
 widget 98
 Restart on FACE Errors option 139
 restarting Design process 347
 -restore 348, 417
 restore 417
 -restore option 141
 Restore state 347
 Restore state option 140
 -revision 411
 revision 411
 RGB 102, 363, 366
 rgb.txt 103
 Rogue Wave style 299
 root 125
 in creation scripts 131

- S**
- save messages 55
 - Save Prefs 53, 212
 - classes 267
 - groups 193
 - resources 95
 - saving files 52
 - .fmp files 54, 257
 - C code files 55
 - default name 53
 - fmp files 53
 - interface files 52, 256
 - message files 55
 - optional files 52
 - options 55
 - RDB files 55
 - renaming files 52
 - UIL files 55
 - user code 55
 - script (keyword) 115
 - scripts
 - active value 135
 - callback 128
 - creation 131
 - debugging 136
 - errors in 137
 - Eval 114
 - FACE 124
 - FACE types in 247
 - named 115, 132
 - naming 133
 - setting breakpoints 137
 - shared 133
 - searching
 - for a resource name 82
 - select button 111
 - selecting gadgets 59
 - selecting widgets 59
 - self 125
 - server 417
 - server 417
 - serverHost 417
 - serverhost 417
 - servername 417
 - set scripts 135, 159
 - for C++ class active values 171, 295
 - for drop site active values 168, 391
 - for interface active values 159, 165
 - for widget class active values 170
 - Set Values option 83
 - SetActiveValue() 177
 - shell 414
 - Shift-click 20
 - show 73
 - shrink widget tree 65
 - silentmode 416
 - silentmode 416
 - socket addresses 345
 - socketFile 417
 - socketfile 417
 - socketPort 417
 - socketport 417
 - standalone C code 250
 - standc 415
 - standc 415
 - starting XFaceMaker 40
 - startup 415
 - startup 415
 - startupClasses 411
 - status indicator 49
 - step-by-step mode 137
 - storage 158
 - allocating 160
 - strings
 - edit box 98
 - internationalizing 98, 404
 - structures 397
 - subclass 261
 - superclass 261
 - synchronous 418
 - synchronous 418
- T**
- target 384
 - targetEnvironment 418

-
- targets 418
 - technical support 20
 - templates 196
 - applying 203
 - applying in Resource window 86
 - defining 197
 - deleting 200
 - icon in resource list 90
 - inherited resources 201
 - instances 196
 - loading 199
 - modifying 200
 - resource modes 201
 - saving 197
 - testing
 - the application 205
 - testing the interface 45
 - text editors
 - choosing an editor 127
 - the Wx text editor 127
 - vi 127
 - TextGetString() 179
 - token 413
 - token 413
 - toolkit 417
 - toolkit 47
 - toolkitClasses 417
 - TopLevelShell 155
 - tracking widget names 71
 - TransientShell 58
 - translations 111
 - augment mode 113
 - editing 112
 - override mode 113
 - replace mode 113
 - trusted 417
 - trusted 417
 - Trusted state 347
 - trusted state 55
 - Trusted state option 140
 - try 413
 - try 154, 155, 156, 413
 - Try mode 45
 - type (keyword) 323
 - types
 - declaring 245
 - U**
 - uid 354, 373
 - uil 416
 - uil 353, 416
 - UIL files
 - and C code 354
 - compiler 354
 - contents 355
 - Dialogs 359, 369
 - FACE scripts in 357
 - generating 352
 - icons 363, 366
 - identifiers 367
 - options 352
 - procedures 368
 - structure 355
 - templates in 204
 - to Fm files 362
 - values 364
 - Uil() 362
 - uil2fm 351, 362
 - uialogs 416
 - uialogs 359, 416
 - UilDumpSymbolTable() 362
 - undo 413
 - undo 413
 - undo a project 259
 - unnamed.fm 53
 - V**
 - verbose 418
 - verbose 418
 - View.h++ library 293
 - W**
 - widget class
 - class record 277
 - deleting 267
 - editing 267
 - generating C code 268

- get and set scripts. 266
- limited number 267, 331
- loading 267
- methods. 266
- using active values 264
- widget instance record. 277
- widget classe 261
- widget classes. See also classes
- Widget List 66
- Widget Tree 64
- widgets
 - aligning 74
 - attachments 116
 - copying 68
 - creating 58
 - cutting 68
 - deleting 69
 - displaying 73
 - duplicating 69
 - handles 59
 - hiding 73
 - hierarchy 67
 - laying out. 58
 - lowering 72
 - moving 67
 - naming 62, 85
 - pasting 69
 - raising 72
 - reference name 63
 - reparenting 68
 - resizing 71
 - resources 81, 98
 - saving 52
 - selecting 59
 - tracking names 71
 - x,y position 67
- widgetstores. 47
- window manager 39

X

- X server 240, 414
- XAPPLRESDIR 377, 422
- XBMLANGPATH 110, 422
- XDrawMaker
 - editing draw files in XFM 425

- XFaceMaker
 - alternate libraries 339
 - building a new 338
 - extending 311
 - launching 343
 - restart Design Process 347
- XFaceMaker process 141, 343
 - running several 345
- XFM
 - compilation flag. 270, 340
- xfm 40, 343
- XFM Entry 413
- XFM Full 413
- XFM IDT 413
- xfm.color 103
- XFM_LICENSE_NAME. 40, 422
- XFM_TARGET_MK. 209
- XFM_TARGET_OS 209
- xfm3 413
- xfm3/demo 413
- xfm3/el 413
- xfm3/idt 413
- XfmApplication 121, 411
- xfmCreateCallback 131, 386
- XfmDb. 412
- xfmextend 311, 315
- xfmname 315
- xfmwf 40, 343, 425
- XKeysymDB 113
- XmDisplay 48, 100
- XmList. 87
- XmScreen. 48, 100
- Xt Intrinsics 269
- Xt Intrinsics classes. 261
- XtAddConverter() 333, 338
- XtCreateWidget 236
- XWD 363, 366
- xwd command 110
- XWD format. 110
- xwud command. 110

